# AI 模型推理引擎

从 0 到 1 完全教程

推理引擎深度解析

2026 年 5 月 19 日

# 前言

本书旨在为具备良好编程和数学基础的读者提供一份从零开始构建 AI 模型推理引擎的完整教程。全书内容涵盖从底层硬件优化到上层框架设计的完整技术栈，力求对每个关键概念和实现细节进行深入剖析。

推理引擎是 AI 模型部署的核心组件。它负责将训练好的模型高效地运行在各种硬件平台上，处理用户请求并返回结果。与训练不同，推理更关注延迟、吞吐量和资源消耗。一个优秀的推理引擎需要在算法、系统、硬件三个层面进行深度协同设计。

本书组织如下：

- **第一篇基础篇** (第 1–4 章): 介绍 AI 模型的核心架构、模型文件格式、张量系统设计以及计算图原理。
- **第二篇核心实现篇** (第 5–9 章): 深入算子实现、矩阵运算、内存管理、任务调度和模型加载流程。
- **第三篇优化技术篇** (第 10–15 章): 覆盖量化、向量化、汇编优化、缓存与访存优化、JIT 编译和稀疏优化。
- **第四篇 Transformer 专项优化篇** (第 16–17 章): 专门讨论注意力机制优化、动态形状与批处理技术。
- **第五篇分布式与硬件篇** (第 18 章, 第 22–23 章): 讲解分布式推理策略和 GPU/专用硬件优化。
- **第六篇框架深度剖析篇** (第 19–21 章): 选取 llama.cpp、ONNX Runtime 和 vLLM 三个代表性框架进行完整剖析。

*“优化永无止境，但每一步都值得。”*

# 目录


前言 i

第一部分　基础篇：推理引擎的理论基石 1

第一章　AI 模型核心架构 3

1.1 Transformer 架构概览 . . . 3

1.2 自注意力机制 . . . 4

1.2.1 缩放点积注意力——数学形式 . . . 4

1.2.2 为何缩放因子为 $\sqrt{d_k}$？——梯度分析 . . . 4

1.2.3 因果掩码（Causal Masking） . . . 5

1.3 多头注意力（Multi-Head Attention, MHA） . . . 6

1.3.1 多头分解 . . . 6

1.3.2 多头注意力的参数量 . . . 7

1.4 分组查询注意力与多查询注意力 . . . 7

1.4.1 GQA 与 MQA 的核心思想 . . . 7

1.4.2 GQA 的张量形状变换 . . . 7

1.5 前馈网络（FFN）与门控变体 . . . 8

1.5.1 标准 FFN . . . 8

1.5.2 门控 FFN：SwiGLU 与 GeGLU . . . 9

1.6 归一化层：LayerNorm 与 RMSNorm . . . 9

1.6.1 经典 LayerNorm . . . 9

1.6.2 RMSNorm . . . 10

1.6.3 Pre-Norm vs Post-Norm . . . 10

1.7 旋转位置编码（RoPE） . . . 11

1.7.1 从 2D 旋转到高维扩展 . . . 11

1.7.2 频率分布与长程衰减 . . . 12

1.7.3 NTK-Aware 缩放与长上下文扩展 . . . 13

1.8 嵌入层与权重绑定 . . . 13

1.8.1 Token 嵌入与位置嵌入 . . . 13

1.8.2 权重绑定（Weight Tying） . . . 14

1.9 混合专家（Mixture of Experts, MoE） . . . 14
1.9.1 稀疏 MoE 的形式化 . . . 14
1.9.2 负载均衡与专家容量 . . . 15
1.10 KV Cache 机制 . . . 16
1.10.1 KV Cache 的存储与更新 . . . 16
1.10.2 KV Cache 显存计算 . . . 16
1.10.3 KV Cache 的转置与内存布局 . . . 17
1.11 完整前向传播流程 . . . 17

**第二章 模型文件格式详解 19**
2.1 概述 . . . 19
2.2 GGUF 格式（llama.cpp 生态） . . . 19
2.2.1 文件结构概览 . . . 19
2.2.2 Metadata 键值对详解 . . . 20
2.2.3 Tensor 命名约定 . . . 21
2.2.4 量化 Tensor 的块级存储 . . . 22
2.2.5 GGUF 读取器实现要点 . . . 23
2.2.6 GGML 到 GGUF 的迁移 . . . 24
2.3 ONNX 格式（开放神经网络交换） . . . 25
2.3.1 Protobuf-based 结构 . . . 25
2.3.2 算子描述与属性 . . . 26
2.3.3 外部数据存储（External Data） . . . 27
2.4 HuggingFace 与 safetensors . . . 28
2.4.1 PyTorch checkpoints 与安全缺陷 . . . 28
2.4.2 safetensors 格式详解 . . . 28
2.4.3 config.json 与 tokenizer 文件 . . . 29
2.5 其他推理格式 . . . 31
2.6 格式对比与选择指南 . . . 32
2.7 格式转换与工具链 . . . 32

**第三章 张量系统设计 35**
3.1 张量抽象 . . . 35
3.1.1 核心属性 . . . 35
3.1.2 内存布局与性能 . . . 36
3.2 步长张量与视图 . . . 37
3.2.1 泛化的步长机制 . . . 37
3.2.2 零拷贝视图操作 . . . 37
3.3 量化张量 . . . 39
3.3.1 量化-反量化（QDQ）范式 . . . 39

3.4 张量操作 API 设计 . . . 40
3.4.1 原地操作与可变语义 . . . 41
3.5 设备抽象与内存管理 . . . 42
3.5.1 设备层级 . . . 42
3.5.2 缓冲区管理 . . . 42
3.5.3 内存管理策略 . . . 43
3.6 FP16 与 BF16 存储 . . . 44
3.6.1 IEEE 754 半精度 vs BFloat16 . . . 44
3.6.2 格式转换与混合精度推理 . . . 44
3.7 张量系统实现考量 . . . 45

**第四章 计算图原理与优化 47**
4.1 计算图基础 . . . 47
4.1.1 有向无环图（DAG）作为计算表示 . . . 47
4.1.2 静态图 vs 动态图 . . . 47
4.2 图中间表示（IR）设计 . . . 49
4.2.1 算子模式定义 . . . 49
4.2.2 图的拓扑结构构建 . . . 49
4.3 图优化 Pass . . . 50
4.3.1 常量折叠（Constant Folding） . . . 50
4.3.2 算子融合（Operator Fusion） . . . 51
4.3.3 死代码消除（Dead Code Elimination, DCE） . . . 51
4.3.4 公共子表达式消除（CSE） . . . 52
4.3.5 代数简化（Algebraic Simplification） . . . 52
4.3.6 布局优化（Layout Optimization） . . . 53
4.3.7 内存规划（Memory Planning） . . . 53
4.4 图分割 . . . 54
4.4.1 异构计算的分割 . . . 54
4.5 执行调度 . . . 55
4.5.1 拓扑排序与并行执行 . . . 55
4.5.2 依赖分析：RAW、WAR、WAW . . . 55
4.6 形状推断 . . . 56
4.6.1 静态形状推断 . . . 56
4.6.2 符号形状与动态形状 . . . 56
4.7 图中的控制流 . . . 57
4.8 子图替换与模式匹配 . . . 57
4.9 图序列化 . . . 58
4.10 从图到执行：完整流水线 . . . 59

**第二部分　核心实现篇：引擎的内部机制** **61**

**第五章　算子实现原理** **63**
5.1 算子接口设计 . . . 63
5.2 矩阵乘法算子 . . . 64
5.2.1 朴素实现与缓存分析 . . . 64
5.2.2 循环顺序分析 . . . 64
5.2.3 分块（Tiling）策略 . . . 65
5.2.4 块大小选择策略 . . . 65
5.2.5 微内核（Micro-Kernel） . . . 66
5.2.6 打包（Packing）策略 . . . 67
5.2.7 批量矩阵乘法 . . . 68
5.3 卷积算子 . . . 68
5.3.1 直接卷积 . . . 68
5.3.2 Im2col + GEMM 方法 . . . 69
5.3.3 Winograd 卷积 . . . 69
5.3.4 FFT 基卷积 . . . 70
5.3.5 深度可分离卷积 . . . 70
5.4 注意力算子 . . . 71
5.5 激活函数算子 . . . 72
5.5.1 精确公式 . . . 72
5.5.2 高效近似 . . . 72
5.6 归一化算子 . . . 73
5.6.1 LayerNorm . . . 73
5.6.2 RMSNorm . . . 73
5.7 逐元素算子 . . . 73
5.8 归约算子 . . . 74
5.9 嵌入查找与位置编码 . . . 75
5.9.1 嵌入查找 . . . 75
5.9.2 RoPE 旋转位置编码 . . . 75
5.9.3 KV 缓存更新 . . . 76
5.10 算子融合 . . . 76

**第六章　矩阵分解与转换** **79**
6.1 Roofline 模型与算术强度 . . . 79
6.1.1 Roofline 模型定义 . . . 79
6.1.2 用 Roofline 分析矩阵乘法 . . . 80
6.2 低秩分解与模型压缩 . . . 80
6.2.1 SVD 分解 . . . 80

6.2.2 二因子分解 . . . 81
6.2.3 在注意力和 FFN 中的应用 . . . 81
6.3 QR 分解与 Cholesky 分解 . . . 82
6.3.1 QR 分解 . . . 82
6.3.2 Cholesky 分解 . . . 82
6.4 量化与矩阵结构 . . . 83
6.4.1 仅权重量化 vs 权重 + 激活量化 . . . 83
6.4.2 量化矩阵乘法的实现 . . . 83
6.5 张量核心利用 . . . 84
6.5.1 NVIDIA 张量核心架构 . . . 84
6.5.2 张量核心编程模型 . . . 84
6.6 混合精度矩阵乘法 . . . 85
6.7 分级分块策略 . . . 85
6.7.1 分级分块 . . . 85
6.7.2 自动参数调优 . . . 86
6.8 矩阵变换汇总 . . . 87

**第七章 内存管理 89**
7.1 内存层次结构概览 . . . 89
7.2 内存分配策略 . . . 90
7.2.1 竞技场分配器 . . . 90
7.2.2 池分配器 . . . 90
7.2.3 内存重用与生命周期分析 . . . 91
7.3 设备内存管理 . . . 92
7.3.1 CUDA 内存类型 . . . 92
7.3.2 GPU 内存分配优化 . . . 92
7.3.3 流序内存分配 . . . 93
7.3.4 统一内存 . . . 93
7.4 推理内存规划 . . . 93
7.4.1 静态内存规划 . . . 93
7.4.2 贪婪大小排序分配 . . . 94
7.5 KV 缓存内存管理 . . . 95
7.5.1 预分配与动态增长 . . . 95
7.5.2 PagedAttention：虚拟内存用于 KV 缓存 . . . 95
7.5.3 碎片化与驱逐策略 . . . 96
7.6 权重内存管理 . . . 96
7.6.1 内存映射加载 . . . 96
7.6.2 NUMA 意识 . . . 97

7.7 内存带宽优化 . . . 97
7.7.1 合并访存 . . . 97
7.7.2 双缓冲 . . . 98
7.7.3 张量引用计数与垃圾回收 . . . 98

**第八章 任务调度 99**
8.1 线程池设计 . . . 99
8.1.1 基础架构 . . . 99
8.1.2 工作窃取 . . . 100
8.1.3 自旋等待 vs 阻塞 . . . 100
8.1.4 CPU 亲和性 . . . 101
8.2 计算图的并行执行 . . . 101
8.2.1 拓扑排序与就绪队列 . . . 101
8.2.2 依赖关系类型 . . . 102
8.3 GPU 流管理 . . . 102
8.3.1 CUDA 流 . . . 102
8.3.2 计算与传输重叠 . . . 102
8.3.3 CUDA 图 . . . 103
8.4 批处理调度 . . . 103
8.4.1 静态批处理 . . . 103
8.4.2 连续批处理 . . . 103
8.4.3 优先级调度 . . . 104
8.5 流水线并行调度 . . . 104
8.5.1 气泡分析 . . . 105
8.6 抢占与状态交换 . . . 105
8.7 解码阶段调度 . . . 105
8.7.1 解码循环的调度挑战 . . . 106
8.7.2 连续批处理在解码时的表现 . . . 106
8.8 负载均衡 . . . 106

**第九章 模型加载与运行流程 109**
9.1 端到端模型加载流水线 . . . 109
9.1.1 第 1 步：检测模型格式 . . . 109
9.1.2 第 2 步：解析模型配置 . . . 110
9.1.3 第 3 步：构建计算图骨架 . . . 111
9.1.4 第 4 步：加载权重 . . . 111
9.1.5 第 5 步：应用图优化 . . . 112
9.1.6 第 6 步：内存规划 . . . 112
9.1.7 第 7 步：创建执行上下文 . . . 112

9.2 权重加载策略详解 . . . 113
9.2.1 显式读取 . . . 113
9.2.2 量化加载 . . . 113
9.2.3 张量名到层的映射 . . . 113
9.3 会话与上下文管理 . . . 114
9.3.1 多会话支持 . . . 114
9.3.2 会话序列化 . . . 115
9.4 推理执行循环 . . . 115
9.4.1 预填充阶段（Prefill Phase） . . . 115
9.4.2 解码阶段（Decode Phase） . . . 116
9.4.3 采样策略 . . . 116
9.4.4 Logit 处理 . . . 117
9.5 批处理执行 . . . 117
9.5.1 预填充批处理 . . . 117
9.5.2 解码批处理 . . . 118
9.5.3 批处理 KV 缓存 . . . 118
9.6 异常处理 . . . 118
9.7 性能分析与调试 . . . 119
9.7.1 逐层计时 . . . 119
9.7.2 内存追踪 . . . 119
9.7.3 算子统计 . . . 120

**第三部分 优化技术篇：极致性能的秘密 121**

**第十章 低精度量化 123**
10.1 量化基础理论 . . . 123
10.1.1 量化问题的数学形式化 . . . 123
10.1.2 对称量化与非对称量化 . . . 123
10.1.3 粒度量化的层次 . . . 124
10.1.4 动态量化与静态量化 . . . 125
10.1.5 量化感知训练与后训练量化 . . . 125
10.2 整数矩阵乘法的量化计算 . . . 125
10.2.1 量化的 GEMM 运算 . . . 125
10.2.2 累加与再量化 . . . 126
10.3 GGUF 量化类型详解 . . . 126
10.3.1 Q4_0：4 位对称逐块量化 . . . 126
10.3.2 Q4_1：4 位非对称逐块量化 . . . 127
10.3.3 Q5_0 与 Q5_1 . . . 127

10.3.4 Q8_0 与 Q8_1 . . . 127
10.3.5 K-Quants：Q2_K 至 Q6_K . . . 127
10.3.6 IQ 系列：重要性感知量化 . . . 128
10.4 GPTQ：精确后训练量化 . . . 129
10.4.1 OBQ 基础理论 . . . 129
10.4.2 GPTQ 算法流程 . . . 129
10.4.3 Cholesky 重正化与组大小 . . . 130
10.4.4 性能特性 . . . 130
10.5 AWQ：激活感知权重量化 . . . 130
10.5.1 核心洞察 . . . 130
10.5.2 逐通道缩放方法 . . . 131
10.5.3 与逐组量化的集成 . . . 131
10.6 SmoothQuant . . . 131
10.6.1 问题描述 . . . 131
10.6.2 数学等效变换 . . . 132
10.7 LLM.int8() . . . 132
10.7.1 混合精度分解 . . . 132
10.8 KV 缓存量化 . . . 133
10.8.1 键缓存量化 . . . 133
10.8.2 值缓存量化 . . . 133
10.8.3 先进方案：KIVI 与 KVQuant . . . 134
10.9 FP8 与块浮点 . . . 134
10.9.1 FP8 数据格式 . . . 134
10.9.2 块浮点（Block Floating Point） . . . 134
10.10 量化硬件支持 . . . 134
10.10.1 NVIDIA Tensor Core DP4A . . . 134
10.10.2 ARM NEON SDOT/UDOT . . . 135
10.10.3 x86 VNNI . . . 135
10.10.4 量化建模实践总结 . . . 135

**第十一章 向量化与 SIMD 137**

11.1 SIMD 基础 . . . 137
11.1.1 SIMD 范式 . . . 137
11.1.2 寄存器宽度演进 . . . 137
11.1.3 SIMD 数据类型 . . . 138
11.2 x86 SIMD Intrinsic 详解 . . . 138
11.2.1 基本加载与存储操作 . . . 138
11.2.2 浮点计算指令 . . . 139

11.2.3 Int8 整数点积：VNNI . . . 139
11.2.4 Shuffle 与重排操作 . . . 140
11.2.5 Gather 与 Scatter . . . 140
11.2.6 掩码操作（AVX-512） . . . 141
11.3 向量化 MatMul 微内核 . . . 141
11.3.1 微内核设计原则 . . . 141
11.3.2 6x16 fp32 微内核（AVX2） . . . 141
11.3.3 处理奇数尺寸 . . . 142
11.4 ARM NEON 向量化 . . . 143
11.4.1 NEON 基础类型与指令 . . . 143
11.4.2 NEON 矩阵乘法示例 . . . 144
11.5 自动向量化 . . . 144
11.5.1 编译器自动向量化能力 . . . 144
11.5.2 阻止自动向量化的常见模式 . . . 145
11.5.3 编译器 Hint 机制 . . . 145
11.5.4 SLP 向量化 . . . 145
11.6 SIMD 优化策略 . . . 146
11.6.1 循环向量化的层次 . . . 146
11.6.2 交错数据布局 . . . 146
11.6.3 归约向量化 . . . 146
11.6.4 部分向量化与边界处理 . . . 147
11.6.5 SIMD 性能调试 . . . 147

**第十二章 汇编级别优化 149**
12.1 为何需要汇编级别优化 . . . 149
12.1.1 编译器生成的局限 . . . 149
12.1.2 手写汇编的实际收益 . . . 149
12.2 x86-64 汇编基础 . . . 150
12.2.1 寄存器集 . . . 150
12.2.2 SIMD 指令编码 . . . 150
12.2.3 FMA 指令变体 . . . 150
12.2.4 寻址模式 . . . 151
12.3 调用约定 . . . 151
12.3.1 System V AMD64 ABI（Linux/macOS） . . . 151
12.3.2 Windows x64 ABI . . . 151
12.4 寄存器分配技术 . . . 152
12.4.1 GEMM 微内核的寄存器需求分析 . . . 152
12.4.2 寄存器溢出与填充交换 . . . 152

12.4.3 寄存器复用调度 . . . 153
12.5 指令调度 . . . 153
12.5.1 CPU 流水线与乱序执行 . . . 153
12.5.2 指令选择技巧 . . . 153
12.5.3 循环展开 . . . 154
12.5.4 软件流水线 . . . 154
12.5.5 硬件预取与软件预取 . . . 155
12.6 CISC 特殊指令 . . . 155
12.6.1 高速内存复制 . . . 155
12.6.2 BMI 位操作指令 . . . 155
12.7 内联汇编与独立汇编 . . . 156
12.7.1 GCC/Clang 内联汇编语法 . . . 156
12.7.2 内联汇编的局限性 . . . 156
12.7.3 独立汇编文件（NASM / GAS） . . . 157
12.8 汇编优化实践 . . . 157

**第十三章 缓存与访存优化 159**
13.1 缓存层次结构深度剖析 . . . 159
13.1.1 现代 CPU 缓存架构 . . . 159
13.1.2 缓存定位与索引 . . . 159
13.1.3 TLB：转换后备缓冲区 . . . 160
13.1.4 缓存一致性协议 . . . 160
13.1.5 非时序存储 . . . 160
13.2 缓存分块（Tiling/Blocking） . . . 161
13.2.1 分块原理 . . . 161
13.2.2 分层分块策略：L3 → L2 → L1 . . . 161
13.2.3 分块大小的解析模型 . . . 162
13.2.4 自动调优（Auto-tuning） . . . 162
13.3 循环变换 . . . 162
13.3.1 循环交换 . . . 162
13.3.2 循环分块（循环 Tiling） . . . 162
13.3.3 循环展开与循环融合/裂变 . . . 163
13.4 矩阵数据布局 . . . 163
13.4.1 行主序与列主序 . . . 163
13.4.2 Z 序（Morton Order） . . . 164
13.4.3 分块/打包布局 . . . 164
13.5 预取技术 . . . 164
13.5.1 硬件预取器 . . . 164

13.5.2 软件预取 . . . 165
13.5.3 GPU 上的预取 . . . 165
13.6 NUMA 优化 . . . 166
13.6.1 NUMA 架构 . . . 166
13.6.2 First-Touch 策略 . . . 166
13.6.3 numactl 与 libnuma . . . 166
13.7 页面大小优化 . . . 167
13.7.1 4KB 与巨页 . . . 167
13.7.2 透明巨页的局限性 . . . 167
13.7.3 显式巨页分配 . . . 167
13.8 访存优化的整体图景 . . . 168

**第十四章 JIT 编译 169**
14.1 JIT 编译基础 . . . 169
14.1.1 JIT 在推理中的角色 . . . 169
14.1.2 AOT 与 JIT 的对比 . . . 169
14.2 JIT 编译架构 . . . 170
14.2.1 推理引擎中的 JIT 管线 . . . 170
14.2.2 JIT 代码缓存管理 . . . 170
14.3 编译后端 . . . 171
14.3.1 C 级 JIT . . . 171
14.3.2 LLVM JIT（ORC JIT） . . . 171
14.3.3 直接机器码生成 . . . 172
14.3.4 编译后端选择指南 . . . 172
14.4 核函数代码生成 . . . 173
14.4.1 可特化的参数 . . . 173
14.4.2 常量传播的价值 . . . 173
14.4.3 寄存器分配在 JIT 时的优势 . . . 173
14.5 基于模板的 JIT . . . 174
14.5.1 XNNPACK 风格的模板 JIT . . . 174
14.5.2 缓存键与函数调度表 . . . 174
14.6 高层 JIT 框架 . . . 175
14.6.1 Triton 语言方法 . . . 175
14.6.2 TVM/Apache TVM . . . 175
14.6.3 MLIR 与 IREE . . . 175
14.7 JIT 缓存策略详解 . . . 176
14.7.1 磁盘持久缓存 . . . 176
14.7.2 内存缓存管理 . . . 176

14.7.3 预编译和热身 . . . 176
14.8 JIT 编译实践 . . . 177

**第十五章 稀疏优化 179**
15.1 神经网络的稀疏性类型 . . . 179
15.1.1 非结构化稀疏 . . . 179
15.1.2 结构化稀疏：2:4 模式 . . . 179
15.1.3 块稀疏 . . . 180
15.1.4 激活稀疏与注意力稀疏 . . . 180
15.2 稀疏矩阵存储格式 . . . 180
15.2.1 CSR 格式 . . . 180
15.2.2 CSC 格式 . . . 181
15.2.3 COO 格式 . . . 181
15.2.4 ELLPACK 格式 . . . 181
15.2.5 混合格式与高级格式 . . . 181
15.3 稀疏矩阵乘法实现 . . . 182
15.3.1 CSR-based SpMM . . . 182
15.3.2 块 SpMM . . . 182
15.3.3 向量化 SpMM . . . 183
15.4 2:4 结构化稀疏 . . . 183
15.4.1 硬件加速机制 . . . 183
15.4.2 cuSPARSELt 库 . . . 183
15.4.3 2:4 剪枝到目标的流程 . . . 184
15.5 稀疏注意力 . . . 184
15.5.1 局部/滑动窗口注意力 . . . 184
15.5.2 组合稀疏模式 . . . 185
15.5.3 FlashAttention 与块稀疏的集成 . . . 185
15.6 剪枝方法 . . . 185
15.6.1 幅值剪枝（Magnitude Pruning） . . . 185
15.6.2 基于 Hessian 的剪枝：SparseGPT . . . 185
15.6.3 移动剪枝（Movement Pruning） . . . 186
15.7 运行时稀疏操作选择 . . . 186
15.7.1 阈值驱动的核函数选择 . . . 186
15.7.2 格式转换开销 . . . 186
15.7.3 稀疏的内存节省 . . . 187
15.8 推理引擎中的稀疏张量表示 . . . 187
15.8.1 稀疏张量的 IR 表示 . . . 187
15.8.2 稀疏感知的操作分发 . . . 187

15.8.3 混合执行策略 . . . . . 188

**第四部分 Transformer 专项优化篇 189**

**第十六章 Attention 机制专项优化 191**

16.1 注意力的计算与存储瓶颈 . . . . . 191
16.1.1 标准注意力实现 . . . . . 191
16.1.2 存储压力分析 . . . . . 192
16.2 FlashAttention：分块计算的革命 . . . . . 193
16.2.1 在线 Softmax 算法 . . . . . 193
16.2.2 FlashAttention-v1 分块算法 . . . . . 194
16.3 FlashAttention-2：并行度与调度优化 . . . . . 195
16.4 FlashAttention-3：Hopper 架构特化 . . . . . 196
16.5 PagedAttention 与 vLLM . . . . . 198
16.5.1 KV Cache 碎片问题 . . . . . 198
16.5.2 PagedAttention 的分页实现 . . . . . 199
16.5.3 Copy-on-Write 内存共享 . . . . . 200
16.6 KV Cache 优化技术总览 . . . . . 201
16.6.1 多头注意力的变体 . . . . . 201
16.6.2 KV Cache 压缩技术 . . . . . 201
16.7 投机解码 . . . . . 202
16.7.1 基本投机解码框架 . . . . . 202
16.7.2 投机解码的演进 . . . . . 203
16.8 注意力优化在推理引擎中的实践 . . . . . 204
16.8.1 混合注意力算子选择策略 . . . . . 204
16.8.2 预填充与解码的混合调度 . . . . . 205

**第十七章 动态形状与批处理 207**

17.1 动态形状问题的本质 . . . . . 207
17.1.1 可变序列长度的挑战 . . . . . 207
17.1.2 序列打包 . . . . . 207
17.2 连续批处理 . . . . . 208
17.2.1 静态批处理 vs 连续批处理 . . . . . 209
17.2.2 迭代级调度器 . . . . . 210
17.2.3 Prefill-Decode 混合调度 . . . . . 212
17.3 预填充-解码分离架构 . . . . . 213
17.3.1 Mooncake 式分离架构 . . . . . 213
17.3.2 SGLang 的 RadixAttention . . . . . 214

17.4 动态形状的算子支持 . . . 215
17.4.1 形状无关的内核 . . . 215
17.4.2 动态形状的内存规划 . . . 216
17.4.3 变长注意力核函数 . . . 216
17.5 批处理的效率分析 . . . 217
17.5.1 吞吐量建模 . . . 217
17.5.2 请求分布的负载均衡 . . . 219

**第五部分 分布式与硬件篇 221**

**第十八章 分布式推理 223**
18.1 张量并行 . . . 223
18.1.1 线性层的张量并行 . . . 223
18.1.2 Transformer 层的张量并行 . . . 224
18.1.3 张量并行的通信分析 . . . 225
18.1.4 序列并行 . . . 226
18.2 流水线并行 . . . 227
18.2.1 流水线并行的基本概念 . . . 227
18.2.2 流水线调度策略 . . . 227
18.3 专家并行 . . . 228
18.3.1 MoE 路由与专家分配 . . . 229
18.3.2 All-to-All 通信 . . . 229
18.3.3 专家负载均衡 . . . 230
18.4 数据并行 . . . 230
18.5 混合并行与自动搜索 . . . 231
18.5.1 TP+PP+DP+EP 的组合策略 . . . 231
18.5.2 自动并行策略搜索 . . . 232
18.6 通信集合原语 . . . 233
18.6.1 NCCL 与集合通信算法 . . . 233
18.6.2 互联技术 . . . 234
18.6.3 通信与计算的重叠 . . . 235
18.6.4 分布式 KV Cache . . . 236

**第十九章 GPU 编程与 CUDA 优化 239**
19.1 GPU 架构与执行模型 . . . 239
19.1.1 流多处理器架构 . . . 239
19.1.2 Warp 与 SIMT 执行模型 . . . 240
19.1.3 显存层级 . . . 240

19.2 CUDA Kernel 优化技术 . . . 241
19.2.1 占用率优化 . . . 241
19.2.2 共享内存编程 . . . 242
19.2.3 合并内存访问 . . . 243
19.2.4 CUDA 流与异步执行 . . . 244
19.3 Tensor Core 编程 . . . 245
19.3.1 Tensor Core 基础 . . . 245
19.3.2 Hopper 的异步 Tensor Core . . . 247
19.4 Kernel 融合 . . . 248
19.4.1 Kernel 融合的原理与收益 . . . 248
19.4.2 协作组 . . . 249
19.5 多 GPU 推理的 CUDA 实现 . . . 251
19.5.1 NCCL 与 CUDA 的集成 . . . 251
19.5.2 MIG 与 GPU 分区 . . . 252
19.6 推理 Kernel 的最佳实践 . . . 253
19.6.1 GEMV 优化（增量解码核心） . . . 253
19.6.2 激活函数与归一化的向量化实现 . . . 253

**第二十章 专用硬件与异构计算 255**
20.1 NVIDIA GPU 架构演进 . . . 255
20.1.1 从 Volta 到 Blackwell . . . 255
20.1.2 Transformer Engine 与 FP8 . . . 256
20.2 AMD GPU 与 ROCm 生态 . . . 257
20.2.1 MI250X 与 MI300X 架构 . . . 257
20.2.2 HIP 与 CUDA 的互操作性 . . . 258
20.3 专用 AI 推理硬件 . . . 259
20.3.1 Google TPU . . . 259
20.3.2 Groq LPU . . . 260
20.3.3 Cerebras WSE . . . 261
20.3.4 其他专用硬件 . . . 262
20.4 异构计算架构 . . . 264
20.4.1 算子放置策略 . . . 264
20.4.2 llama.cpp 的异构执行 . . . 264
20.5 移动端与边缘推理 . . . 266
20.5.1 移动端推理的特殊约束 . . . 266
20.5.2 移动端推理框架对比 . . . 267
20.6 硬件感知 Kernel 设计 . . . 267
20.6.1 跨硬件的编程抽象 . . . 267

20.6.2 硬件自动调优 . . . . . 268
20.7 新兴趋势 . . . . . 269
20.7.1 超低精度推理 . . . . . 269
20.7.2 稀疏计算的硬件支持 . . . . . 270
20.7.3 存内计算与光学互联 . . . . . 270

**第六部分 框架深度剖析篇 273**

**第二十一章 llama.cpp 框架深度剖析 275**
21.1 项目概览与设计哲学 . . . . . 275
21.1.1 起源与动机 . . . . . 275
21.1.2 架构总览 . . . . . 276
21.1.3 支持的模型家族 . . . . . 276
21.2 GGUF 格式深度剖析 . . . . . 277
21.2.1 从 GGML 到 GGUF 的演进 . . . . . 277
21.2.2 二进制格式规范 . . . . . 277
21.2.3 张量信息数组 . . . . . 279
21.2.4 字节序和对齐 . . . . . 281
21.3 GGML 张量库 . . . . . 281
21.3.1 核心数据结构 . . . . . 281
21.3.2 操作类型系统 . . . . . 282
21.3.3 计算图的构建与执行 . . . . . 283
21.3.4 内存管理 . . . . . 284
21.4 后端系统 . . . . . 284
21.4.1 后端抽象层 . . . . . 284
21.4.2 CPU 后端 . . . . . 285
21.4.3 CUDA 后端 . . . . . 286
21.4.4 Metal 后端 . . . . . 286
21.4.5 Vulkan 后端 . . . . . 286
21.4.6 多后端执行与张量分割 . . . . . 286
21.5 模型加载与架构实例化 . . . . . 287
21.5.1 加载流程 . . . . . 287
21.5.2 核心数据结构 . . . . . 288
21.5.3 架构适配 . . . . . 288
21.5.4 量化加载 . . . . . 288
21.6 前向传播实现 . . . . . 289
21.6.1 推理主循环 . . . . . 289
21.6.2 注意力机制与 KV 缓存 . . . . . 292

21.6.3 Flash Attention 实现 . . . 292
21.6.4 位置编码（RoPE 及其扩展） . . . 293
21.7 关键优化技术 . . . 294
21.7.1 量化矩阵乘法 . . . 294
21.7.2 多线程并行 . . . 294
21.7.3 KV 缓存量化 . . . 294
21.7.4 内存映射模型加载 . . . 295
21.7.5 服务端优化——连续批处理与流式输出 . . . 295
21.7.6 Mamba/SSM 架构支持 . . . 295
21.8 性能特征 . . . 296
21.8.1 CPU 推理性能 . . . 296
21.8.2 GPU 加速效果 . . . 296
21.8.3 量化对质量的影响 . . . 296
21.9 构建推理引擎的经验总结 . . . 296
21.9.1 C 优于 C++ 的选择 . . . 296
21.9.2 自研张量库的选择 . . . 297
21.9.3 量化优先的设计 . . . 298
21.9.4 后端抽象的价值 . . . 298
21.9.5 工程实践总结 . . . 298

**第二十二章 ONNX Runtime 框架深度剖析 301**
22.1 项目概览 . . . 301
22.1.1 起源与使命 . . . 301
22.1.2 架构层次 . . . 301
22.1.3 ONNX Runtime 的核心模块 . . . 303
22.2 ONNX 格式与 ORT 中的模型表示 . . . 303
22.2.1 ONNX 协议缓冲区格式 . . . 303
22.2.2 外部数据格式 . . . 304
22.2.3 ORT 格式 . . . 305
22.3 Execution Provider（EP）系统 . . . 305
22.3.1 EP 抽象接口 . . . 305
22.3.2 内置 Execution Provider 一览 . . . 306
22.3.3 图分区算法 . . . 306
22.3.4 内存管理 . . . 307
22.4 图优化与变换 . . . 307
22.4.1 图优化概述 . . . 307
22.4.2 规则的图优化 . . . 308
22.4.3 关键融合模式 . . . 308

22.4.4 常量折叠 . . . 309
22.4.5 布局优化：NCHW 到 NHWC . . . 309
22.5 CPU Execution Provider（MLAS） . . . 309
22.5.1 MLAS 概述 . . . 309
22.5.2 SGEMM 实现 . . . 309
22.5.3 整数量化 GEMM（QGEMM） . . . 310
22.5.4 SIMD 指令集优化 . . . 310
22.5.5 卷积优化：im2col + SGEMM . . . 311
22.5.6 线程调度与亲和性 . . . 311
22.6 CUDA 和 TensorRT Execution Provider . . . 312
22.6.1 CUDA EP 架构 . . . 312
22.6.2 CUDA Graph 集成 . . . 312
22.6.3 TensorRT EP . . . 312
22.7 Session 与运行时模型 . . . 313
22.7.1 Session 的生命周期 . . . 313
22.7.2 SessionOptions 关键参数 . . . 313
22.7.3 执行计划（Execution Plan） . . . 313
22.7.4 IO 绑定（IO Binding） . . . 314
22.7.5 并发执行 . . . 315
22.8 ONNX Runtime 中的量化 . . . 315
22.8.1 概述 . . . 315
22.8.2 QDQ 格式 . . . 315
22.8.3 QDQ 优化过程 . . . 316
22.8.4 量化推理的内核 . . . 316
22.9 关键设计经验 . . . 316
22.9.1 EP 抽象的多面性 . . . 316
22.9.2 图优化是差异化价值 . . . 317
22.9.3 QDQ 作为量化中间表示 . . . 317
22.9.4 内存规划是性能的基础 . . . 317
22.9.5 图缓存的工程价值 . . . 317

**第二十三章 vLLM 框架深度剖析 319**

23.1 项目概览 . . . 319
23.1.1 起源与核心创新 . . . 319
23.1.2 架构总览 . . . 319
23.2 PagedAttention：KV 缓存的虚拟内存 . . . 320
23.2.1 传统 KV 缓存的问题 . . . 320
23.2.2 PagedAttention 的核心思想 . . . 320

23.2.3 块大小选择的权衡 . . . 321
23.2.4 块表与内核实现 . . . 321
23.2.5 内存共享与写时拷贝（Copy-on-Write） . . . 324
23.2.6 块分配与释放 . . . 324
23.2.7 抢占（Preemption） . . . 324
23.3 连续批处理（Continuous Batching） . . . 324
23.3.1 静态批处理的局限性 . . . 324
23.3.2 连续批处理的工作方式 . . . 325
23.3.3 Prefill 与 Decode 的调度 . . . 326
23.4 LLMEngine 与核心执行流程 . . . 327
23.4.1 LLMEngine . . . 327
23.4.2 Scheduler . . . 328
23.4.3 Sequence 抽象 . . . 328
23.4.4 BlockSpaceManager . . . 328
23.4.5 ModelRunner . . . 329
23.5 分布式执行 . . . 329
23.5.1 张量并行（Tensor Parallelism） . . . 329
23.5.2 张量并行与 PagedAttention 的融合 . . . 330
23.5.3 流水线并行（Pipeline Parallelism） . . . 330
23.5.4 分布式 Worker 管理 . . . 330
23.6 内存管理策略 . . . 331
23.6.1 GPU 内存的划分 . . . 331
23.6.2 KV 缓存块预算 . . . 331
23.6.3 前缀缓存（Prefix Caching） . . . 331
23.7 关键优化技术 . . . 331
23.7.1 FlashAttention 集成 . . . 331
23.7.2 融合内核（Fused Kernels） . . . 332
23.7.3 CUDA Graph 捕获 . . . 332
23.7.4 CPU 与 GPU 重叠 . . . 333
23.8 在线推理服务特性 . . . 333
23.8.1 OpenAI 兼容 API . . . 333
23.8.2 采样策略 . . . 333
23.8.3 结构化输出和约束解码 . . . 334
23.8.4 Multi-LoRA . . . 334
23.8.5 工具调用（Function Calling） . . . 334
23.9 性能特征 . . . 334
23.9.1 吞吐量对比 . . . 334
23.9.2 内存效率 . . . 335

23.9.3 扩展性 . . . . . 335
23.10 构建推理引擎的经验总结 . . . . . 336
23.10.1 KV 缓存是核心资源 . . . . . 336
23.10.2 连续批处理是不可或缺的 . . . . . 336
23.10.3 Prefill 和 Decode 需要不同对待 . . . . . 336
23.10.4 前缀共享是巨大优化 . . . . . 336
23.10.5 开源 + 生产特性 = 巨大影响力 . . . . . 337

**参考资料** **339**

# 第一部分

# 基础篇：推理引擎的理论基石

# 第一章　AI 模型核心架构

## 1.1　Transformer 架构概览

Transformer 架构自 2017 年提出以来，已成为现代大语言模型（LLM）的基础骨架。当前主流推理引擎所面对的模型——无论 LLaMA、Mistral、Qwen 还是 DeepSeek——均以 Transformer 为核心构建。

**Transformer 的两大范式**

**Encoder-Decoder（编码器-解码器）：**

- 原始 Transformer（Vaswani et al., 2017）和 T5 采用此结构
- 编码器对输入序列做双向注意力，生成上下文表示
- 解码器以自回归方式生成输出，通过交叉注意力（cross-attention）关注编码器输出
- 适用场景：机器翻译、文本摘要等需要完整理解输入后生成的任务
- 推理时需维护编码器的 KV Cache（不可忽视的显存开销）

**Decoder-Only（仅解码器）：**

- GPT 系列、LLaMA、Mistral、Gemma 等主流 LLM 均采用此结构
- 去掉编码器和交叉注意力，仅保留自注意力层 + FFN 层
- 因果掩码（causal mask）确保自回归性质：token $i$ 只能关注 token $j \leq i$
- 每个 token 的表示通过堆叠的 decoder 层逐步精化
- 训练效率高（无需编码器），推理时仅需维护自注意力的 KV Cache

**为何现代 LLM 几乎统一为 Decoder-Only？**

1. **训练效率**：每个 token 可以作为训练目标（下一个 token 预测），所有 token 的前向传播可并行化——这与 encoder-decoder 不同，后者需等待编码器完成后才能开始解码器训练
2. **扩展性**：decoder-only 模型可以无缝扩展到极大参数规模，训练损失随参数量的幂律关系（scaling law）更加平滑
3. **上下文学习**：GPT 系列证明了 decoder-only 模型在上下文学习（in-context learning）上的惊人能力，encoder-decoder 模型在此能力上明显弱于 decoder-only
4. **统一的接口**：一切任务都可以转化为" 文本补全"——这是 decoder-only 模型的自然界面

## 1.2 自注意力机制

### 1.2.1 缩放点积注意力——数学形式

自注意力（Self-Attention）是 Transformer 的核心计算单元。给定输入序列表示 $\boldsymbol{X} \in \mathbb{R}^{n\times d}$（$n$ 为序列长度，$d$ 为模型维度），自注意力通过三个可学习的投影矩阵生成查询（Query）、键（Key）和值（Value）：

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^Q, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^K, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^V \tag{1.1}$$

其中 $\boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V \in \mathbb{R}^{d\times d}$（单头情况下）。缩放点积注意力的核心计算为：

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{1.2}$$

其中 $d_k = d/h$ 为每个注意力头的维度（$h$ 为头数），缩放因子 $\sqrt{d_k}$ 用于防止点积值过大导致 softmax 进入饱和区。

### 1.2.2 为何缩放因子为 $\sqrt{d_k}$？——梯度分析

**缩放因子的数学解释**

假设 $\boldsymbol{q}$ 和 $\boldsymbol{k}$ 的各分量独立同分布，均值为 0，方差为 1。则点积 $\boldsymbol{q}^\top\boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i$ 的方差为：

$$\text{Var}(\boldsymbol{q}^\top\boldsymbol{k}) = \sum_{i=1}^{d_k}\text{Var}(q_i k_i) = \sum_{i=1}^{d_k}\text{Var}(q_i)\cdot\text{Var}(k_i) = d_k$$

因此，未缩放的点积值分布在均值 0、标准差 $\sqrt{d_k}$ 附近。当 $d_k$ 较大时（如 64 或 128），

点积值可能达到 $\pm 20$ 以上。

**softmax 在饱和区的梯度消失：**对于 $\text{softmax}(z)_i = e^{z_i}/\sum_j e^{z_j}$，其梯度为：

$$\frac{\partial \,\text{softmax}(z)_i}{\partial z_j} = \text{softmax}(z)_i(\delta_{ij} - \text{softmax}(z)_j)$$

当 $z$ 的某些分量远大于其他分量时，$\text{softmax}(z)$ 趋近于 one-hot 向量，梯度趋近于零矩阵——所有注意力权重都集中在极少数位置上，模型失去学习能力。

**缩放 $\sqrt{d_k}$ 的效果：**除以 $\sqrt{d_k}$ 使点积方差归一化为 1，确保 softmax 输入保持在合理的数值范围，梯度可以有效传递。

### 1.2.3　因果掩码（Causal Masking）

在 decoder-only 架构中，token $t$ 只能关注位置 $1, 2, \ldots, t$ 的 token。这种自回归约束通过*因果掩码*实现：

$$\boldsymbol{M}_{ij} = \begin{cases} 0, & j \le i \\ -\infty, & j > i \end{cases}$$

最终的注意力计算变更为：

$$\text{Attention}_{\text{causal}}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V}$$

掩码矩阵 $\boldsymbol{M}$ 的上三角部分为 $-\infty$，经过 $\exp(-\infty) = 0$ 后，对应位置的注意力权重为零——从而保证 token $t$ 的表示仅由位置 1 到 $t$ 的 token 决定。

**掩码的数值实现**

实践中不会创建 $n \times n$ 的 $-\infty$ 矩阵。通常的做法是：

1. 先在 SRAM 中计算 $\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{d_k}$
2. 对于 $j > i$ 的位置，将 $\boldsymbol{S}_{ij}$ 设为 $-\infty$（或一个极大的负数如 $-65504$，即 FP16 的最小值）
3. FlashAttention 中，因果掩码通过限制内层循环的 $j$ 范围实现：只遍历 $j \le i$ 的 $\boldsymbol{K}, \boldsymbol{V}$ 块

# 1.3 多头注意力（Multi-Head Attention, MHA）

## 1.3.1 多头分解

单头注意力的表达力有限——所有 token 之间的关系由同一组投影矩阵决定。多头注意力将模型维度 $d$ 均匀分配到 $h$ 个头上，每个头在低维子空间（$d_k = d/h$）中独立计算注意力，最后拼接所有头的输出。

**MHA 的完整张量变换流程**

输入 $\boldsymbol{X} \in \mathbb{R}^{B\times S\times d}$，其中 $B$ 为 batch 大小，$S$ 为序列长度。

**步骤 1——线性投影：**

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^Q \in \mathbb{R}^{B\times S\times d}, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^K \in \mathbb{R}^{B\times S\times d}, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^V \in \mathbb{R}^{B\times S\times d}$$

**步骤 2——重塑为多头格式：**

$$\boldsymbol{Q} \to \mathbb{R}^{B\times S\times h\times d_k} \to \mathbb{R}^{B\times h\times S\times d_k}$$

（对 $\boldsymbol{K}, \boldsymbol{V}$ 同理）这一步将 $d$ 维度拆分为 $(h, d_k)$，然后将 $h$ 维提前。

**步骤 3——逐头缩放点积注意力：**对每个头 $i \in \{1, \ldots, h\}$：

$$\boldsymbol{O}_i = \text{softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}_i^\top}{\sqrt{d_k}}\right)\boldsymbol{V}_i \in \mathbb{R}^{B\times S\times d_k}$$

**步骤 4——拼接与输出投影：**

$$\boldsymbol{O} = \text{Concat}(\boldsymbol{O}_1, \ldots, \boldsymbol{O}_h) \in \mathbb{R}^{B\times S\times d}$$

$$\boldsymbol{O}_{\text{final}} = \boldsymbol{O}\boldsymbol{W}^O \in \mathbb{R}^{B\times S\times d}, \quad \boldsymbol{W}^O \in \mathbb{R}^{d\times d}$$

**张量形状速查表**

以 LLaMA-2-7B 为例：$d = 4096$，$h = 32$，$d_k = 128$。

| 操作 | 形状 |
|---|---|
| 输入 $\boldsymbol{X}$ | $(B, S, 4096)$ |
| 投影后 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ | $(B, S, 4096)$ |
| 重塑为多头 | $(B, 32, S, 128)$ |
| $\boldsymbol{Q}_i\boldsymbol{K}_i^\top$ | $(B, 32, S, S)$ |
| 注意力权重 | $(B, 32, S, S)$ |
| 每头输出 $\boldsymbol{O}_i$ | $(B, 32, S, 128)$ |
| 拼接后 $\boldsymbol{O}$ | $(B, S, 4096)$ |
| 输出投影后 | $(B, S, 4096)$ |

### 1.3.2　多头注意力的参数量

给定 $d$ 和 $h$，MHA 的总参数量为：

$$N_{\text{MHA}} = 4 \times d \times d = 4d^2$$

（四个投影矩阵 $\boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V, \boldsymbol{W}^O$ 各 $d \times d$）。对于 LLaMA-2-7B（$d = 4096$），单个 MHA 层约 $4 \times 4096^2 \approx 67$M 参数。加上 FFN 层的 $3 \times 4096 \times 11008 \approx 135$M 参数（若含 gating 则为 $3 \times d \times d_{\text{ff}}$），每层总计约 202M 参数，32 层共计 $\approx 6.5$B（不含嵌入层）。

## 1.4　分组查询注意力与多查询注意力

### 1.4.1　GQA 与 MQA 的核心思想

随着模型规模的增长，KV Cache 的显存成为推理瓶颈。KV Cache 的大小正比于 $2 \times L \times h_{\text{kv}} \times d_k \times S$（$L$ 为层数，$h_{\text{kv}}$ 为 KV 头数，$S$ 为序列长度）。减少 KV 头数可以直接降低 KV Cache 显存。

**三种注意力方案的对比**

设 query 头数为 $h_q$，KV 头数为 $h_{\text{kv}}$。

**MHA（Multi-Head Attention）**：$h_{\text{kv}} = h_q$。每个 query 头有独立的 K 和 V。KV Cache $= 2 \times L \times h_q \times d_k \times S$。

**GQA（Grouped-Query Attention）**：$h_{\text{kv}} < h_q$，且 $h_q$ 是 $h_{\text{kv}}$ 的整数倍。将 $h_q$ 个 query 头分为 $h_{\text{kv}}$ 组，每组内的 query 头共享同一组 KV。例如 LLaMA-2-70B 使用 $h_q = 64$，$h_{\text{kv}} = 8$ 的 GQA（每组 8 个 query 头共享 1 组 KV），KV Cache 减小到 1/8。

**MQA（Multi-Query Attention）**：$h_{\text{kv}} = 1$ 的极限情况。所有 query 头共享唯一的一组 KV。KV Cache 最小，但模型质量下降明显。

表 1.1: MHA、GQA、MQA 对比

| 特性 | MHA | GQA (8:1) | MQA |
|---|---|---|---|
| KV 头数 | $h_q$ | $h_q/8$ | 1 |
| KV Cache 大小 | 100% | 12.5% | $1/h_q$ |
| 注意力质量 | 最优 | 接近 MHA | 轻微退化 |
| 适用模型 | 小模型 | 中型/大型模型 | 追求极致速度 |
| 代表模型 | LLaMA-1, GPT-3 | LLaMA-2/3-70B, Mistral | PaLM, Gemini |

### 1.4.2　GQA 的张量形状变换

在 GQA 中，K 和 V 的投影矩阵尺寸减小。设 $h_q = 64$，$h_{\text{kv}} = 8$，$d = 8192$，$d_k = 128$：

- $\boldsymbol{W}^Q$: $(d, h_q \cdot d_k) = (8192, 8192)$
- $\boldsymbol{W}^K$: $(d, h_{\text{kv}} \cdot d_k) = (8192, 1024)$
- $\boldsymbol{W}^V$: $(d, h_{\text{kv}} \cdot d_k) = (8192, 1024)$
- $\boldsymbol{W}^O$: $(h_q \cdot d_k, d) = (8192, 8192)$

重塑后的形状：

- $\boldsymbol{Q}$: $(B, 64, S, 128)$
- $\boldsymbol{K}$: $(B, 8, S, 128)$
- $\boldsymbol{V}$: $(B, 8, S, 128)$

计算注意力时，每个 query 头需要找到其对应的 KV 组：对于 query 头 $i$，使用 KV 头 $\lfloor i \cdot h_{\text{kv}}/h_q \rfloor$。实际上，通过将 K 和 V 在头维度上扩展（repeat_interleave）来与 Q 匹配：

$$\boldsymbol{K}_{\text{exp}} = \text{repeat_interleave}(\boldsymbol{K}, \text{repeats} = h_q/h_{\text{kv}}, \text{dim} = 1) \in \mathbb{R}^{B\times 64\times S\times 128}$$

然后按照标准 MHA 流程计算注意力。

**GQA 的质量-效率权衡理论**

GQA 之所以能在大幅减少 KV Cache 的同时保持接近 MHA 的质量，基于以下理论依据：

**1. KV 表示的信息冗余**：研究发现，MHA 中不同头的 K 和 V 表示存在高度相关性，许多头的 KV 矩阵几乎线性相关。GQA 通过分组共享来消除这种冗余。

**2. 训练过程中的 KV 上训练（Uptraining）**：多查询/分组查询模型从 MHA checkpoint 开始，将原有多个 KV 头均值池化或随机选一作为初始共享 KV，然后通过少量训练步数（数百万 tokens）微调，使共享 KV 头适配分组 query 头。

**3. 典型质量损失**：GQA (8:1) 相比 MHA，在标准 benchmark 上的困惑度增加通常 $< 0.05$，在下游任务上的性能下降通常 $< 0.5\%$。

## 1.5 前馈网络（FFN）与门控变体

### 1.5.1 标准 FFN

标准 Transformer 中的 FFN 是一个简单的两层 MLP：

$$\text{FFN}(\boldsymbol{x}) = \sigma(\boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$

其中 $\boldsymbol{W}_1 \in \mathbb{R}^{d\times d_{\text{ff}}}$，$\boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{ff}}\times d}$。通常 $d_{\text{ff}} = 4d$（如 LLaMA 中 $d = 4096$，$d_{\text{ff}} = 11008$）。激活函数 $\sigma$ 传统上使用 ReLU 或 GELU。

参数量：$2 \times d \times d_{\text{ff}} \approx 8d^2$。在 LLaMA-2-7B 中约占每层参数的 67%。

### 1.5.2　门控 FFN：SwiGLU 与 GeGLU

现代 LLM（LLaMA、Mistral、Gemma 等）普遍使用门控激活函数替代简单的两层的 FFN。SwiGLU 是最流行的选择。

**SwiGLU 门控机制**

门控 FFN 的通用形式为：

$$\mathrm{FFN}_{\mathrm{gated}}(\boldsymbol{x}) = (\mathrm{Gate}(\boldsymbol{x}) \odot \mathrm{Up}(\boldsymbol{x})) \cdot \boldsymbol{W}_{\mathrm{down}}$$

其中：

$$\mathrm{Gate}(\boldsymbol{x}) = \mathrm{SiLU}(\boldsymbol{x}\boldsymbol{W}_{\mathrm{gate}}), \quad \mathrm{Up}(\boldsymbol{x}) = \boldsymbol{x}\boldsymbol{W}_{\mathrm{up}}$$

$\mathrm{SiLU}(x) = x \cdot \sigma(x)$（即 Swish 激活函数），$\odot$ 表示逐元素乘积。

**参数量**：三个投影矩阵各 $d \times d_{\mathrm{ff}}$，总计 $3dd_{\mathrm{ff}}$。相对于标准 FFN 的 $2dd_{\mathrm{ff}}$ 增加了约 50% 的 FFN 参数。为了保持总参数量不变，通常将 $d_{\mathrm{ff}}$ 从 $4d$ 调整为 $\frac{8}{3}d$。

表 1.2: 常见门控变体

| 变体 | 门控函数 | 说明 |
|---|---|---|
| SwiGLU | $\mathrm{SiLU}(x\boldsymbol{W}_g) \odot (x\boldsymbol{W}_u)$ | LLaMA 首选 |
| GeGLU | $\mathrm{GELU}(x\boldsymbol{W}_g) \odot (x\boldsymbol{W}_u)$ | 早期 GPT 变体 |
| ReGLU | $\mathrm{ReLU}(x\boldsymbol{W}_g) \odot (x\boldsymbol{W}_u)$ | 简单快速 |
| GEGLU | $\mathrm{GELU}(x\boldsymbol{W}_g)$ | 无独立 up 投影 |

**为何 SwiGLU 优于 ReLU/GELU？**

**平滑性**：SiLU（$x\sigma(x)$）在整个实轴上连续可导，梯度流动平滑。ReLU 在 $x = 0$ 处不可导，GELU 虽可导但趋于零的速度不同。

**门控的语义**：门控机制赋予了网络" 选择性传递信息" 的能力——当 $\mathrm{Gate}(\boldsymbol{x}) \approx 0$ 时，该通道被抑制；当 $\mathrm{Gate}(\boldsymbol{x})$ 较大时，该通道被放大。这种动态路由能力类似于注意力机制的软选择，但作用在特征维度而非序列维度。

**经验结果**：在同等计算量下，SwiGLU 在语言建模困惑度和下游任务中一致优于 ReLU 和 GELU。T5 实验显示 GEGLU/SwiGLU 在下游任务上平均提升 $1\% \sim 2\%$。

## 1.6　归一化层：LayerNorm 与 RMSNorm

### 1.6.1　经典 LayerNorm

Layer Normalization 对每个 token 的表示向量进行归一化，使不同特征维度的尺度保持一致：

$$\text{LayerNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x} - \mu}{\sigma} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}$$

其中：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 + \epsilon}$$

$\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ 为可学习的缩放和平移参数，$\epsilon$ 为防止除零的小常数（如 $10^{-5}$）。

### 1.6.2 RMSNorm

**RMSNorm——简化但高效的归一化**

RMSNorm（Root Mean Square Layer Normalization）去掉了均值中心化（re-centering）步骤，仅使用均方根进行缩放：

$$\text{RMSNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\text{RMS}(\boldsymbol{x})} \odot \boldsymbol{\gamma}$$

其中 $\text{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$。

**与 LayerNorm 的关键差异：**

1. **无均值减除**：不计算 $\mu$，减少一次归约操作（reduction），计算量降低约 33%
2. **无可学习平移参数 $\boldsymbol{\beta}$**：仅保留缩放参数 $\boldsymbol{\gamma}$，参数量减半
3. **数值实验**：LLaMA 论文报告 RMSNorm 相比于 LayerNorm 在训练速度和最终效果上无显著差异，但推理速度提升可观

| 操作 | LayerNorm | RMSNorm | 节省 |
|---|---|---|---|
| 均值归约 | 1 次 | 0 次 | 100% |
| 平方归约 | 1 次 | 1 次 | 0% |
| 逐元素减 | 1 次 | 0 次 | 100% |
| 逐元素乘 | 1 次 | 1 次 | 0% |
| 可学习参数 | $2d$ | $d$ | 50% |

### 1.6.3 Pre-Norm vs Post-Norm

**残差连接位置的选择**

残差连接与归一化的顺序分为两种范式：

**Post-Norm（原始 Transformer）：**

$$\boldsymbol{x}_{l+1} = \text{LayerNorm}(\boldsymbol{x}_l + \text{Sublayer}(\boldsymbol{x}_l))$$

归一化在残差加法之后。训练时梯度需要通过归一化层，可能导致梯度消失。

**Pre-Norm（现代 LLM 的主流）：**

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \text{Sublayer}(\text{RMSNorm}(\boldsymbol{x}_l))$$

归一化在子层之前。残差路径上始终保持恒等映射，梯度可以无衰减地反向传播。

**Pre-Norm 的优势：**

1. **训练稳定性：**深层 Transformer 的训练发散风险大幅降低，无需复杂的学习率预热策略
2. **梯度流：**$\frac{\partial \boldsymbol{x}_{l+1}}{\partial \boldsymbol{x}_l} = \boldsymbol{I} + \frac{\partial\,\text{Sublayer}(\text{RMSNorm}(\boldsymbol{x}_l))}{\partial \boldsymbol{x}_l}$，接近 $\boldsymbol{I}$ 保证了梯度不会消失
3. **可逆性：**pre-norm 使得残差网络的每一层近似可逆，有助于信息的跨层传播

# 1.7　旋转位置编码（RoPE）

## 1.7.1　从 2D 旋转到高维扩展

RoPE（Rotary Position Embedding）是目前 LLM 中最主流的位置编码方式，同时被 LLaMA、Mistral、Qwen、Gemma 等模型采用。

**RoPE 的数学推导**

**出发点——2D 旋转：**位置 $m$ 的 token，我们希望将位置信息编码为旋转操作。在 2D 空间中，位置 $m$ 处的向量可表示为旋转矩阵 $\boldsymbol{R}_m$ 作用于基础向量：

$$\boldsymbol{R}_m = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix}$$

**高维扩展：**将 $d$ 维向量拆分为 $d/2$ 个 2D 子空间（即 $d/2$ 对相邻维度），每一对有对应的旋转频率 $\theta_i$：

$$\theta_i = b^{-2i/d}, \quad i = 0, 1, \ldots, d/2 - 1$$

其中 $b$ 为基频（base frequency），LLaMA 使用 $b = 10000$，有些模型使用 $b = 500000$ 以获得更长的上下文支持。

**应用 RoPE：**对于 query 向量 $\boldsymbol{q}_m$ 在位置 $m$ 和 key 向量 $\boldsymbol{k}_n$ 在位置 $n$，RoPE 编码为：

$$\boldsymbol{q}'_m = \boldsymbol{R}_m \boldsymbol{q}_m, \quad \boldsymbol{k}'_n = \boldsymbol{R}_n \boldsymbol{k}_n$$

则注意力分数变为：

$$\boldsymbol{q}'^{\top}_m \boldsymbol{k}'_n = \boldsymbol{q}^{\top}_m \boldsymbol{R}^{\top}_m \boldsymbol{R}_n \boldsymbol{k}_n = \boldsymbol{q}^{\top}_m \boldsymbol{R}_{n-m} \boldsymbol{k}_n$$

**关键性质：**注意力分数仅依赖于相对位置 $n-m$ 而非绝对位置——这正是 RoPE 的核心优势。

### 1.7.2 频率分布与长程衰减

**频带特性**

基频 $b=10000$ 产生了从高频（$\theta_0=1.0$）到低频（$\theta_{d/2-1}=10000^{-(d-2)/d}\approx 10^{-4}$）的频带分布：

- **高频分量**（低维对，$i$ 小）：可以捕捉局部 token 间的位置差，梯度对临近位置敏感
- **低频分量**（高维对，$i$ 大）：对远距离相对位置仍有可区分性，支持长程依赖

**长程衰减（Long-Range Decay）：**当两个 token 的相对距离 $|n-m|$ 增大时，RoPE 编码的点积值会自然衰减。这是因为 RoPE 隐含了 $q^\top k$ 与 $\sum_i \cos((n-m)\theta_i)$ 的关联，当 $n-m$ 较大时不同频率的余弦项之间会发生破坏性干涉，导致整体点积值降低。这种属性使得注意力分数天然随距离衰减，符合语言的局部性假设。

**RoPE 的 CUDA 伪代码实现**

**输入：** $\boldsymbol{x}\in\mathbb{R}^{S\times d}$（输入），$m$（起始位置）
**输出：** $\boldsymbol{x}'$（应用 RoPE 后的输出）

**for** $i=0$ **to** $d/2-1$ **do**
  $\theta_i \leftarrow b^{-2i/d}$ ▷ 预计算的频率
**for** $s=0$ **to** $S-1$ **do**
  $pos \leftarrow m+s$
  **for** $i=0$ **to** $d/2-1$ **do**
    $\cos_val \leftarrow \cos(pos\cdot\theta_i)$
    $\sin_val \leftarrow \sin(pos\cdot\theta_i)$
    $x_{2i}\leftarrow \boldsymbol{x}[s,2i],\ x_{2i+1}\leftarrow \boldsymbol{x}[s,2i+1]$
    $\boldsymbol{x}'[s,2i]\leftarrow x_{2i}\cdot\cos_val - x_{2i+1}\cdot\sin_val$
    $\boldsymbol{x}'[s,2i+1]\leftarrow x_{2i}\cdot\sin_val + x_{2i+1}\cdot\cos_val$
**return** $\boldsymbol{x}'$

### 1.7.3　NTK-Aware 缩放与长上下文扩展

**NTK-Aware Interpolation**

当模型需要在超出训练上下文长度的序列上推理时，简单的线性插值（将位置索引缩放）会导致高频分量的信息丢失。NTK-Aware 插值通过非线性缩放基频 $b$ 来更好地保留高频信息：

$$b' = b \cdot \left( \frac{S_{\text{new}}}{S_{\text{train}}} \right)^{\frac{d}{d-2}}$$

其中 $S_{\text{new}}$ 为目标上下文长度，$S_{\text{train}}$ 为原始训练长度。该公式通过放缩基频来" 压缩" 旋转速度，使得位置编码在更长的序列范围内保持可区分。
常用变体：

- **NTK-Aware**：如上所述，非线性缩放 $b$
- **YaRN**：结合线性插值和 NTK-Aware，对高频分量少缩放、低频分量多缩放
- **Code LLaMA**：直接增大 $b$ 到 $10^6$，简单有效

## 1.8　嵌入层与权重绑定

### 1.8.1　Token 嵌入与位置嵌入

输入 token 首先经过词汇嵌入矩阵 $\boldsymbol{E}_{\text{tok}} \in \mathbb{R}^{V \times d}$（$V$ 为词表大小）映射到 $d$ 维向量。对于使用 RoPE 的模型，位置编码直接在注意力层的 Q/K 向量上施加，因此不需要单独的可学习位置嵌入表。

**嵌入层形状**

输入 token ID 序列 $\boldsymbol{t} \in \mathbb{Z}^{B \times S}$：

1. $\boldsymbol{X}_{\text{emb}} = \text{EmbeddingLookup}(\boldsymbol{t}, \boldsymbol{E}_{\text{tok}}) \in \mathbb{R}^{B \times S \times d}$
2.（可选）若使用绝对位置嵌入：$\boldsymbol{X} = \boldsymbol{X}_{\text{emb}} + \boldsymbol{E}_{\text{pos}}[:S, :]$
3. 对于 LLaMA 等 RoPE 模型：$\boldsymbol{X} = \boldsymbol{X}_{\text{emb}}$，位置信息由后续 RoPE 提供

### 1.8.2 权重绑定（Weight Tying）

**输入-输出嵌入共享**

许多 LLM 将输入嵌入矩阵 $\boldsymbol{E}_{\text{tok}}$ 与输出投影矩阵（lm_head）绑定为同一矩阵：

$$\boldsymbol{W}_{\text{out}} = \boldsymbol{E}_{\text{tok}} \in \mathbb{R}^{V \times d}$$

即输出 logits 计算为 $\text{logits} = \boldsymbol{X}_{\text{final}} \boldsymbol{E}_{\text{tok}}^{\top} \in \mathbb{R}^{B \times S \times V}$。

**优势：**

- 参数节省：$V \times d \approx 32000 \times 4096 \approx 131\text{M}$ 参数（对于 LLaMA-2-7B）
- 正则化效果：输入和输出共享语义空间，鼓励 token 嵌入的一致性
- 梯度效率：嵌入层获得来自输入和输出两方面的梯度信号

**不适用场景：**当输入和输出的”语义范围”差异过大时（如翻译任务中源语言和目标语言不同），权重绑定可能有害。decoder-only 的 LM 任务中绑定通常是有效的。

## 1.9 混合专家（Mixture of Experts, MoE）

### 1.9.1 稀疏 MoE 的形式化

传统 dense 模型的每个 token 经过所有 FFN 参数。MoE 将 FFN 层替换为 $N$ 个并行的”专家”，每个 token 仅激活其中 $k$ 个（$k \ll N$，通常 $k = 2$）：

$$\text{MoE}(\boldsymbol{x}) = \sum_{i \in \mathcal{S}} g_i(\boldsymbol{x}) \cdot E_i(\boldsymbol{x}) \tag{1.3}$$

其中 $\mathcal{S} \subseteq \{1, \ldots, N\}$ 为 top-$k$ 路由选中的专家索引集合，$g_i(\boldsymbol{x})$ 为门控权重。

**路由机制的数学细节**

门控网络将输入 $\boldsymbol{x} \in \mathbb{R}^d$ 映射到 $N$ 个 logit：

$$\boldsymbol{r} = \boldsymbol{x}\boldsymbol{W}_r \in \mathbb{R}^N, \quad \boldsymbol{W}_r \in \mathbb{R}^{d \times N}$$

Top-$k$ 选择：

$$\mathcal{S} = \arg\text{top}_k(\boldsymbol{r})$$

Softmax 归一化（仅在选中的专家上）：

$$g_i(\boldsymbol{x}) = \frac{\exp(r_i)}{\sum_{j \in \mathcal{S}} \exp(r_j)}, \quad \forall i \in \mathcal{S}$$

对于 $i \notin \mathcal{S}$，$g_i(\boldsymbol{x}) = 0$（完全不参与计算和反向传播）。

### 1.9.2　负载均衡与专家容量

**辅助损失——处理专家坍塌**

**专家坍塌（Expert Collapse）**：训练过程中，门控网络倾向于重复选择少数几个专家，导致其他专家几乎不被使用——这些专家的梯度接近零，参数退化。

**负载均衡损失**（Switch Transformer 提出）：

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

其中 $f_i = \frac{1}{T}\sum_{t=1}^{T} \mathbb{I}[\text{token } t \text{ 被路由到专家} i]$（实际分配比例），$P_i = \frac{1}{T}\sum_{t=1}^{T} \text{softmax}(\boldsymbol{r}_t)_i$（门控概率均值），$\alpha$ 为平衡系数（如 0.01）。

当路由均匀分布时，$f_i = P_i = 1/N$，$\mathcal{L}_{\text{aux}} = \alpha$（最小值）。

**专家容量**：限制每个专家在一个 batch 内最多处理 $C = \lceil \frac{T \cdot k}{N} \cdot \gamma \rceil$ 个 token，其中 $\gamma \geq 1$ 为容量因子。超出容量的 token 被" 丢弃"——通过残差连接直接传递到下一层。

**Mixtral 8x7B 的具体配置**

Mixtral 是 MoE 在开源领域最具代表性的模型：

- 8 个专家，每个 token 激活 top-2
- 每个专家为标准的 SwiGLU FFN（$d_{\text{ff}} = 14336$）
- 总参数量 46.7B，但每 token 活跃参数仅 12.9B
- 推理速度 $\approx$ 13B dense 模型的速度
- MoE 层仅在每隔一层替换 FFN（非每层）
- 在多数 benchmark 上优于或接近 LLaMA-2-70B（总参数 140B 的 dense 模型）

**MoE 推理的挑战**

**显存瓶颈**：所有专家参数必须驻留在 GPU 显存中。Mixtral 8x7B 虽然活跃参数仅 13B，但显存需求约为 47B 参数模型级别（FP16 下约 94GB）。对于推理部署，这可能要求多卡部署。

**Batch 效率低**：小 batch 下每个专家只处理极少数 token，GPU 利用率低。MoE 擅长大 batch 场景（高吞吐推理服务）。

**通信开销**：在多卡 TP/EP（expert parallelism）部署中，token 需要跨卡路由到对应专家所在的 GPU，引入额外通信。

**DeepSeekMoE 的创新**：将专家进一步细粒度化（如 64 个更小的专家）+ 引入共享专家（shared experts，所有 token 均激活），在激活参数和总参数的效率上取得了更

好的平衡。

# 1.10 KV Cache 机制

## 1.10.1 KV Cache 的存储与更新

自回归解码阶段（decode），每生成一个新的 token，需要计算该 token 对所有历史 token 的注意力。为避免重复计算，所有历史 token 的 K 和 V 被缓存：

**KV Cache 的增量更新**

生成过程（时间步 $t$）：

**初始化**（prefill 阶段，$t = S_{\text{prompt}}$）：

- 输入 prompt 的 $n$ 个 token，一次前向计算所有层的 K 和 V
- 缓存每层的 $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{B \times h_{\text{kv}} \times n \times d_k}$

**增量步骤**（decode 阶段，$t > S_{\text{prompt}}$）：

- 仅对当前生成的 token 计算 $\boldsymbol{q}_t, \boldsymbol{k}_t, \boldsymbol{v}_t$
- 将新的 $\boldsymbol{k}_t, \boldsymbol{v}_t$ 追加到缓存中：$\boldsymbol{K}_{[1:t]} = [\boldsymbol{K}_{[1:t-1]}; \boldsymbol{k}_t]$
- 计算注意力：$\boldsymbol{o}_t = \text{softmax}(\boldsymbol{q}_t \boldsymbol{K}_{[1:t]}^\top / \sqrt{d_k}) \boldsymbol{V}_{[1:t]}$

## 1.10.2 KV Cache 显存计算

$$M_{\text{KV}} = 2 \times L \times h_{\text{kv}} \times d_k \times S \times B_{\text{dtype}} \tag{1.4}$$

**显存估算示例**

以 LLaMA-2-7B 为例（$L = 32$，$h_{\text{kv}} = 32$，$d_k = 128$，FP16 $B_{\text{dtype}} = 2$ 字节）：

| 序列长度 $S$ | KV Cache 显存 | 说明 |
|---|---|---|
| 512 | 0.25 GB | 短对话 |
| 2048 | 1.00 GB | 标准上下文 |
| 4096 | 2.00 GB | 长文本 |
| 8192 | 4.00 GB | 长上下文扩展后 |
| 32768 | 16.00 GB | 超长上下文 |

以 LLaMA-2-70B 为例（$L = 80$，$h_{\text{kv}} = 8$（GQA），$d_k = 128$）：

| 序列长度 $S$ | KV Cache 显存 |
|---|---|
| 4096 | 5.00 GB |
| 32768 | 40.00 GB |

**KV Cache 导致的显存瓶颈**

**Memory-Bound 推理：**decode 阶段每次只处理 1 个 token。计算量为 $O(d^2 + Sd_kh)$，而显存读取量为 $O(Sd_khL)$（读取所有层的 KV Cache）。当 $S$ 较大时，显存读取时间远超计算时间——这就是 LLM 推理本质上是*显存带宽受限*而非算力受限的原因。

**MFU2% 以下：**decode 阶段的浮点运算利用率通常仅为 1% ∼ 3%。绝大部分时间 GPU 在等待 HBM 数据的到来。这就是 KV Cache 量化和 PagedAttention 等技术如此关键的原因。

### 1.10.3　KV Cache 的转置与内存布局

**内存布局优化**

KV Cache 通常在内存中以 $(B, h_{\text{kv}}, S, d_k)$ 的形状存储。但在注意力计算时，$\boldsymbol{q}_t\boldsymbol{K}^\top$ 需要 $\boldsymbol{K}$ 以 $(B, h_{\text{kv}}, d_k, S)$ 的形式（最后两个维度交换）以利用 CUDA Tensor Core 的 GEMM。

**两种策略：**

- **转置写入：**在追加 KV 到缓存时就以 $(B, h_{\text{kv}}, d_k, S)$ 存储。代价是内存不连续（$S$ 维度增长需要重新分配连续空间）。
- **转置读取：**以 $(B, h_{\text{kv}}, S, d_k)$ 存储，在注意力计算时做转置——这可以利用 FlashAttention kernel 在 SRAM 内高效完成转置，避免 HBM 层面的显式转置。

PagedAttention 中，每个 KV 块独立存储，块内布局为 $(B_{\text{block}}, h_{\text{kv}}, d_k)$，读取时在 SRAM 内做转置。

## 1.11　完整前向传播流程

**Decoder-Only 模型的完整前向（以 LLaMA 为例）**

给定输入 token 序列 $\boldsymbol{t} \in \mathbb{Z}^{B\times S}$：

1. **嵌入**：$\boldsymbol{X}_0 = \text{Embed}(\boldsymbol{t}) \in \mathbb{R}^{B\times S\times d}$
2. **逐层处理**（$l = 0, \ldots, L-1$）：

(a) **Pre-Norm**: $\boldsymbol{X}_l^{\text{norm}} = \text{RMSNorm}(\boldsymbol{X}_l)$

(b) **投影**: $\boldsymbol{Q} = \boldsymbol{X}_l^{\text{norm}}\boldsymbol{W}_l^Q$, $\boldsymbol{K} = \boldsymbol{X}_l^{\text{norm}}\boldsymbol{W}_l^K$, $\boldsymbol{V} = \boldsymbol{X}_l^{\text{norm}}\boldsymbol{W}_l^V$

(c) **RoPE**: 对 $\boldsymbol{Q}, \boldsymbol{K}$ 施加 RoPE 位置编码

(d) **因果注意力**: $\boldsymbol{O}_{\text{attn}} = \text{Attention}_{\text{causal}}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})$

(e) **输出投影**: $\boldsymbol{O}_{\text{attn}} \leftarrow \boldsymbol{O}_{\text{attn}}\boldsymbol{W}_l^O$

(f) **残差**: $\boldsymbol{X}_l \leftarrow \boldsymbol{X}_l + \boldsymbol{O}_{\text{attn}}$

(g) **Pre-Norm**: $\boldsymbol{X}_l^{\text{norm2}} = \text{RMSNorm}(\boldsymbol{X}_l)$

(h) **SwiGLU FFN**: $\boldsymbol{O}_{\text{ffn}} = \text{SiLU}(\boldsymbol{X}_l^{\text{norm2}}\boldsymbol{W}_l^{\text{gate}}) \odot (\boldsymbol{X}_l^{\text{norm2}}\boldsymbol{W}_l^{\text{up}})\boldsymbol{W}_l^{\text{down}}$

(i) **残差**: $\boldsymbol{X}_l \leftarrow \boldsymbol{X}_l + \boldsymbol{O}_{\text{ffn}}$

3. **最终归一化**: $\boldsymbol{X}_L = \text{RMSNorm}(\boldsymbol{X}_{L-1})$
4. **输出投影**: $\text{logits} = \boldsymbol{X}_L\boldsymbol{W}_{\text{out}}^\top \in \mathbb{R}^{B\times S\times V}$
5. **采样**: $\text{token} \sim \text{softmax}(\text{logits}[:, -1, :]/T)$

**推理引擎需要关注的架构细节总结**

对每个 Transformer 层，需要实现的核心算子序列为：

1. RMSNorm → QKV 投影（GEMM）→ RoPE → 因果注意力（FlashAttention kernel）→ 输出投影（GEMM）→ 残差加法
2. RMSNorm → SwiGLU FFN（3 个 GEMM + SiLU + 逐元素乘）→ 残差加法

其中 GEMM 和注意力是最耗时的部分（约占总时间的 95% 以上）。高效的 GEMM 库（cuBLAS, CUTLASS）和注意力 kernel（FlashAttention）是推理引擎性能的基石。

# 第二章　模型文件格式详解

## 2.1　概述

大模型的部署和推理离不开模型文件的存储和交换。不同的推理框架和生态系统定义了各自的模型格式，每种格式在功能、性能和安全方面各有侧重。本章深入剖析主流模型文件格式的内部结构、设计动机和实现细节，为推理引擎中的模型加载模块提供知识基础。

**格式分类**

按设计理念可分为三类：

- **原生框架格式**：PyTorch checkpoint（.pt/.pth/.bin）、TensorFlow SavedModel——紧密耦合训练框架的序列化机制
- **交换格式**：ONNX（开放神经网络交换）——跨框架互操作的中间表示
- **推理专用格式**：GGUF（CPU 推理）、TensorRT Engine（NVIDIA GPU 推理）、OpenVINO IR（Intel 推理）——经过优化的推理就绪格式
- **安全存储格式**：safetensors——解决 PyTorch pickle 安全隐患的纯数据格式

## 2.2　GGUF 格式（llama.cpp 生态）

GGUF 是 GGML 文件格式的后继者，由 llama.cpp 项目维护，是目前 CPU 推理和大模型分发中最常用的格式。其设计哲学是**自包含**：单个文件包含模型的所有信息——参数字典、架构超参数、tokenizer 数据和量化权重——下载即可用。

### 2.2.1　文件结构概览

**GGUF 二进制布局**

GGUF 文件由以下区域顺序组成：

1. **文件头（File Header）**：魔数、版本、tensor 数量和 metadata 的 KV 对数
2. **Metadata 键值对**：以字符串键组织的模型超参数字典

3. **Tensor 信息表**：每个 tensor 的名称、维度、offset 和量化类型
4. **数据对齐填充**：确保 tensor 数据区域对齐到指定边界（通常为 32 字节）
5. **Tensor 数据块**：按序排列的原始 tensor 数据（量化的或未量化的）

表 2.1: GGUF 文件头结构（v3）

| 字段 | 类型 | 描述 |
|---|---|---|
| magic | u32 | 魔数 `0x47 0x47 0x55 0x46`（"GGUF"） |
| version | u32 | 格式版本号（当前为 3） |
| tensor_count | u64 | 文件中包含的 tensor 总数 |
| metadata_kv_count | u64 | metadata 键值对的数量 |

### 2.2.2 Metadata 键值对详解

**必需的 Metadata 字段**

GGUF 的 metadata 使用类型化键值对存储。值类型包括：u8/i8/u16/i16/u32/i32/f32/f64/bool/string/array。

**架构标识：**

- `general.architecture`: string —如"llama", "mistral", "falcon", "gpt2", "mpt"
- `general.name`: string —模型名称
- `general.file_type`: u32 —量化类型编号

**模型尺寸（对 LLaMA 系列至关重要）：**

- `{arch}.context_length`: u32 —最大上下文长度
- `{arch}.embedding_length`: u32 —模型维度 $d$
- `{arch}.block_count`: u32 —Transformer 层数 $L$
- `{arch}.feed_forward_length`: u32 —FFN 中间维度 $d_{\text{ff}}$
- `{arch}.attention.head_count`: u32 —Query 头数 $h_q$
- `{arch}.attention.head_count_kv`: u32 —KV 头数 $h_{\text{kv}}$（GQA/MQA）
- `{arch}.attention.layer_norm_epsilon`: f32 —归一化 $\epsilon$
- `{arch}.rope.dimension_count`: u32 —RoPE 应用的维度数

- `{arch}.rope.freq_base`: f32 —RoPE 基频 $b$

**Tokenizer 信息：**

- `tokenizer.ggml.model`: string —tokenizer 类型（如"llama", "gpt2", "bpe"）
- `tokenizer.ggml.tokens`: array[string] —词表 token 列表
- `tokenizer.ggml.scores`: array[f32] —token 分数（如 BPE 的合并优先级）
- `tokenizer.ggml.token_type`: array[i32] —token 类型
- `tokenizer.ggml.bos_token_id`: u32 —开始符 ID
- `tokenizer.ggml.eos_token_id`: u32 —结束符 ID
- `tokenizer.ggml.add_bos_token`: bool —是否自动添加 BOS
- `tokenizer.ggml.add_eos_token`: bool —是否自动添加 EOS

**量化参数：**

- `general.quantization_version`: u32 —量化方案的版本号
- `general.file_type`: u32 —枚举值指示量化类型（Q4_0=2, Q4_1=3, Q8_0=7, etc.）

### 2.2.3　Tensor 命名约定

**GGUF 中的 Tensor 命名规范**

GGUF 使用点分隔的层级命名体系：

**嵌入层：**

- `token_embd.weight`: 输入 token 嵌入矩阵，形状 $(V, d)$
- （可选）`output.weight`: 输出投影矩阵（如与 token_embd 绑定则为 shared）

**每层权重**（$i = 0, 1, \ldots, L-1$）：

- `blk.{i}.attn_norm.weight`: 注意力子层前的 RMSNorm 权重，形状 $(d,)$
- `blk.{i}.attn_q.weight`: Query 投影，形状 $(h_q \cdot d_k, d)$
- `blk.{i}.attn_k.weight`: Key 投影，形状 $(h_{\text{kv}} \cdot d_k, d)$
- `blk.{i}.attn_v.weight`: Value 投影，形状 $(h_{\text{kv}} \cdot d_k, d)$

- `blk.{i}.attn_output.weight`: 注意力输出投影，形状 $(d, h_q \cdot d_k)$
- `blk.{i}.ffn_norm.weight`: FFN 子层前的 RMSNorm 权重，形状 $(d,)$
- `blk.{i}.ffn_gate.weight`: SwiGLU 门控投影，形状 $(d_{\text{ff}}, d)$
- `blk.{i}.ffn_up.weight`: SwiGLU up 投影，形状 $(d_{\text{ff}}, d)$
- `blk.{i}.ffn_down.weight`: SwiGLU down 投影，形状 $(d, d_{\text{ff}})$

**最终归一化：**

- `output_norm.weight`: 最终 RMSNorm 权重，形状 $(d,)$

**命名约定与模型架构的关系**

不同架构（LLaMA, Mistral, Falcon 等）的 tensor 命名可能略有差异。GGUF 使用 `general.architecture` 字段中的架构名来动态确定命名模式。推理引擎中通常维护一个" 架构名 →tensor 名映射表" 来支持多架构加载。
常用映射关系：

- LLaMA: `blk.{i}.attn_q.weight` → `model.layers.{i}.self_attn.q_proj.weight`（PyTorch 命名）
- Mistral: 由于使用 SWA，额外有 `blk.{i}.attn_q.weight` 等
- Qwen2: 额外有 `blk.{i}.attn_q.bias`, `blk.{i}.attn_k.bias` 等（使用 bias）

### 2.2.4 量化 Tensor 的块级存储

**GGUF 的量化块结构**

GGUF 支持多种量化格式（Q4_0, Q4_1, Q5_0, Q8_0, Q2_K, Q3_K, Q4_K, Q5_K, Q6_K 等）。以最常见的 Q4_0 为例：
每个量化块包含 $B = 32$ 个元素。存储结构：

- `d`: f16 —块的缩放因子（scale），即该块中最大绝对值
- `qs`: u8[16] —$B/2$ 字节，每个字节存储两个 4-bit 量化值（低 4 位在前，高 4 位在后）

**块大小：** $2 + 16 = 18$ 字节 vs 原始 FP16 的 64 字节，压缩比 3.56×。
**反量化公式**（对块内第 $j$ 个元素）：

$$w_j = d \cdot \text{q}[j]$$

其中 $\mathrm{q}[j] \in [-8, 7]$（有符号 4-bit 整数）。
对于非对称量化（Q4_1），额外存储块内最小值 $m$（f16），反量化公式为：

$$w_j = d \cdot \mathrm{q}[j] + m$$

**K-Quant 系列**

llama.cpp 在 GGUF 中引入了 K-Quant 系列，使用更大的超级块（super-block，256 个元素），并在超级块内使用次级量化：
**Q4_K 的存储布局：**

- 超级块级：d_min_max（6-bit 量化的全局缩放信息）
- 子块级（16 个子块，每个 16 个元素）：各自独立的 4-bit 量化值和 scale
- scale 本身也被量化（6-bit），进一步节省空间

K-Quant 系列在相同压缩比下提供了比简单 Q4_0 显著更好的困惑度表现，尤其是 Q4_K_M 和 Q4_K_S（中等和小尺寸两种变体）。

### 2.2.5　GGUF 读取器实现要点

**GGUF 文件加载流程**

**输入：** GGUF 文件路径 $path$
**输出：** 模型参数映射表 $\mathcal{W}$，模型配置 $\mathcal{C}$

$f \leftarrow \mathrm{OpenFile}(path)$
$magic \leftarrow \mathrm{ReadU32}(f)$
**if** $magic \neq$ `0x47475546` **then**
    **error** ” 非法的 GGUF 魔数”
$version \leftarrow \mathrm{ReadU32}(f)$
$n_tensors \leftarrow \mathrm{ReadU64}(f)$
$n_kv \leftarrow \mathrm{ReadU64}(f)$
$\mathcal{C} \leftarrow \{\}$
**for** $i = 1$ **to** $n_kv$ **do**
    $key \leftarrow \mathrm{ReadString}(f)$
    $type \leftarrow \mathrm{ReadU32}(f)$
    $value \leftarrow \mathrm{ReadValue}(f, type)$
    $\mathcal{C}[key] \leftarrow value$
$tensor_offsets \leftarrow \{\}$
**for** $i = 1$ **to** $n_tensors$ **do**

$name \leftarrow \text{ReadString}(f)$
$n_dims \leftarrow \text{ReadU32}(f)$
$dims \leftarrow$ array of $n_dims$ u64 values
$ggml_type \leftarrow \text{ReadU32}(f)$
$offset \leftarrow \text{ReadU64}(f)$
$tensor_offsets[name] \leftarrow (dims, ggml_type, offset)$
$alignment \leftarrow$ 根据当前文件位置计算对齐填充
**跳至**对齐后的 tensor 数据起始位置
**for** 每个 $(name, info)$ **in** $tensor_offsets$ **do**
  $size \leftarrow \text{CalcTensorSize}(info.dims, info.ggml_type)$
  $\mathcal{W}[name] \leftarrow$ 内存映射或读取 $size$ 字节到缓冲区
**return** $(\mathcal{W}, \mathcal{C})$

**内存映射 vs 缓冲区读取**

GGUF Reader 设计中的一个关键选择是**内存映射**（memory-mapping）还是**缓冲区读取**（buffered read）。

**内存映射优点：**

- 操作系统按需加载页面，支持惰性加载
- 多个进程可以共享同一文件的物理内存页
- 对于大模型，可以避免一次性分配全部显存/内存

**缓冲区读取优点：**

- 不受文件系统页对齐和 THP（Transparent Huge Pages）问题影响
- 更容易实现自定义的异步预取策略
- 某些平台（如 WASM, Android）不支持可靠的 mmap

通常建议：桌面/服务器平台首选 mmap，嵌入式/移动平台使用缓冲区读取。

### 2.2.6 GGML 到 GGUF 的迁移

**GGML vs GGUF 关键差异**

GGUF 于 2023 年引入，替代了早期的 GGML 格式：

| 特性 | GGML | GGUF |
|---|---|---|
| Metadata 存储 | 非标准化，混在文件头字段中 | 类型化键值对系统，可扩展 |
| Tensor 命名 | 硬编码的枚举 ID | 字符串命名，人类可读 |
| 版本兼容性 | 每个版本需要新解析代码 | 前向兼容的版本号机制 |
| Tokenization | 存储在外部文件 | 内嵌在文件 metadata 中 |
| 对齐填充 | 无保证 | 对齐到指定边界 |
| 量化灵活性 | 有限的量化类型 | 可扩展的量化类型系统 |

## 2.3　ONNX 格式（开放神经网络交换）

### 2.3.1　Protobuf-based 结构

ONNX（Open Neural Network Exchange）使用 Protocol Buffers（protobuf）作为序列化格式，类似于一个有向无环图（DAG）的计算图描述。

**ONNX 的顶层结构**

一个 ONNX 模型文件（.onnx）对应一个 `ModelProto` 消息：

Listing 2.1: ONNX ModelProto 结构（简化版）

```
message ModelProto {
  int64 ir_version = 1;
  repeated OperatorSetIdProto opset_import = 8;
  string producer_name = 2;
  string producer_version = 3;
  GraphProto graph = 7;
  repeated StringStringEntryProto metadata_props = 14;
}
```

核心是 `GraphProto`，定义了完整的计算图：

Listing 2.2: GraphProto 结构

```
message GraphProto {
  repeated NodeProto node = 1;          // 计算节点（算子）
  string name = 2;
  repeated TensorProto initializer = 5; // 权重（常量）
  repeated ValueInfoProto input = 11;   // 图输入
  repeated ValueInfoProto output = 12;  // 图输出
  repeated ValueInfoProto value_info = 13; // 中间值信息（含形状）
}
```

### 2.3.2 算子描述与属性

**NodeProto——计算节点的完整表示**

每个算子的 protobuf 结构：

Listing 2.3: NodeProto

```
message NodeProto {
  repeated string input = 1;    // 输入名称列表
  repeated string output = 2;   // 输出名称列表
  string name = 3;              // 节点名称
  string op_type = 7;           // 算子类型
  string domain = 4;            // 算子域（空=标准ONNX域）
  repeated AttributeProto attribute = 5; // 属性列表
}
```

常见算子类型示例：

| 算子 | op_type | 域 |
|---|---|---|
| 矩阵乘法 | MatMul | ""（标准域） |
| 2D 卷积 | Conv | "" |
| 批归一化 | BatchNormalization | "" |
| 激活函数 | Relu, Sigmoid, Gelu | "" |
| 重塑 | Reshape | "" |
| 自定义注意力 | MultiHeadAttention | "com.microsoft" |
| 量化操作 | DequantizeLinear | "ai.onnx" |

**TensorProto——权重和数据的序列化**

ONNX 中的 tensor：

Listing 2.4: TensorProto

```
message TensorProto {
  repeated int64 dims = 1;      // 形状，如 [2, 512, 512]
  int32 data_type = 2;          // 数据类型枚举
  SegmentProto segment = 3;     // 分段（用于大模型拆分）
  // 以下为不同数据类型的存储字段
  repeated float float_data = 4;
  repeated int32 int32_data = 5;
```

```
  bytes raw_data = 9;           // 原始字节（推荐，最紧凑）
  string name = 8;              // tensor名称
  string doc_string = 12;
  repeated string external_data = 13; // 外部数据引用（大模型）
}
```

### 2.3.3　外部数据存储（External Data）

**ONNX 大模型的外部数据拆分**

对于超过 protobuf 2GB 限制的模型，ONNX 支持将 tensor 数据存储在外部分片中：

**外部数据引用**（TensorProto 中的 `external_data` 字段）：

```
external_data {
  key: "location" value: "model_data.bin"
  key: "offset"   value: "0"
  key: "length"   value: "16777216"
}
```

**典型文件布局：**

- `model.onnx`: 图结构 + metadata（通常几 MB 到几十 MB）
- `model.onnx.data` 或 `model_data.bin`: tensor 原始数据（GB 级别）

推理引擎加载时先解析.onnx 文件获取图结构和偏移信息，再用内存映射或缓冲区读取外部数据文件中的 tensor 数据。

**形状推断与类型检查**

ONNX 标准定义了每个算子的形状推断规则。例如 `MatMul`：

- 输入 A: `[..., M, K]`
- 输入 B: `[..., K, N]`
- 输出: `[..., M, N]`

`Reshape` 的推断规则：

- 输入 data: `[D0, D1, ..., Dn]`
- 输入 shape: 1D tensor of int64
- 输出: 根据 shape 值和原始元素总数推断完整形状（`-1` 表示自动推断维度）

静态形状推断：在加载模型时，遍历图的所有节点，基于 initializer 的已知形状逐步推导每个中间 tensor 的形状。这不需要实际执行计算，是图优化（如内存规划）的前提。

## 2.4 HuggingFace 与 safetensors

### 2.4.1 PyTorch checkpoints 与安全缺陷

**PyTorch checkpoint 的安全隐患**

传统 PyTorch 模型使用 Python 的 `pickle` 序列化机制存储 `state_dict`：

Listing 2.5: 加载 PyTorch checkpoint 的不安全方式

```
import torch
# 危险：torch.load会执行pickle中嵌入的任意Python代码！
model = torch.load("model.bin")
```

`pickle` 允许在数据中嵌入可执行的 Python 代码。恶意模型文件可以在加载时执行任意系统命令——这对于模型分发平台（HuggingFace Hub）构成了严重的安全威胁。这也是 safetensors 被引入的直接原因。

### 2.4.2 safetensors 格式详解

**safetensors——纯数据的张量格式**

safetensors 采用极简的两部分结构：

**文件布局：**

1. **1 个 u64 整数：** header 的长度 $N$（字节数）
2. **header**（$N$ 字节）：JSON 格式的 metadata，描述每个 tensor 的 dtype、shape 和偏移
3. **tensor data：** 原始二进制数据块，按 header 中指定的 offset 排列

**Header JSON 结构：**

Listing 2.6: safetensors header 示例

```
{
  "model.layers.0.self_attn.q_proj.weight": {
    "dtype": "F16",
    "shape": [4096, 4096],
    "data_offsets": [0, 33554432]
```

```
  },
  "model.layers.0.self_attn.k_proj.weight": {
    "dtype": "F16",
    "shape": [1024, 4096],
    "data_offsets": [33554432, 37748736]
  }
}
```

`data_offsets` 是一个两元素数组 `[start, end]`，指示该 tensor 在 data 区域的起止字节偏移。

**安全保证：**

- Header 仅包含 JSON——JSON 解析器不会执行代码
- Tensor 数据是纯二进制 blob——无任何代码执行风险
- 完全不依赖 pickle 或任何 Python 执行

### 2.4.3　config.json 与 tokenizer 文件

**HuggingFace 模型仓库的标准文件**

一个典型的 HuggingFace 模型仓库包含以下文件：

**模型配置——`config.json`：**

Listing 2.7: LLaMA config.json 示例

```
{
  "architectures": ["LlamaForCausalLM"],
  "hidden_size": 4096,
  "intermediate_size": 11008,
  "num_attention_heads": 32,
  "num_hidden_layers": 32,
  "num_key_value_heads": 8,
  "rms_norm_eps": 1e-05,
  "rope_theta": 10000.0,
  "vocab_size": 32000,
  "max_position_embeddings": 4096,
  "hidden_act": "silu",
  "torch_dtype": "float16"
}
```

**权重文件：**

- `model.safetensors` 或 `model-00001-of-00002.safetensors`（分片）
- `model.safetensors.index.json`：分片索引文件，记录每个 tensor 在哪个分片中

**Tokenizer 文件：**

- `tokenizer.json`：完整的 tokenizer 定义（token 列表、BPE merges、特殊 token 等）
- `tokenizer_config.json`：tokenizer 的配置参数（截断策略、padding 方向等）
- `vocab.json` + `merges.txt`：GPT-2/BPE 老式 tokenizer 使用
- `special_tokens_map.json`：BOS/EOS/PAD/UNK 等特殊 token 的映射
- `added_tokens.json`：用户添加的自定义 token

**safetensors 的分片机制**

对于超大模型（70B+），单个 safetensors 文件可能过大。HuggingFace 通过 `safetensors.index.json` 支持分片：

Listing 2.8: 分片索引示例

```
{
  "metadata": {"total_size": 139847390208},
  "weight_map": {
    "model.layers.0.input_layernorm.weight":
      "model-00001-of-00002.safetensors",
    "model.layers.15.self_attn.q_proj.weight":
      "model-00001-of-00002.safetensors",
    "model.layers.31.mlp.down_proj.weight":
      "model-00002-of-00002.safetensors",
    ...
  }
}
```

每个分片是一个标准的 safetensors 文件，索引文件提供了”tensor 名 → 分片文件” 的映射。

## 2.5　其他推理格式

**专用推理格式一览**

**CoreML (.mlmodel / .mlpackage)：**

- Apple 生态系统的模型格式
- 自动利用 Apple Silicon 的 ANE（Apple Neural Engine）、GPU 和 CPU
- 支持量化（FP16、INT8）和模型加密
- 通过 `coremltools` 从 PyTorch/TensorFlow/ONNX 转换

**TensorFlow SavedModel：**

- TF 的部署标准格式，包含 `saved_model.pb`（图）+ `variables/`（权重）
- 支持签名（Signature）：定义推理的输入/输出接口
- 与 TF Serving 直接集成

**TensorFlow Lite (.tflite)：**

- 移动/嵌入式和微控制器部署的优化格式
- FlatBuffers 序列化（比 protobuf 更轻量）
- 内置 INT8、FP16 量化支持
- 对 Transformer 架构的支持有限（主要在 CNN 类模型上成熟）

**OpenVINO IR (.xml + .bin)：**

- Intel 推理引擎的格式
- XML 描述图结构，BIN 包含权重数据
- 针对 Intel CPU（含 AVX-512/VNNI）和 GPU/iGPU/VPU 优化
- 量化支持完善的 INT8 实现

**TensorRT Engine：**

- NVIDIA GPU 专用，通过 TensorRT builder 从 ONNX/PyTorch 编译生成
- 包含硬件特化（针对特定 GPU 架构和 Tensor Core 代数）的算子实现
- 不可移植（为特定 GPU 型号编译），但提供极致性能

## 2.6 格式对比与选择指南

表 2.2: 主流模型格式特性对比

| 特性 | GGUF | ONNX | safetensors | PyTorch .bin | TFLite | CoreML |
|---|---|---|---|---|---|---|
| 自包含 | ✓ | ✓ | ×（需 config） | ×（需 config） | ✓ | ✓ |
| 安全（无代码执行） | ✓ | ✓ | ✓ | ×（pickle） | ✓ | ✓ |
| 内置量化 | ✓ | 通过 Q/DQ 节点 | 无 | 无 | ✓ | ✓ |
| 内置 tokenizer | ✓ | × | × | × | × | × |
| 跨框架 | ×（llama.cpp） | ✓ | ✓ | ×（PyTorch） | ×（TF） | ×（Apple） |
| 人类可读 | 部分 | 部分 | 部分（JSON 头） | × | × | × |

**推理引擎中的格式选择建议**

1. **CPU 推理（桌面/服务器）**：GGUF——内置量化、tokenizer、架构识别，下载即用
2. **CPU 推理（移动/嵌入式）**：GGUF 或 TFLite——根据模型类型选择
3. **GPU 推理（NVIDIA）**：
   - 追求性能：TensorRT Engine（编译时优化）
   - 追求灵活性：ONNX + ONNX Runtime（需要额外量化步骤）
4. **GPU 推理（Apple Silicon）**：CoreML 或 GGUF（MLX 后端）
5. **模型安全分发**：safetensors（无代码执行风险）
6. **跨框架互操作**：ONNX

**多格式支持的架构设计**：推理引擎中通常采用” 格式检测器 + 格式解析器” 的插件架构。文件加载时先读取头部魔数字节判断格式类型，然后路由到对应的解析器。解析器统一输出内部 tensor 映射表接口。

## 2.7 格式转换与工具链

**常用格式转换路径**

**从 HuggingFace 到 GGUF：**

Listing 2.9: convert-hf-to-gguf.py

```
python convert-hf-to-gguf.py path/to/hf-model \
    --outfile model.gguf \
    --outtype q4_k_m
```

**从 HuggingFace 到 ONNX：**

Listing 2.10: 使用 optimum 导出 ONNX

```
from optimum.onnxruntime import ORTModelForCausalLM
model = ORTModelForCausalLM.from_pretrained("meta-llama/llama-2-7b"
    , export=True)
model.save_pretrained("llama-2-7b-onnx")
```

**从 PyTorch 到 safetensors：**

Listing 2.11: 安全存储转换

```
from safetensors.torch import save_file
save_file(model.state_dict(), "model.safetensors")
```

**从 ONNX 到 TensorRT Engine：**

Listing 2.12: trtexec 命令行工具

```
trtexec --onnx=model.onnx --saveEngine=model.engine --fp16
```

**从 PyTorch 到 CoreML：**

Listing 2.13: coremltools 转换

```
import coremltools as ct
traced_model = torch.jit.trace(model, example_input)
coreml_model = ct.convert(traced_model,
    inputs=[ct.TensorType(shape=(1, 128))],
    compute_units=ct.ComputeUnit.ALL)
coreml_model.save("model.mlpackage")
```

**格式转换中的数值精度损失**

跨格式转换时需注意：

- **FP32 → FP16：**直接截断，相对误差约为 $10^{-3} \sim 10^{-4}$。对推理结果影响通常可忽略
- **FP16 → BF16：**尾数位数从 10 bit 降至 7 bit，对某些对精度敏感的权重可能有影响
- **量化（FP16 → INT4/INT8）：**需要通过校准数据（calibration data）优化 scale/zero-point，直接线性量化可能导致显著精度下降
- **ONNX → TensorRT：**builder 会根据目标 GPU 的 Tensor Core 代数选择不

同的 kernel 实现，可能在数值精度上有微小差异（通常 $< 10^{-5}$）

# 第三章　张量系统设计

## 3.1　张量抽象

张量（Tensor）是推理引擎中最基本的数据结构。一个设计良好的张量抽象既要提供简洁的 API，又要暴露足够的底层信息以支持高性能计算。

### 3.1.1　核心属性

**张量的六要素**

一个张量 $T$ 由以下属性唯一确定：

1. **形状（shape）**：一个整数向量 $\boldsymbol{s} = (s_1, s_2, \ldots, s_n) \in \mathbb{N}^n$，$n$ 为维度数（rank）。元素总数为 $\prod_i s_i$。

2. **数据类型（dtype）**：元素的标量类型。常见值：float32, float16, bfloat16, int32, int8, int4, uint8, bool。

3. **步长（strides）**：每个维度的内存步长 $\boldsymbol{st} = (st_1, st_2, \ldots, st_n)$。元素 $\boldsymbol{T}[i_1, i_2, \ldots, i_n]$ 的线性偏移为：

$$\text{offset}(i_1, \ldots, i_n) = \sum_{k=1}^{n} st_k \cdot i_k$$

   对于”稠密”（contiguous）张量：$st_k = \prod_{j=k+1}^{n} s_j$（行优先）或 $st_k = \prod_{j=1}^{k-1} s_j$（列优先）。

4. **数据指针**：指向包含张量元素的连续内存区域的指针。

5. **设备（device）**：数据所在的物理设备。如：CPU（主机内存）、CUDA:0（GPU0 显存）、Metal（Apple GPU）、Vulkan。

6. **偏移（offset）**：数据指针起始位置到第一个有效元素的字节偏移。通常为 0，但在视图（view）操作中非零。

**张量形状与步长示例**

考虑一个形状为 $(3, 4, 5)$ 的稠密 FP32 张量：

- 行优先（row-major / C-order）步长：$st = (20, 5, 1)$
  - $T[1, 2, 3]$ 的偏移 $= 1 \cdot 20 + 2 \cdot 5 + 3 \cdot 1 = 33$，位于 $33 \times 4 = 132$ 字节处
- 列优先（column-major / Fortran-order）步长：$st = (1, 3, 12)$
  - $T[1, 2, 3]$ 的偏移 $= 1 \cdot 1 + 2 \cdot 3 + 3 \cdot 12 = 43$，位于 $43 \times 4 = 172$ 字节处

同一个形状的张量，layout 不同时步长完全不同，但形状语义不变。

### 3.1.2 内存布局与性能

**行优先 vs 列优先：对 GEMM 性能的影响**

矩阵乘法 $C = A \times B$ 中，两个输入矩阵的布局直接影响内存访问模式：
**行优先的 $A$：** $A[i, :]$ 的 $K$ 个元素在内存中连续。读取 $A$ 的一行是一次连续的、64/128 字节对齐的读取——理想的内存合并访问。
**列优先的 $A$：** $A[i, :]$ 的 $K$ 个元素在内存中步长为 $M$。读取 $A$ 的一行需要跳过 $M$ 个元素——cache miss 率高，带宽利用率低。
**实际策略：**

- GEMM kernel 在加载 tile 时会先检查 layout，若不匹配则执行隐式转置
- 推理引擎中通常强制所有权重矩阵采用 huggingface 默认的 layout（通常为行优先），避免运行时判断
- 对来自 ONNX 的模型（ONNX 默认可为不同 layout），需要预处理统一 layout

**图像张量的 NCHW vs NHWC：**

- **NCHW**（batch, channel, height, width）：通道维度在空间维度前。适合单个通道内的空间操作（池化、卷积核应用）。
- **NHWC**（batch, height, width, channel）：通道维度在最内层。GPU CUDA kernel 中的线程可以连续访问同一空间位置的所有通道值——有利于利用 Tensor Core 和 SIMD。
- 现代 GPU 推理通常偏好 NHWC（如 TensorRT 默认使用 NHWC）。

## 3.2　步长张量与视图

### 3.2.1　泛化的步长机制

**步长张量的偏移计算**

给定形状 $\boldsymbol{s} = (s_1, \ldots, s_n)$ 和步长 $\boldsymbol{st} = (st_1, \ldots, st_n)$，元素 $\boldsymbol{T}[i_1, \ldots, i_n]$ 在内存中的线性偏移为：

$$\text{addr}(i_1, \ldots, i_n) = \text{base_addr} + \text{offset} \cdot \text{elem_size} + \sum_{k=1}^{n} st_k \cdot i_k \cdot \text{elem_size}$$

对于稠密行优先张量，$st_k = \prod_{j=k+1}^{n} s_j$。但步长可以是任意整数，从而支持零拷贝的视角变换：

- **切片**：$T[2:5, :] \to$ 新形状 $(3, s_2)$，$st'_1 = st_1$，$st'_2 = st_2$，offset 增加 $2 \cdot st_1$
- **转置**：$T^\top \to$ 新形状 $(s_2, s_1)$，$st'_1 = st_2$，$st'_2 = st_1$。完全不移动数据。
- **广播**：将形状 $(1, d)$ 广播为 $(B, d)$：$st'_1 = 0$（沿第 0 维步长为 0——所有 $B$ 个元素指向同一个内存位置）
- **重塑（reshape）**：当张量连续时，reshape 只需更改形状 $(s'_1, s'_2)$ 和相应步长，不移动数据

**连续性（Contiguity）与可重塑性**

张量 $T$ 是**行优先连续**的当且仅当步长满足：

$$st_n = 1, \quad st_k \geq s_{k+1} \cdot st_{k+1} \ \forall k < n$$

只有连续张量才能保证 reshape 不产生数据冲突。对于非连续张量（如转置后的张量），reshape 之前必须先调用 `contiguous()` 制作一个连续副本。推理引擎中，对权重张量的 reshape（如将 $(d, h_q \cdot d_k)$ 重塑为 $(d, h_q, d_k)$）必须在张量连续的条件下进行。

### 3.2.2　零拷贝视图操作

**零拷贝张量视图的实现**

**输入：** 原张量 $T$（data, shape, strides, offset, dtype, device）
**输出：** 新张量视图 $V$

1: **函数** SLICE($T$, dim, start, end, step=1):
2: $new_shape[dim] \leftarrow \lceil (end - start)/step \rceil$
3: $new_strides[dim] \leftarrow strides[dim] \cdot step$

```
    new_offset ← offset + start · strides[dim]
    return TensorView(data, new_shape, new_strides, new_offset, dtype, device)
函数 TRANSPOSE(T, dim0, dim1):
    swap(shape[dim0], shape[dim1])
    swap(strides[dim0], strides[dim1])
    return TensorView(data, shape, strides, offset, dtype, device)
函数 BROADCAST(T, new_shape):
    new_strides ← []

for i = 0 to len(new_shape) − 1 do

    if 原 shape[i] = 1 且 new_shape[i] > 1 then
            new_strides[i] ← 0                          ▷ 步长为 0 = 广播

    else
            new_strides[i] ← strides[mapped_dim]

    return TensorView(data, new_shape, new_strides, offset, dtype, device)
```

**步长张量的陷阱**

**1. 不连续访问的 GEMM：**将转置后的矩阵直接用于 GEMM 时，内存访问模式从连续变为步长跳跃。这会导致：

- CUDA global memory 的合并访问失败
- L1/L2 cache 的有效利用率急剧下降
- 性能下降可达 $5\times \sim 10\times$

**解决方案：**在 GEMM 之前检测连续性，对不连续的输入先制作连续副本。这引入了额外的内存复制开销，但通常远小于不连续访问的代价。

**2. 广播与实际内存膨胀：**广播张量的 strides 中存在 0，导致多个逻辑元素映射到同一物理元素。如果算法需要原地修改广播张量（如 `add_`），会意外地多次累加到同一个内存位置。需要特别处理或禁止对广播张量做原地操作。

## 3.3　量化张量

### 3.3.1　量化-反量化（QDQ）范式

**量化张量的内部表示**

量化张量额外携带量化和反量化所需的参数：
量化操作（FP32 → INT8）：

$$q = \text{round}\left(\frac{x - \text{zero_point}}{\text{scale}}\right), \quad q \in [-128, 127] \text{ (signed INT8)}$$

反量化操作（INT8 → FP32）：

$$\hat{x} = q \cdot \text{scale} + \text{zero_point}$$

**量化张量存储：**数据区域存储整数（INT8/INT4），额外存储 scale（FP32/FP16）和 zero_point（整数或 FP32）。量化方案可以是：

- **per-tensor：**整个张量一个 (scale, zero_point)
- **per-channel：**每个输出通道一个 (scale, zero_point)，如权重 $(d_{\text{out}}, d_{\text{in}})$ 的 scale 形状为 $(d_{\text{out}},)$
- **per-group：**每组 $g$ 个元素一个 (scale, zero_point)，如 GPTQ 使用的 group size 128

**反量化 kernel 伪代码（per-group INT4）**

**输入：** $q_data$ (uint8[$\lceil N/2 \rceil$]), $scales$ (f16[$N/G$]), $N$ 元素, $G$ 组大小
**输出：** $fp_data$ (f16[$N$])

**for** $i = 0$ **to** $N - 1$ **do**
  $byte_idx \leftarrow i/2$
  $is_low \leftarrow (i \bmod 2) = 0$
  **if** $is_low$ **then**
    $q \leftarrow q_data[byte_idx]$ & `0x0F`　▷ 提取低 4 位
  **else**
    $q \leftarrow (q_data[byte_idx] \gg 4)$ & `0x0F`　▷ 提取高 4 位
  $q \leftarrow q - 8$　▷ 将无符号 4-bit 转为有符号 $[-8, 7]$
  $group_idx \leftarrow i/G$
  $fp_data[i] \leftarrow (f16)q \cdot scales[group_idx]$
**return** $fp_data$

**即时反量化 vs 缓存反量化**

**即时反量化（Dequantize-on-the-Fly）：**

- 在矩阵乘法 kernel 中进行，读取量化权重的整数后立即反量化，然后乘累加
- 显存占用：仅存储量化权重 + scales/zero_points
- 计算开销：每次前向都需要反量化——但对于 memory-bound 的推理来说，这通常不是瓶颈
- 典型实现：llama.cpp 的 GGML GEMM kernel

**缓存反量化（Cached Dequantization）：**

- 加载时一次性反量化，存储 FP16 副本
- 显存占用翻倍（量化版本保留 + FP16 副本）
- 每次前向直接使用 FP16 权重——最大程度利用硬件 GEMM（cuBLAS）
- 权衡：显存效率 vs 计算效率

推理引擎通常提供两种模式：内存受限模式使用即时反量化，速度优先模式使用缓存反量化。

## 3.4 张量操作 API 设计

**张量操作分类**

推理引擎的张量操作库通常按操作类型组织：

| 类别 | 示例操作 | 计算特性 |
|---|---|---|
| 逐元素 | `add`, `mul`, `relu`, `silu`, `exp` | 内存密集型 |
| 归约 | `sum`, `mean`, `max`, `argmax` | 中等 |
| 矩阵乘法 | `matmul`, `linear` | 计算密集型 |
| 数据移动 | `reshape`, `transpose`, `concat`, `split` | 内存密集型 |
| 索引 | `gather`, `scatter`, `index_select` | 内存密集型 |
| 归一化 | `rms_norm`, `layer_norm`, `softmax` | 中等 |
| 随机 | `bernoulli`, `multinomial`, `randint` | 极少量 |

**广播规则（Broadcasting Semantics）**

两个张量 $A$（形状 $\boldsymbol{s}_A$）和 $B$（形状 $\boldsymbol{s}_B$）进行逐元素二元运算时，短形状从右侧对齐：

1. 若 $\text{len}(\boldsymbol{s}_A) \neq \text{len}(\boldsymbol{s}_B)$，在较短的形状左侧补 1
2. 对每个维度 $d$，若 $s_A[d] = s_B[d]$，该维度兼容；若其中一个为 1，该维度广播为另一个的值；否则不兼容
3. 若任一维度不兼容，抛出错误

**典型广播场景：**

- $(B, 1, d) + (1, S, d) \to (B, S, d)$ ——batch 和序列都广播
- $(B, d) + (d,) \to (B, d)$ ——bias 向量广播到所有 batch 条目
- $(B, h, S, d_k) + (h, 1, d_k) \to (B, h, S, d_k)$ ——RoPE 的 cos/sin 广播

### 3.4.1　原地操作与可变语义

**原地操作的设计考量**

**原地操作**直接修改现有张量的数据而不创建新张量，用后缀 `_inplace` 或 `_ip` 标识：

- `add_inplace(A, B)`：$A \leftarrow A + B$
- `silu_inplace(A)`：$A \leftarrow \text{SiLU}(A)$

**使用场景：**

- 残差连接：$\boldsymbol{X} \leftarrow \boldsymbol{X} + \boldsymbol{O}_{\text{attn}}$ ——修改 $\boldsymbol{X}$ 而无需分配新缓冲区
- 激活函数：$\boldsymbol{A} \leftarrow \text{SiLU}(\boldsymbol{A})$ ——可以直接覆写输入

**安全性约束：**

- 若输出是某个其他张量的视图，原地操作可能意外修改其他张量
- 广播张量的原地累加会导致同一个物理元素被多次累加
- 推理引擎通常对原地操作施加限制：仅允许对最近创建的、没有任何活跃视图的张量做原地操作

# 3.5 设备抽象与内存管理

## 3.5.1 设备层级

**设备抽象层**

推理引擎的设备抽象通常包含以下接口：

| 设备 | 后端 | 内存分配器 |
|---|---|---|
| CPU | 主机内存 | `malloc/aligned_alloc` |
| CUDA | NVIDIA GPU | `cudaMalloc/cudaMallocAsync` |
| Metal | Apple GPU | `MTLBuffer` |
| Vulkan | 跨平台 GPU | `VkBuffer` |
| ROCm | AMD GPU | `hipMalloc` |

设备抽象的核心职责：

1. **内存分配与释放**：在目标设备上分配指定大小和对齐的内存
2. **数据传输**：跨设备的异步/同步数据拷贝（如 `cudaMemcpy`, `blitCommandEncoder`）
3. **同步原语**：流/队列、事件等异步执行控制
4. **设备查询**：设备属性（SM 数量、显存大小、最大线程数、warp 大小）

## 3.5.2 缓冲区管理

**所有权与生命周期**

**拥有型缓冲区（Owning Buffer）**：张量内部持有的、负责释放的缓冲区。当张量的引用计数归零时，拥有型缓冲区被释放。
**非拥有型缓冲区（Non-Owning Buffer）**：张量仅持有数据指针，不负责释放。典型场景：

- 张量视图（view）：与原张量共享同一个缓冲区
- 外部内存（user-provided）：用户自己管理的缓冲区，如 Python numpy 数组的底层内存
- 映射的内存区域（mmap）：操作系统管理的内存映射

**内存对齐要求：**

- CPU：通常要求 64 字节对齐（cache line 大小）

- CUDA：256 字节对齐（某些 tensor core 操作要求 128 或 256 字节）
- AVX-512 指令：要求 64 字节对齐
- 量化块的对齐：GGUF 中 tensor 数据区域对齐到 32 字节

### 3.5.3　内存管理策略

**引用计数与 COW（Copy-on-Write）**

**引用计数**是管理张量缓冲区共享的简单方案：

- 每个缓冲区维护一个原子引用计数（atomic refcount）
- 创建视图时引用计数 +1，销毁视图时-1
- 引用计数归零时触发缓冲区释放

**COW（写时复制）**：多线程/异步场景下的保护机制：

- 当对引用计数 $> 1$ 的缓冲区发起的原地写入请求时，先制作缓冲区副本
- 后续写入在副本上进行，不影响其他引用该缓冲区的张量
- 这在推理引擎中较为少见（推理通常是确定性的单 thread 流），在训练引擎中更常见

**内存池（Memory Pool）**：高性能推理引擎通常不直接使用 `malloc/cudaMalloc`，而是预先分配大块内存池，按需从中切分。好处：

- 避免频繁的内存分配/释放带来的 OS/驱动开销
- 复用已分配的内存块（如中间激活 buffer 的重用）
- 更好的内存局部性

# 3.6 FP16 与 BF16 存储

## 3.6.1 IEEE 754 半精度 vs BFloat16

**两种半精度浮点格式**

**FP16 (IEEE 754 binary16)**：1 位符号 + 5 位指数 + 10 位尾数

- 范围：$\pm 6.55 \times 10^{-5}$ 到 $\pm 6.55 \times 10^{4}$
- 精度：约 $10^{-3} \sim 10^{-4}$（3.3 位十进制有效数字）
- 问题：范围较小，容易发生上溢/下溢。训练时需要用 loss scaling

**BF16 (Bfloat16 / Brain Floating Point)**：1 位符号 + 8 位指数 + 7 位尾数

- 范围：$\pm 1.18 \times 10^{-38}$ 到 $\pm 3.39 \times 10^{38}$（与 FP32 相同）
- 精度：约 $10^{-2}$（2.1 位十进制有效数字）
- 优势：与 FP32 范围相同，转换简单（截断 FP32 的低 16 位或在高 16 位补零）

| 格式 | 符号位 | 指数位 | 尾数位 |
|---|---|---|---|
| FP32 | 1 | 8 | 23 |
| FP16 | 1 | 5 | 10 |
| BF16 | 1 | 8 | 7 |

## 3.6.2 格式转换与混合精度推理

**FP32 与 BF16 的转换规则**

BF16 的设计使其与 FP32 之间可以通过简单的截断/扩展进行转换：

**FP32 → BF16：**

1: 检查 FP32 的指数是否为全 1（NaN/Inf），若是则保留特殊值
2: 将 FP32 的 16 位 mantissa 截断为 7 位（简单截断，非舍入）
3: 保持符号位和指数位不变
4: **return** BF16 值

**BF16 → FP32：**

1: 符号和指数保持不变
2: 将 7 位 mantissa 左移到 FP32 mantissa 的高位，低位补零
3: **return** FP32 值

**混合精度推理策略：**

- **权重**：BF16 或 FP16 存储，计算时转换为 FP32 或 FP16
- **激活**：FP16 或 BF16，保持与权重的精度匹配
- **累加器**：FP32——矩阵乘法的内部累加使用 FP32 防止精度损失
- **注意力 softmax**：FP32——softmax 的指数计算对精度敏感
- **RMSNorm**：FP32 内部计算，输出转回 FP16/BF16

## 3.7　张量系统实现考量

**推理引擎中张量系统的设计权衡**

1. **细粒度 vs 粗粒度**：每个操作单独分配张量（细粒度，灵活但开销大）vs 预分配所有中间缓冲区并在图优化阶段复用（粗粒度，高性能但实现复杂）。推理引擎通常采用后者。
2. **同步 vs 异步**：同步 API 简单但阻塞调用。异步 API（stream/callback based）可以重叠计算和数据传输，但错误处理和 debug 更复杂。
3. **设备统一 vs 显式管理**：统一的张量接口隐藏设备差异（如 PyTorch）vs 为每种设备提供独立的张量类型（更底层、更高效）。
4. **梯度记录**：推理引擎通常不需要 autograd/梯度记录。移除梯度追踪可以：
   - 省去 tape/计算图的构建开销
   - 允许更积极的原地优化
   - 减少每个张量的元数据存储
5. **多线程安全**：推理服务（如 vLLM）中多个请求并发运行。张量系统需要保证引用计数的原子操作和缓冲区访问的线程安全。

**从 PyTorch 看张量系统的核心教训**

PyTorch 张量系统（基于 ATen/C10）的设计为推理引擎提供重要参考：
**优点**：

- 简洁的 strides + shape + offset 模型支持零拷贝视图
- dispatch 机制将算子自动路由到正确的后端实现
- 与 Python 的良好互操作性

**缺点（对推理而言）**：

- 每个张量携带过多的元数据（autograd 节点、设备对象等），在大量小张量操作中开销显著
- 动态 dispatch 引入了无法被编译器优化的函数调用开销
- CPU 端的 Python/C++ 交互开销在低延迟推理中不可接受

**推理专用张量系统**应当：精简元数据（仅 shape + strides + dtype + data + device），预编译操作序列，消除 Python overhead。

# 第四章　计算图原理与优化

## 4.1　计算图基础

### 4.1.1　有向无环图（DAG）作为计算表示

计算图是一种将数值计算抽象为图结构的表示方法：**节点**（node）代表操作（操作符），**边**（edge）代表数据流动（张量）。形式上，计算图 $G = (V, E)$ 是一个有向无环图（DAG），其中 $V$ 为操作节点集合，$E \subseteq V \times V$ 为有向边集合。

**计算图的数学形式化**

令 $\mathcal{T}$ 为所有可能的张量值集合，$\mathcal{O}$ 为所有可能的操作符集合。计算图中的一个操作节点 $v \in V$ 代表一个函数：

$$f_v : \mathcal{T}^k \to \mathcal{T}^m$$

其中 $k$ 为输入边数，$m$ 为输出边数。

**拓扑序：**若 $(u, v) \in E$，则 $u$ 必须在 $v$ 之前执行。DAG 保证了存在至少一个拓扑序——这就是静态图中" 定义后运行" 的基础。

**入度/出度：**$\deg^-(v)$ 为输入数量，$\deg^+(v)$ 为输出数量。叶节点 $\deg^-(v) = 0$ 对应图的初始输入（输入张量或常量/权重）。

**示例：**表达式 $C = \mathrm{ReLU}(\boldsymbol{A}\boldsymbol{B} + \boldsymbol{c})$ 的计算图包含以下节点：

$$\boldsymbol{D} = \mathrm{MatMul}(\boldsymbol{A}, \boldsymbol{B}), \quad \boldsymbol{E} = \mathrm{Add}(\boldsymbol{D}, \boldsymbol{c}), \quad \boldsymbol{C} = \mathrm{ReLU}(\boldsymbol{E})$$

### 4.1.2　静态图 vs 动态图

**两种图范式对比**

**静态图（Define-then-Run）：**

- **代表框架：**TensorFlow 1.x, ONNX, TensorRT, OpenVINO, TVM
- **工作流程：**(1) 使用 API 定义完整的计算图 (2) 编译/优化图 (3) 执行图
- **优点：**

  - 全局图信息可用于激进的优化（算子融合、内存规划、常量折叠）
  - 图的序列化和可移植性
  - 执行时无 Python overhead

- **缺点：**
  - 不支持 Python 原生的控制流（if/while 需要专门的图构造 API，如 tf.cond / tf.while_loop）
  - Debug 困难——错误发生在图执行阶段而非定义阶段
  - 动态形状处理需要额外机制

**动态图（Define-by-Run）：**

- **代表框架：** PyTorch (eager mode), TensorFlow 2.x (eager mode), JAX
- **工作流程：** (1) 执行 Python 代码 (2) 每次操作即时执行并构建 autograd 图 (3) 图在运行时动态生成
- **优点：**
  - Python 控制流的原生支持
  - 交互式 debug（标准 Python debugger）
  - 动态形状天然支持
- **缺点：**
  - 缺乏全局优化知识，只能做局部优化
  - Python 在每个操作上的解释开销
  - 难以序列化为自包含的部署格式

**推理引擎的取舍：**推理引擎几乎总是使用静态图。导出（export）步骤将 PyTorch eager 图转换为静态图（通过 torch.jit.trace, torch.export, 或 ONNX export），然后对此静态图执行优化和部署。

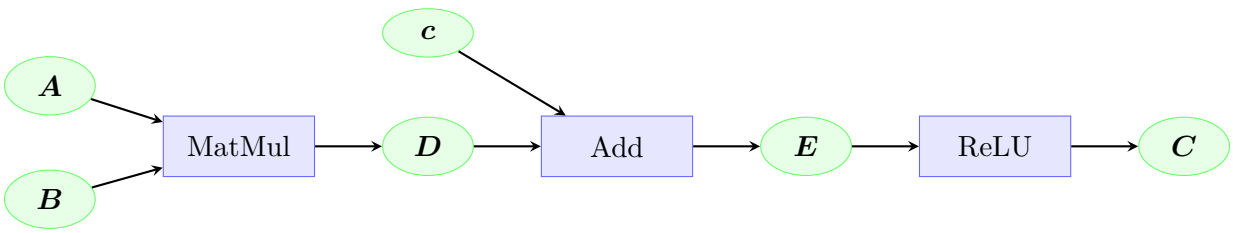



图 4.1: 表达式 $\boldsymbol{C} = \mathrm{ReLU}(\boldsymbol{A}\boldsymbol{B} + \boldsymbol{c})$ 的计算图。椭圆表示张量，矩形表示算子。

## 4.2　图中间表示（IR）设计

### 4.2.1　算子模式定义

**算子模式（Operator Schema）的字段**

推理引擎的图 IR 中，每个算子由一个模式（schema）定义。核心字段包括：

| 字段 | 类型 | 描述 |
|---|---|---|
| `op_type` | `string` | 算子类型标识符 |
| `domain` | `string` | 算子命名空间（默认为空或"ai.onnx"） |
| `inputs` | `list[string]` | 输入张量名称列表 |
| `outputs` | `list[string]` | 输出张量名称列表 |
| `attributes` | `dict[string, attr]` | 属性键值对（非张量参数） |
| `since_version` | `int` | 算子引入的 opset 版本 |

属性（attribute）类型包括：整数、浮点数、字符串、整数列表、浮点数列表、字符串列表、张量（小型常量）、图（子图）。

**ONNX 示例：** `op_type="MatMul", inputs=["A","B"], outputs=["C"]`，无属性。**带属性的算子：** `op_type="Reshape", inputs=["data","shape"], outputs=["reshaped"], attributes={"allowzero": 0}`。

### 4.2.2　图的拓扑结构构建

**从模型定义构建计算图**

**输入：** 模型 $M$ 的层序列 $[L_0, L_1, \ldots, L_{N-1}]$（如 PyTorch nn.Module 的 children）
**输出：** 计算图 $G = (V, E)$

$V \leftarrow \{\}$，$E \leftarrow \{\}$
prev_output $\leftarrow$ "input_tokens" ▷ 图的初始输入
counter $\leftarrow 0$
**for** 每层 $l$ **in** $[0, \ldots, N-1]$ **do**
    将层 $l$ 展开为基本操作列表 $[op_1, op_2, \ldots]$
    **for** 每个操作 $op$ **do**
        node_name $\leftarrow$ $op$.type + "_" + counter
        output_name $\leftarrow$ node_name + "_$out$"
        $V \leftarrow V \cup \{\text{node_name}\}$
        记录节点 node_name 的输入为 prev_output，属性为 $op$.attrs
        $E \leftarrow E \cup \{(\text{prev_output}, \text{node_name})\}$ ▷ 边（上一步输出 → 当前节点）
        prev_output $\leftarrow$ output_name

13: counter ← counter + 1
14: **return** $G$

## 4.3 图优化 Pass

图优化是推理引擎性能的关键。以下按重要性排列核心优化 pass。

### 4.3.1 常量折叠（Constant Folding）

**常量折叠的原理与实现**

**定义：**在模型加载时，识别出所有输入均为常量的子图，提前计算其结果并替换为常量节点。

**典型可折叠模式：**

- `Add(Constant(a), Constant(b))` → `Constant(a+b)`
- `Reshape(Constant(W), Constant([d1,d2]))` → `Constant(W_reshaped)`
- `Transpose(Constant(W))` → `Constant(W.T)`
- 位置编码的预计算：`Sin(Positions)` → 直接存储预计算的结果

**输入：**计算图 $G = (V, E)$
**输出：**优化后的图 $G'$

1: $changed \leftarrow$ **True**
2: **while** $changed$ **do**
3: $changed \leftarrow$ **False**
4: **for** 每个节点 $v \in V$ **do**
5: **if** $v$ 的所有输入都是 `Constant` 节点 **then**
6: 直接执行 $v$ 的操作，得到输出张量
7: 用新的 `Constant` 节点替换 $v$
8: $changed \leftarrow$ **True**
9: **return** 删除无用常量后的 $G'$

**注意事项：**常量折叠受限于可用内存。对于极大常量（如 embedding 表 $V \times d = 32000 \times 4096$），折叠后占用的显存可能超过原始表示，需评估后再折叠。

### 4.3.2　算子融合（Operator Fusion）

**关键融合模式**

**1. Conv + BatchNorm + ReLU 融合**（CNN 经典）：

$$\text{Conv}(x) \to \text{BN} \to \text{ReLU} \quad \longrightarrow \quad \text{ConvBnReLU}(x)$$

BN 的 scale/shift 在推理时是常量，可以在编译时” 折叠” 到卷积权重中：

$$\boldsymbol{W}_{\text{fused}} = \boldsymbol{W}_{\text{conv}} \cdot \frac{\gamma}{\sigma}, \quad \boldsymbol{b}_{\text{fused}} = \boldsymbol{b}_{\text{conv}} \cdot \frac{\gamma}{\sigma} + \beta - \frac{\gamma\mu}{\sigma}$$

**2. MatMul + Bias + Activation 融合**（LLM 常用）：

$$\text{MatMul}(x, W) + b \to \text{SiLU} \quad \longrightarrow \quad \text{GemmBiasSiLU}(x, W, b)$$

一次 kernel launch 完成矩阵乘法、bias 加法和激活函数，避免多次显存往返。

**3. LayerNorm/RMSNorm + MatMul 融合：**

$$\text{RMSNorm}(x) \to \text{MatMul}(x', W) \quad \longrightarrow \quad \text{NormedGemm}(x, W, \gamma)$$

将 RMSNorm 的缩放结果直接传递给 GEMM kernel，减少一个中间 buffer 的写入和读取。

**4. 残差 + 激活融合：**

$$y = x + \text{SiLU}(\text{Gemm}(x, W)) \quad \longrightarrow \quad \text{GemmSiLUResidual}(x, W, y)$$

将残差加法融入 kernel，避免单独分配残差输出 buffer。

| 融合模式 | 减少的 kernel launch | 减少的显存 I/O |
|---|---|---|
| MatMul + Bias + SiLU | $3 \to 1$ | 2 个中间 buffer |
| RMSNorm + MatMul | $2 \to 1$ | 1 个中间 buffer |
| Attention + OutputProj | $2 \to 1$ | 1 个中间 buffer |
| RoPE + MatMul | $2 \to 1$ | 无（RoPE 已是 in-place） |

### 4.3.3　死代码消除（Dead Code Elimination, DCE）

**DCE 的实现**

**定义：**删除那些输出从未被任何后续节点消费的节点。

**算法：**

**输入：**图 $G = (V, E)$，输出节点集合 $O \subseteq V$

1: $visited \leftarrow \{\}$

2: $queue \leftarrow O$ ▷ 从输出节点开始反向 *BFS*
3: **while** $queue \neq \emptyset$ **do**
4: $v \leftarrow queue.\text{pop}()$
5: **if** $v \notin visited$ **then**
6: $visited \leftarrow visited \cup \{v\}$
7: **for** 每个输入边 $(u, v) \in E$ **do**
8: $queue.\text{push}(u)$
9: 删除所有 $v \notin visited$ 的节点 ▷ 未被任何输出可达的节点
10: **return** $G_{\text{pruned}}$

**LLM 中的典型场景：**

- 训练时辅助的 dropout 层在推理时输出未被使用
- 模型导出时不小心包含的调试辅助输出
- 条件分支中未被选中的路径

### 4.3.4 公共子表达式消除（CSE）

**CSE 的原理**

**定义：**若两个或多个节点具有相同的操作类型、相同的输入和相同的属性，则它们是等价的——可以合并为一个节点，后续消费者共享同一输出。

**LLM 中的应用：**

- 共享权重矩阵（如嵌入层权重绑定，weight tying）：输入嵌入和输出投影使用同一个 $\boldsymbol{E}_{\text{tok}}$。在图中表示为两个 `MatMul` 节点使用同一个 `Constant` weight。
- 多个注意力头若共享 K/V（GQA），则 K/V 投影节点是共享的。

**实现：**哈希表的键 = (`op_type`, `inputs`, `attributes`)。插入每个节点时检查是否已有等价节点，若有则替换引用。

### 4.3.5 代数简化（Algebraic Simplification）

**代数恒等式的图级应用**

常见代数简化规则：

| 原模式 | 简化后 | 条件 |
|---|---|---|
| `Add(x, 0)` | `Identity(x)` | — |
| `Mul(x, 1)` | `Identity(x)` | — |
| `Mul(x, 0)` | `Constant(0)` | x 非 NaN |
| `Sub(x, x)` | `Constant(0)` | — |
| `Div(x, 1)` | `Identity(x)` | — |
| `Reshape(x, x.shape)` | `Identity(x)` | — |
| `Transpose(Transpose(x))` | `Identity(x)` | — |
| `Slice(x, 0, len(x))` | `Identity(x)` | — |
| `Squeeze(Unsqueeze(x))` | `Identity(x)` | 维度匹配 |

`Identity` 节点本身可以在后续 pass 中被消除（将其输入直接连接到其消费者）。

### 4.3.6　布局优化（Layout Optimization）

**消除冗余转置**

**问题：**不同框架/导出工具可能插入大量的 `Transpose` 操作。ONNX 导出有时会产生 `Transpose`→`MatMul`→`Transpose` 的冗余序列。
**消除规则：**

- $\text{Transpose}(\text{Transpose}(x)) \to \text{Identity}(x)$
- $\text{MatMul}(\text{Transpose}(A), \text{Transpose}(B)) \to \text{Transpose}(\text{MatMul}(B, A))$（某些情况下可以减少一次转置）
- 将 `Transpose` 推入后续算子：`Transpose(MatMul(A,B))` 可以变为调整 MatMul 的输入顺序

**布局传播：**为每个 tensor 标注其内存布局（NHWC/NCHW, RowMajor/ColMajor），然后在图中传播并最小化显式转置的次数。这在 TVM/Relay 的 AutoScheduler 中有系统化的实现。

### 4.3.7　内存规划（Memory Planning）

**缓冲区复用优化的形式化**

**目标：**在给定算子执行顺序（拓扑序）下，最小化所有中间张量的峰值总显存占用。
**问题建模：**每个张量 $t$ 有一个生命周期区间 $[\text{first_use}, \text{last_use}]$。若两个张量的生命周期不重叠，它们可以共享同一块物理内存。

**贪心内存分配算法：**

**输入：**按拓扑序排列的算子列表 $[op_1, \ldots, op_K]$

```
free_list ← 空列表                                  ▷ 可用的空闲 buffer
allocated ← {}                                      ▷ 张量名 → buffer 映射
refcount[t] ← 每个张量的消费者数量
for k = 1 to K do
    for 每个 op_k 的输出张量 t do
        if free_list ≠ 空 then
            buf ← free_list.pop()
        else
            buf ← 新分配的内存
        allocated[t] ← buf
    for 每个 op_k 的输入张量 t do
        refcount[t] ← refcount[t] − 1
        if refcount[t] = 0 then
            free_list.push(allocated[t])            ▷ 释放无后续引用的 buffer
return allocated
```

这个算法等价于经典的图着色寄存器分配问题，其最优解是 NP 完全的。贪心算法在实践中表现良好，峰值显存通常接近最优的 1.1× 以内。

**LLM 推理中的应用：**推理引擎中每个模型的中间激活 buffer 在执行计划阶段被分配。KV Cache 的管理使用更复杂的 PagedAttention 策略，因为 KV Cache 的生命周期跨越整个生成过程。

## 4.4 图分割

### 4.4.1 异构计算的分割

**图分割（Graph Partitioning）**

**动机：**不同算子在不同设备上的效率差异巨大。例如：

- GPU 擅长 GEMM 和大规模并行操作
- CPU 擅长控制流、tokenization、数据预处理
- NPU/DSP 擅长特定类型的卷积和量化操作

**分割策略：**将图 $G$ 划分为不相交的子图 $G_1, G_2, \ldots, G_m$，每个子图分配给一个设备。分割点需要插入 `Memcpy` 节点以在设备间传输数据。

**成本函数**：分割的代价包括每个子图的计算时间和设备间数据传输时间。图分割问题也是 NP 完全的，但对 Transformer 类模型，子图划分通常是自然的（如全部 GEMM/注意力分配给 GPU，tokenizer 分配给 CPU）。

## 4.5　执行调度

### 4.5.1　拓扑排序与并行执行

**基于数据依赖的并行调度**

给定 DAG $G = (V, E)$，多个独立的节点（互相之间没有路径连接）可以并行执行：

**调度策略**：

- **单流顺序执行**：一个 CUDA stream 顺序执行所有 kernel。实现简单。
- **多流并行**：将独立子图分配到不同 CUDA stream，利用 GPU 的 stream 并发能力。GPU 硬件支持多个 stream 的 kernel 在 SM 间交错执行。
- **图捕获（CUDA Graph）**：将整个执行序列" 捕获" 为一个 CUDA Graph，消除单次 kernel launch 的 CPU 开销。对 latency-sensitive 的推理至关重要。

**典型 Transformer 的并行机会**：

- 不同 attention head 之间（在 MHA 实现为单次 GEMM 之前）
- 不同层的 QKV 投影可以和上一层 FFN 的后续计算重叠（流水线）
- Attention 计算和 FFN 计算在不同的 layer 间可以流水线化

但在实际的高性能推理中，单层内的操作通常被融合成一个或两个大的 GPU kernel，跨 kernel 的并行化收益有限。CUDA Graph 的使用（将多次 kernel launch 合并为单次图启动）更有价值。

### 4.5.2　依赖分析：RAW、WAR、WAW

**三种数据冒险**

类似于 CPU 流水线的数据冒险，计算图的执行也存在类似依赖：

| 冒险类型 | 含义 | 示例 |
|---|---|---|
| RAW (Read-After-Write) | 真依赖——节点 B 读取节点 A 的输出 | 必须顺序执行 |
| WAR (Write-After-Read) | 反依赖——节点 B 覆写节点 A 的输入 | 不可并行（但可重命名解决 |
| WAW (Write-After-Write) | 输出依赖——节点 B 覆写节点 A 的输出 | 不可并行（但可消除） |

WAR 和 WAW 可以通过张量重命名（分配不同的 buffer）解决——这就是内存规划 pass 的另一个目的。

## 4.6 形状推断

### 4.6.1 静态形状推断

**形状推断的算子级规则**

形状推断在不执行计算的前提下，基于输入形状和算子属性推导输出形状：

| 算子 | 输入形状 | 输出形状 |
|---|---|---|
| MatMul | $(M, K) \times (K, N)$ | $(M, N)$ |
| Add | $(D_1, \ldots, D_n) + (D_1, \ldots, D_n)$ | $(D_1, \ldots, D_n)$ (广播后) |
| Reshape | $(D_1, \ldots, D_n)$, target $[t_1, \ldots, t_m]$ | $(t_1, \ldots, t_m)$ |
| Concat | $[(B, S, d), (B, S', d)]$, dim=1 | $(B, S + S', d)$ |
| Softmax | $(D_1, \ldots, D_n)$, axis=a | $(D_1, \ldots, D_n)$ (形状不变) |
| RMSNorm | $(B, S, d)$ | $(B, S, d)$ (形状不变) |

**实现：**从图的输入节点开始，按拓扑序遍历所有节点，为每个节点的输出推断形状。这使后续的 buffer 大小确定和内存规划成为可能。

### 4.6.2 符号形状与动态形状

**动态维度的处理**

当 batch size 或序列长度在每次推理中变化时，完全静态的形状推断无法完成。解决方案：

**1. 符号形状（Symbolic Shape）：**某些维度用符号变量（而非具体整数）表示，如 $(B, S, d)$ 中 $B$ 和 $S$ 是符号变量。形状推断在符号层面进行，产生符号的输出形状。在运行时，用实际值替换符号变量以确定 buffer 大小。

**2. 动态形状的多 profile**（TensorRT 做法）：对于动态维度，预先定义一组 profile（如 $S \in \{128, 512, 2048, 4096\}$），为每个 profile 编译优化 kernel。运行时选择最佳匹

配的 profile。

**3. 批处理调度**（vLLM 做法）：将不同长度序列的请求”填充”（padding）为统一的最大长度。但这会浪费显存。更高效的做法是使用 CUDA Graph 的动态形状支持或连续批处理（continuous batching）。

## 4.7　图中的控制流

**IF/ELSE 与循环的图表示**

静态图对控制流的支持需要专门的图构造节点：

**条件分支（If）：**

- ONNX: `op_type="If"`, 输入条件 bool + 两个子图 (then_branch, else_branch)
- 运行时根据条件值执行其中一个子图
- 图优化中，若条件是常量，可以消除分支直接展开

**循环（Loop/While）：**

- ONNX: `op_type="Loop"`, 输入循环次数 + 条件 + 子图 (body)
- 循环体子图在每次迭代时执行
- 静态展开（loop unrolling）：若循环次数是常量且较小，可以在编译时展开

**LLM 推理中的控制流需求：**

- 自回归生成循环：标准做法是在推理引擎的调度层（C++/Python）处理循环，不在计算图中
- 投机解码：验证阶段的”树注意力”可能使用条件选择
- 束搜索（beam search）：多个候选序列可以 batch 化为图中的一个并行维度

## 4.8　子图替换与模式匹配

**图重写的模式匹配框架**

子图替换是图优化 pass 的通用框架：

**模式定义**：定义一个模式图 $P = (V_P, E_P)$，其中节点和边可能包含通配符和约束条件。

**模式匹配：**在主图 $G$ 中查找所有同构于 $P$ 的子图。由于子图同构是 NP 完全问题，实际中限制搜索空间（如按拓扑序限制候选节点）。
**替换规则：**将匹配到的子图替换为目标子图。
**LLM 中的典型重写规则：**

1. **SwiGLU 合并：**$\text{SiLU}(xW_g) \odot (xW_u) \to W_d \longrightarrow$ 单个 fused SwiGLU kernel
2. **Attention 替换：**MHA 的 5 步操作（QKV 投影 + reshape + 注意力 + reshape + 输出投影）$\longrightarrow$ 单个 FlashAttention 调用
3. **RMSNorm 重写：**将除方根 + 除法 + 逐元素乘的模式替换为优化的 RMSNorm kernel
4. **数值精度提升：**将不稳定实现（除小数的 `Div`）替换为数值稳定的版本（`Add Epsilon` 后再 `Div`）

## 4.9 图序列化

**优化后图的持久化**

**序列化需求：**

- 优化耗时长（TensorRT engine build 可能数分钟到数小时），不希望每次启动都重新优化
- 分发优化后的图给用户（类似 TensorRT engine 文件）
- 版本一致性：确保序列化的图与当前推理引擎版本兼容

**序列化格式：**

- **Protobuf**（ONNX 方式）：将 GraphProto 序列化到文件，完整保留图结构
- **FlatBuffers**：比 Protobuf 更快解析，适合嵌入式
- **自定义二进制**：自定义紧凑格式（类似 GGUF），包含算子类型、输入输出映射、属性和权重数据

**关键挑战：**

- **浮点权重精度**：FP32 → FP16 存储时可能有微小的舍入误差
- **架构特化**：为特定 GPU 架构（SM80, SM89, SM90）编译的 kernel 无法在其他架构上执行
- **版本兼容**：图结构（算子类型、属性名）可能在不同推理引擎版本间变化

## 4.10　从图到执行：完整流水线

**推理引擎的图处理流水线**

模型的完整处理流程：

1. **导入**：加载模型文件（GGUF, ONNX, safetensors），解析为内部 IR 图
2. **调平（Leveling）**：将框架特有的高层操作展开为基本操作（如将 `nn.Linear` 展开为 `MatMul` + `Add`）
3. **常量折叠**：预计算所有常量子图
4. **形状推断**：推导所有中间张量的形状
5. **算子融合**：运行融合 pass，减少 kernel 数量
6. **布局优化**：消除冗余转置，统一内存布局

7. **内存规划：**为中间张量分配/复用缓冲区
8. **调度：**生成拓扑序执行计划
9. **代码生成 / Kernel 选择：**为每个算子选择或编译最优 kernel 实现
10. **（可选）序列化：**将优化后的图保存为缓存文件
11. **执行：**按照执行计划运行模型

| 阶段 | 耗时占比（典型） |
|---|---|
| 加载 + 反量化 | 20% ∼ 60%（取决模型大小和存储速度） |
| 图优化 | 1% ∼ 5%（通常很快） |
| 首次推理 | 剩余时间 |
| 后续推理 | （利用了预编译的 kernel 缓存） |

**图优化的核心洞见**

1. **融合 > 其他一切：**减少 kernel launch 数量是推理性能提升的最大单一因素。一个 fused kernel 可能比 3 个单独 kernel 快 2× ∼ 3×。
2. **内存是最稀缺的资源：**图优化的首要目标是减少显存占用（峰值 + 持续占用），而非减少 FLOPs。KV Cache 和中间激活是显存的消费大头。
3. **静态信息在编译期利用：**运行时的动态 dispatch 代价高昂。能在编译期确定的（形状、layout、量化参数）就不要留到运行时。
4. **图优化和 kernel 优化是互补的：**图优化决定” 做什么计算”，kernel 优化决定” 怎么计算”。两者共同决定最终性能。
5. **通用优化 vs 模型特化：**针对特定模型架构（如 LLaMA 风格 decoder-only）可以设计比通用图优化器更激进、更高效的优化 pass。

# 第二部分

# 核心实现篇：引擎的内部机制

# 第五章　算子实现原理

算子（operator），在很多框架中也称为核函数（kernel），是计算图中的基本执行单元。每一个算子接收一个或多个输入张量，执行特定的数值计算，产生一个或多个输出张量。正如细胞是生物体的基本结构单位，算子就是推理引擎的基本计算单位。本章将系统性地剖析各类核心算子的数学原理和高效实现方法。

## 5.1　算子接口设计

一个良好设计的算子系统需要统一的接口抽象，使得上层框架可以透明地调度不同后端（CPU、GPU、NPU 等）的实现。

**算子抽象接口**

算子基类通常包含以下虚函数接口：

**compute()** 执行实际计算。接收输入张量列表，产生输出张量列表。这是算子的核心函数。

**infer_shape()** 根据输入张量的形状推断输出张量的形状。在计算图构建阶段调用，不需要实际数据。

**memory_plan()** 估算算子执行所需的内存大小。包括临时缓冲区和工作空间。

**supports_inplace()** 算子是否支持原地操作。例如 ReLU 可以将输出写入输入张量的内存空间。

**backend()** 返回算子当前运行的后端类型，如 `Backend::CPU`、`Backend::CUDA` 等。

**算子注册机制**

推理引擎通常使用注册模式将算子名与实现类解耦：

```
REGISTER_OP("MatMul")
    .Input("A: float32")
    .Input("B: float32")
    .Output("C: float32")
```

```
    .Attr("transpose_a: bool = false")
    .Attr("transpose_b: bool = false")
    .SetShapeFn([](InferenceContext* ctx) { ... });
```

这样，当计算图中出现特定类型的算子时，引擎可以在运行时查找对应的实现并实例化。

## 5.2 矩阵乘法算子

矩阵乘法（Matrix Multiplication）是神经网络推理中最重要的算子，没有之一。在 Transformer 模型中，全连接层、注意力投影、嵌入查找等都以矩阵乘法为核心。一个推理引擎的性能上限在很大程度上由其 `MatMul`/GEMM 的实现质量决定。

### 5.2.1 朴素实现与缓存分析

**朴素矩阵乘法**

给定 $\boldsymbol{A} \in \mathbb{R}^{M\times K}$，$\boldsymbol{B} \in \mathbb{R}^{K\times N}$，矩阵乘法 $\boldsymbol{C} = \boldsymbol{AB} = \boldsymbol{C} \in \mathbb{R}^{M\times N}$ 的朴素实现为三重循环：

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} \cdot B_{kj}, \quad 0 \le i < M,\ 0 \le j < N \tag{5.1}$$

时间复杂度为 $O(MNK)$，空间复杂度为 $O(MN)$（输出矩阵）。

朴素实现虽然正确，但在现代处理器上性能极差。以一个 $1024 \times 1024 \times 1024$ 的矩阵乘法为例，它需要执行约 $10^9$ 次乘加运算，但实际运行时间可能比优化版本慢 $10 \sim 100$ 倍。造成性能差异的根本原因在于**缓存未命中**（cache miss）。

考虑以 `i-j-k` 顺序遍历的朴素实现：

- 当访问 $\boldsymbol{A}$ 的第 $i$ 行时，如果 $K$ 很大，每次取 $A_{i,0}$ 到 $A_{i,K-1}$ 可以顺序访问，缓存命中较好。
- 但访问 $\boldsymbol{B}$ 的第 $j$ 列时，$B_{0,j}, B_{1,j}, \ldots, B_{K-1,j}$ 在行优先存储中是间隔 $N$ 个元素分布的，缓存行利用率极低。如果 $N$ 超过 L1 缓存大小，每次访问几乎都会导致缓存未命中。
- 对 $\boldsymbol{C}$ 的累加也存在类似问题。

### 5.2.2 循环顺序分析

矩阵乘法的六个循环顺序中，只有两个是缓存友好的：

表 5.1: 六种循环顺序的缓存行为分析（假设行优先存储）

| 循环顺序 | 内层累加 | 访存模式 | 缓存效率 |
|---|---|---|---|
| ijk | `for k` | A 连续, B 跨步, C 固定 | 差 |
| ikj | `for j` | A 固定, B 连续, C 连续 | 好 |
| jik | `for k` | A 跨步, B 连续, C 固定 | 差 |
| jki | `for i` | A 连续, B 固定, C 跨步 | 差 |
| kij | `for j` | A 固定, B 连续, C 连续 | 好 |
| kji | `for i` | A 连续, B 固定, C 跨步 | 差 |

**最佳循环顺序的直觉理解**

在 `ikj` 和 `kij` 顺序中，最内层循环在 $\boldsymbol{B}$ 的同一条行和 $\boldsymbol{C}$ 的同一条行上遍历，这两个访问在行优先存储中都是连续的。而 $\boldsymbol{A}$ 的元素在最内层循环中保持不变（可保持在寄存器中）。因此这两种顺序具有最佳的缓存局部性。反直觉的是：**将归约循环 $k$ 放在最外层反而比放在最内层更好**。

### 5.2.3　分块（Tiling）策略

分块是提升矩阵乘法性能的核心技术。基本思想是将大矩阵划分为多个小矩阵块，使得每个块的数据都可以完全放入高速缓存（L1/L2）中。

**分块矩阵乘法**

**输入:** $\boldsymbol{A} \in \mathbb{R}^{M\times K}$, $\boldsymbol{B} \in \mathbb{R}^{K\times N}$, 块大小 $T_m, T_n, T_k$
**输出:** $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$

**for** $i_o \leftarrow 0$ **to** $M-1$ **step** $T_m$ **do**
  **for** $j_o \leftarrow 0$ **to** $N-1$ **step** $T_n$ **do**
    初始化 $\boldsymbol{C}$ 子块为 0
    **for** $k_o \leftarrow 0$ **to** $K-1$ **step** $T_k$ **do**
      将 $\boldsymbol{A}$ 子块 $[i_o : i_o + T_m, k_o : k_o + T_k]$ 加载到缓存
      将 $\boldsymbol{B}$ 子块 $[k_o : k_o + T_k, j_o : j_o + T_n]$ 加载到缓存
      计算 $\boldsymbol{C}$ 子块 $+=$ $\boldsymbol{A}$ 子块 $\times$ $\boldsymbol{B}$ 子块

当子块 $T_m \times T_k$ 和 $T_k \times T_n$ 完全适配到 L1 缓存时，每个子块只需要从主存加载一次，而块内的元素可以被重复使用 $T_m \times T_n/(T_m \times T_k + T_k \times T_n)$ 次。这极大减少了主存访问次数。

### 5.2.4　块大小选择策略

块大小的选择本身是一个优化问题。假设处理器有：

- 寄存器数量 $R$（例如 ARM NEON 有 32 个 128 位寄存器，x86-64 AVX-512 有 32 个 512 位寄存器）
- L1 数据缓存大小 $L_1$（典型值 32KB）
- L2 缓存大小 $L_2$（典型值 256KB~1MB）
- L3 缓存大小 $L_3$（典型值 8MB~32MB）

**分块空间约束**

一个 $T_m \times T_n$ 的输出块需要 $T_m \times T_n \times \text{sizeof(element)}$ 字节的寄存器空间来累加。一个 $T_m \times T_k$ 的 $\boldsymbol{A}$ 块和一个 $T_k \times T_n$ 的 $\boldsymbol{B}$ 块共需要

$$(T_m \times T_k + T_k \times T_n) \times \text{sizeof(element)} \leq L_1 \tag{5.2}$$

的空间来避免 L1 缓存未命中。类似地，更大的块可以适配到 L2 和 L3 约束。

实际调优中常见的做法是：将 $T_k$ 设得较大（因为 $k$ 是归约维度，增大 $T_k$ 可以增加数据复用），将 $T_m$ 和 $T_n$ 适配到寄存器文件的约束。

### 5.2.5 微内核（Micro-Kernel）

微内核是分块矩阵乘法的最内层实现。它通常对非常小的块（例如 $6 \times 16$）进行极度优化，使用汇编或内联函数将寄存器使用推到极限。

**一个典型的 ARM 微内核结构**

对于 $M_r \times N_r$ 大小的微内核（例如 $6 \times 8$），实现为：

- 将 $\boldsymbol{A}$ 的 $M_r$ 行 $\times$ $K_c$ 列的块按列打包（Packing），确保每个 $M_r \times 1$ 的列微面板在内存中连续。
- 将 $\boldsymbol{B}$ 的 $K_c$ 行 $\times$ $N_r$ 列的块按行打包，确保每个 $1 \times N_r$ 的行微面板在内存中连续。
- 最内层循环在 $K_c$ 上，每次取 1 列（A 微面板）和 1 行（B 微面板），执行外积并累加到 $M_r \times N_r$ 的寄存器块中。

**微内核伪代码**

```
// 微内核: C_6x8 += A_6xK * B_Kx8
// 假设所有数据已打包为连续内存
void micro_kernel_6x8(
    float* packed_A,  // 6 x kc, 列优先连续
```

```
    float* packed_B,  // kc x 8, 行优先连续
    float* C, int ldc,
    int kc
) {
    float32x4_t c_reg[12];  // 6x8 输出，使用12个128位寄存器
    // 清零累加寄存器
    for (int p = 0; p < kc; p++) {
        float32x4_t a_0 = vld1q(packed_A + p*6 + 0);
        float32x4_t a_1 = vld1q(packed_A + p*6 + 4);
        float32x4_t b_0 = vld1q(packed_B + p*8 + 0);
        float32x4_t b_1 = vld1q(packed_B + p*8 + 4);
        // FMLA: fused multiply-add
        c_reg[0] = vfmaq_laneq_f32(c_reg[0], b_0, a_0, 0);
        // ... 更多FMLA指令展开
    }
    // 将寄存器写回C矩阵
}
```

### 5.2.6　打包（Packing）策略

打包是矩阵乘法优化的关键步骤。它重新排列输入数据的内存布局，使得微内核可以以完全顺序的方式访问数据。

**打包的目的**

对于行优先存储的 $\boldsymbol{B} \in \mathbb{R}^{K\times N}$，当微内核需要访问 $B[k:k+T_k, j:j+N_r]$ 子块时，自然的内存布局是跨步的（stride 为 $N$）。打包将子块重排为连续内存：

$$\text{packed_B}[p\cdot N_r + j'] = B[k+p][j+j'],\quad p\in[0,T_k), j'\in[0,N_r) \tag{5.3}$$

这样微内核内层循环只需顺序访问 `packed_B`，所有加载都是缓存友好的。

**打包的代价**

打包本身需要额外的内存拷贝操作。对于大型矩阵乘法，打包的时间开销通常可以忽略（因为它们增加了数据复用的收益），但对于小型矩阵乘法（如 GEMV，即矩阵-向量乘），打包开销可能超过收益。因此实际实现中通常设置一个尺寸阈值，小矩阵直接使用非打包版本。

### 5.2.7 批量矩阵乘法

在神经网络推理中，我们经常需要同时处理多个独立的矩阵乘法。例如在 Transformer 的多头注意力中，所有注意力头的 $Q, K, V$ 投影可以批量化。

**批量矩阵乘法 BMM**

给定 $\boldsymbol{A} \in \mathbb{R}^{B\times M\times K}$，$\boldsymbol{B} \in \mathbb{R}^{B\times K\times N}$，批量矩阵乘法定义为：

$$C_{b,i,j} = \sum_{k=0}^{K-1} A_{b,i,k} \cdot B_{b,k,j}, \quad \forall b \in [0, B) \tag{5.4}$$

其中 $B$ 是批量维度。可以通过以下方法实现：

- **批量串行**: 循环遍历 $B$，对每个批次的 $M \times K$ 和 $K \times N$ 调用标准 GEMM。
- **展平**: 当所有批次具有相同的矩阵尺寸时，可以将其视为一个 $(B \times M) \times K$ 和 $K \times (B \times N)$ 的 GEMM。注意这仅在 $B \times M$ 和 $B \times N$ 的结果与展平前一致时才有效。
- **跨批次分块**: 在每个分块内同时处理多个批次的子块，增加并行度。

## 5.3 卷积算子

### 5.3.1 直接卷积

在输入特征图 $\boldsymbol{X} \in \mathbb{R}^{C_{in}\times H_{in}\times W_{in}}$ 上使用卷积核 $\boldsymbol{W} \in \mathbb{R}^{C_{out}\times C_{in}\times K_h\times K_w}$ 进行卷积运算，输出特征图 $\boldsymbol{Y} \in \mathbb{R}^{C_{out}\times H_{out}\times W_{out}}$。

**直接卷积公式**

$$Y_{o,h,w} = \sum_{c=0}^{C_{in}-1} \sum_{i=0}^{K_h-1} \sum_{j=0}^{K_w-1} W_{o,c,i,j} \cdot X_{c,h\cdot S_h+i,w\cdot S_w+j} \tag{5.5}$$

其中 $S_h, S_w$ 分别为高和宽方向的步长。输出尺寸为：

$$H_{out} = \left\lfloor \frac{H_{in} - K_h + 2P_h}{S_h} \right\rfloor + 1 \tag{5.6}$$

直接卷积需要 7 层嵌套循环（$o, h, w, c, i, j$ 加上输入通道累加），实现简单但缓存局部性差。当面临小型卷积核（如 $3 \times 3$）时，im2col 方法可能不如直接法高效（因为展开的内存膨胀因子太大）。

### 5.3.2　Im2col + GEMM 方法

im2col（image to column）是将卷积转化为矩阵乘法的经典技术。

**Im2col 转换**

**输入:** 输入 $\boldsymbol{X} \in \mathbb{R}^{C_{in} \times H_{in} \times W_{in}}$，卷积核 $K_h \times K_w$，步长 $S_h, S_w$，填充 $P_h, P_w$
**输出:** 展开矩阵 $\boldsymbol{X}_{col} \in \mathbb{R}^{C_{in}K_hK_w \times H_{out}W_{out}}$

对 $\boldsymbol{X}$ 进行填充
**for** $h \leftarrow 0$ **to** $H_{out} - 1$ **do**
  **for** $w \leftarrow 0$ **to** $W_{out} - 1$ **do**
    **for** $c \leftarrow 0$ **to** $C_{in} - 1$ **do**
      **for** $i \leftarrow 0$ **to** $K_h - 1$ **do**
        **for** $j \leftarrow 0$ **to** $K_w - 1$ **do**
          $\boldsymbol{X}_{col}[c \cdot K_hK_w + i \cdot K_w + j][h \cdot W_{out} + w] \leftarrow \boldsymbol{X}[c][h \cdot S_h + i][w \cdot S_w + j]$

将卷积核重塑为 $\boldsymbol{W}_{col} \in \mathbb{R}^{C_{out} \times C_{in}K_hK_w}$，则卷积等价于：

$$\boldsymbol{Y}_{col} = \boldsymbol{W}_{col} \cdot \boldsymbol{X}_{col}, \quad \boldsymbol{Y}_{col} \in \mathbb{R}^{C_{out} \times H_{out}W_{out}} \tag{5.7}$$

随后将 $\boldsymbol{Y}_{col}$ 重塑为 $\mathbb{R}^{C_{out} \times H_{out} \times W_{out}}$。

**Im2col 的内存开销**

im2col 的内存膨胀因子为 $K_h \times K_w$。对于 $3 \times 3$ 卷积，输入放大 9 倍；对于 $7 \times 7$ 卷积，放大 49 倍。在大型网络或大输入分辨率下，这一开销可能是不可接受的。

### 5.3.3　Winograd 卷积

Winograd 算法是卷积计算的一个重要优化方向。对于小卷积核（特别是 $3 \times 3$），Winograd $F(2,3)$ 和 $F(4,3)$ 可以显著减少乘法次数。

**Winograd 最小滤波算法**

Winograd $F(m,r)$ 算法用 $r$ 个元素的滤波器计算 $m$ 个输出。一维版本所需的乘法次数为 $m + r - 1$，而直接法需要 $m \times r$ 次。二维版本（通过嵌套）所需的乘法次数为 $(m + r - 1)^2$，而直接法需要 $m^2 \times r^2$ 次。

**标题**

对于输入 $d_0, d_1, d_2, d_3$ 和滤波器 $g_0, g_1, g_2$，一维 $F(2,3)$ 计算两个输出 $r_0, r_1$。

变换矩阵为：

$$\boldsymbol{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 0 & 0 & 0 \end{bmatrix}, \quad \boldsymbol{G} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{B} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & -1 \\ 0 & -1 \end{bmatrix} \tag{5.8}$$

计算步骤：

1. 滤波器变换: $\boldsymbol{U} = \boldsymbol{G}\boldsymbol{g}\boldsymbol{G}^T$（离线完成）
2. 输入变换: $\boldsymbol{V} = \boldsymbol{B}^T\boldsymbol{d}\boldsymbol{B}$
3. 逐元素乘法: $\boldsymbol{M} = \boldsymbol{U} \odot \boldsymbol{V}$（Hadamard 积）
4. 输出逆变换: $\boldsymbol{r} = \boldsymbol{A}^T\boldsymbol{M}\boldsymbol{A}$

**Winograd 的数值稳定性**

Winograd 变换矩阵包含 $\frac{1}{2}$ 等分数元素，在定点/整数量化场景下可能导致精度问题。因此在实际部署中，量化模型常使用 im2col+GEMM 而非 Winograd，或使用特殊设计的 Winograd-friendly 量化方案。

对于 $F(2,3)$，4 次输出需要 36 次乘法（直接法），而 Winograd 只需 16 次（$\frac{16}{36} = 44.4\%$ 的乘加运算）。对于 $F(4,3)$，加速比更为显著。

### 5.3.4 FFT 基卷积

对于大卷积核，基于快速傅里叶变换的方法可以将卷积的时域计算转化为频域的点乘：

$$\boldsymbol{Y} = \mathcal{F}^{-1}\big(\mathcal{F}(\boldsymbol{W}_{padded}) \odot \mathcal{F}(\boldsymbol{X}_{padded})\big) \tag{5.9}$$

其复杂度从 $O(N^2K^2)$ 降低到 $O(N^2 \log N)$。但在现代神经网络中，卷积核通常较小（$3 \times 3$ 或 $1 \times 1$），FFT 的优势不明显甚至更差。

### 5.3.5 深度可分离卷积

深度可分离卷积将标准卷积分解为逐通道的空间卷积（depthwise）和点卷积（pointwise，即 $1 \times 1$ 卷积）：

$$Y_{o,h,w} = \sum_{c=0}^{C_{in}-1} \left( \sum_{i,j} W_{c,i,j}^{depth} \cdot X_{c,hS_h+i,wS_w+j} \right) \cdot W_{o,c}^{point} \tag{5.10}$$

计算量从 $C_{out} \cdot C_{in} \cdot K_h \cdot K_w \cdot H_{out} \cdot W_{out}$ 降低到 $C_{in} \cdot K_h \cdot K_w \cdot H_{out} \cdot W_{out} + C_{out} \cdot C_{in} \cdot H_{out} \cdot W_{out}$。

## 5.4　注意力算子

**缩放点积注意力**

给定查询 $\boldsymbol{Q} \in \mathbb{R}^{N\times d_k}$，键 $\boldsymbol{K} \in \mathbb{R}^{M\times d_k}$，值 $\boldsymbol{V} \in \mathbb{R}^{M\times d_v}$：

$$\text{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{5.11}$$

其中 $N$ 是查询（query）序列长度，$M$ 是键（key）/值（value）的序列长度。注意 $QK^T$ 生成一个 $N \times M$ 的注意力矩阵，其内存复杂度为 $O(NM)$，在长序列下成为瓶颈。

**注意力的计算分解**

标准注意力算子的计算分解为以下子步骤：

1. $\mathbf{QK}^T$ 矩阵乘法: $\boldsymbol{S} = \boldsymbol{Q}\cdot\boldsymbol{K}^T \in \mathbb{R}^{N\times M}$
2. **缩放**: $\boldsymbol{S} \leftarrow \boldsymbol{S}/\sqrt{d_k}$
3. **掩码**（如因果掩码）: $\boldsymbol{S}[i,j] \leftarrow -\infty$ 当 $j > i$（自回归解码）
4. **Softmax**（按行）: $\boldsymbol{P} = \text{softmax}(\boldsymbol{S})$
5. **PV 矩阵乘法**: $\boldsymbol{O} = \boldsymbol{P}\cdot\boldsymbol{V} \in \mathbb{R}^{N\times d_v}$

在多线程/GPU 上，计算密集型步骤（1 和 5）使用 GEMM；内存密集型步骤（3 和 4）使用 element-wise 核函数。

FlashAttention 类算法通过将注意力计算在线进行（online softmax），避免将完整的 $N \times M$ 注意力矩阵写入 HBM，从而将内存复杂度从 $O(N^2)$ 降低到 $O(N)$。其核心思想将在第 16 章详细讨论。

## 5.5 激活函数算子

### 5.5.1 精确公式

$$\mathrm{ReLU}(x) = \max(0, x) \tag{5.12}$$

$$\mathrm{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right] \tag{5.13}$$

$$\mathrm{SiLU/Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{5.14}$$

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{5.15}$$

其中 $\Phi(x)$ 为标准正态分布的累积分布函数，$\sigma(x)$ 为 Sigmoid 函数。

### 5.5.2 高效近似

精确计算 erf 和 exp 函数较为昂贵。实际推理引擎通常使用快速近似：

**快速 GELU 近似**

$$\mathrm{GELU}_{approx}(x) = 0.5x \cdot \left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^{3}\right)\right]\right) \tag{5.16}$$

此近似在 $|x| < 10$ 范围内高度精确，并且避免了 erf 的计算。

**快速指数函数**

对于 Sigmoid 和 SiLU 中的 exp 计算，可以使用以下技巧：

- **查表法**: 预计算 exp 在网格点上的值，运行时线性插值。
- **exp2f 指令**: 某些平台 `exp2f(x)` 比 `expf(x)` 快，结合 $\exp(x) = 2^{x/\ln 2}$ 使用。
- **Schraudolph 近似**: 利用 IEEE 754 浮点表示的位操作技巧。不过精度较低，在推理中需谨慎使用。

# 5.6　归一化算子

## 5.6.1　LayerNorm

**LayerNorm**

对输入 $\boldsymbol{x} \in \mathbb{R}^d$（通常沿最后一维），LayerNorm 定义为：

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \tag{5.17}$$

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 \tag{5.18}$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \tag{5.19}$$

$$y_i = \gamma_i \cdot \hat{x}_i + \beta_i \tag{5.20}$$

其中 $\gamma, \beta \in \mathbb{R}^d$ 是可学习的仿射参数，$\epsilon$ 是数值稳定常数（通常为 $10^{-5}$ 或 $10^{-6}$）。

实际实现中，需要进行两次遍历输入向量：第一次计算 $\mu$ 和 $\sigma^2$，第二次应用归一化和仿射。但可以通过 Welford 算法的在线版本在一次遍历中完成。

## 5.6.2　RMSNorm

**RMSNorm**

RMSNorm 是 LayerNorm 的简化版本，省略了均值中心化步骤：

$$\mathrm{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}, \quad y_i = \frac{x_i \cdot \gamma_i}{\mathrm{RMS}(\boldsymbol{x})} \tag{5.21}$$

RMSNorm 被 LLaMA 系列模型广泛采用。它在推理时比 LayerNorm 快约 $15\% \sim 25\%$，因为省略了均值计算和 $\beta$ 偏置项。

# 5.7　逐元素算子

**逐元素操作**

逐元素算子对输入张量的每个元素独立应用相同的标量运算。常见的有：

- **Scale**: $y = \alpha \cdot x$
- **Add**: $y = a + b$（Broadcasting 规则适用）
- **Multiply**: $y = a \cdot b$

- **Divide**: $y = a/b$
- **Tanh/ReLU 等**: 见上节激活函数。

逐元素算子的计算密度（计算量/访存量）极低——每个元素只需要 1∼2 次浮点运算，却需要加载 1∼2 个输入和写入 1 个输出。因此它们是典型的**内存密集型**（memory-bound）算子。

**逐元素算子的优化策略**

由于计算密度低，优化的核心是**减少内存访问**：

1. **算子融合**: 将多个连续的逐元素算子融合为一个核函数，如将 ReLU(SiLU(x) + y) 在一次内核中完成。
2. **自动向量化**: 使用 SIMD 指令一次处理 4∼16 个元素。
3. **预取**: 利用预取指令提前将数据加载到缓存。
4. **Cache bypass**: 对于仅被访问一次的数据，使用非时间性存储（non-temporal store）绕过缓存直接写入内存。

## 5.8 归约算子

**归约操作**

归约算子沿指定轴对张量进行聚合计算。例如沿第 2 维求和：

$$Y_{i_0,i_1} = \sum_{i_2=0}^{D-1} X_{i_0,i_1,i_2}, \quad \text{输出形状从}(N_0, N_1, D)\text{变为}(N_0, N_1) \tag{5.22}$$

常见归约操作包括 Sum、Max、Min、Mean、Prod、Argmax。

高效实现并行归约是 GPU 编程的经典问题。在 CUDA 中，warp 级别的归约使用洗牌指令（shuffle）：

```
// Warp-level sum reduction (32 threads per warp)
val += __shfl_down_sync(0xFFFFFFFF, val, 16);
val += __shfl_down_sync(0xFFFFFFFF, val, 8);
val += __shfl_down_sync(0xFFFFFFFF, val, 4);
val += __shfl_down_sync(0xFFFFFFFF, val, 2);
val += __shfl_down_sync(0xFFFFFFFF, val, 1);
```

这比使用共享内存的树形归约更为高效，因为 shuffle 指令的延迟仅约 5 个周期，而共享内

存访问则需要约 30 个周期。

## 5.9　嵌入查找与位置编码

### 5.9.1　嵌入查找

嵌入查找是一种特殊的聚集操作：给定一个词汇 ID 索引向量 $\boldsymbol{ids} \in \mathbb{Z}^{B\times S}$ 和权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{V\times d}$（$V$ 为词表大小，$d$ 为嵌入维度），输出为：

$$\boldsymbol{E}[b,s,:] = \boldsymbol{W}[\boldsymbol{ids}[b,s],:] \tag{5.23}$$

在 GPU 上，实现嵌入查找需要特别处理，因为大词汇量（$V$ 可达 32K∼128K 甚至 256K）意味着 $W$ 可能占用数百 MB。Gather 操作本质上是随机访问，缓存效率极低。一种优化是使用**嵌入量化和压缩**，例如将权重矩阵量化为 int4/int8 并在查找时解量化。

### 5.9.2　RoPE 旋转位置编码

RoPE 通过对 $Q$ 和 $K$ 的每对相邻维度施加旋转变换来编码位置信息。对于位置 $p$ 和维度对 $2i, 2i+1$：

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(p\cdot\theta_i) & -\sin(p\cdot\theta_i) \\ \sin(p\cdot\theta_i) & \cos(p\cdot\theta_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix} \tag{5.24}$$

其中 $\theta_i = \texttt{base}^{-2i/d}$，`base` 通常为 10000。

**原地 RoPE 实现**

**输入：** $\boldsymbol{X} \in \mathbb{R}^{N\times d}$（$Q$ 或 $K$），位置 $p$
**输出：** $\boldsymbol{X}$ 原地旋转

**for** $i \leftarrow 0$ **to** $d/2-1$ **do**
  $\theta \leftarrow p \cdot \texttt{base}^{-2i/d}$
  $\cos_val \leftarrow \cos(\theta),\ \sin_val \leftarrow \sin(\theta)$
  **for** $n \leftarrow 0$ **to** $N-1$ **do**
    $x_0 \leftarrow \boldsymbol{X}[n,2i],\ x_1 \leftarrow \boldsymbol{X}[n,2i+1]$
    $\boldsymbol{X}[n,2i] \leftarrow x_0\cdot\cos_val - x_1\cdot\sin_val$
    $\boldsymbol{X}[n,2i+1] \leftarrow x_0\cdot\sin_val + x_1\cdot\cos_val$

### 5.9.3 KV 缓存更新

**KV 缓存更新算子**

在自回归解码中，每生成一个新 token，我们需要将新计算的 $\boldsymbol{k}_{new} \in \mathbb{R}^{1\times d_k}$（或 $H$ 个注意力头、$B$ 个样本的批量）拼接到已缓存的 $\boldsymbol{K}_{cache} \in \mathbb{R}^{L\times d_k}$ 上：

$$\boldsymbol{K}'_{cache}[0:L,:] = \boldsymbol{K}_{cache}[0:L,:]; \quad \boldsymbol{K}'_{cache}[L,:] = \boldsymbol{k}_{new} \tag{5.25}$$

对于 $V$ 也进行相同的操作。KV 缓存的形状为 $(B, H, L_{max}, d_{head})$，其中 $L_{max}$ 是最大支持的上下文长度。

在实际实现中,KV 缓存更新通常写成简单的内存复制操作。但在 PagedAttention（vLLM）中使用块表（Block Table）实现虚拟内存映射，更新操作变为块表条目更新而非数据拷贝。这一点将在第 7 章和第 21 章深入讨论。

## 5.10 算子融合

算子融合（Operator Fusion）是将多个连续的小算子合并为一个综合算子的技术，其动机在于减少内存访问次数。

**典型的融合模式**

- **线性层 + 激活**: 将 MatMul 和后续的 ReLU/GELU 融合，避免将中间结果写回内存。
- **归一化 + 残差连接**: `RMSNorm(x) + residual` 是 Transformer decoder 中的标准模式。
- **注意力融合**: 将 $\mathrm{QK}^T$、softmax、PV 三个步骤融合为一个 FlashAttention 核函数。
- **量化-解量化融合**: 将 Dequantize + MatMul + Quantize 在同一核函数中完成。

**本章要点总结**

1. 矩阵乘法是推理引擎的性能核心。分块、打包、微内核三级优化是获得接近硬件峰值性能的关键。
2. 卷积可通过 im2col 转换为矩阵乘法，但在小核场景 Winograd 更具优势。
3. 注意力算子的内存瓶颈在于 $O(N^2)$ 的注意力矩阵，需使用分块/在线算法规避。
4. 逐元素和归约算子的优化核心是减少内存访问：融合和自动向量化。

5. 归一化算子（LayerNorm/RMSNorm）在现代模型中大量使用，其高效实现直接影响端到端延迟。

# 第六章　矩阵分解与转换

矩阵运算是神经网络推理的绝对核心。在 Transformer 模型中，超过 95% 的浮点运算量集中在矩阵乘法上。因此，理解矩阵运算的优化空间和变换技术对于构建高性能推理引擎至关重要。本章将从硬件性能模型出发，系统性地讨论矩阵分解、量化和分块策略。

## 6.1　Roofline 模型与算术强度

### 6.1.1　Roofline 模型定义

Roofline 模型是分析程序性能的基本工具。它将可达到的峰值性能与一道算法实现可获得的实际性能之间建立了联系。

**Roofline 模型**

Roofline 模型的纵轴为可达到的性能（FLOPS），横轴为算术强度（FLOP/Byte）：

$$\text{Attainable Performance} = \min \begin{pmatrix} \text{Peak Compute} \\ \text{Peak Bandwidth} \times \text{Arithmetic Intensity} \end{pmatrix} \tag{6.1}$$

其中：

- 算术强度（Arithmetic Intensity, AI）= 总 FLOP 数 / 总访存字节数。
- 峰值计算（Peak Compute）= 处理器在理想条件下的最大 FLOPS。
- 峰值带宽（Peak Bandwidth）= 处理器在理想条件下的最大内存传输速率（Bytes/sec）。

**RTX 4090 的 Roofline 示算**

RTX 4090 纸面规格：FP16 峰值 $\approx$ 330 TFLOPS（Tensor Core），HBM2e 带宽 $\approx$ 1008 GB/s。其 Roofline 转折点（ridge point）为：

$$\text{AI}_{ridge} = \frac{330 \times 10^{12}\ \text{FLOP/s}}{1008 \times 10^{9}\ \text{Byte/s}} \approx 327\ \text{FLOP/Byte} \tag{6.2}$$

意味着任何算术强度低于 327 FLOP/Byte 的算子都是内存受限的，高于此值才是计算受限的。

### 6.1.2 用 Roofline 分析矩阵乘法

对于矩阵乘法 $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$，其中 $\boldsymbol{A} \in \mathbb{R}^{M\times K}$, $\boldsymbol{B} \in \mathbb{R}^{K\times N}$：

**矩阵乘法的算术强度**

$$\text{FLOPs} = 2MNK \quad \text{(每次乘加算 2 个 FLOP)} \tag{6.3}$$

$$\text{Bytes} = (MK + KN + MN) \times \text{sizeof(element)} \tag{6.4}$$

$$\text{AI} = \frac{2MNK}{(MK + KN + MN) \times \text{sizeof(element)}} \tag{6.5}$$

当 $M, N \gg K$ 或 $M, K \gg N$ 等多种情况下，AI 的渐近行为不同。
对于较大的方阵（$M = N = K = n$）：

$$\text{AI}(n \times n \times n) = \frac{2n^3}{3n^2 \times 4} = \frac{n}{6} \quad \text{FLOP/Byte} \quad \text{(FP32)} \tag{6.6}$$

**操作类型的判定规则**

- **内存受限**: $\text{AI} < \text{AI}_{\text{ridge}}$。优化方向：减少内存访问（融合、量化），而不是优化计算。
- **计算受限**: $\text{AI} > \text{AI}_{\text{ridge}}$。优化方向：提高计算效率（向量化、Tensor Core、更好的并行性）。

**典型模式**: 小型批处理（b=1）的推理中，GEMV 操作（$M = 1$, $N$ 或 $K$ 任意）是极端内存受限的。大批量推理中，GEMM（$M, N, K$ 都较大）可能变成计算受限。

## 6.2 低秩分解与模型压缩

### 6.2.1 SVD 分解

**奇异值分解 SVD**

对于任意实矩阵 $\boldsymbol{W} \in \mathbb{R}^{m\times n}$（$m \geq n$），存在 SVD 分解：

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^T \tag{6.7}$$

其中 $\boldsymbol{U} \in \mathbb{R}^{m\times n}$ 和 $\boldsymbol{V} \in \mathbb{R}^{n\times n}$ 是正交矩阵（$\boldsymbol{U}^T\boldsymbol{U} = \boldsymbol{V}^T\boldsymbol{V} = \boldsymbol{I}$），$\boldsymbol{\Sigma} = \mathrm{diag}(\sigma_1, \sigma_2, \ldots, \sigma_n)$ 是对角矩阵，且 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n \geq 0$。

**截断 SVD 与低秩近似**

保留前 $r$ 个最大的奇异值，截断 SVD 给出 $\boldsymbol{W}$ 的最优秩-$r$ 近似（Eckart-Young-Mirsky 定理）：

$$\boldsymbol{W} \approx \boldsymbol{W}_r = \boldsymbol{U}_{[:,:r]} \cdot \mathrm{diag}(\sigma_1, \ldots, \sigma_r) \cdot \boldsymbol{V}^T_{[:,:r]} \tag{6.8}$$

这种近似在 Frobenius 范数意义上是最优的：$\|\boldsymbol{W} - \boldsymbol{W}_r\|_F^2 = \sum_{i=r+1}^{n} \sigma_i^2$。

### 6.2.2　二因子分解

**二因子分解**

将权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ 替换为两个更小矩阵的乘积：

$$\boldsymbol{W} \approx \boldsymbol{A} \cdot \boldsymbol{B}, \quad \boldsymbol{A} \in \mathbb{R}^{m\times r}, \boldsymbol{B} \in \mathbb{R}^{r\times n} \tag{6.9}$$

参数数量从 $m \times n$ 降至 $r \times (m + n)$。压缩比：

$$\text{Compression Ratio} = \frac{mn}{r(m+n)} \tag{6.10}$$

当 $r < \frac{mn}{m+n}$ 时，参数数量减少。如果 $m = n$，需要 $r < n/2$ 才能获得压缩。

**LLaMA-7B 中的二因子分解应用**

LLaMA-7B 中，Attention 层的 $Q, K, V$ 投影矩阵和 FFN 层的参数矩阵可以通过低秩分解来压缩。然而，实践表明，LLM 中的权重矩阵通常具有较高的有效秩（large effective rank），直接应用低秩分解会导致较大的精度损失。因此，低秩分解更多用于辅助微调（如 LoRA）而非直接的模型压缩。

### 6.2.3　在注意力和 FFN 中的应用

Transformer 模型中有两个主要的矩阵乘法密集区：

- **注意力投影**: $Q, K, V, O$ 四个线性投影。在 GQA 中，$K, V$ 的头数更少，其投影矩阵维度也更小，但 $Q$ 和 $O$ 仍然是全头数的。

- **FFN**: 两个全连接层，通常维度为 $d_{model} \times d_{ff}$ 和 $d_{ff} \times d_{model}$，其中 $d_{ff} \approx 4 \times d_{model}$。这两层占据了模型约 $\frac{2}{3}$ 的参数量。

低秩分解在 FFN 层上通常更有效，因为 FFN 权重矩阵的高维性质使其存在更多的秩冗余。但需要注意，随着模型规模的增大（如 LLaMA-70B）、训练数据的增多，模型的秩分布趋于平坦，低秩分解的收益降低。

## 6.3 QR 分解与 Cholesky 分解

### 6.3.1 QR 分解

**QR 分解**

对于一个 $m \times n$ 矩阵 $\boldsymbol{A}$（$m \geq n$），QR 分解给出：

$$\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{R} \tag{6.11}$$

其中 $\boldsymbol{Q}$ 是 $m \times m$ 正交矩阵（$\boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{I}$），$\boldsymbol{R}$ 是 $m \times n$ 上三角矩阵。QR 分解在求解最小二乘问题、特征值计算和某些量化方案中有重要应用。

在模型量化中，QR 分解可用于分析权重矩阵的正交性质。例如，基于最优脑手术（OBS）的量化方法（如 GPTQ）等效于在权重矩阵上迭代执行 QR 分解来消除量化误差的累积。

### 6.3.2 Cholesky 分解

**Cholesky 分解**

对于对称正定矩阵 $\boldsymbol{A} \in \mathbb{R}^{n \times n}$，存在唯一的下三角矩阵 $\boldsymbol{L}$ 使得：

$$\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^T \tag{6.12}$$

Cholesky 分解在整个推理过程中频繁出现，例如在计算协方差矩阵的逆、求解正规方程以及更广义的线性系统求解中。

在 GPTQ 等量化方法中，需要求解 Hessian 矩阵或其近似矩阵的逆。通过 Cholesky 分解可以将求解复杂度从 $O(n^3)$ 降低至更可管理的 $O(n^2)$（在增量更新时）。

# 6.4　量化与矩阵结构

## 6.4.1　仅权重量化 vs 权重 + 激活量化

**仅权重量化（WOQ）**

$W$ 被量化为低频宽格式（如 int4），但在矩阵乘法阶段解量化为 fp16/bfloat16。访存模式：

$$\boldsymbol{Y} = \text{dequant}(\boldsymbol{W}_{int4}) \cdot \boldsymbol{X}_{fp16} \tag{6.13}$$

此方法节省了权重加载的带宽，但计算仍在高精度下进行。对于内存受限的推理场景（小批量），这一策略非常有效。

**权重 + 激活量化：W+A**

$W$ 和 $X$ 都被量化。矩阵乘法直接以低精度整数运算：

$$\boldsymbol{Y} = \alpha \cdot (\boldsymbol{W}_{int8} \cdot \boldsymbol{X}_{int8}) \quad \text{（对称量化）} \tag{6.14}$$

这种方法同时节省了内存带宽和计算时间，但精度损失更大。需要仔细的校准来确保最终输出质量。

## 6.4.2　量化矩阵乘法的实现

**Int8 矩阵乘法**

**输入：** $\boldsymbol{W}_{int8} \in \mathbb{Z}^{M\times K}$, $s_w \in \mathbb{R}^{M\times 1}$（每行或每块的缩放因子）

**输入：** $\boldsymbol{X}_{int8} \in \mathbb{Z}^{K\times N}$, $s_x \in \mathbb{R}^{1\times N}$（缩放因子）

**输出：** $\boldsymbol{Y}_{fp32} \in \mathbb{R}^{M\times N}$

$\boldsymbol{Y}_{int32} \leftarrow \boldsymbol{W}_{int8} \cdot \boldsymbol{X}_{int8}$　▷ *在 int32 中累加，避免溢出*

$\boldsymbol{Y}_{fp32} \leftarrow \text{float}(\boldsymbol{Y}_{int32})$

**for** $i \leftarrow 0$ **to** $M-1$ **do**

　　**for** $j \leftarrow 0$ **to** $N-1$ **do**

　　　　$\boldsymbol{Y}_{fp32}[i,j] \leftarrow \boldsymbol{Y}_{fp32}[i,j] \times s_w[i] \times s_x[j]$

# 6.5 张量核心利用

## 6.5.1 NVIDIA 张量核心架构

**张量核心指令**

NVIDIA Tensor Cores 引入了一种硬件级别的矩阵乘加指令（`mma.sync`），能在单个周期内计算大小为 $M \times N \times K$ 块的部分积。以 A100 为例：

- FP16 输入 → FP16/FP32 累加: `mma.sync.aligned.m16n8k16`
- INT8 输入 → INT32 累加: `mma.sync.aligned.m16n8k32`
- TF32 输入 → FP32 累加: `mma.sync.aligned.m16n8k8`
- BF16 输入 → FP32 累加: `mma.sync.aligned.m16n8k16`

每个 SM（流式多处理器）有 4 个张量核心，可同时执行 4 条 `mma` 指令。

**张量核心的吞吐量优势**

在 H100 上，FP16 张量核心可提供约 990 TFLOPS 的理论峰值（带稀疏性）。相比之下，传统 CUDA Core 的 FP16 吞吐量仅为约 60 TFLOPS。差距约为 16×，这就是为什么在高性能推理中必须使用张量核心。

## 6.5.2 张量核心编程模型

使用 CUDA 中的 WMMA（Warp Matrix Multiply Accumulate）API 或直接编写 PTX 代码来使用张量核心。

**WMMA API 使用示例**

```
#include <cuda_fp16.h>
#include <mma.h>
using namespace nvcuda;

// 声明fragment
wmma::fragment<wmma::matrix_a, 16, 16, 16,
               half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16,
               half, wmma::col_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16,
               half> c_frag;
```

```
// 加载和计算
wmma::load_matrix_sync(a_frag, A_ptr, ldA);
wmma::load_matrix_sync(b_frag, B_ptr, ldB);
wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
wmma::store_matrix_sync(C_ptr, c_frag, ldC,
                        wmma::mem_row_major);
```

**Fragment 布局约束**

WMMA 的 fragment 类型依赖于矩阵在内存中的行/列主序布局。如果数据布局不匹配，需要先通过转置操作重新排列。这种布局要求是张量核心编程的一个主要工程难点，但 CUTLASS 等库已经抽象了这种复杂性。

## 6.6　混合精度矩阵乘法

**混合精度矩阵乘法**

输入为 FP16/BF16，但累加器使用 FP32 精度：

$$\boldsymbol{C}_{fp32} \leftarrow \boldsymbol{C}_{fp32} + \mathrm{fp32}\big(\boldsymbol{A}_{fp16} \times \boldsymbol{B}_{fp16}\big) \tag{6.15}$$

这利用了” 乘积在 FP32 中累加” 的事实：FP16 的乘法积必须转为 FP32 才能进行更高精度的累加，否则在大型内积（$K$ 很大）中会遭遇快速精度损失。

在典型的推理场景中，矩阵乘法的输入和输出都可以保持在 FP16 中，但在 kernel 内部始终用 FP32 累加。当 $K > 2048$ 时尤其重要，因为 FP16 的尾数仅为 10 位，而 FP32 的尾数为 23 位，提供了额外的 $\sim$13 位精度来积累中间和。

## 6.7　分级分块策略

### 6.7.1　分级分块

现代优化 GEMM 采用多级缓存层次的分块策略：

**分级分块框架**

| 缓存级别 | 典型大小 | 分块用途 |
|---|---|---|
| L3 (~8-32 MB) | 宏分块（$M_{macro} \times N_{macro}$） | 将全局的 $K$ 维度完整遍历限制在 L3 内 |
| L2 (~256KB-1MB) | 中分块（$M_{med} \times N_{med}$） | 在 L2 中完成子块乘法 |
| L1 (~32-128KB) | 细分块（$M_{l1} \times N_{l1}$） | 微内核的输入数据 |
| 寄存器 (~32-256 regs) | 微分块（$M_r \times N_r$） | 微内核的累加区域 |

**不同缓存的利用策略**

- **L3 缓存**: 主要由软件的线程间划分隐式利用，而非显式编程。足够大的 L3 可以减少重复从主存加载数据的次数。
- **L2 缓存**: 通过 OpenMP 的 `parallel for` 在不同 NUMA 节点上由不同的核使用。
- **L1 缓存**: 通过编译器内置函数 `__builtin_prefetch` 进行软件预取。
- **寄存器**: 通过手写汇编或内联函数进行精确控制。

### 6.7.2 自动参数调优

分块大小的最优选择取决于矩阵尺寸和硬件特性。手工调优费时费力，因此自动搜索成为了实际部署中的标准做法。

**网格搜索自动调优**

**输入:** 搜索空间 $\mathcal{S}$（各维度的块大小候选集）

**输入:** 目标矩阵尺寸 $M, N, K$

**输出:** 最优块大小组合 $(T_m^*, T_n^*, T_k^*)$

best_gflops $\leftarrow 0$
**for** $(T_m, T_n, T_k) \in \mathcal{S}$ **do**
    生成特化的矩阵乘法代码（带打包和微内核）
    运行基准测试，记录 GFLOPS
    **if** GFLOPS$(T_m, T_n, T_k) >$ best_gflops **then**
        best_gflops $\leftarrow$ GFLOPS
        $(T_m^*, T_n^*, T_k^*) \leftarrow (T_m, T_n, T_k)$
**return** $(T_m^*, T_n^*, T_k^*)$

**自动调优的搜索空间爆炸**

如果 $T_m, T_n, T_k$ 各有 10 个候选值（$T_m \in \{2, 4, 6, 8, 12, 16, 24, 32, 48, 64\}$），总的组合数为 $10^3 = 1000$ 种。对于每个尺寸动态（不同的 $M, N, K$），需要对 1000 种配置进行基准测试。在实际系统中，通常使用以下策略之一：

- **预计算查找表**: 在安装时对典型的 $(M, N, K)$ hash 表进行一轮离线搜索。
- **成本模型**: 训练一个机器学习模型来预测给定 $(M, N, K, T_m, T_n, T_k)$ 的 GFLOPS，避免实测。
- **分层搜索**: 先粗搜（大步长），再在最优区域细搜（小步长）。

## 6.8　矩阵变换汇总

表 6.1: 推理引擎中常用的矩阵变换技术总结

| 技术 | 数学形式 | 主要用途 | 复杂度 |
|---|---|---|---|
| 低秩分解 | $W \approx AB$ | 模型压缩、减少参数量 | $O(mnr)$（SVD 计算） |
| 量化 | $W_q = \text{round}(W/s)$ | 减少内存占用和带宽 | $O(mn)$ |
| 混合精度 | FP16 乘 + FP32 加 | 求精度速度平衡 | $O(mnk)$ |
| Tiling | 分块 GEMM | 提高缓存利用率 | 自动化 |
| Tensor Core | $M \times N \times K$ MMA | 专用加速器 | 硬件实现 |

**本章要点总结**

1. Roofline 模型提供了一种统一的框架来判断算子是计算受限还是内存受限，从而指导优化方向。
2. 低秩分解（SVD、二因子分解）可用于模型压缩，但现代 LLM 的有效秩较高，直接分解可能导致性能显著下降。
3. 张量核心的吞吐量远超标准 CUDA 核心（$10\times \sim 16\times$），但编程模型更复杂。
4. 分级分块和参数自动调优是达到高性能 GEMM 的关键工程实践。
5. 量化（仅权重或权重 + 激活）在很大程度上改变矩阵运算的访存和计算模式，在内存受限场景尤为有效。

# 第七章　内存管理

内存管理是推理引擎中最具系统性的挑战。一个错误的分配策略可能导致内存碎片化、频繁的分配/释放开销，甚至 OOM（内存溢出）。在 GPU 上，内存管理还涉及复杂的设备内存层次和有限的显存容量。本章将从理论到实践全面覆盖推理引擎中的内存管理技术。

## 7.1　内存层次结构概览

**CPU 端内存层次**

| 层次 | 延迟（周期） | 延迟（ns） | 典型容量 |
|---|---|---|---|
| 寄存器 | 1 | 0.3 | 几百字节 |
| L1 缓存 | 4-5 | 1-2 | 32 KB（数据） |
| L2 缓存 | 12-15 | 5-8 | 256 KB ~ 1 MB |
| L3 缓存 | 40-50+ | 15-20+ | 8 ~ 32 MB |
| 主存 RAM | 200-300+ | 80-120+ | 16 ~ 512 GB |
| SSD/磁盘 | $10^6$+ | $10^5$+ | TB 级 |

**GPU 端内存层次（以 NVIDIA A100 为例）**

| 层次 | 延迟（周期） | 大小/SM | 带宽 |
|---|---|---|---|
| 寄存器 | 1 | 256 KB | ~8 TB/s |
| 共享内存 (SRAM) | ~20 | 164 KB (可配) | ~128 TB/s |
| L1 缓存 | ~30 | 同共享内存 | ~128 TB/s |
| L2 缓存 | ~200 | 40 MB (全局) | ~4 TB/s |
| HBM2e | ~500-600+ | 40/80 GB | 1.5 ~ 2.0 TB/s |

**” 内存墙” 问题**

现代处理器的计算能力每代增长 (1.5 ~ 2)×，而内存带宽增长仅 (1.1 ~ 1.2)×。这一不对称增长导致处理器在大量操作上闲置等待数据——这就是内存墙（Memory Wall）。对于推理引擎，这意味着我们必须将尽可能多的数据保持在更接近计算单元的位置，

并在调度中考虑数据本地性。

## 7.2 内存分配策略

### 7.2.1 竞技场分配器

竞技场分配器（Arena/Bump Allocator）是最简单的分配器：一个大的预分配内存块、一个偏移指针。

**竞技场分配器**

**输入：** 预分配内存池 `pool`，大小 `pool_size`

```
1: offset ← 0
2: function ALLOC(size)
3:     if offset + size > pool_size then
4:         return nullptr                          ▷ 空间不足
5:     ptr ← pool + offset
6:     offset ← offset + size
7:     return ptr
8: function RESET
9:     offset ← 0                                  ▷ 一次性重置整个区域
```

**竞技场分配器的优缺点**

优点：极快的分配速度（仅指针自增），极高的内存利用率（序列分配时无碎片）。缺点：无法释放单个对象——只能整体重置。这意味着竞技场分配器最适合**推理的整个生命周期**：在初始化阶段分配所有需要的张量，推理阶段仅使用（不分配不释放），会话结束时一次性重置。

### 7.2.2 池分配器

**池分配器**

池分配器（Pool Allocator）管理一群固定大小的内存块。通常实现为空闲链表：

```
struct Block { Block* next; }; // 当空闲时
Block* free_list;              // 链表头

void* allocate() {
    Block* block = free_list;
    free_list = free_list->next;
```

```
    return block;
}
void deallocate(void* p) {
    Block* block = (Block*)p;
    block->next = free_list;
    free_list = block;
}
```

这种方式对于频繁分配和释放的小张量（例如每次解码迭代中的中间张量）非常高效。

### 7.2.3　内存重用与生命周期分析

在计算图中，不同张量具有不同的生命周期。如果两个张量的生命周期不重叠，它们可以共享同一块内存。

**张量生命周期分析**

对于计算图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$，张量 $t$ 的生命周期定义为从产生它的节点到最后一个使用它的节点之间的活跃区间：

$$\text{lifetime}(t) = [\text{producer}(t), \text{last_consumer}(t)] \tag{7.1}$$

两个张量 $t_1, t_2$ 可以共享内存当且仅当它们生命周期不重叠：

$$\text{lifetime}(t_1) \cap \text{lifetime}(t_2) = \emptyset \tag{7.2}$$

这一问题数学上等价于图着色问题的变体：每个张量是图中的节点，若两个张量生命周期重叠则存在边。求最少的”颜色”（内存块）数即求图的色谱数——这个问题是 NP 难的。实践中使用贪婪算法。

**基于大小排序的贪婪内存分配**

**输入：** 张量列表 $\mathcal{T}$，按字节大小降序排列

**输入：** 内存池，容量 $C$

1: 初始化空闲区间列表 $\mathcal{F} \leftarrow [(0, C)]$
2: **for** tensor $t \in \mathcal{T}$（按内存大小降序） **do**
3: 　$l \leftarrow t.\text{lifetime_start},\ r \leftarrow t.\text{lifetime_end}$
4: 　在 $\mathcal{F}$ 中查找第一个大小 $\geq \text{sizeof}(t)$ 的区间 $(a, b)$，且在该区间上 $t$ 不与任何已分配张量生命周期重叠。
5: 　**if** 找到 **then**

```
        分配地址 a 给 t
        从 F 中移除分配的区间或拆分余下的
    else
        分配失败：内存不足
```

## 7.3 设备内存管理

### 7.3.1 CUDA 内存类型

**CUDA 内存类型详解**

- **全局内存（Global Memory / HBM）**：GPU 外部的 DRAM。容量最大（80GB 在 A100），但延迟最高。所有线程块都可以访问。
- **共享内存（Shared Memory / SRAM）**：SM 上的片上内存。容量小（164KB/SM），延迟极低（约 20 周期）。同一线程块内的线程共享。这是 FlashAttention 等高性能 kernel 的核心内存层级。
- **纹理内存（Texture Memory）**：具有空间局部性优化的只读全局内存区域，带硬件级别缓存。
- **常量内存（Constant Memory）**：64KB 的只读内存，对所有线程广播访问有优化。

### 7.3.2 GPU 内存分配优化

**cudaMalloc 的开销**

`cudaMalloc` 和 `cudaFree` 是昂贵的同步操作：每次调用需要与进程 GPU 内核驱动程序通信，引入显著延迟。在 CUTLASS 库的微基准测试中，一次 `cudaMalloc` 可花费数毫秒。因此在高频分配/释放的模式下（如每次解码迭代都分配 KV 缓存空间），必须避免频繁的 `cudaMalloc` 调用。

解决方案是 **GPU 缓存分配器**（Caching Allocator）：在宿主代码中维护一个大池，在 `cudaMalloc` 时先从池中分配，在 `cudaFree` 时放回池中而不是还给操作系统。PyTorch 等框架默认使用这种分配器。

### 7.3.3　流序内存分配

**流序分配（Stream-Ordered Allocation）**

CUDA 11.2 开始支持的内存分配新语义：内存分配不再与默认流同步，而是相对于特定的 CUDA 流有序。这意味着：

1. 在流 $S$ 上分配的内存可以被同一流上的后续操作安全使用。
2. 其他流上的操作不能访问此内存，直到通过事件同步。
3. 释放操作也是流序的，在释放前必须确保同一流上所有使用方已完成。

流序分配避免了隐式的设备同步，极大地提升了多流并发的效率。

### 7.3.4　统一内存

统一内存（Unified Memory，即 Managed Memory）允许从 CPU 和 GPU 代码中透明地访问同一指针。底层通过**页面迁移**（page migration）实现：当 CPU 或 GPU 访问不在本地物理页面中的数据时，会触发页面故障将数据迁移过去。

**统一内存的陷阱**

对于推理引擎，统一内存存在两个主要问题：

1. **隐式迁移开销**: 页面迁移发生在访问时，很难预测。一个意外的页面故障可能增加数十微秒甚至更长的延迟。
2. **并发迁移冲突**: 多个并发流或 CPU 线程可能引发大量页面迁移，导致总线（PCIe/NVLink）饱和。

对于延迟敏感的服务型推理，统一内存目前很少被采用。对于吞吐敏感的离线推理，它可能用于模型的懒加载和卸载。

## 7.4　推理内存规划

### 7.4.1　静态内存规划

静态规划是最简单也最确定性的方法：在模型加载后、推理开始前，一次性计算整个执行过程所需的峰值内存，并一次性分配。

**静态规划的步骤**

1. **遍历计算图**: 分析每个算子的输入、输出和临时张量尺寸。
2. **计算峰值内存**. 通过遍历计算图的拓扑序、模拟张量分配与释放来计算。
3. **预分配**. 使用竞技场分配器分配峰值内存量。
4. **地址偏移分配**. 为每个张量分配一个在此池中的偏移地址。

**LLaMA-7B 的静态内存规划示例**

以 LLaMA-7B FP16 推理为例：

- 模型权重: $7\text{B} \times 2$ bytes $= 14$ GB
- 中间激活（单层）用于 $B = 1$: 约 $4 \times B \times d_{model} \times \text{hidden_size} = 4 \times 1 \times 4096 \times 4096 \times 2 \approx 128$ MB
- KV 缓存: $2 \times L \times H \times L_{max} \times d_{head} \times 2 = 2 \times 32 \times 32 \times 4096 \times 128 \times 2 \approx 2$ GB ($L_{max} = 4096$)
- 框架开销: 约 $100 \sim 200$ MB

总计约 16.5 GB。将峰值规划为 18 GB 即可在 24GB GPU（如 RTX 4090）中安全运行。

### 7.4.2 贪婪大小排序分配

**贪婪大小排序分配 (Greedy by-size)**

**输入:** 张量列表 $\mathcal{T}$，按内存占用降序排列

**输入:** 内存池基地址 `base`，总大小 `total_size`

$\mathcal{F} \leftarrow \{(0, \texttt{total_size})\}$ ▷ 空闲区间列表

$\mathcal{A} \leftarrow \emptyset$ ▷ 已分配区间列表，带生命周期

**for** tensor $t \in \mathcal{T}$ **do**

  将 $\mathcal{A}$ 中生命周期已结束的区间放回 $\mathcal{F}$

  addr $\leftarrow$ 在$\mathcal{F}$ 中查找第一个 $\geq$ sizeof($t$)的区间

  **if** addr $\neq$ `null` **then**

    将 $t$ 的区间加入 $\mathcal{A}$，从 $\mathcal{F}$ 中移除或拆分

  **else**

    **return** 内存不足

**return** $\mathcal{A}$ ▷ 分配计划

先分配大张量可以有效减少碎片化，因为小张量更容易填充剩余的间隙。

## 7.5　KV 缓存内存管理

### 7.5.1　预分配与动态增长

KV 缓存的需求取决于运行时的上下文长度（包括输入 prompt 长度和生成的 token 数）。两种基本策略：

**KV 缓存分配策略**

- **全量预分配**: 在会话创建时就分配 $2 \times L \times H \times L_{max} \times d_{head}$ 大小的缓存。$L_{max}$ 为模型支持的最大上下文长度。确定性高，但浪费大——实际使用中可能远小于 $L_{max}$。
- **按需增长**: 初始分配小块（如 128 个 token），当缓存满时扩大。类似 `vector` 的动态扩容，但需处理 GPU 端的重分配与数据迁移。

### 7.5.2　PagedAttention：虚拟内存用于 KV 缓存

PagedAttention 是 vLLM 框架的核心贡献，它受到 OS 中的分页虚拟内存启发。

**PagedAttention 核心概念**

- KV 缓存被划分为固定大小的**块**（block），每块包含固定数量 $B$ 个 token 的 KV 对。
- 一个**块表**（Block Table）维护逻辑位置到物理块 ID 的映射。
- 物理块是非连续分配的，消除了静态分配中产生的内部碎片。
- 逻辑块表条目在被使用时标记为**引用计数** $> 0$，当生成任务完成时递减引用计数。引用计数为 0 的物理块被释放回空闲池。

**PagedAttention 的数据结构**

```
struct PhysicalBlock {
    float16_t k_cache[BLOCK_SIZE][NUM_HEADS][HEAD_DIM];
    float16_t v_cache[BLOCK_SIZE][NUM_HEADS][HEAD_DIM];
};

struct BlockTable {
    int32_t physical_block_ids[MAX_LOGICAL_BLOCKS];
    int32_t num_logical_blocks;
};
```

```
struct KVCacheManager {
    std::vector<PhysicalBlock> physical_blocks;
    std::vector<int32_t> free_list;
    int32_t block_size;  // 如 16 tokens per block
};
```

### 7.5.3 碎片化与驱逐策略

在多请求并发处理中（一个服务同时处理多个对话），不同请求产生不同长度的序列，物理块的分配和释放会产生外部碎片。

**驱逐（Eviction）策略**

当空闲块不足时，需要驱逐某些正在处理的请求。常用策略：

- **FIFO**: 先到先驱逐。简单但可能驱逐到关键请求。
- **LRU（Least Recently Used）**: 驱逐最长时间未使用的序列。对于多轮对话来说，保留最近活跃对话的 KV 缓存。
- **动态优先级**: 高优先级请求永不驱逐，标准优先级请求在内存紧张时被换出（swapped to CPU）。

## 7.6 权重内存管理

### 7.6.1 内存映射加载

**mmap 加载模型权重**

使用操作系统的 `mmap()`（或 Windows 上的 `MapViewOfFile()`）系统调用将模型文件映射到进程虚拟地址空间。这样：

1. **零拷贝**: 初始不读入任何页面。内核在进程访问某一页（引发页面故障）时才将对应磁盘块加载到内存。
2. **页面缓存复用**: 由 OS 维护的页面缓存可以使同一文件的多个 mmap 映射共享物理页面。这在多进程场景（如负载均衡的多个推理服务器进程加载相同模型）中极大地节省了内存。
3. **懒加载**: 在进程不访问某些层时，对应的磁盘块永远不会被加载，节省了内存。

**Linux 上 mmap 的局限性**

在 Linux 上，mmap 文件的写回机制可能导致回写（writeback）延迟和页面刷新风暴（page flush storms）。不过对于只读模型权重文件，不存在写回问题，mmap 是在大型模型上实现共享和懒加载的标准方法。

### 7.6.2　NUMA 意识

**NUMA（Non-Uniform Memory Access）**

在多插槽服务器上，内存被划分到不同的 NUMA 节点中。访问本地节点的时间显著少于远程节点（差异可达 $1.5 \sim 2\times$）。推理引擎应该：

- 将权重张量分配到负载 balanced 的 NUMA 节点
- 将处理该张量的线程绑定到同一 NUMA 节点的核心上（CPU 亲和性）
- 使用 `numactl` 或 `libnuma` 进行显式绑定

## 7.7　内存带宽优化

### 7.7.1　合并访存

**GPU 上的合并内存访问**

在 CUDA 中，同一个 warp 内的 32 个线程同时访问全局内存。如果它们访问的地址连续，GPU 硬件可以将它们合并为一次或几次内存事务（memory transaction）。如果地址不规则，则可能需要多达 32 次独立事务，带宽利用率极低。

**行优先 vs 列优先访问**

对于行优先存储的矩阵：

- **行访问**（线程 $t$ 访问第 $t$ 列）：地址连续 $\rightarrow$ 合并访问
- **列访问**（线程 $t$ 访问第 $t$ 行）：地址跨步 $\rightarrow$ 非合并访问

当内部循环必须列访问时（如矩阵转置或按列计算），需要先进行数据重排或使用共享内存缓冲来聚合访问。

### 7.7.2 双缓冲

**双缓冲**

使用两个内存区域交替作为源和目的。当内核 A 在缓冲区 1 上工作时，DMA 引擎（如 GPU 的复制引擎）将下一批数据加载到缓冲区 0 中。一旦 A 完成、数据也已加载，进行切换。

$$T_{total} = \max(T_{compute}, T_{transfer}) + T_{kern_switch} \tag{7.3}$$

理想情况下，双缓冲可以实现计算与传输的完全重叠。

### 7.7.3 张量引用计数与垃圾回收

**张量引用计数**

每个张量维护一个引用计数。当创建时，引用计数初始化为 1。当一个新张量引用它时，递增；当一个张量被释放或不需再读时，递减。当引用计数降为 0 时，释放支撑内存。

$$\text{refcount} \leftarrow \text{refcount} + 1 \quad \text{(ref)}; \qquad \text{refcount} \leftarrow \text{refcount} - 1 \quad \text{(unref)} \tag{7.4}$$

然而在具有竞技场分配的推理引擎中，显式引用计数和释放通常是不必要的：内存池在推理结束时整体重置。

**本章要点总结**

1. CPU 和 GPU 都有深刻的内存层次，离计算单元越近的内存延迟越低、容量越小。推理引擎需要显式地管理这些层次。
2. 竞技场分配器最适合推理的确定论内存模式；池分配器适合频繁和不可预测的分配。
3. 张量生命周期分析可以将内存需求降约 50% 甚至更多（重用非重叠张量的内存）。
4. PagedAttention 将 KV 缓存的碎片问题转化为虚拟内存管理问题，是现代服务系统的基础。
5. 合并访存和双缓冲是在 GPU 上达到高内存带宽利用率的关键工程实践。

# 第八章　任务调度

推理引擎需要在异构硬件上高效地调度大量计算任务。与训练不同，推理的延迟要求更为严格（用户期望在毫秒级内得到响应），同时系统的资源利用必须保证高吞吐。本章系统性地覆盖线程池设计、计算图并行执行、GPU 流管理、批处理调度和流水线并行等主题。

## 8.1　线程池设计

### 8.1.1　基础架构

**推理引擎线程池**

一个典型的高性能线程池由以下组件构成：

- $N$ 个固定工作线程（worker threads），$N$ 通常设置为 `std::thread::hardware_concurrency()` 或较之小 1 以避免与系统争夺资源。
- 每个工作线程拥有一个**工作队列**（work queue），通常是多生产者单消费者（MPSC）的无锁队列。
- 一个**任务对象**：封装了一个可调用对象（函数指针、lambda 或 `std::function`）及其参数。

**工作线程主循环**

**输入：** 工作队列 $Q$，全局线程池状态

```
while 未收到停止信号 do
    task ← Q.try_pop()
    if task ≠ null then
        task.execute()
    else
        尝试从其他线程的工作队列窃取任务
        if stolen_task ≠ null then
            stolen_task.execute()
```

```
9:        else
10:           std::this_thread::yield()                    ▷ 或条件变量等待
```

### 8.1.2 工作窃取

**工作窃取（Work Stealing）**

工作窃取是解决负载不均衡的核心机制。当一个线程完成自己的工作队列中的所有任务后，它从其他更忙的线程队列的**尾部**窃取任务。这种"LIFO（被窃方）+ FIFO（窃取方）"策略使得：

- 每个线程主要处理自己的队列和窃取来的大块任务
- 被窃取的线程因为从自己队列的尾部取任务，保持了 LIFO 顺序也即良好的缓存局部性

**Chase-Lev 双端队列**

经典的无锁工作窃取双端队列实现：

- 拥有者线程从**底部**（bottom）推入和弹出任务（LIFO）。
- 窃取线程从**顶部**（top）弹出任务（FIFO）。
- top 和 bottom 指针由原子变量维护，确保在无锁条件下正确同步。

在大量推理任务需要动态分配的情况下，Chase-Lev 双端队列是实现高效工作窃取的底层数据结构。

### 8.1.3 自旋等待 vs 阻塞

**两种等待策略的权衡**

- **自旋等待（Spinning）**：工作线程在发现无任务时短暂自旋（如 `PAUSE` 指令 100 次），若仍无工作则休眠。自旋消耗 CPU 但避免了上下文切换开销（$\sim 5-20\mu s$）。适合低延迟推理场景。
- **阻塞等待（Blocking）**：工作线程立即通过条件变量进入睡眠状态。无 CPU 浪费，但有唤醒延迟。适合大批量处理或空闲时期。

**混合策略**（Adaptive Spinning）：根据历史空闲时长动态选择自旋和休眠。

### 8.1.4　CPU 亲和性

**CPU 亲和性设置**

将工作线程**固定**（pin）到特定 CPU 核心可以避免 OS 调度器的重新调度带来的缓存失效和 NUMA 远程访问。工业级推理引擎（如 TensorRT、OpenVINO）都重度使用 CPU 亲和性。

```
cpu_set_t cpuset;
CPU_ZERO(&cpuset);
CPU_SET(core_id, &cpuset);
pthread_setaffinity_np(thread.native_handle(),
                       sizeof(cpuset), &cpuset);
```

此外，当启用超线程（Hyper-Threading/SMT）时，应将工作线程分配到物理核心上，确保每个核心仅运行一个推理线程。

## 8.2　计算图的并行执行

### 8.2.1　拓扑排序与就绪队列

**就绪队列机制**

给定一个计算图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$，我们需要找到一个执行计划 $\sigma = (v_{i_1}, v_{i_2}, \ldots, v_{i_{|\mathcal{V}|}})$。$\sigma$ 是有效的当且仅当对于每一条边 $(u, v) \in \mathcal{E}$，$u$ 在 $\sigma$ 中出现在 $v$ 之前（拓扑序）。
并行执行时，$\sigma$ 不是顺序序列，而是一系列**批**（wavefront）。在每个 wavefront 的所有节点都是在当前时刻就绪的。

**就绪队列的并行调度**

**输入：** 计算图 $\mathcal{G}$，线程池 $P$

初始化就绪队列 $R \leftarrow \{v \in \mathcal{V} \mid \text{indegree}(v) = 0\}$
初始化依赖计数器数组 $C[v] \leftarrow \text{indegree}(v)$
**while** $R \neq \emptyset$ **do**
    对 $R$ 中的每个节点 $v$，将 execute($v$) 提交到线程池 $P$
    等待 $P$ 中的所有提交完成　▷ *wavefront barrier*
    **for** 每个完成的 $v$ **do**
        **for** 每个后继 $w$ of $v$ **do**
            $C[w] \leftarrow C[w] - 1$
            **if** $C[w] = 0$ **then**
                $R \leftarrow R \cup \{w\}$

### 8.2.2 依赖关系类型

**两类依赖**

- **数据依赖**（Data Dependency）：算子 $B$ 需要算子 $A$ 的输出张量作为输入。这是由计算图结构确定的。
- **控制依赖**（Control Dependency）：即使两个算子之间没有数据流，但为了正确性必须保证执行顺序。例如，两个算子使用同一块内存作工作缓冲区，必须先完成一个再执行另一个。

推理引擎在构建执行计划时，需要同时考虑这两种依赖。

## 8.3 GPU 流管理

### 8.3.1 CUDA 流

**CUDA 流**

CUDA 流是一个命令队列——在流上的操作按顺序执行，不同流上的操作可以并发执行。在推理引擎中：

- **计算流**: 承载所有核心 kernel（矩阵乘法、注意力、激活等）。
- **复制流**: 承载主机到设备（H2D）和设备到主机（D2H）的数据传输。
- **流同步**: 通过 CUDA 事件（`cudaEvent`）在流之间建立依赖关系，例如，在复制流完成内存传输后，在计算流上的 Kernel 才可以开始。

### 8.3.2 计算与传输重叠

**计算和数据传输重叠示例**

```
// 复制流: 将输入数据从CPU传输到GPU
for (int i = 0; i < num_layers; i++) {
    cudaMemcpyAsync(gpu_input[i], cpu_input[i],
                    size, cudaMemcpyHostToDevice, copy_stream);
    cudaEventRecord(copy_done[i], copy_stream);
}

// 计算流: 等待数据完成，然后计算
for (int i = 0; i < num_layers; i++) {
    cudaStreamWaitEvent(compute_stream, copy_done[i], 0);
```

```
    launch_kernel<<<grid, block, 0, compute_stream>>>(...);
}
```

在复制流忙于传输第 $i+1$ 层数据时，计算流已经在处理第 $i$ 层的 kernel。当计算多层时，这可以隐藏大部分数据传输延迟。

### 8.3.3　CUDA 图

**CUDA 图**

CUDA Graph 允许将一个包含多个 kernel 和操作（包括流同步）的有向无环图捕获为单一的图对象，然后通过一次调用来重放整个图。这消除了一次 kernel 启动和流同步的 CPU 开销（约 $3-7\mu s$ 每次）。
特别是在推理中，计算图在模型加载后是固定的——只有输入数据和 KV 缓存的状态变化。CUDA Graph 在 decode 阶段（重复生成 token）可以显著降低延迟，因为它将每次迭代的 kernel 启动开销从 $O(N_k)$ 降到 $O(1)$。

## 8.4　批处理调度

### 8.4.1　静态批处理

**静态批处理**

传统批处理策略:等待一段时间收集用户请求,将收集到的所有请求合并为一个 batch，送入模型进行推理。此方案在两个冲突的目标之间存在权衡：

- **最大延迟**: 等待时间 + 推理时间
- **最大批大小**: 更大的批量提升 GPU 使用率，但增加了等待延迟

最大化吞吐目标是无限制地将所有可用请求编批；但延迟目标设置了上限（通常是几秒内）。

### 8.4.2　连续批处理

**连续/飞行批处理（Continuous/In-flight Batching）**

与静态批处理不同，连续的批处理在每次解码 iteration 可以添加和移除请求动态地。vLLM 是广泛应用这一策略的框架：

1. 当用户发出提示词时，请求加入运行中的 batch
2. 每生成一个 token 后，如果请求已完成（EOS token、达到 max length 等），请

求从 batch 中移除

3. 新请求可以随时加入（前提是有可用的内存和 KV 缓存空间）

这消除了” 等待 batch 填充” 的延迟，同时保留了调度的灵活性。

### 8.4.3 优先级调度

**优先级机制**

服务层面的推理调度器可以支持：

- **高优先级队列**: 来自 VIP 用户的请求或实时交互请求，跳过排队直接分配到解码槽位。
- **最长等待时间（Age-Based）**: 推进已等待时间最长的请求，避免请求饿死。
- **最短作业优先**: 优先完成短 prompt/短生成的任务，提高系统的整体吞吐。

## 8.5 流水线并行调度

当模型过大无法放入单个 GPU 设备时，可以将模型的不同层分配到不同设备上（流水线并行）。调度问题演化为如何编排微批次（micro-batch）的前向传播和后向传播。

虽然本书主要讨论推理（无反向传播），但理解流水线调度对推理的模型并行部署有帮助。

**推理中的流水线调度**

在纯推理（无反向传播）中，只需调度前向传播。对于在 $P$ 个设备上划分的模型：

1. 第一个设备处理一批 token 输入（embedding + 前几层），然后将其中间激活发送给第二个设备
2. 第二个设备继续处理，以此类推
3. 同时，第一个设备可以开始处理下一批 token，形成流水线

这称为 **1F（One Forward）**调度。与训练中的 1F1B 不同，没有反向传播步骤。

**推理流水线调度**

**输入:** $P$ 个设备，$S$ 个微批次，前向函数 $f_i$ 在第 $i$ 个设备上

1: **for** $s \leftarrow 0$ **to** $S-1$ **do** ▷ 并行启动所有微批次
2: 　在设备 0 上启动 $f_0(\text{microbatch}_s)$

3: **for** $i \leftarrow 0$ **to** $P-2$ **do**
4: 　**for** $s \leftarrow 0$ **to** $S-1$ **do**
5: 　　等待设备 $i$ 完成 $f_i(\text{microbatch}_s)$
6: 　　将结果传输到设备 $i+1$
7: 　　在设备 $i+1$ 上启动 $f_{i+1}(\text{microbatch}_s)$

### 8.5.1　气泡分析

**流水线气泡**

流水线气泡是指流水线中的某些设备处于空闲状态的时期：

$$\text{Bubble Fraction} = \frac{P-1}{S+P-1} \tag{8.1}$$

当微批次数量 $S$ 很大时，气泡比例接近于 0。但在推理场景中，$S$ 通常很小（prompt 的 batch size= 1 在交互式对话中），气泡是一个重要考虑因素。

## 8.6　抢占与状态交换

**推理抢占**

当 GPU 内存满（例如并发请求的 KV 缓存总和超过可用 GPU 内存）时，需要将某些运行中的请求抢占（evict）。状态可以：

- **交换（Swapping）**：将 KV 缓存放回 CPU 内存，等待内存空闲时重新分配并恢复。
- **重计算（Recomputation）**：丢弃 KV 缓存，当恢复该请求时从最开始的 prompt 和所有历史 token 重新计算。这避免了额外的内存拷贝，但增加了重计算的延时。

在 vLLM 中，基于 PagedAttention 的块管理允许在会话级别进行非常细粒度的抢占。

## 8.7　解码阶段调度

解码（decode）阶段是自回归推理的核心循环。每个迭代中，模型接收一个新 token 的嵌入输入，处理所有 Transformer 层，并将结果投影到词表 logits 上。

### 8.7.1 解码循环的调度挑战

**解码循环的开销来源**

- **Kernel 启动延迟**: 在 GPU 上，每个解码迭代启动的大量 kernel。如果模型有 $L$ 层，每层有 5~10 个 kernel，总共达到 $5L \sim 10L$ 个 kernel 启动，每个 $\sim 3-7\mu s$。
- **通信开销**: 若模型跨多个 GPU 分布，每个解码迭代都要进行跨 GPU 的数据传输。
- **IO 等待**: 在解码循环中，最后一步是 logits 的反量化/处理，这涉及相对较小的数据传输，但对延迟有直接贡献。

**解决方案**: 使用 CUDA Graph 封装整个解码循环，将 $O(L)$ 的启动开销减至 $O(1)$。

### 8.7.2 连续批处理在解码时的表现

连续批处理在解码阶段展现其真正的威力：

**解码阶段连续批处理示例**

在 $t = 0$ 时收到 2 个请求。$t = (0, 1, 2)$ 表明生成阶段：

- 第 1 步：2 个请求都在运行（batch size=2），各生成 1 个 token
- 第 2 步：请求 1 生成了 EOS（完成），请求 2 继续运行（batch size=1）
- 此时一个新请求到达。如果可用内存允许，新请求直接加入（batch size=2）

整个过程没有” 等待 batch 填充” 的延迟，旧请求立即继续。

## 8.8 负载均衡

**变长序列的负载均衡**

不同的用户请求可能具有非常不同的 prompt 长度和生成长度。在处理这些请求时，均衡不同设备或不同工作线程的负载是难点：

- **Token 级并行**: 不将整个请求分配给单个 worker，而是以 token 为粒度在 workers 之间分配工作。
- **动态拆分**: 当一个请求的 prompt 超过预设长度（如 1024 tokens），将其拆分为多个子任务在内核级别处理。
- **预分块**: 在 KV 缓存预分配空间时，根据 prompt 长度进行初始容量调整。

**本章要点总结**

1. 线程池和工作窃取是现代推理引擎的标准并发范型。CPU 亲和性和自旋等待是低延迟的关键。
2. 计算图的并行执行围绕就绪队列展开——每个 wavefront 中的所有就绪节点可以并行调度。
3. GPU 流管理和 CUDA Graph 极大地减少了 kernel 启动和同步的 CPU 开销。
4. 连续批处理消除了传统静态批处理的等待延迟，同时保持了吞吐。
5. 调度需求因推理的阶段而异：prompt 阶段是计算密集型，decode 阶段是内存密集型，调度器应差异化对待。

# 第九章　模型加载与运行流程

前面章节已经分别讨论了算子系统、矩阵运算、内存管理和任务调度。本章将这些部分整合成一个完整的端到端流程：从磁盘上的模型文件到回答用户请求。我们将覆盖模型加载流水线、权重加载策略、会话管理、推理执行循环、批处理执行以及异常处理和性能分析。

## 9.1　端到端模型加载流水线

一个完整的模型加载流程通常包括以下七个步骤：

**模型加载七步流水线**

1. **检测模型格式**: 自动从文件扩展名和魔数字节（magic bytes）推断模型格式。
2. **解析模型配置**: 读取模型类型、各维度（$d_{model}, N_{heads}, L_{layers}$ 等）、注意力类型、归一化类型等。
3. **构建计算图骨架**: 基于架构配置，构建不包含权重的计算图结构体。
4. **加载权重**: 将训练好的权重矩阵/向量从文件载入内存。
5. **应用图优化**: 融合、常量折叠、消除死代码等。
6. **内存规划与分配**: 计算峰值内存需求，预分配池。
7. **创建执行上下文/会话**: 初始化 KV 缓存、输入输出缓冲区等。

### 9.1.1　第 1 步：检测模型格式

**格式自动检测**

推理引擎通常支持多种模型格式：

- **PyTorch (.pt, .pth, .bin)**: Python pickle + zip 归档。可通过 `torch.load()` 或自定义 zip 解包器解析。
- **SafeTensors (.safetensors)**: 安全的零拷贝格式。开头为 JSON 头（描述 tensor

元信息），后面是连续的原始二进制数据。

- **GGUF (.gguf)**: llama.cpp 使用的格式。二进制格式，包含自描述的键-值元数据和连续的张量数据块。
- **ONNX (.onnx)**: protobuf 序列化的模型图 + 初始器（tensor 值）。
- **TensorFlow (.pb)**: protobuf 中的冻结图。

检测逻辑可以通过读取前几个字节的 magic 值或文件签名来实现。

**SafeTensors 格式检测**

SafeTensors 文件的前 8 字节固定为 JSON 头长度的 u64 小端整数。读取后即可判断：

```
uint64_t header_len;
fread(&header_len, sizeof(uint64_t), 1, f);
// header_len 应小于文件大小
char* json_buf = malloc(header_len);
fread(json_buf, 1, header_len, f);
// 解析JSON，获得所有tensor名称、dtype、shape和偏移
```

### 9.1.2 第 2 步：解析模型配置

**典型模型配置项**

Transformer 系列模型的配置通常包含以下字段：

| 参数 | 符号 | 说明 |
|---|---|---|
| `hidden_size` | $d_{model}$ | 隐藏层维度，如 4096 |
| `num_attention_heads` | $H$ | 注意力头数，如 32 |
| `num_key_value_heads` | $H_{kv}$ | KV 头数（GQA），如 8 |
| `num_hidden_layers` | $L$ | Transformer 层数，如 32 |
| `intermediate_size` | $d_{ff}$ | FFN 中间层维度，如 11008 |
| `max_position_embeddings` | $L_{max}$ | 最大位置编码，如 4096 |
| `vocab_size` | $V$ | 词表大小，如 32000 |
| `rms_norm_eps` | $\epsilon$ | RMSNorm 稳定项 |
| `rope_theta` | $\theta_{base}$ | RoPE 基频，通常 10000 |
| `hidden_act` | - | 激活函数类型：silu, gelu 等 |

配置可以来自文件中的 `config.json`、GGUF 中的元数据键值对，或 ONNX 中的 `metadata_props`。

### 9.1.3　第 3 步：构建计算图骨架

根据配置参数，推理引擎构建层和算子的图结构：

**构建 Transformer 计算图骨架**

**输入：** 模型配置 `config`
**输出：** 计算图 $\mathcal{G}$

$\mathcal{G}$.add_node("Embedding", config.`vocab_size`, config.`hidden_size`)
**for** $l \leftarrow 0$ **to** config.`num_layers` $- 1$ **do**
    $\mathcal{G}$.add_node("RMSNorm", prefix $= l$)
    $\mathcal{G}$.add_node("QKV Projection", prefix $= l$)
    $\mathcal{G}$.add_node("RoPE", prefix $= l$)
    $\mathcal{G}$.add_node("Attention", prefix $= l$)
    $\mathcal{G}$.add_node("O Projection", prefix $= l$)
    $\mathcal{G}$.add_node("Residual", prefix $= l$)
    $\mathcal{G}$.add_node("RMSNorm", prefix $= l$)
    $\mathcal{G}$.add_node("FFN Gate/Up", prefix $= l$)
    $\mathcal{G}$.add_node("Activation", prefix $= l$)
    $\mathcal{G}$.add_node("FFN Down", prefix $= l$)
    $\mathcal{G}$.add_node("Residual", prefix $= l$)
$\mathcal{G}$.add_node("RMSNorm")
$\mathcal{G}$.add_node("LM Head", config.`hidden_size`, config.`vocab_size`)
**return** $\mathcal{G}$

此时各节点关联的张量只有形状信息，没有实际数据。

### 9.1.4　第 4 步：加载权重

这是模型加载中最具挑战性的步骤。对于 LLaMA-70B 这样的大型模型（∼140GB FP16），完整加载可能耗时数分钟，且需要巨大的连续内存空间。

**权重加载策略对比**

| 策略 | 初始加载时间 | 内存占用 | 适用场景 |
|---|---|---|---|
| 全量读入 | 长（IO bound） | 完整权重内存 | 小模型、无内存压力 |
| 内存映射 | 零（懒加载） | 增量（按需） | 大模型、多进程共享 |
| 量化加载 | 中等 | $1/2 \sim 1/4$ 权重 | 低内存设备 |
| 流式加载 | 中等 | 中间 | 网络传输场景 |

### 9.1.5 第 5 步：应用图优化

一旦计算图构建完成并且权重的形状已知，推理引擎运行图 pass 对计算图进行变换：

**常见的图优化 Pass**

- **算子融合**: 连续的 MatMul + BiasAdd + ReLU 融合为单一算子。
- **常量折叠**: 预计算仅依赖常量的子图，用一个常量张量替代。
- **死代码消除**: 移除输出不被任何后续节点使用的子图。
- **布局传播**: 在图中传播最佳内存布局，必要时插入 `Transpose` 节点。
- **量化插入**: 在需要量化的算子前后插入 `Quantize/Dequantize` 节点。

### 9.1.6 第 6 步：内存规划

**推理内存规划的确定性**

与训练不同，推理中的张量尺寸在模型加载后是完全确定的。因此：

- **根据最大 batch size $B_{max}$ 和最大上下文长度 $L_{max}$ 预分配.**
- 推理期间绝不再分配或释放内存，避免引入不确定性延迟。
- 极限内存量在配置阶段即可计算，可以作为约束条件进行检查。

### 9.1.7 第 7 步：创建执行上下文

执行上下文（session/context）将所有权重与特定推理会话的参数绑定。对于同一个加载模型，可以创建多个上下文（每个用户一个），它们共享权重但拥有独立的 KV 缓存。

## 9.2　权重加载策略详解

### 9.2.1　显式读取

**显式读取流程**

1. 打开文件，使用 `fread` 或 `pread` 按顺序读取各张量数据到预分配的缓冲区。
2. 优点：可预测的延迟，精确控制何时加载哪一层。
3. 缺点：需要足够的内存来容纳完整权重；大模型必须分块加载。

对于多文件模型（如分片的 SafeTensors），遍历每个分片文件按序读取并分配到正确的缓冲区。

### 9.2.2　量化加载

**加载时量化**

在从文件读取权重后，立即进行量化转换，只将量化版本保留在内存中：

$$W_{int4}[i] = \text{round}\left(\frac{W_{fp16}[i]}{s}\right) \tag{9.1}$$

其中 $s = \max(|W_{fp16}|)/7$（对于 int4 对称量化）。
主要优点：内存占用从 FP16 的 2 字节/元素降至 Int4 的 0.5 字节/元素，约节省 4×。

**Int4 打包格式**

Int4 权重在内存中通常是打包的：2 个 Int4 值打包为 1 个字节。解包可以在矩阵乘法微内核中直接使用（无需显式转换），通过位操作提取：

```
uint8_t packed = packed_weights[idx];
int8_t w0 = (packed & 0x0F) - 8;  // 低4位
int8_t w1 = (packed >> 4) - 8;    // 高4位
```

### 9.2.3　张量名到层的映射

不同框架对同一逻辑层的张量命名不同。推理引擎需要一个健壮的映射系统。

**张量名映射策略**

| 逻辑层 | PyTorch (HuggingFace) | GGUF |
|---|---|---|
| 嵌入权重 | `model.embed_tokens.weight` | `token_embd.weight` |
| 第 $l$ 层输入归一化 | `model.layers.`$l$`.input_layernorm.weight` | `blk.`$l$`.attn_norm.weight` |
| 第 $l$ 层 QKV 投影 | `model.layers.`$l$`.self_attn.q_proj.weight` | `blk.`$l$`.attn_q.weight` |

推导引擎使用正则表达式或前缀匹配自动建立映射。对于未知架构，回退到基于形状和名称部分的启发式规则。

## 9.3 会话与上下文管理

**会话对象设计**

一个会话对象封装了一次推理会话的所有可变状态：

- **共享权重**（只读）：指向模型权重内存的指针（多会话共享）。
- **KV 缓存**（独占）：$(B, L, H, L_{max}, d_{head})$ 形状的缓存，存储键和值。
- **输入缓冲区**：当前 token 的嵌入输入。
- **中间缓冲区**：每一层的中间激活（可重用）。
- **状态变量**：当前位置（position id）、已生成的 token 数、是否完成（done）等。

### 9.3.1 多会话支持

**权重共享模型**

在服务场景中，同一个模型可能同时被多个用户请求使用。所有会话共享同一个权重加载实例（只读），大大节省显存。分离点在于：

- 每个会话拥有独立的 KV 缓存块分配（PagedAttention 管理的）
- 每个会话拥有独立的输入/输出缓冲
- 推理循环逐个切换：在一次解码迭代中同时处理多个会话的 batch

### 9.3.2　会话序列化

**会话保存/恢复**

在某些场景下（抢占、跨步迁移），会话的状态需要保存和恢复：

- **保存**: 将 KV 缓存和生成状态序列化到磁盘/发送到另一个节点
- **恢复**: 从序列化文件重新加载状态，继续中断的生成

在 PagedAttention 系统中，保存仅需记录块表映射和每个物理块的内容。

## 9.4　推理执行循环

推理的执行分为两个不对称的阶段：预填充阶段和解码阶段。

### 9.4.1　预填充阶段（Prefill Phase）

**预填充阶段**

用户输入 prompt 的 $P$ 个 token 被一次性处理：

1. **嵌入**: $P$ 个 token 索引 $\rightarrow$ 嵌入矩阵查找 $\rightarrow P \times d_{model}$ 输入
2. **逐层前向传播**: 对每层 $l = 0, \ldots, L-1$：
   - 归一化 $\rightarrow$ Q,K,V 投影 $\rightarrow$ RoPE
   - 注意力（因果掩码）$\rightarrow$ O 投影 $\rightarrow$ 残差连接
   - 归一化 $\rightarrow$ FFN Gate/Up $\rightarrow$ 激活 $\rightarrow$ FFN Down $\rightarrow$ 残差连接
3. **在计算注意力时**: 将每个头的 K,V（$P \times d_{head}$）写入 KV 缓存。
4. **最终归一化** $\rightarrow$ **LM Head** 投影: 输出 $(P \times V)$ logits。
5. **仅保留最后位置**的 logits 用于采样下一个 token（解码开始时）。

预填充是计算密集型的：$P$ 个 token 并行处理，利用矩阵乘法的批量优势。

### 9.4.2 解码阶段（Decode Phase）

**自回归解码循环**

**输入：** 会话上下文 $C$（含 KV 缓存），最大生成长度 $L_{max}$，EOS token ID
**输出：** 生成的 token 序列 $G$

$G \leftarrow []$ ▷ 空的生成列表
**while** $|G| < L_{max}$ **do**
  emb $\leftarrow$ embedding_lookup($C$.last_token)
  pos $\leftarrow$ $C$.current_position()
  **for** $l \leftarrow 0$ **to** $L-1$ **do**
    emb $\leftarrow$ rms_norm(emb) ▷ 或 *LayerNorm*
    $q, k, v \leftarrow$ qkv_project(emb)
    对 $q, k$ 应用 RoPE 在位置 pos
    attn_out $\leftarrow$ attention($q$, K_cache[$l$], V_cache[$l$])
    更新 K_cache[$l$] 和 V_cache[$l$] 在位置 pos
    emb $\leftarrow$ residual(emb, attn_out)
    emb $\leftarrow$ rms_norm(emb)
    ffn_out $\leftarrow$ ffn_forward(emb)
    emb $\leftarrow$ residual(emb, ffn_out)
  logits $\leftarrow$ lm_head(rms_norm(emb))
  next_token $\leftarrow$ sample(logits, temperature, top_k, top_p)
  **if** next_token = EOS 或 next_token = max_length **then**
    **break**
  $G$.append(next_token)
  $C$.advance_position()
**return** $G$

解码的核心特征是：每次迭代只处理一个 token。这使得它极其**内存受限**——大部分操作是矩阵-向量乘法（GEMV）或低维度矩阵乘，它们的算术强度远低于计算单元的峰值。

### 9.4.3 采样策略

**常用采样策略详解**

在解码循环的最后一步，logits 被转换为下一个 token：

**贪心采样（Greedy）** next_token $= \arg\max_i \text{logits}[i]$。确定性高，但文本多样性差。

**温度采样（Temperature）** probs $= \text{softmax}(\text{logits}/T)$。$T \to 0$ 近似贪心；$T \to \infty$ 近似均匀采样。

**Top-K 采样** 仅保留概率最高的 $K$ 个 token，对其重新归一化后采样。

**Top-P（核采样 Nucleus）** 选择最小的 token 集合使得累计概率 $\geq p$。

**Min-P** 将概率低于 max_prob × min_p_ratio 的 token 设为 0。

**典型采样（Typical）** 选择熵接近自然语言分布的 token。

**Mirostat** 自适应调节 $K$ 值以保持目标困惑度（perplexity）。

### 9.4.4 Logit 处理

在应用采样策略之前，logits 通常经过一系列惩罚和偏置处理：

**Logit 处理流水线**

1. **Logit 偏置（Bias）**：对某些 token 施加固定偏移（例如强制避免某些敏感词）。
2. **存在惩罚（Presence Penalty）**：$L[i] -= \alpha_p$ 如果 token $i$ 已出现在序列中。
3. **频率惩罚（Frequency Penalty）**：$L[i] -= \alpha_f \times \text{count}(i)$。
4. **重复惩罚（Repetition Penalty）**：若前几个 token 中存在 $i$，则降低其概率。

这些处理对生成的多样性、重复性和安全性有直接影响。

## 9.5 批处理执行

### 9.5.1 预填充批处理

**预填充批处理**

多个 prompt 同时处理时，不同 prompt 的长度通常不等。处理方法：

- **左侧填充（Left Padding）**：在较短的 prompt 左侧填充 PAD token 直到 max length。这确保解码时的最后一个有效位置在同一时间步。
- **掩码处理**: 在注意力计算中为填充位置设置 $-\infty$ 掩码，防止它们影响有效 token 的注意力。
- **分批策略**: 将长度接近的 prompt 分配给相同 batch，减少填充浪费。

### 9.5.2 解码批处理

**解码批处理的差异化特性**

与预填充不同，解码时每个样本已独立运行，具有不同的 KV 缓存长度：

- 每个样本的 K,V 缓存长度可能不同。
- 注意力掩码需要独立处理每个样本的有效长度。
- 在连续批处理中，随时可能有新样本加入和旧样本退出。

形状布局：K 缓存 $\sim (B, L, H, L_{max}, d_{head})$。在 PagedAttention 中，各块的逻辑表独立管理，注意力 kernel 能按需访问物理块。

### 9.5.3 批处理 KV 缓存

**批处理 KV 缓存形状**

对于 $B$ 个批处理样本，$L$ 个 Transformer 层，$H$ 个注意力头，最大序列长度 $L_{max}$，头维度 $d_{head}$：

$$\text{K_cache} : (B, L, H, L_{max}, d_{head}), \quad \text{V_cache} : (B, L, H, L_{max}, d_{head}) \tag{9.2}$$

总内存占用（FP16）：

$$\text{Mem}_{KV} = 2 \times B \times L \times H \times L_{max} \times d_{head} \times 2 \text{ bytes} \tag{9.3}$$

例如 $B = 4, L = 32, H = 32, L_{max} = 4096, d_{head} = 128$: $\text{Mem}_{KV} = 2 \times 4 \times 32 \times 32 \times 4096 \times 128 \times 2 \approx 8.5$ GB。

## 9.6 异常处理

**推理引擎异常类型**

| 异常类型 | 发生场景 | 处理策略 |
|---|---|---|
| 无效输入 | Token ID 超出词表、输入为空 | 返回空或回退至 PAD/UNK token |
| OOM | 分配 KV 缓存失败 | 抢占低优请求、拒绝新请求 |
| 数据不匹配 | 权重形状与配置不符 | 尝试重塑或报错 |
| 硬件错误 | CUDA 错误、内存故障 | 重试、回退至 CPU 模式 |

**优雅降级原则**

推理引擎应遵循"fail gracefully" 原则：如果 OOM，不应崩溃，而应尝试驱逐、交换或重计算。如果无法满足最小内存需求，应尽早报错并提供解决建议，而不是在推理中 OOM 后直接崩溃。

## 9.7　性能分析与调试

### 9.7.1　逐层计时

**逐层性能分析**

在开发推理引擎时，需要在每个 Transformer 层记录执行时间：

```
for (int l = 0; l < n_layers; l++) {
    auto t0 = high_resolution_clock::now();
    forward_layer(l, input);
    auto t1 = high_resolution_clock::now();
    layer_timings[l] += duration(t1 - t0);
    // 区分 prefill 和 decode 阶段的计时
}
```

这帮助识别性能瓶颈：某些层可能因为维度不匹配或数据布局问题耗时异常。

### 9.7.2　内存追踪

**GPU 内存使用追踪**

在 CUDA 上，使用 `cudaMemGetInfo` 追踪空闲/已用内存；使用 NVIDIA Nsight Systems 或 Nsight Compute 进行更细粒度的分析。关键指标：

- 每层的 KV 缓存内存占比
- 中间激活的峰值内存
- 内存分配与释放的时间分布
- 内存传输（H2D/D2H）的带宽利用率

### 9.7.3 算子统计

**算子执行统计**

在每次推理运行后，汇总：

- 每个算子的总执行时间（以毫秒计）
- 每个算子占 GPU 时间的百分比
- 算子级别的计算密度（FLOPs/Byte）
- 算子使用的内存总量（包含临时内存）

这形成了一个性能画像，可用于针对性优化最昂贵的算子。

**本章要点总结**

1. 模型加载是一个七步流水线：检测格式 → 解析配置 → 构建图 → 加载权重 → 优化 → 内存规划 → 创建上下文。
2. 权重加载策略（全量/mmap/量化）的权衡影响启动时间、内存占用和多进程共享能力。
3. 推理循环分为预填充（一次性处理所有 prompt tokens）和解码（逐 token 生成）两个不对称阶段。
4. 解码阶段是内存受限的，连续批处理是实现高吞吐的关键。
5. 采样和 logit 处理是控制生成质量和多样性的最后防线，需仔细调校。
6. 全面的性能分析（逐层计时、内存追踪、算子统计）是持续优化的基础。

# 第三部分

# 优化技术篇：极致性能的秘密

# 第十章　低精度量化

## 10.1　量化基础理论

### 10.1.1　量化问题的数学形式化

量化（Quantization）是指将连续值（或高精度浮点值）映射到一组离散的整数格点上的过程。在深度学习推理中，量化的核心目标是在尽可能减少信息损失的条件下，降低模型的存储占用和计算带宽需求。形式化地，给定一个浮点张量 $\boldsymbol{X} \in \mathbb{R}^{m\times n}$，我们期望找到一个整数张量 $\boldsymbol{X}_q \in \mathbb{Z}^{m\times n}$ 和一个逆映射，使得

$$\|\boldsymbol{X} - \mathcal{D}(\mathcal{Q}(\boldsymbol{X}))\|_F \tag{10.1}$$

最小化，其中 $\mathcal{Q}$ 表示量化算子，$\mathcal{D}$ 表示反量化算子。在实际应用中，由于硬件对整数运算的支持远优于浮点运算，量化可以在适当的精度损失下带来 $2\times$–$4\times$ 的吞吐量提升和显著的能耗降低。

**仿射量化定义**

仿射量化（Affine Quantization）是最通用的量化方案。对于浮点值 $x$，其量化到 $b$ 位整数的过程为：

$$q = \operatorname{round}\left(\frac{x}{\Delta}\right) + z, \quad \Delta = \frac{x_{\max} - x_{\min}}{2^b - 1}, \quad z = -\operatorname{round}\left(\frac{x_{\min}}{\Delta}\right) \tag{10.2}$$

其中 $\Delta$ 为比例因子（scale），$z$ 为零点（zero_point）。反量化将整数恢复为浮点近似值：

$$\hat{x} = \Delta \cdot (q - z) \tag{10.3}$$

若 $x_{\min} = -x_{\max}$ 且 $z = 0$，则退化为对称量化（Symmetric Quantization）。

### 10.1.2　对称量化与非对称量化

对称量化强制零点为零，即 $z = 0$，映射范围以原点对称展开。具体地，若量化位宽为 $b$，则整数域为 $[-2^{b-1}, 2^{b-1} - 1]$（有符号）或 $[0, 2^b - 1]$（无符号）。对称量化的比例因子为

$$\Delta_{\text{sym}} = \frac{\max(|x|)}{2^{b-1} - 1} \tag{10.4}$$

对称量化的优势在于实现简单——反量化时的零点偏移为零，矩阵乘法可直接在整数域进行（纯整数 GEMM），无需额外的零点校正项。然而，对于非对称分布（如 ReLU 激活值恒为非负），对称量化会浪费一半的整数表示范围，导致有效精度减半。

非对称量化保留了零点 $z \neq 0$，允许量化区间完全覆盖实际的数据范围。对于非负分布，这意味着所有整数编码位都可以用来表示有效数据。其代价是矩阵乘法中需要引入零点偏移的补偿项：

$$\begin{aligned} \hat{y}_i &= \Delta_w \Delta_x \left[ \sum_j \left( W_q[i,j] - z_w \right) \left( X_q[j] - z_x \right) \right] \\ &= \Delta_w \Delta_x \left[ \sum_j W_q[i,j] X_q[j] - z_x \sum_j W_q[i,j] - z_w \sum_j X_q[j] + K z_w z_x \right] \end{aligned} \tag{10.5}$$

其中 $K$ 为归约维度的大小。

**量化方式选择建议**

- 权重通常呈对称分布，适合对称量化（有符号整数）。
- 激活值（尤其是 ReLU 之后）呈非对称分布，非对称量化可提供更好的精度。
- 在硬件支持的情况下（如 Qualcomm Hexagon、部分 ARM NEON 路径），优先使用非对称量化以最大化表示能力。

### 10.1.3 粒度量化的层次

量化粒度（Granularity）直接决定了精度与开销之间的权衡。从小到大，量化粒度可分为以下几类：

**逐张量量化（Per-Tensor Quantization）：**整个张量共享一个比例因子 $\Delta$ 和一个零点 $z$。这是最简单的方案，存储开销仅需两个标量。但对于内部数值分布差异大的张量，精度损失严重——权重矩阵中不同输出通道的数值范围可能相差数个数量级。

**逐通道量化（Per-Channel Quantization）：**对于权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{C_o \times C_i}$，每个输出通道（行）拥有独立的 $(\Delta_i, z_i)$。此为卷积核量化的事实标准，因其能在极低精度（如 int4）下仍保持与全精度接近的模型精度。存储开销为 $C_o$ 个比例参数，相比权重本身可忽略不计。

**逐块/分组量化（Per-Block / Per-Group Quantization）：**将张量划分为固定大小的连续块（block size $B$ 通常取 32, 64, 128, 256），每个块拥有独立的量化参数。这是 GGUF 格式家族（如 Q4_0, Q4_K, IQ 系列）的核心思想。块大小越小，量化误差越小，但元数据开销（比例因子和零点的存储）越高。例如，Q4_0 使用 $B = 32$ 的块大小，每个块存储块内所有 4 位权重值及一个 fp16 比例因子。

**GGUF 量化类型总览**

GGUF 是 llama.cpp 框架使用的模型文件格式，定义了数十种量化类型。其核心设计思想是逐块量化——将权重划分成小块，每块独立量化，在推理时按需反量化。以下将逐一剖析各类量化类型的存储布局、反量化算法和性能特性。

### 10.1.4　动态量化与静态量化

**静态量化（Static Quantization）：**在推理前预先使用校准数据集（calibration dataset）统计每个张量的数值范围 $[x_{\min}, x_{\max}]$，并将 $\Delta$ 和 $z$ 固定下来。推理时直接使用预计算的量化参数，无需实时统计。这是部署中最常用的方式。

**动态量化（Dynamic Quantization）：**推理时根据实际输入实时计算量化范围。优点是不需要校准步骤，且对分布偏移的数据具有鲁棒性。缺点是需要在线计算 $x_{\min}$ 和 $x_{\max}$，增加了额外的归约开销。PyTorch 的 torch.quantization 提供了两种模式的统一接口。

### 10.1.5　量化感知训练与后训练量化

**后训练量化（Post-Training Quantization, PTQ）：**在预训练完成的完整精度模型上直接应用量化。所需数据极少（通常仅需数百张图片/数段文本作为校准数据），计算开销极小。PTQ 的有效性依赖于模型本身的量化鲁棒性——某些模型在 int8 精度下几乎无损，但在 int4 下可能需要更复杂的处理（如 GPTQ、AWQ）。

**量化感知训练（Quantization-Aware Training, QAT）：**在训练过程中插入伪量化节点（FakeQuant），模拟量化误差的前向和反向传播。模型在训练中就适应了量化的离散性，通常可达到比 PTQ 更低的量化误差。代价是需要完整的训练流程和数据集。

## 10.2　整数矩阵乘法的量化计算

### 10.2.1　量化的 GEMM 运算

考虑完整的量化矩阵乘法流程。设权重矩阵 $\boldsymbol{W}$ 量化为 int8 张量 $\boldsymbol{W}_q$，输入激活 $\boldsymbol{X}$ 量化为 int8（或 uint8）张量 $\boldsymbol{X}_q$。目标是在整数域完成计算后反量化回浮点输出。

**纯整数 GEMM：**当采用对称量化且零点为零时，GEMM 可简化为

$$\boldsymbol{Y} = (\Delta_w \Delta_x) \cdot (\boldsymbol{W}_q \times \boldsymbol{X}_q) \tag{10.6}$$

仅需一次整数矩阵乘法和一次浮点标量乘法。

**带零点的 GEMM：**当零点非零时，需引入校正项：

$$\boldsymbol{Y} = \Delta_w \Delta_x \cdot \left( \boldsymbol{W}_q \boldsymbol{X}_q - z_x \boldsymbol{W}_q \mathbf{1} - z_w \mathbf{1}^\top \boldsymbol{X}_q + K z_w z_x \right) \tag{10.7}$$

其中 $\mathbf{1}$ 为全一向量，$K$ 为内维长度。在实践中，$\boldsymbol{W}_q \mathbf{1}$ 和 $z_w \mathbf{1}^\top$ 可以预计算并存储。

### 10.2.2 累加与再量化

整数 GEMM 的内积累加在 int32 精度中进行，以容纳更大的中间结果而无溢出风险。具体地，int8 的点积 $\sum w_q \cdot x_q$ 最多产生 $-128 \times -128 \times K$ 到 $127 \times 127 \times K$ 的值。对于 $K \leq 2^{24}$ 的情况，int32 的表示范围 $[-2^{31}, 2^{31}-1]$ 足够容纳。

再量化（Requantization）将 int32 累加器映射回 int8 输出，为下一层准备输入：

$$q_{\text{out}} = \text{round}\left(\frac{\Delta_w \Delta_x}{\Delta_{\text{out}}} \cdot \texttt{acc_int32}\right) + z_{\text{out}} \tag{10.8}$$

其中 $\Delta_{\text{out}}$ 和 $z_{\text{out}}$ 是下一层激活的量化参数。在实际实现中，比例因子的除法可转化为定点乘法和位移：

$$\frac{\Delta_w \Delta_x}{\Delta_{\text{out}}} \approx \frac{M}{2^n}, \quad M \in [2^{31}, 2^{32}) \tag{10.9}$$

**融合量化 GEMM 过程**

1. 加载 int8 权重块 $\boldsymbol{W}_q$ 和 int8 激活块 $\boldsymbol{X}_q$。
2. 执行 int8 $\times$ int8 $\to$ int32 的矩阵乘法累积（如通过 VNNI 指令：vpdpbusd）。
3. 加载预计算的偏移补偿向量（若使用非对称量化）。
4. 对累加器施加偏移补偿：$\texttt{acc} \mathrel{-}= z_x \cdot \mathbf{s}_w + z_w \cdot \mathbf{s}_x - K z_w z_x$。
5. 再量化：$\texttt{out} = \text{saturate}(\texttt{multiply_and_shift}(\texttt{acc}, M, n) + z_{\text{out}})$。
6. 将结果裁剪到 int8 范围 $[-128, 127]$ 或 uint8 范围 $[0, 255]$。
7. 写回输出张量。

## 10.3 GGUF 量化类型详解

GGUF（GGML Universal Format）定义了一套丰富的量化类型，以逐块量化为核心设计理念。每种类型在比特数、量化方案（对称/非对称）和块结构上有所不同。

### 10.3.1 Q4_0：4 位对称逐块量化

Q4_0 是最基础的 4 位量化格式。其存储布局为：每个块包含 $B = 32$ 个 4 位权重值，以及一个 fp16 比例因子 $\Delta$，共 $B/2 + 2 = 18$ 字节，有效位宽为 $18 \times 8/32 = 4.5$ bpw。

**Q4_0 反量化算法**

设读取的块数据为 $\{q_0, q_1, \ldots, q_{31}\}$（低 4 位与高 4 位交错打包），比例因子为 $\Delta$（fp16）。对于第 $j$ 个元素（$j \in [0, 31]$）：

$$w_j = (q_j - 8) \cdot \Delta \tag{10.10}$$

其中 $q_j \in [0, 15]$ 表示 4 位无符号整数。「减去 8」的操作将无符号范围 $[0, 15]$ 映射到以零为中心的对称范围 $[-8, 7]$，实现对称量化。

### 10.3.2　Q4_1：4 位非对称逐块量化

Q4_1 在 Q4_0 的基础上增加了最小值参数（minimum），实现了非对称量化。每块存储：16 字节 4 位权重（32 个值）+ 2 字节 fp16 比例因子 $\Delta$ + 2 字节 fp16 最小值 $m$，共 20 字节，有效位宽 5.0 bpw。反量化公式为

$$w_j = q_j \cdot \Delta + m \tag{10.11}$$

其中 $q_j \in [0, 15]$。由于引入了非对称映射，Q4_1 对非对称分布的权重（如某些注意力层的输出投影）精度优于 Q4_0，但额外 2bpw 的代价使其在总模型体积上不如 Q4_0 经济。

### 10.3.3　Q5_0 与 Q5_1

Q5_0 为 5 位对称量化，与 Q4_0 类似但每块包含 32 个 5 位值。打包方式为：第一个 16 字节存低 4 位（交错打包），后跟 4 字节存储 32 个值的第 5 位（高位位）。加上 2 字节 fp16 比例，总计 $16 + 4 + 2 = 22$ 字节，有效位宽 5.5 bpw。Q5_1 同理增加最小值字段，为 5 位非对称量化，有效位宽 6.0 bpw。

### 10.3.4　Q8_0 与 Q8_1

Q8_0 为 8 位对称逐块量化（$B = 32$）。每块存储 32 个 int8 值（32 字节）和一个 fp16 比例因子（2 字节），有效位宽 8.5 bpw。在多数实际应用中，Q8_0 的精度损失微乎其微——困惑度（perplexity）增量在 0.01 以内，非常适合对精度敏感的场景。

### 10.3.5　K-Quants：Q2_K 至 Q6_K

K-Quant 家族引入了超块（super-block）结构，在单个超块内使用不同粒度的量化参数。该设计由 llama.cpp 团队提出，在给定总位宽约束下显著提升了量化精度。

**超块结构：**每个超块包含 $S = 256$ 个权重值，被划分为 $G = 16$ 个子块，每子块 $B_s = 16$ 个元素。超块级别存储一个 fp16 的全局比例因子 $\Delta_{\text{super}}$ 和一个 fp16 的全局最小值 $m_{\text{super}}$。每个子块再存储一个相对于超块参数的 6 位增量比例因子 $\delta_i$ 和 6 位增量最小值 $\varepsilon_i$。最终每个子块的比例和最小值为

$$\Delta_i = \Delta_{\text{super}} \cdot 2^{\delta_i/64}, \quad m_i = m_{\text{super}} \cdot 2^{\varepsilon_i/64} \tag{10.12}$$

**Q4_K 存储布局**

以 Q4_K 为例，256 个权重值的存储结构为：

- 2 字节 fp16 超块比例因子 $\Delta_{\text{super}}$。
- 2 字节 fp16 超块最小值 $m_{\text{super}}$。
- 12 字节：16 个子块的 6 位比例增量（$16 \times 6 = 96$ 位 $= 12$ 字节）打包。
- 12 字节：16 个子块的 6 位最小值增量打包。
- 128 字节：256 个 4 位权重值（$256 \times 4/8 = 128$ 字节）。

总计 $2 + 2 + 12 + 12 + 128 = 156$ 字节 / 256 个权重 $\Longrightarrow$ $156 \times 8/256 \approx 4.875$ bpw。

各 K-Quant 类型的位宽近似值为：Q2_K $\approx$ 2.56 bpw，Q3_K $\approx$ 3.44 bpw，Q4_K $\approx$ 4.88 bpw，Q5_K $\approx$ 5.66 bpw，Q6_K $\approx$ 6.56 bpw。在实际大语言模型评估中，Q4_K 的零样本（zero-shot）精度与 fp16 的差距通常在 1% 以内，而 Q2_K 的差距约为 3%-5%。

### 10.3.6 IQ 系列：重要性感知量化

IQ（Integer Quantization）系列引入了重要性矩阵（Importance Matrix）来指导量化过程。其核心思想是：并非所有权重对模型输出的贡献相同，应该在重要性高的权重上分配更多精度。

重要性矩阵 $\boldsymbol{H}$ 通常取权重激活的 Fisher 信息矩阵或 Hessian 矩阵的对角近似。IQ 算法求解一个加权最小二乘问题：

$$\min_{\hat{\boldsymbol{W}}} \sum_{i} H_{ii}(W_{ii} - \hat{W}_{ii})^2 \tag{10.13}$$

在高重要性权重上容忍更小的量化误差，在低重要性权重上可以更激进地量化。

IQ 系列的常见变体包括：

- **IQ1_S**：约 1.56 bpw，使用 2 位查找表（LUT）+1 位符号。存储从超块的比例因子中构建的 LUT。
- **IQ2_XXS**：约 2.06 bpw，极低比特推力的代表。
- **IQ2_XS**：约 2.31 bpw，更注重精度的 2 位级量化。
- **IQ3_XXS**：约 3.06 bpw，3 位级别的最激进选项。
- **IQ4_XS**：约 4.25 bpw，4 位级别、重要性感知，在很多模型上接近 Q4_K 的精度。

**GGUF 量化类型位宽与精度对比**

| 量化类型 | 有效 bpw | 困惑度增量 | 模型体积比（fp16） |
|---|---|---|---|
| Q2_K | 2.56 | 1.0 – 2.0 | ∼16% |
| IQ3_XXS | 3.06 | 0.5 – 1.0 | ∼19% |
| Q3_K_M | 3.91 | 0.3 – 0.6 | ∼24% |
| IQ4_XS | 4.25 | 0.2 – 0.4 | ∼27% |
| Q4_K_M | 4.88 | 0.1 – 0.3 | ∼31% |
| Q5_K_M | 5.66 | 0.05 – 0.15 | ∼35% |
| Q6_K | 6.56 | 0.02 – 0.07 | ∼41% |
| Q8_0 | 8.50 | <0.01 | ∼53% |

注：困惑度增量为近似区间，实际值因模型架构和训练数据而异。

## 10.4　GPTQ：精确后训练量化

### 10.4.1　OBQ 基础理论

GPTQ（Post-Training Quantization for Generative Pre-trained Transformers）建立在最优脑量化（Optimal Brain Quantization, OBQ）的理论框架之上。OBQ 的思想源于最优脑损伤（Optimal Brain Damage, OBD）和最优脑外科（Optimal Brain Surgeon, OBS）——即贪心地、逐个地量化权重，并在每次量化后更新剩余权重以补偿引入的误差。

对于线性层 $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$，目标是最小化量化前后输出的均方误差：

$$E = \|\boldsymbol{W}\boldsymbol{x} - \hat{\boldsymbol{W}}\boldsymbol{x}\|_2^2 = \|(\boldsymbol{W} - \hat{\boldsymbol{W}})\boldsymbol{x}\|_2^2 \tag{10.14}$$

利用 Hessian 矩阵 $\boldsymbol{H} = 2\boldsymbol{X}\boldsymbol{X}^\top$（基于校准数据），权重 $w_q$ 被量化为 $\tilde{w}_q$ 后，误差为 $\delta_q = w_q - \tilde{w}_q$，对所有未量化权重 $\boldsymbol{W}_{:,q+1:}$ 施加更新：

$$\boldsymbol{W}_{:,q+1} \leftarrow \boldsymbol{W}_{:,q+1} - \frac{\delta_q}{\boldsymbol{H}^{-1}_{q,q}} \cdot \boldsymbol{H}^{-1}_{:,q} \tag{10.15}$$

这保证了量化一个权重后，输出误差在最小二乘意义下被其余权重完全补偿。

### 10.4.2　GPTQ 算法流程

GPTQ 将 OBQ 扩展为逐列批量量化（batch quantization of columns），以避免 OBQ 每量化一个权重就更新整个 Hessian 逆的极高开销。其核心流程如下：

**GPTQ 量化算法**

**输入：**权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{d_{\text{row}} \times d_{\text{col}}}$，校准数据 $\boldsymbol{X}$，块大小 $B = 128$。
**输出：**量化权重 $\hat{\boldsymbol{W}}$。

1. 计算 Hessian 逆的 Cholesky 分解：$\boldsymbol{H}^{-1} = \boldsymbol{L}\boldsymbol{L}^{\top}$，其中 $\boldsymbol{H} = 2\boldsymbol{X}\boldsymbol{X}^{\top}$。
2. 对每个列块 $i = 0, B, 2B, \ldots, d_{\text{col}}$：
   (a) 对块内每列 $j = i, i+1, \ldots, \min(i+B-1, d_{\text{col}})$：
      i. 量化该列：$\hat{\boldsymbol{W}}_{:,j} = \mathcal{Q}(\boldsymbol{W}_{:,j})$。
      ii. 计算量化误差：$\boldsymbol{\delta} = \boldsymbol{W}_{:,j} - \hat{\boldsymbol{W}}_{:,j}$。
      iii. 更新剩余列（$k = j+1, \ldots, d_{\text{col}}$）：$\boldsymbol{W}_{:,k} \mathrel{-}= \boldsymbol{\delta} \cdot \boldsymbol{H}^{-1}_{j,k} / \boldsymbol{H}^{-1}_{j,j}$。
   (b) 重新计算块 $i$ 之后的 Hessian 逆（Cholesky 更新），确保数值稳定性。
3. 返回量化后的权重 $\hat{\boldsymbol{W}}$。

### 10.4.3 Cholesky 重正化与组大小

直接使用 Hessian 逆的增量更新会导致数值误差的累积（由于浮点舍入）。GPTQ 通过每 $B = 128$ 列后重新计算 Cholesky 分解来重置数值误差，这一技巧被称为 Cholesky 重正化（Cholesky reformulation）。

此外，GPTQ 引入了组大小（group_size）参数。在每个大小为 $g$（通常 $g = 128$）的连续行组内，重新计算量化参数 $(\Delta, z)$ 以适应权重在不同行组间的数值分布变化。这等价于逐组量化与 Hessian 补偿的结合，显著提升了 int4 及以下比特率的重建精度。

### 10.4.4 性能特性

GPTQ 一次性处理整个模型，仅需少量未标注的校准数据（如 128 个样本）。量化速度较快——对 7B 参数模型，在单 GPU 上约需 4 小时即可完成 int4 量化。与原始权重相比，GPTQ-int4 的困惑度增量通常在 0.2–0.5 范围，远优于朴素的 RTN（Round-to-Nearest）量化。

## 10.5 AWQ：激活感知权重量化

### 10.5.1 核心洞察

AWQ（Activation-aware Weight Quantization）的研究者对 LLaMA 等模型的权重和激活分布进行了系统分析，发现了一个关键现象：**并非所有权重通道（channel）同等重要**。大约 1% 的输入通道（对应于权重矩阵的列）拥有异常大的激活量级，这些通道的权重量化误差会被放大，成为量化精度的主导因素。

**AWQ 的关键观察**

如果对所有权重通道使用相同的量化参数，则高激活通道的量化误差在输出中被大激活值放大（$\hat{y} - y = \delta_w \cdot x$），主导输出误差。因此，应该对这些重要通道施加更精确的量化（等价于使用更小的比例因子）。

### 10.5.2　逐通道缩放方法

AWQ 引入逐通道的等效缩放变换：对权重矩阵的一个输入通道 $j$，将权重向量 $\boldsymbol{W}_{:,j}$ 乘以因子 $s > 1$，同时将对应的激活值 $x_j$ 除以 $s$：

$$\boldsymbol{W}_{:,j} \leftarrow s \cdot \boldsymbol{W}_{:,j}, \quad x_j \leftarrow x_j / s \tag{10.16}$$

由于 $\boldsymbol{W}_{:,j} \cdot (x_j/s) = (s \cdot \boldsymbol{W}_{:,j}) \cdot (x_j/s) = \boldsymbol{W}_{:,j} \cdot x_j$，该变换在数学上等价（保真），但改变了量化行为——权重的数值范围被放大，相对量化误差 $\delta_w/(s \cdot W)$ 减小。

**AWQ 缩放因子的确定**

缩放因子 $s_j$ 通过在网格上最小化量化前后输出的 L2 误差确定：

$$s_j = \arg\min_{s \in \mathcal{S}} \left\| \boldsymbol{W}_{:,j} \boldsymbol{X}_{j,:} - \mathcal{Q}(s \cdot \boldsymbol{W}_{:,j}) \cdot (\boldsymbol{X}_{j,:}/s) \right\|_F^2 \tag{10.17}$$

其中 $\mathcal{S} = \{1, 2^{-\alpha}, 2^{-2\alpha}, \ldots, 2^{-K\alpha}\}$ 为预定义的缩放因子候选集，$\alpha$ 为超参数控制搜索粒度。通常 $s_j \in [0.5, 2.0]$。

### 10.5.3　与逐组量化的集成

AWQ 通常与逐组量化（per-group quantization，group_size = 128）结合使用：先应用逐通道缩放变换，然后在每个 128 个权重的组内执行量化。这样不需要在推理时存储额外的缩放因子——缩放已融入权重值和后续的量化参数中。

实验表明，AWQ 在 int4 精度下将 LLaMA-7B 的困惑度从 RTN 量化的严重退化恢复到接近全精度的水平，性能提升接近 GPTQ，但无需计算 Hessian 矩阵，实现更简单。

## 10.6　SmoothQuant

### 10.6.1　问题描述

在大语言模型中，激活值中普遍存在「离群值通道」——某些特征维度上的激活值比其他维度大 100×–1000×。当量化为 int8 时，这些离群通道主导了比例因子的确定，导致大多数正常通道的量化精度严重不足。

### 10.6.2 数学等效变换

SmoothQuant 提出通过平滑变换迁移量化难度：将激活离群通道的量级「平滑地」迁移到对应的权重通道上。对每个通道 $j$，引入平滑因子 $\alpha_j \in [0,1]$：

$$\boldsymbol{W}_{:,j} \leftarrow \boldsymbol{W}_{:,j} \cdot s_j, \quad \boldsymbol{X}_{j,:} \leftarrow \boldsymbol{X}_{j,:}/s_j \tag{10.18}$$

其中 $s_j = \left(\frac{\max|x_j|}{\max|w_j|}\right)^{-\alpha_j}$。平滑因子控制了量级迁移的强度：

- $\alpha = 1.0$：将所有量级迁移到权重，激活变得平滑（近似均匀），有利于激活量化但权重可能被拉伸太多。
- $\alpha = 0.0$：不做迁移，所有困难留在激活侧。
- $\alpha = 0.5$：均衡迁移，实践中常为最优。

**SmoothQuant 平滑因子计算**

常见实现中，$\alpha$ 作为全局超参数（如 0.5），平滑因子 $s_j$ 通过下式计算：

$$s_j = \frac{\max(|X_{j,:}|)^{\alpha}}{\max(|W_{:,j}|)^{1-\alpha}} \tag{10.19}$$

此公式确保了在 $\alpha = 0.5$ 时，「权重侧的量级方差」和「激活侧的量级方差」达到近似均衡。

SmoothQuant 的一个显著优势是：变换是数学上精确等价的（在前向推理中完全保持原模型的输出），仅在量化发生时改变了误差分布。其为 W8A8（权重 int8、激活 int8）的纯整数量化铺平了道路。

## 10.7 LLM.int8()

### 10.7.1 混合精度分解

LLM.int8() 由 Dettmers 等人提出，针对大语言模型中激活离群值导致 int8 量化失败的问题，引入混合精度分解策略。

具体做法是将输入矩阵 $\boldsymbol{X}$ 按特征维度（列）分解为两部分：含离群值的列（outlier_columns）和常规列。离群值的判定采用绝对阈值：若某列的绝对值最大值超过阈值 $\tau$（通常 $\tau = 6.0$，即 $\max_j |x_{:,j}| > 6$），则该列被视为离群列。

**LLM.int8() 前向推理**

**输入：** 权重 $\boldsymbol{W}$，输入特征 $\boldsymbol{X}$，离群阈值 $\tau$。
**输出：** 输出 $\boldsymbol{Y}$。

1. 检测离群特征列 $\mathcal{O} = \{j \mid \max|x_{:,j}| > \tau\}$。

2. 构造常规特征子矩阵 $\boldsymbol{X}_{\neg\mathcal{O}}$ 和离群特征子矩阵 $\boldsymbol{X}_{\mathcal{O}}$。

3. 将 $\boldsymbol{X}_{\neg\mathcal{O}}$ 量化为 int8，执行 int8 GEMM 与对应权重子块 $\boldsymbol{W}_{\neg\mathcal{O}}$。

4. 将 $\boldsymbol{X}_{\mathcal{O}}$ 保持为 fp16，执行 fp16 GEMM 与对应权重子块 $\boldsymbol{W}_{\mathcal{O}}$。

5. 累加两个部分的输出：$\boldsymbol{Y} = \boldsymbol{Y}_{\neg\mathcal{O}} + \boldsymbol{Y}_{\mathcal{O}}$。

6. 反量化 $\boldsymbol{Y}_{\neg\mathcal{O}}$ 后与 $\boldsymbol{Y}_{\mathcal{O}}$ 相加。

在典型 Transformer 模型中，仅约 0.1% 的特征列被标记为离群列，因此 fp16 GEMM 的额外开销可以忽略，而常规列可以被高效地量化为 int8。在 OPT、BLOOM 等模型上，LLM.int8() 实现了与 fp16 完全一致的生成质量，推理速度提升约 2×。

## 10.8　KV 缓存量化

### 10.8.1　键缓存量化

Transformer 的推理过程需要在自回归生成阶段访问大量历史键值（Key-Value cache）。随着序列长度增长（如 128K tokens），KV 缓存的内存占用成为推理吞吐的主要瓶颈。KV 缓存量化旨在降低该占用的精度需求。

键（Key）向量的数值分布通常较为均匀，不含极端离群值，因此对量化相对鲁棒。将键缓存量化为 int8 或 fp8 通常不会显著影响注意力计算的精度。常用的量化方案包括：

- 逐通道量化（per-channel）：每个注意力头独立设置比例因子。

- 逐令牌量化（per-token）：每个时间步独立设置比例因子，适应序列中不同位置的分布变化。

### 10.8.2　值缓存量化

值（Value）缓存对量化更为敏感，因为值向量直接参与输出的加权求和，量化误差会累加传播到最终的 logits。研究发现，值缓存量化的精度损失比键缓存大约 2×–3×。因此，许多方案对值缓存使用更高精度（如 int8 而非 int4），或仅对键缓存进行激进量化。

**KV 缓存量化的精度陷阱**

值缓存使用逐令牌量化时需特别注意——序列末尾的值其量级可能与序列开头差异巨大。建议在量化参数计算中使用滑动窗口统计，而非全局范围，以避免因早期大值而挤压后续小值的表示精度。

### 10.8.3 先进方案：KIVI 与 KVQuant

KIVI（Key-Value Cache Quantization with Importance-guided Calibration）引入了重要性感知的量化校准：根据键值在注意力计算中的实际「使用频率」分配量化精度，将更多精度分配给被频繁访问的早期 token 的 KV 对。

KVQuant 则引入了非均匀量化码本（如 K-means 训练的码本或查找表），替代传统的均匀量化。在 2 位和 3 位的极端压缩率下，非均匀量化能保留更多信息（因为可利用实际分布的不均匀性），但需要额外的解量化查表开销。

## 10.9 FP8 与块浮点

### 10.9.1 FP8 数据格式

FP8（8 位浮点）在 NVIDIA Hopper 架构（H100）中被首次引入。有两种标准格式：

- **E4M3**：4 位指数 + 3 位尾数。范围窄（$\approx \pm 448$），精度高（尾数可区分 8 个值）。适用于需要精度的权重和前向传播。
- **E5M2**：5 位指数 + 2 位尾数。范围宽（$\approx \pm 57344$），精度低。适用于需要动态范围的梯度反向传播。

两种格式都保留一位符号位。FP8 GEMM 在 H100 上由专用的张量核（Tensor Core）提供硬件支持，吞吐量可达 fp16 BF16 的 2×。

### 10.9.2 块浮点（Block Floating Point）

块浮点（BFP）是定点与浮点的混合方案：一个块内的所有数据共享一个公共指数（exponent），每个元素拥有独立的尾数。形式化地，对于块 $\{x_1, \ldots, x_B\}$，使用共享指数 $E$：

$$\hat{x}_i = m_i \cdot 2^{E}, \quad E = \text{floor}(\log_2 \max_i |x_i|) \tag{10.20}$$

其中 $m_i$ 为 $b$ 位有符号整数尾数。块浮点结合了定点（计算简单）和浮点（动态范围大）的优点，Intel Gaudi 的深度学习加速器就采用了块浮点作为其原生数据格式。

## 10.10 量化硬件支持

### 10.10.1 NVIDIA Tensor Core DP4A

NVIDIA 的 Tensor Core 通过 DP4A（Dot Product of 4 Integers and Accumulate）指令支持 int8 GEMM。DP4A 计算两个 4 元素 int8 向量的点积并累加到 int32：

$$\text{DP4A}(a, b, c) = c + \sum_{i=0}^{3} a_i \cdot b_i \tag{10.21}$$

在 CUDA 级别，可通过 PTX 指令 `dp4a` 或 cuBLAS int8 API 进行调用。

### 10.10.2　ARM NEON SDOT/UDOT

ARMv8.2-A 指令集引入了有符号点积（SDOT）和无符号点积（UDOT）指令：

- `sdot vD.4S, vA.16B, vB.16B`：将两个 4-元素 int8 向量的点积累加到 int32 累加器。
- `udot` 同理，用于 uint8 输入。

这些指令是移动端 int8 推理的核心加速原语。

### 10.10.3　x86 VNNI

Intel 的 VNNI（Vector Neural Network Instructions）扩展提供了高效的 int8 点积累加指令：

- AVX-VNNI（256 位）：`vpdpbusd`（uint8 × int8 点积）、`vpdpwssd`（int16 × int16）。
- AVX-512-VNNI（512 位）：`vpdpbusd zmm, zmm, zmm` 一次处理 16 对 int8 元素的点积。

**指令选择建议**

对于 llm.cpp 等纯 CPU 推理场景，应在编译时探测 CPU 特性（`CPUID`）并动态选优：

- 支持 AVX-512-VNNI：使用 `vpdpbusd` 512 位路径。
- 支持 AVX2 + VNNI：使用 `vpdpbusd` 256 位路径。
- 仅有 NEON：使用 `sdot/udot` 路径。
- 无 SIMD 点积指令：退化为 int16 乘法 + 归约（开销约 4×）。

### 10.10.4　量化建模实践总结

**量化管线标准流程**

在工业级推理引擎中，一个完整的量化管线通常包含：

1. **校准阶段**：在代表数据集上运行模型，收集每层激活的数值范围和分布。
2. **参数计算**：根据目标位宽和量化方案计算 $\Delta$ 和 $z$。对于逐块方案，此步骤在每个块上独立执行。
3. **权重转换（离线）**：将全精度权重按照计算得到的参数进行量化和打包（如 GGUF 文件生成）。
4. **前向调整（可选）**：对量化后的权重施加 GPTQ/AWQ 补偿，微调量化参数以减小输出误差。

5. **运行时推理：**加载打包的量化权重，在每一层按需反量化，执行 int8/int4 GEMM。需注意权重重排以适应缓存友好的访问模式（见第十三章）。

# 第十一章　向量化与 SIMD

## 11.1　SIMD 基础

### 11.1.1　SIMD 范式

单指令多数据（Single Instruction Multiple Data, SIMD）是高性能计算的核心范式。与标量处理（每条指令仅操作一个数据元素）不同，SIMD 指令同时操作一个向量寄存器中的多个数据元素——一条浮点乘法指令即可同时完成 4 个、8 个甚至 16 个单精度浮点数的乘法。

**SIMD 吞吐量优势**

设标量处理器每条指令处理 1 个 $N$ 位数据，SIMD 宽度为 $W$（即每条指令处理 $W$ 个数据元素）。在相同频率下，SIMD 的理论峰值吞吐量是标量的 $W$ 倍。例如，AVX-512 的 $W = 16$（对于 fp32），理想加速比为 16×。实际加速比受内存带宽和指令依赖限制，通常在 4×–12× 范围。

### 11.1.2　寄存器宽度演进

现代 CPU 的 SIMD 指令集经历了数代宽度的扩展：

- **SSE（Streaming SIMD Extensions）：** 128 位寄存器（XMM），处理 4 个 fp32 或 2 个 fp64。最早由 Intel Pentium III 引入（1999 年）。
- **AVX / AVX2：** 256 位寄存器（YMM），处理 8 个 fp32 或 4 个 fp64。AVX2（2013 年，Haswell）增加了整数 SIMD 指令和 FMA（Fused Multiply-Add）。
- **AVX-512：** 512 位寄存器（ZMM），处理 16 个 fp32 或 8 个 fp64。首次引入掩码操作（mask register）用于条件执行。SKX（Skylake-X）及后续服务器平台支持。
- **ARM NEON：** 128 位寄存器（Q 寄存器），处理 4 个 fp32。ARMv7 及以后版本均支持。
- **ARM SVE / SVE2：** 可变宽度（scalable），从 128 位到 2048 位硬件实现均可使用同一套二进制指令。

**AVX-512 频率降频问题**

早期 AVX-512 实现（如 Skylake-X）在执行 512 位指令时会触发 CPU 降频（AVX-512 turbo license），导致非 SIMD 代码也受到影响。Ice Lake 及之后的微架构优化了这一问题，降频幅度显著降低。在混合工作负载场景下，应评估纯 256 位（AVX2）与 512 位的实际端到端吞吐。

### 11.1.3 SIMD 数据类型

SIMD 指令集支持多种数据精度，在同一寄存器内可处理不同数量的元素：

| 数据类型 | 元素宽度 | AVX2（256 位）元素数 | AVX-512（512 位）元素数 |
|---|---|---|---|
| fp32（float） | 32 bits | 8 | 16 |
| fp16（half） | 16 bits | 16 | 32 |
| bf16（bfloat） | 16 bits | 16 | 32 |
| int32 | 32 bits | 8 | 16 |
| int16 | 16 bits | 16 | 32 |
| int8 | 8 bits | 32 | 64 |

## 11.2 x86 SIMD Intrinsic 详解

### 11.2.1 基本加载与存储操作

Intel Intrinsics 提供了对底层 SIMD 指令的 C/C++ 级抽象。以下以 AVX2 为例说明核心操作的语义。

**对齐加载与未对齐加载：**

- `_mm256_load_ps(float const* p)`：从 32 字节对齐的内存地址加载 8 个 fp32 值到 YMM 寄存器。若地址未 32 字节对齐则触发通用保护异常（#GP）。
- `_mm256_loadu_ps(float const* p)`：从未对齐地址加载，硬件自动处理跨缓存行边界的读取，但可能增加 1 个周期的延迟。
- `_mm256_store_ps(float* p, __m256 v)`：对齐存储。
- `_mm256_storeu_ps(float* p, __m256 v)`：未对齐存储。

在 GEMM 微内核等性能关键路径中，应确保数据对齐以使用对齐加载——可节省一个 $\mu$op 并避免缓存行分裂（cache line split）的惩罚。

**广播操作：**

- `_mm256_broadcast_ss(float const* p)`:从内存加载一个标量 fp32 值,广播到 YMM 的 8 个通道。等价于 `_mm256_set1_ps(*p)`，但使用内存操作数而非寄存器。

- `_mm256_permute2f128(__m256 a, __m256 b, int imm8)`：从两个 256 位寄存器中各选取一个 128 位段，组合成新的 256 位结果。可用于跨通道数据重排。

### 11.2.2　浮点计算指令

**FMA 指令核心**

AVX2 的 FMA（Fused Multiply-Add）指令在一次操作中完成 $(a \times b) + c$，且中间结果保留无限精度，仅最后做一次舍入。这相比单独执行乘法再加法（vmulps + vaddps）减少了舍入误差，且指令数减半。基础变体包括：

- `_mm256_fmadd_ps(a, b, c)` $\rightarrow a \times b + c$
- `_mm256_fmsub_ps(a, b, c)` $\rightarrow a \times b - c$
- `_mm256_fnmadd_ps(a, b, c)` $\rightarrow -(a \times b) + c$
- `_mm256_fnmsub_ps(a, b, c)` $\rightarrow -(a \times b) - c$

对于通用矩阵乘（GEMM），FMA 是核心原语。每个周期可以发射一条 FMA 指令，在 Skylake 上每条 FMA 产生 8 次 fp32 运算（对于 256 位），双 FMA 单元每个周期可产出 $2 \times 8 = 16$ FLOP，实现了每个周期 16 次浮点运算的吞吐。

**其他浮点指令：**

- `_mm256_add_ps(a, b)`：向量加法。
- `_mm256_mul_ps(a, b)`：向量乘法。
- `_mm256_max_ps(a, b)`：逐元素最大值——在激活函数实现中常用。
- `_mm256_min_ps(a, b)`：逐元素最小值。
- `_mm256_sqrt_ps(a)`：逐元素平方根，延迟较高（约 12 周期），应避免在热路径中使用。
- `_mm256_rcp_ps(a)`：近似倒数（12 位精度），延迟 5 周期。适合对精度不敏感的场景。
- `_mm256_rsqrt_ps(a)`：近似平方根倒数，广泛用于归一化中。

### 11.2.3　Int8 整数点积：VNNI

AVX-512-VNNI（以及更简化的 AVX-VNNI）提供了专为神经网络推理设计的 int8 点积累加指令。

`_mm256_dpbusd_epi32` 的语义（注意操作数的类型转换）：

- 第一个源：uint8 × 4 元素向量（无符号）。

- 第二个源：int8 × 4 元素向量（有符号）。
- 目标/第三个源：int32 累加器，将 4 个乘积加入累加器。

在一发指令中计算：$\texttt{acc}[k]\ +=\ \sum_{i=0}^{3} \texttt{src0}[4k+i](uint8) \times \texttt{src1}[4k+i](int8)$，其中 $k \in [0,3]$ 为 256 位版本中 4 个 32 位通道的索引。

512 位版本 `_mm512_dpbusd_epi32` 进一步将通道数扩展至 8 个。

**VNNI 在推理中的应用**

典型 int8 推理的微内核内循环：

1. 加载激活值（uint8，适合 ReLU 非负输出）到 ZMM 寄存器。
2. 加载 int8 权重值到 ZMM 寄存器。
3. 依次执行 `vpdpbusd` 累积到 ZMM 累加器。
4. 内维 K 循环结束后，对 int32 累加器执行再量化和截断。

在 Ice Lake Xeon 上，每个周期可发射两条 `vpdpbusd`，实现每周期 $2 \times 4 \times 16 = 128$ 次 int8 运算（对于 512 位）的峰值吞吐。

### 11.2.4 Shuffle 与重排操作

数据重排（permute 和 shuffle）指令在 SIMD 优化中不可或缺，用于处理打包数据、转置微块和广播向量通道。

- `_mm256_shuffle_ps(a, b, imm8)`：在两对 128 位段内，根据立即数选择 $a$ 和 $b$ 的各两个通道。
- `_mm256_permutevar8x32_ps(v, idx)`：根据索引向量任意重排 8 个 32 位通道。
- `_mm256_extractf128_ps(v, imm8)`：提取高或低 128 位段为 `__m128`。
- `_mm256_insertf128_ps(a, b, imm8)`：将 `__m128` 插入 256 位的低或高段。

重排指令通常消耗 1 个周期（在端口 5 执行），但某些复杂的跨 128 位段重排可能消耗 3 个周期。在设计微内核时应尽量减少重排指令的数量。

### 11.2.5 Gather 与 Scatter

对于不规则访问模式（如稀疏矩阵运算），AVX2 提供了 Gather 指令：

- `_mm256_i32gather_ps(base, idx, scale)`：从 `base + idx[i]*scale` 加载 8 个 fp32 值。比例因子 scale 可取 1, 2, 4, 8。

Gather 的硬件实现依赖于将索引发送到加载单元，逐元素执行加载。吞吐量约为每 2-3 周期 1 条指令，远慢于连续加载。对于可预测的稀疏模式，批量化连续加载或手动重构访问模式可能更优。

### 11.2.6　掩码操作（AVX-512）

AVX-512 引入了 8 个 64 位掩码寄存器（k0–k7），允许对 SIMD 操作的每个通道单独启用或禁用（零化或合并）。这从根本上消除了旧方法中为处理非对齐结束条件而使用标量尾循环的需求。

- `_mm512_mask_load_ps(src, mask, addr)`：仅在 mask 对应位为 1 的通道加载数据，其余通道保留 src 的值。
- `_mm512_maskz_load_ps(mask, addr)`：仅在 mask 对应位为 1 的通道加载，其余通道设为零。
- `_mm512_mask_fmadd_ps(a, mask, b, c)`：仅对 mask 选中的通道执行 FMA，其余通道保留 $a$ 的值。

## 11.3　向量化 MatMul 微内核

### 11.3.1　微内核设计原则

矩阵乘法的向量化实现通常采用分块的微内核（micro-kernel）架构——最内层循环处理非常小的子块（如 $M_r \times N_r$），数据完全驻留在 SIMD 寄存器中，仅在最内层循环前后与缓存交互。微内核的设计空间包括：

- **寄存器块大小：** $(M_r, N_r)$ 的选择决定了寄存器占用和重用率。
- **累加器交错：**应维持足够多的独立 FMA 链（通常 4-8 条）以隐藏 FMA 的流水线延迟。
- **打包布局：**输入矩阵 A 和 B 的打包（packing）方式决定了微内核的数据加载模式。

### 11.3.2　6x16 fp32 微内核（AVX2）

以下以 AVX2 上的一个 $6 \times 16$ fp32 微内核为例，展示完整的设计过程。该微内核处理 $\bar{\boldsymbol{C}} \mathrel{+}= \bar{\boldsymbol{A}} \cdot \bar{\boldsymbol{B}}$，其中 $\bar{\boldsymbol{C}}$ 为 $6 \times 16$ 寄存器块。

**寄存器分配（16 个 YMM 寄存器）：**

- 12 个寄存器用于 $\bar{\boldsymbol{C}}$ 的累积：$M_r = 6$ 行，$N_r = 16$ 列。每行需要 $16/8 = 2$ 个 YMM（每个 YMM 容纳 8 个 fp32），$6 \times 2 = 12$ 个寄存器。

- 1 个寄存器用于广播 $\bar{\boldsymbol{A}}$ 的一个列元素。
- 2 个寄存器用于加载 $\bar{\boldsymbol{B}}$ 的一行。
- 1 个寄存器空闲（可用于预取下一行 $\bar{\boldsymbol{B}}$）。

总计 16 个 YMM 寄存器全部分配，无溢出。

**内循环伪代码：**

```
// C[6][16] += A[6][K] * B[K][16]
// 寄存器: c00,c01,...,c50,c51 对应 C 的 12 个 8-元素段
//         a_broadcast 为广播 A 元素的寄存器
//         b0, b1 为加载 B 行

for k = 0 to K-1:
    // 加载 B 的下一行到 b0, b1
    b0 = load(B + k*16 + 0)
    b1 = load(B + k*16 + 8)

    // 逐行处理 A
    a_val = A[k + 0*ldA]
    a_broadcast = broadcast(a_val)
    c00 = fmadd(a_broadcast, b0, c00)
    c01 = fmadd(a_broadcast, b1, c01)

    a_val = A[k + 1*ldA]
    a_broadcast = broadcast(a_val)
    c10 = fmadd(a_broadcast, b0, c10)
    c11 = fmadd(a_broadcast, b1, c11)

    // ... 对 A 的第2至第5行重复相同操作 ...
end for
```

**延迟隐藏分析：**内循环中每条 FMA 指令的目标寄存器在下一次被同一 FMA 写入前，有至少 $M_r = 6$ 条 FMA 指令的执行周期——足够覆盖 FMA 的 4-5 周期流水线延迟，实现完全流水线化执行。

### 11.3.3 处理奇数尺寸

当矩阵维度不是微内核块大小的整数倍时，需要处理「碎边」（remainder）。常见策略包括：

- **剥离循环（Peel Loop）**：在主循环前进整块，剩余元素由小的剥离循环处理。
- **掩码写回**：（AVX-512）使用掩码寄存器，仅写回有效通道的结果，避免覆盖目标矩阵中不应被修改的元素。
- **补零填充**：将输入打包缓冲区的剩余位置零填充，执行完整微内核操作，通过索引限制仅写回有效结果。

## 11.4　ARM NEON 向量化

### 11.4.1　NEON 基础类型与指令

ARM NEON 提供 128 位 SIMD 能力，在 ARM Cortex-A 系列和 Apple Silicon 上广泛使用。NEON 内联函数（intrinsics）定义在 `<arm_neon.h>` 中。

**浮点操作：**

- `float32x4_t vld1q_f32(float32_t const* ptr)`：加载 4 个 fp32。
- `void vst1q_f32(float32_t* ptr, float32x4_t val)`：存储 4 个 fp32。
- `float32x4_t vfmaq_f32(float32x4_t a, float32x4_t b, float32x4_t c)`：$a \leftarrow a + b \times c$（FMA）。
- `float32x4_t vaddq_f32(a, b)`：加法。
- `float32x4_t vmulq_f32(a, b)`：乘法。

**int8 点积（需要 ARMv8.2-a+dotprod）：**

- `int32x4_t vdotq_s32(int32x4_t c, int8x16_t a, int8x16_t b)`：计算四个有符号 int8 × int8 点积累加到 int32。
- `uint32x4_t vdotq_u32(uint32x4_t c, uint8x16_t a, uint8x16_t b)`：无符号版本。

**半精度浮点（需要 ARMv8.2-a+fp16）：**

- `float16x8_t vfmaq_f16(float16x8_t a, float16x8_t b, float16x8_t c)`:fp16 FMA。
- `float32x4_t vcvt_f32_f16(float16x4_t a)`：fp16 到 fp32 的转换。

**BF16 点积（需要 ARMv8.6-a+bf16）：**

- `float32x4_t vbfdotq_f32(float32x4_t c, bfloat16x8_t a, bfloat16x8_t b)`:BF16 点积累加到 fp32。

### 11.4.2 NEON 矩阵乘法示例

以下展示使用 NEON 的 int8 点积指令实现 $4 \times 16$ 的量化微内核：

```
// C[4][16] in int32 += A[4][K] (uint8) * B[K][16] (int8)
// NEON寄存器: c00-c03 为每行的4个int32累加器 (4*4=16 int32 使用4个Q寄存器)
//             a0, a1, a2, a3 为加载的A列 (各16个uint8)
//             b0 为加载的B行 (16个int8)

for (k = 0; k < K; k += 16) {
    a0 = vld1q_u8(A + 0*ldA + k);
    a1 = vld1q_u8(A + 1*ldA + k);
    a2 = vld1q_u8(A + 2*ldA + k);
    a3 = vld1q_u8(A + 3*ldA + k);

    for (n = 0; n < 16; n += 4) {
        b0 = vld1q_s8(B + (k+0)*ldB + n);
        c00 = vdotq_u32(c00, a0, b0);  // uint8 * int8
        c01 = vdotq_u32(c01, a1, b0);
        c02 = vdotq_u32(c02, a2, b0);
        c03 = vdotq_u32(c03, a3, b0);
        // ... 继续内维循环 ...
    }
}
```

## 11.5 自动向量化

### 11.5.1 编译器自动向量化能力

现代编译器（GCC, Clang, MSVC）在 `-O3`（及 `-march=native`）下会自动尝试向量化循环。自动向量化的成功取决于循环体是否满足以下条件：

**自动向量化的前提条件**

满足以下条件的循环通常可被自动向量化：

1. 循环次数可静态确定（或至少可计算上界）。
2. 最内层循环的迭代间无数据依赖（即无循环携带的依赖，loop-carried dependency）。
3. 内存访问为连续/单位步长（stride-1），或编译器能识别的规律步长。

4. 循环体内无函数调用（或仅为内置纯函数）。
5. 无复杂的控制流（简单 if-else 可能被两种模式处理：全路径执行后选择，或使用掩码）。
6. 无指针别名问题——编译器需要确定不同指针指向不同内存区域。

### 11.5.2　阻止自动向量化的常见模式

- **数据依赖**：迭代 $i$ 的结果被迭代 $i+1$ 读取。例如：`a[i] = a[i-1] + b[i]`。
- **间接索引（gather/scatter）**：`a[idx[i]] = b[i]`——编译器无法确定所有索引互不重叠。
- **非单位步长**：`for(i=0; i<n; i+=2)`——步长不为 1，需要特殊处理。
- **提前退出**：循环体内的 `break` 或 `return` 使迭代数不确定。
- **指针别名不确定**：当 `a` 和 `b` 可能指向重叠区域时，编译器保守地放弃向量化。
- **混合类型**：循环体内频繁转换数据类型会破坏向量化模式。

### 11.5.3　编译器 Hint 机制

- `#pragma omp simd`：指导编译器按 SIMD 方式向量化此循环，忽略其成本模型中的保守估计。
- `__restrict`（或 C99 `restrict`）：向编译器保证特定指针是访问其内存区域的唯一途径，消除别名顾虑。
- `__attribute__((aligned(32)))`：声明数据对齐，使编译器可使用对齐加载指令。
- `#pragma GCC ivdep`：忽略循环携带的向量依赖（程序员保证无真依赖）。

然而，自动向量化有其根本局限——编译器不能改变数据布局，也不能改变算法结构。例如，对三角矩阵的稀疏处理、交错打包的 GEMM 等，自动向量化基本无能为力。先进的推理引擎（如 llamafile、XNNPACK）普遍使用手写 intrinsics 或汇编而非依赖编译器自动向量化。

### 11.5.4　SLP 向量化

超字级并行（Superword Level Parallelism, SLP）向量化是编译器中另一种自动向量化技术。它不向量化循环，而是识别基本块（basic block）内的多个独立标量操作并将其打包（pack）为 SIMD 指令。例如：

```
a0 = b0 + c0;
a1 = b1 + c1;
a2 = b2 + c2;
a3 = b3 + c3;
```

SLP 可将以上 4 条标量加法打包为一条 SIMD 加法。对循环展开后的代码，SLP 尤为有效。

## 11.6 SIMD 优化策略

### 11.6.1 循环向量化的层次

向量化优化应在嵌套循环的多个层次上考虑：

- **最内层循环向量化**：若内层循环本身就是可向量化的（如逐元素加法），直接应用 SIMD 即可。
- **外层循环展开 + 内层向量化**：对于矩阵乘法，内层是归约维度，外层按行或列展开。通过在外层做适度展开（unrolling），在内层维持多个向量化的累加器。
- **多维寄存器块**：如 6x16 微内核，在寄存器级别将两个维度的展开都向量化。

### 11.6.2 交错数据布局

将数据结构从数组结构（AoS, Array of Structures）转换为结构数组（SoA, Structure of Arrays）可以大幅提升 SIMD 效率。例如，将 $\{(x_i, y_i, z_i)\}$ 转为 $\{x_i\}, \{y_i\}, \{z_i\}$ 三个独立数组，使得对 $x$ 分量的操作可以连续向量加载，而非分散的 gather 操作。

### 11.6.3 归约向量化

归约操作（求 sum, max, min 等）需要特殊处理。归约向量化的常见模式是：

1. 使用一个向量累加器，每条迭代执行向量的元素级操作（如向量加法）。
2. 循环结束后，通过水平归约（horizontal reduction）指令将累加器的各通道合并为标量结果。
3. 例如：AVX 的 `_mm256_hadd_ps` 可做两两加法，配合 `_mm256_permute2f128` 实现完整的 8 路归约。

**性能提示**：水平归约（hadd）的执行延迟较高（2-3 周期），应避免在热循环内使用。相比将循环完全展开为标量，向量化归约通常仍能带来 3×-4× 的加速。

### 11.6.4　部分向量化与边界处理

当数据元素数量 $n$ 不是 SIMD 宽度 $W$ 的整数倍时，余下的 $n \bmod W$ 个元素需要单独处理。AVX-512 的掩码操作在此场景尤为便利：

```
// 使用掩码处理最后几个元素
uint16_t mask = (1 << (n % 16)) - 1;  // 仅对余下的通道启用
__m512 result = _mm512_maskz_load_ps(mask, addr);
```

不带掩码的平台上，常见模式是：

```
// 主循环处理 8 的倍数
for (i = 0; i + 7 < n; i += 8) { /* SIMD body */ }
// 尾循环处理剩余 0-7 个元素
for (; i < n; i++) { /* scalar body */ }
```

### 11.6.5　SIMD 性能调试

**SIMD 优化清单**

1. **检查生成的汇编：**使用 `objdump -d` 或 godbolt（Compiler Explorer）确认编译器确实生成了预期的 SIMD 指令。
2. **Profile 以识别热点：**使用 `perf record` / `perf report` 定位计算最密集的函数。
3. **检查 FMA 的合法化：**确认浮点收缩（-ffp-contract=fast）已启用，编译器能将 mul+add 合并为 FMA。
4. **监视线程与 SIMD 结合：**在多线程下，确保每个核心的 SIMD 单元都被有效利用（通过硬件性能计数器 IPC 和 SIMD 利用率）。
5. **注意混用 AVX 和 SSE 的惩罚：**在 AVX 代码路径中混用遗留 SSE 指令会导致「SSE-to-AVX 过渡惩罚」（ 70 周期）。使用 `vzeroupper` 或编译器标志 `-mno-vzeroupper` 管理状态。

# 第十二章　汇编级别优化

## 12.1　为何需要汇编级别优化

### 12.1.1　编译器生成的局限

现代优化编译器（GCC 14, Clang 18, ICC 2024）在自动生成 SIMD 代码方面已经非常成熟，但在处理器微架构的极端调优方面仍存在系统性不足：

- **寄存器分配次优**：编译器基于图着色算法进行寄存器分配，但不能完全利用微架构知识（如哪些寄存器端口共用）。在 GEMM 微内核等需要精确寄存器分配的代码中，编译器常引入不必要的栈溢出（spill），使关键路径的吞吐量降低 15%-30%。

- **指令选择非最优**：编译器可能选择 `vmulps` + `vaddps` 而非 `vfmadd231ps`，尤其在浮点关联性（-fassociative-math）未显式启用时。

- **指令调度欠考虑**：编译器不了解某些特定指令（如跨 128 位 lane 的 permute）在具体微架构上的端口约束，导致流水线停顿。

- **打包/解包（pack/unpack）低效**：量化推理中频繁的 int8 打包/解包操作，编译器的标量代码往往比手工 seq 多了不必要的数据移动。

### 12.1.2　手写汇编的实际收益

实测数据表明，针对 BLIS（类 BLAS 框架）和 OpenBLAS 的对比：关键 GEMM 微内核在纯 AVX2 汇编优化后，矩阵乘法吞吐量比同一编译器（-O3 -march=native）生成的 intrinsics 版本高出 15%-35%。原因集中在更优的寄存器分配和预取调度。

**汇编优化的代价**

手写汇编不具有可移植性——x86-64, ARM64, RISC-V 需要完全不同的实现。调试和维护难度极高。现代实践倾向于使用 intrinsics 覆盖 90% 的热点，仅对最核心的 5%-10% 函数使用汇编实现（通常为矩阵乘法微内核和量化解包装器）。

## 12.2 x86-64 汇编基础

### 12.2.1 寄存器集

x86-64 架构在 64 位模式（long mode）下提供以下寄存器资源：

**通用寄存器：** RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8–R15（共 16 个 64 位寄存器）。这些寄存器可用于地址计算、循环计数和整数运算。

**SIMD 寄存器（AVX2）：** YMM0–YMM15（共 16 个 256 位寄存器）。在 AVX-512 下为 ZMM0–ZMM31（共 32 个 512 位寄存器）——但需注意，操作系统必须启用 AVX-512 状态保存（XSAVE），否则访问高 16 个 ZMM 寄存器会导致故障。

**掩码寄存器（AVX-512）：** K0–K7（8 个 64 位掩码寄存器）。K0 为硬连线全 1，不可被修改，用作「无掩码」的便捷替代。

### 12.2.2 SIMD 指令编码

AVX/AVX-512 指令使用多字节编码方案：

- **遗留 SSE 前缀：** 2-3 字节操作码（66h, F2h, F3h）。
- **VEX 前缀：** 2 字节或 3 字节前缀，编码目标寄存器、源寄存器、立即数和向量长度。AVX/AVX2 使用此方案。
- **EVEX 前缀：** 4 字节前缀（AVX-512），除 VEX 内容外还编码掩码寄存器、压缩/广播模式和舍入控制。

例如,指令 `vfmadd231ps ymm0, ymm1, ymm2` 的机器码为 `C4 E2 75 B8 C2`（VEX.256.66.0F38.W0 B8 /r），编码了操作、三个寄存器和 256 位宽度。

### 12.2.3 FMA 指令变体

AVX/AVX2 的 FMA 指令有三种操作数顺序变体，区别在于哪个源寄存器被覆盖（作为目标）：

| 助记符 | 操作 | 用途 |
|---|---|---|
| `vfmadd132ps D, A, B` | $D \leftarrow D \times B + A$ | 覆盖第一个操作数 |
| `vfmadd213ps D, A, B` | $D \leftarrow A \times D + B$ | 覆盖第二个操作数 |
| `vfmadd231ps D, A, B` | $D \leftarrow A \times B + D$ | 覆盖第三个操作数（累加器） |

在 GEMM 微内核中，累加器是需要在循环中反复更新的寄存器，因此 `vfmadd231ps` 是最自然的变体：累加器作为目标（第三操作数），新乘积作为源。

### 12.2.4　寻址模式

x86-64 支持复杂的寻址模式：

$$\text{addr} = \text{base_reg} + \text{index_reg} \times \text{scale} + \text{displacement} \tag{12.1}$$

其中 scale 可取 1, 2, 4, 8。例如 `vmovaps ymm0, [rax + rcx*4 + 64]` 将基址在 rax、索引在 rcx、步长为 4、偏移 64 的地址处的数据加载到 ymm0。这种复杂的寻址模式在一步中完成了乘法和加法，是 CISC 架构的优势。

## 12.3　调用约定

### 12.3.1　System V AMD64 ABI（Linux/macOS）

- **整数参数：** RDI, RSI, RDX, RCX, R8, R9（最多 6 个参数通过寄存器传递，剩余通过栈）。
- **浮点/SIMD 参数：** XMM0–XMM7（SSE 参数），返回值在 XMM0（浮点）或 RAX（整数）。
- **调用者保存寄存器（Caller-saved）：** RAX, RCX, RDX, RSI, RDI, R8–R11, XMM0–XMM15（YMM0–YMM15 的高 128 位不保证保存）。
- **被调用者保存寄存器（Callee-saved）：** RBX, RBP, R12–R15。若函数需修改这些寄存器，必须将其保存到栈并在返回前恢复。
- **栈对齐：**进入函数时 RSP 必须 16 字节对齐；在调用另一函数前 RSP 必须对齐。
- **红区（Red zone）：**RSP 以下 128 字节的区域，在信号处理期间不会被异步修改，可用于临时存储（仅在叶函数中）。

### 12.3.2　Windows x64 ABI

Windows 使用不同的调用约定：

- 整数参数：RCX, RDX, R8, R9。
- 阴影空间（Shadow space）：在调用指令前，调用者必须在栈上为被调用者的前 4 个参数预留 32 字节，即使参数少于 4 个。
- 被调用者保存寄存器：RBX, RBP, RDI, RSI, R12–R15, XMM6–XMM15。
- 无红区。

**跨平台汇编的注意事项**

编写跨 Linux/Windows 的汇编函数时，最常见的问题就是调用约定不匹配：在 Linux 上通过 RDI 传递的第一个参数，在 Windows 上在 RCX 中。解决方案是使用条件汇编（`%ifdef __WIN64__`）或编写与调用约定无关的 C 封装函数。

## 12.4 寄存器分配技术

### 12.4.1 GEMM 微内核的寄存器需求分析

以 AVX2 上 $6 \times 16$ fp32 微内核为例，进行精确的寄存器需求计算。假设 $\bar{\boldsymbol{C}}$ 的每个元素都需要一个独立寄存器来进行累积：

- $\bar{\boldsymbol{C}}$ 累积寄存器：$M_r = 6$ 行，$N_r = 16$ 列，每行需要 $N_r/8 = 2$ 个 YMM 寄存器。共 $6 \times 2 = 12$ 个 YMM。
- $\bar{\boldsymbol{A}}$ 广播寄存器：1 个 YMM（复用）。
- $\bar{\boldsymbol{B}}$ 加载寄存器：需要同时持有当前行和下一行（支持软件预取和交错），2 个 YMM。
- 可选——前一行 $\bar{\boldsymbol{B}}$ 的寄存器（用于双缓冲）：1 个 YMM。

总计 $12 + 1 + 2 + 1 = 16$ 个 YMM。恰好用完所有 16 个 YMM0-YMM15 寄存器，零溢出。这是设计微内核的最优状态。

若选择 $8 \times 12$ 的块，则需要 $8 \times \lceil 12/8 \rceil = 8 \times 2 = 16$ 个 YMM 仅用于 $\bar{\boldsymbol{C}}$ 累积，无法容纳 $\bar{\boldsymbol{A}}$ 和 $\bar{\boldsymbol{B}}$ 的寄存器，导致不可避免的溢出。这揭示了微内核块形状选择的核心约束：在给定的寄存器资源下，最大化计算密度同时避免溢出。

### 12.4.2 寄存器溢出与填充交换

当需要的虚拟寄存器数量超过物理寄存器数量时，编译器（或程序员）必须将某些值「溢出」到栈上（spill）并在稍后重新加载（fill）。一次 spill-fill 对的延迟约 7-12 周期（L1 缓存命中时），相当于 3 条 FMA 指令的执行时间。在 GEMM 的最内层循环中，每次迭代都发生 spill 会将有效吞吐减半。

手工分配中避免溢出的策略：

- **缩减块大小**：降低 $M_r$ 或 $N_r$ 以减少累加器数量。
- **循环重排**：将部分累积移动到外层循环。
- **预取线程协作**：将数据加载的任务交给帮助线程，主线程只负责计算。

### 12.4.3　寄存器复用调度

不仅仅是寄存器数量，寄存器复用的时机也至关重要。以微内核的内循环为例：

```
; 每个迭代：
vmovaps    ymm15, [rsi]        ; 加载 B 的当前行
vfmadd231ps ymm0, ymm14, ymm15 ; A广播 * B行 + 累加器
vfmadd231ps ymm2, ymm14, ymm15
; ... 更多行 ...

; 在A广播不变的情况下，遍历B的多行
; 然后再更新A广播（广播寄存器复用）
```

通过在外层固定 $\bar{\boldsymbol{A}}$ 的广播值，在内层遍历 $\bar{\boldsymbol{B}}$ 的各行，可以在不变动广播寄存器的情况下发出大量 FMA——最大化广播寄存器的重用周期。

## 12.5　指令调度

### 12.5.1　CPU 流水线与乱序执行

现代 x86 CPU 的流水线大致分为：

1. **前端（Front-end）：**取指令（fetch）→ 解码（decode，包括宏指令融合）→ 送入指令队列。
2. **执行引擎：**重命名（register renaming）→ 分配（allocate）→ 发射到保留站（reservation station）→ 执行单元执行 → 重排序缓冲区（ROB）提交。
3. **乱序窗口：**ROB 大小决定 CPU 可以同时「飞行」的最多未完成指令数（例如 Skylake 约 224 条、Zen 4 约 320 条）。

**FMA 延迟与吞吐：**在 Skylake-X 上，FMA 指令的执行延迟为 4 周期，每周期可发射 2 条 FMA（双 FMA 单元，端口 0 和端口 5）。要在满吞吐下执行，需维持 $4 \times 2 = 8$ 条处于飞行状态的 FMA 指令，即需要至少 8 条独立的 FMA 链（不同目标寄存器，互无依赖）。

### 12.5.2　指令选择技巧

- **FMA 始终优于 Mul+Add：**`vfmadd231ps` 同时完成乘法和加法，仅消耗 1 个 $\mu$op、1 个发射槽和 1 个执行端口。分别执行 `vmulps` + `vaddps` 需要使用 2 个 $\mu$op 和 2 个端口，且中间结果额外消耗了寄存器和 ROB 槽。
- **LEA 代替 ADD+MUL：**`lea rax, [rcx + rdx*8 + 64]` 在 AGU（地址生成单元）中一步完成，不占用 ALU。

- **零化惯用法：** `xor eax, eax` 较 `mov eax, 0` 节省 1 个字节的编码，且被识别为依赖破坏（不影响标志寄存器）。
- **比较惯用法：** `test eax, eax` 或 `cmp eax, 0` 配合条件跳转。

### 12.5.3 循环展开

循环展开（Loop Unrolling）通过复制循环体多次并调整计数器，减少分支预测次数，增加基本块内的指令级并行机会。

对于微内核的 K 维内循环，展开次数由寄存器资源决定。例如，如果每个迭代处理 $K_U$ 个 K 元素的归约，则寄存器压力增加 $K_U\times$（用于容纳更深流水线）。常见的选择是展开 K 维 2-4 次，同时展开 M 和 N 维以匹配寄存器容量。

- **展开因子的权衡：**展开太少 $\rightarrow$ 分支开销大，ILP 不足。展开太多 $\rightarrow$ 代码体积增大，指令缓存（L1i）溢出，反而降低性能。
- **编译器展开与手工展开：**手工展开允许做编译器无法作的选择——例如在打包缓冲区中交错存储来自不同 K 位置的数据以便在展开的循环中连续加载。

### 12.5.4 软件流水线

软件流水线（Software Pipelining）通过交错相邻迭代的加载、计算和存储阶段，最小化流水线停顿。在 GEMM 微内核中，典型的软件流水线模式是：

```
; 迭代0: 预加载 B 行
vmovaps ymm14, [rsi + 0]
vmovaps ymm15, [rsi + 32]

; 主循环:
.L_inner:
    ; 计算当前迭代（使用已加载的 ymm14/15）
    vfmadd231ps ymm0, ymm13, ymm14
    ...
    ; 预加载下一个迭代的 B 行
    vmovaps ymm14, [rsi + rdx + 0]     ; 下一次迭代的数据
    vmovaps ymm15, [rsi + rdx + 32]
    add rsi, rdx                        ; 推进指针
    sub rcx, 1                          ; 递减计数器
    jne .L_inner
```

这种交错允许加载指令在 FMA 指令执行期间访问缓存，两者的内存端口和执行端口互不冲突。

### 12.5.5　硬件预取与软件预取

**硬件预取：**现代 CPU 拥有多种自动预取器：步幅预取器（检测固定步长的顺序访问）、下一行预取器（自动预取下一个缓存行）、流式预取器。对于顺序的大块数据访问（如 GEMM 中的打包矩阵 B），硬件预取器通常工作良好。

**软件预取：**对于硬件预取无法覆盖的模式（如列优先访问的 A 矩阵），程序员可插入显式预取指令：

- `prefetcht0`：预取到所有缓存级别（L1, L2, L3）。
- `prefetcht1`：预取到 L2 及以上。
- `prefetcht2`：预取到 L3 及以上。
- `prefetchnta`：非时序预取（non-temporal），将数据加载为一次性使用，仅进入一个「非时序」缓存路，避免污染常规缓存。

**预取距离：**太近——数据在使用时可能还在传输中。太远——数据可能在到达后被逐出（L1 缓存被后续数据替换）。对于 GEMM 中 B 矩阵的行访问，经验预取距离约为 4-8 个迭代（即约 256-512 字节）。

## 12.6　CISC 特殊指令

### 12.6.1　高速内存复制

- `rep movsb`：在现代微架构（Ivy Bridge+）上，`rep movsb` 被微码（microcode）实现为增强型快速字符串操作，可达接近全带宽的复制速度。对于大于 128 字节的复制，其性能接近手写 AVX 复制循环。
- `rep stosb`：类似的内存填充操作，用于清零（memset(0)）。

### 12.6.2　BMI 位操作指令

BMI（Bit Manipulation Instructions）提供了一些对量化码的位打包/解包极为有用的指令：

- `blsr dst, src`：重置最低设置位 $\rightarrow dst = src \,\&\, (src - 1)$。用于遍历稀疏位集的迭代。
- `blsi dst, src`：隔离最低设置位 $\rightarrow dst = src \,\&\, -src$。
- `tzcnt/lzcnt`：末尾零计数/前导零计数，在量化值的符号-大小表示转换中常用。
- `pext/pdep`：并行位提取/沉积，在 GGUF 解包（如从打包字节提取 5 位字段）中一次处理 64 位的位级操作。

# 12.7 内联汇编与独立汇编

## 12.7.1 GCC/Clang 内联汇编语法

内联汇编（Inline Assembly）允许在 C/C++ 函数中直接嵌入汇编指令：

```
__m256 my_fma(__m256 a, __m256 b, __m256 c) {
    __m256 result;
    asm volatile(
        "vfmadd231ps %[r], %[a], %[b]"
        : [r] "=v" (result)        // 输出操作数
        : [a] "v" (a), [b] "v" (b), "0" (c)  // 输入操作数; "0" 表示与输出0共享寄存器
        :  // 无额外clobber
    );
    return result;
}
```

操作数约束（Constraint）指定了操作数可以位于何处：

- `"r"`：通用寄存器。
- `"m"`：内存操作数。
- `"x"`：XMM 寄存器。
- `"v"`：任何 SIMD 寄存器（XMM/YMM/ZMM）。
- `"i"`：立即数。
- `"=r"`：仅输出的通用寄存器。

**Clobber 列表：**通知编译器哪些寄存器和状态被指令修改。例如，`"cc"` 表示条件码寄存器被修改，`"memory"` 表示内存可能被修改（编译器会将所有内存驻留变量重新加载）。

## 12.7.2 内联汇编的局限性

- 编译器在优化内联汇编语句时的自由度很低。`asm volatile` 禁止编译器移动或删除该语句，但也会抑制周围的优化。
- 没有 `volatile` 的非 volatile asm 可能被编译器消除（如果其输出未被使用）或与其他代码重排——需要显式指定依赖。
- 在函数内使用过多内联汇编会使寄存器分配器的工作变得困难，可能导致非 asm 代码部分的溢出。

### 12.7.3　独立汇编文件（NASM / GAS）

对于大规模的汇编优化（如完整的 GEMM 微内核），更推荐使用独立的汇编文件：

- **NASM（Netwide Assembler）**：Intel 语法，与 Intel/AMD 的汇编风格一致。示例：

```
section .text
global my_kernel
my_kernel:
    vmovaps ymm0, [rdi]
    vfmadd231ps ymm0, ymm1, [rsi]
    ret
```

- **GAS（GNU Assembler）**：AT&T 语法，与 GCC 工具链天然集成。示例：

```
.globl my_kernel
my_kernel:
    vmovaps (%rdi), %ymm0
    vfmadd231ps (%rsi), %ymm1, %ymm0
    ret
```

两者通过汇编器（NASM: `nasm -f elf64`，GAS: `as -o`）生成目标文件（.o），随后与 C/C++ 编译单元链接。

## 12.8　汇编优化实践

**汇编优化的典型工作流程**

1. **确定目标函数**：通过 profiling（`perf record`）识别总运行时间占比最高的函数。
2. **深入研究编译器输出**：`objdump -d -M intel ./binary` 分析编译器生成的汇编，找出明显的低效模式（不必要的溢出、缺失的 FMA、差的指令调度）。
3. **写 C intrinsics 原型**：使用 intrinsics 写一个最优化版本，作为功能和性能基准。
4. **用汇编重写**：在独立汇编文件中用纯汇编重写，手动控制寄存器分配和指令顺序。使用宏（NASM `%macro`）管理重复展开模式。
5. **微基准测试（Micro-benchmark）**：在隔离的条件下（固定矩阵尺寸，绑定单个核心）测试汇编版本的吞吐量和延迟，与 intrinsics 版本做对比。
6. **集成测试**：将汇编函数集成到主推理管线中，在真实模型（如 LLaMA 7B）上测量端到端延迟和吞吐变化。

7. **持续维护**：为每个汇编函数编写功能测试（对比参考 C 实现的输出），确保在不同编译器版本和 CPU 微架构下产出一致的结果。

**安全与正确性**

手写汇编中常见的 bug 包括：

- 未保存被调用者保存寄存器（callee-saved），在返回后破坏了调用者的上下文。
- 栈未对齐（misaligned stack），在调用 printf 等 C 函数时触发 segfault（仅在传入 SIMD 参数时触发，难于调试）。
- 未正确处理负向栈增长——在函数开始时 `sub rsp, N`，在返回前 `add rsp, N`。
- YMM/ZMM 状态未清理——在 AVX 代码后调用 SSE 遗留函数前需要 `vzeroupper`，否则遗留代码可能误读高 128 位。

使用 Valgrind、AddressSanitizer 和 GDB 等工具进行日常测试至关重要，但需要注意的是某些工具对 AVX/AVX-512 指令的模拟存在局限。

# 第十三章　缓存与访存优化

## 13.1　缓存层次结构深度剖析

### 13.1.1　现代 CPU 缓存架构

处理器与主存之间的速度鸿沟（memory wall）是高性能计算的持久挑战。现代 CPU 通过多级缓存层次来缓解这一差距，每级缓存在容量、延迟和带宽之间做出不同权衡。

**缓存层次典型参数**

以 Intel Xeon Platinum 8480+（Sapphire Rapids）为例的缓存层次：

| 缓存级 | 典型容量 | 相联度 | 缓存行大小 | 命中延迟 |
|---|---|---|---|---|
| L1 数据缓存（L1d） | 48 KB/核 | 12 路 | 64 字节 | 4–5 周期 |
| L1 指令缓存（L1i） | 32 KB/核 | 8 路 | 64 字节 | 4–5 周期 |
| L2 缓存 | 2 MB/核 | 16 路 | 64 字节 | 14–16 周期 |
| L3 缓存（LLC） | 105 MB 共享 | 15 路 | 64 字节 | 45–55 周期 |
| 主存（DDR5） | 512+ GB | – | 64 字节（总线） | 90–110 ns（含排队） |

**缓存行的角色：**每一级缓存以 64 字节的固定大小行（cache line）为单位管理数据。即使程序仅访问单个 4 字节浮点数，整个 64 字节行都将被加载到缓存中。这一空间局部性机制意味着，连续的内存访问几乎「免费」地利用了同一缓存行中的相邻数据。

### 13.1.2　缓存定位与索引

缓存通常采用组相联（set-associative）映射。对于 $S$ 组、$W$ 路（way）的缓存，物理地址的中间位决定了组索引（set index），低 6 位为行内偏移。每个组可以容纳 $W$ 个缓存行；当第 $W+1$ 个映射到同一组的行到来时，最久未使用（LRU）的行被逐出（evicted）。

**缓存颠簸（Cache Thrashing）：**在 GEMM 这类操作中，若矩阵的行长（leading dimension）恰好是缓存组数的倍数（例如，ld = 4096 = 64K 字节，在 8 路 32KB L1 缓存中），则同列不同行的元素频繁映射到同一组，导致非期望的逐出——吞吐可下降 3×–10×。应对措施包括在行维上增加填充（padding）使其步长稍微大于 2 的幂。

### 13.1.3 TLB：转换后备缓冲区

TLB（Translation Lookaside Buffer）缓存虚拟地址到物理地址的转换，分为两级：

- **L1 TLB**：小容量（64 项典型），全相联或高组相联，延迟约 1 周期。
- **L2 TLB**：中等容量（2048 项典型），16 路组相联，延迟约 4-8 周期。
- **TLB 未命中**：触发硬件页表遍历（page-table walk），访问 4 级页表（每个级别可能需一次内存访问），额外增加 30-100 周期的延迟。

对于大规模张量（如 $4096 \times 4096$ 的 fp16 矩阵：$4096 \times 4096 \times 2 = 32$ MB），若使用 4KB 页面，需 32MB/4KB = 8192 个页面——远超 L2 TLB 的容量（仅能覆盖约 8MB），因此必须使用 2MB 或 1GB 的大页（见第13.7节）。

### 13.1.4 缓存一致性协议

在多核共享内存系统中，MESI 协议（及其衍生：MESIF, MOESI）维护各核心私有缓存间的一致性。四个状态为：

- **M**odified：该缓存行的值已被本核心修改，与主存不一致。
- **E**xclusive：该缓存行的值与主存一致，且仅本核心拥有该副本。
- **S**hared：该缓存行可能被多个核心共享，值与主存一致。
- **I**nvalid：该缓存行无效。

在推理场景中，若一个核心写入输出张量而另一核心读取中间结果，一致性协议确保写入可见。然而，一致性流量（snoop traffic）可能成为多线程推理的带宽瓶颈。

### 13.1.5 非时序存储

对于写入目标且后续不再读取的数据，可以使用非时序（non-temporal）存储指令绕过缓存，直接写入主存。这避免了缓存中「读取所有权」（Read-for-Ownership, RFO）的浪费，且不污染缓存（将这些行逐出不会影响后续的热数据访问）。

- `_mm256_stream_si256()`：非时序 256 位对齐写入。
- `movntdq [addr], xmm/ymm/zmm`：底层指令。

在 GEMM 中，输出矩阵 $\boldsymbol{C}$ 可以被非时序存储（因为一般在写完一个子块后不会立即重新读取它），而 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 矩阵作为输入应加载到缓存中。

## 13.2　缓存分块（Tiling/Blocking）

### 13.2.1　分块原理

分块（Blocking 或 Tiling）是应对「数据量远超缓存容量」这一矛盾的核心技术。以矩阵乘法 $\boldsymbol{C} = \boldsymbol{A} \times \boldsymbol{B}$ 为例，朴素的三重循环每次内维迭代都从主存重新加载数据（或从缓存中驱逐之前加载的数据）。分块将矩阵分解为可放入缓存的子块，在子块内完成尽可能多的计算后才移至下一子块。

**矩阵乘法的缓存分块**

设 $\boldsymbol{A}(M \times K)$ 被划为 $m_r \times k_c$ 的块，$\boldsymbol{B}(K \times N)$ 被划为 $k_c \times n_r$ 的块，$\boldsymbol{C}(M \times N)$ 被划为 $m_r \times n_r$ 的块。则：

$$\bar{\boldsymbol{C}}_{i,j} \mathrel{+}= \bar{\boldsymbol{A}}_{i,p} \times \bar{\boldsymbol{B}}_{p,j} \tag{13.1}$$

其中每个 $\bar{\boldsymbol{C}}$ 块由 $K/k_c$ 次块矩阵乘法（GEPP: General Panel-Panel multiply）累加得到。通过使 $\bar{\boldsymbol{A}}_{i,p}$、$\bar{\boldsymbol{B}}_{p,j}$ 和 $\bar{\boldsymbol{C}}_{i,j}$ 的总大小适配特定缓存层次，可最小化对主存的访问。

### 13.2.2　分层分块策略：L3 → L2 → L1

现代高性能 GEMM（如 BLIS 框架）采用多层嵌套分块策略，将数据重用的层次与缓存层次精确对齐：

**L3 层次分块：** $\bar{\boldsymbol{B}}_{p,j}$ 的整块（$k_c \times n_r$）应适配 L3 缓存。在遍历外部 M 维时，$\bar{\boldsymbol{B}}_{p,j}$ 被多次重用（$M/m_r$ 次），将其保持在 L3 中避免了重复从主存加载。

**L2 层次分块：** $\bar{\boldsymbol{A}}_{i,p}$ 的块（$m_r \times k_c$）应适配 L2 缓存。在更新 $\bar{\boldsymbol{C}}_{i,j}$ 时，$\bar{\boldsymbol{A}}_{i,p}$ 中将被遍历 $N/n_r$ 次。

**L1 层次：微内核（micro-kernel）：**最内层计算使用非常小的寄存器块 $\bar{\boldsymbol{C}}_r$（$6 \times 16$ 如第11.3节所述），其完全驻留在寄存器中。微内核从 L1 加载打包的 $\bar{\boldsymbol{A}}$ 和 $\bar{\boldsymbol{B}}$ 微面板（micro-panels）。

**分层分块中每一层的动机**

| 层级 | 分块维度 | 为何在此缓存级别 |
|---|---|---|
| L3（或主存） | $k_c \times n_r$（B 面板） | 在外部 M 循环中重用。若 $k_c \times n_r$ 足够小，可完全驻 |
| L2 | $m_r \times k_c$（A 面板） | 在外部 N 循环中重用。若 $m_r \times k_c$ 适配 L2，则 A 面 |
| L1 | $m_r \times k_u$ 和 $k_u \times n_r$（微面板） | 在内层 K 循环中，这些数据被直接送入 SIMD 寄存 |
| 寄存器 | $m_r \times n_r$（微内核 C） | 最终累积，完全在寄存器内完成。 |

### 13.2.3 分块大小的解析模型

给定缓存大小 $C$，块大小的选择决定了缓存失效率。以将 $\boldsymbol{B}$ 保持在 L3 缓存中为例，条件为：

$$k_c \cdot n_r \cdot \text{element_size} \leq C_{L3} \cdot \alpha \tag{13.2}$$

其中 $\alpha \in [0.5, 0.75]$ 是留给他用（如 $\boldsymbol{A}$ 和 $\boldsymbol{C}$ 的片段、操作系统和其他进程）的容量因子。

同样，$\bar{\boldsymbol{A}}$ 块适配 L2 的条件：

$$m_r \cdot k_c \cdot \text{element_size} \leq C_{L2} \cdot \alpha \tag{13.3}$$

在实际调优中，解析模型仅提供初步估计——最佳块大小需要通过硬件性能计数器（如 `perf stat -e cache-misses,LLC-load-misses`）进行微调确认。

### 13.2.4 自动调优（Auto-tuning）

系统地搜索最优的分块参数是一项计算上可处理的网格搜索问题。给定参数空间 $(m_r, n_r, k_c)$，每个参数候选运行 $20 \times 20 \times 20 = 8000$ 种组合在几分钟内完成（利用 CPU 性能计数器反馈）。ATLAS（Automatically Tuned Linear Algebra Software）是最早实现此方法的系统，而 BLIS 则通过手动建模和专家知识提供接近最优的默认参数。

## 13.3 循环变换

### 13.3.1 循环交换

循环嵌套的顺序决定了访存模式。朴素矩阵乘法的三种循环顺序：

- `i-j-k`（C 行优先，A 行、B 列）：对 $\boldsymbol{C}$ 和 $\boldsymbol{A}$ 为 stride-1 访问，但 $\boldsymbol{B}$ 为列访问（步长 =N），在每步中都可能导致缓存未命中。
- `i-k-j`（A 行，B 行，C 行）：A 和 B 都是 stride-1，最优。
- `j-k-i`（B 列，A 列，C 列）：所有矩阵都是非单位步长，性能极差。

循环交换（Loop Interchange）将最内层循环置为连续访存的维度。通过 `#pragma GCC ivdep` 或等效约束保证交换正确性。

### 13.3.2 循环分块（循环 Tiling）

循环分块将一个循环分割为两个嵌套循环：外层按固定步长遍历（tile loop），内层在一个块内遍历（scan loop）。块大小 $B$ 被选择为匹配缓存容量。例如，将 $\boldsymbol{C}$ 的写操作分块：

```
for (ii = 0; ii < M; ii += Bm)
  for (jj = 0; jj < N; jj += Bn)
    for (kk = 0; kk < K; kk += Bk)
      // 在此块内执行原循环
      for (i = ii; i < min(ii+Bm, M); i++)
        for (j = jj; j < min(jj+Bn, N); j++)
          for (k = kk; k < min(kk+Bk, K); k++)
            C[i*N + j] += A[i*K + k] * B[k*N + j];
```

### 13.3.3　循环展开与循环融合/裂变

- **循环展开（Unrolling）**：如第12.5节所述，通过复制循环体减少分支开销。

- **循环融合（Fusion）**：将两个连续循环（通常逐元素操作不同数组）合并为一个，减少遍历次数。在推理管线中，将卷积后的批量归一化和 ReLU 激活融合到一个循环中可以降低对中间缓冲区的访存。

- **循环裂变（Fission / Distribution）**：将一个结构复杂、混用不同访存模式的循环拆分为多个简单循环。理由是：简单循环对指令缓存更友好，且更可能被编译器自动向量化。

- **循环歪斜（Skewing）**：通过坐标变换使特定维度的访问模式线性化，适用于像卷积这样的操作。

## 13.4　矩阵数据布局

### 13.4.1　行主序与列主序

矩阵在内存中的平铺顺序决定了访问模式的缓存友好性：

- **行主序（Row-major）**：$\boldsymbol{A}[i][j]$ 在内存中是连续的（stride-1）。适合逐行处理的算法。C/C++ 默认。

- **列主序（Column-major）**：$\boldsymbol{A}[i][j]$ 在内存中非连续（stride = 行数）。Fortran/MATLAB 默认。

在 GEMM 中，若 $\boldsymbol{A}$ 为行主序而 $\boldsymbol{B}$ 为列主序，则两者的内积访问都是 stride-1——这是最高效的配置。若两者均为行主序，则 $\boldsymbol{B}$ 的列访问将导致缓存和 TLB 未命中，性能可能下降 10×。

### 13.4.2 Z 序（Morton Order）

Z 序（Morton order）将矩阵递归地划分为四象限，按深度优先的顺序存储，使得空间上相邻的元素在内存中也倾向于相邻。其地址位交错的公式为：

$$\text{morton}(x, y) = \sum_k [(x_k \ll (2k+1)) \mid (y_k \ll 2k)] \tag{13.4}$$

Z 序在矩阵乘法中提供了比行列主序更好的分层空间局部性（跨多个尺度），但由于寻址开销较高，在推理引擎中使用不多。

### 13.4.3 分块/打包布局

**打包（Packing）**技术是高性能 GEMM 的关键步骤——在进入微内核之前，将原始矩阵的块按照微内核期望的格式「打包」为连续缓冲区。

例如，将 $\bar{\boldsymbol{A}}_{i,p}$（一个 $m_r \times k_c$ 的块）打包为以「微面板」（micro-panel）为单位的布局——在微面板内部，按列排列使得每个微内核迭代可以连续加载 A 的一个列的元素。

打包的开销是 $O(M \times K + K \times N)$ 的数据移动，但其被分摊到 $O(M \times N \times K)$ 的计算量上。当 $K$ 足够大时，打包的开销可被完全隐藏。

**矩阵 A 的打包过程**

**输入：**原始矩阵 $\boldsymbol{A}$（$M \times K$，行主序），微内核参数 $m_r, k_c$。
**输出：**打包缓冲区 packed_A。

1. 分配打包缓冲区：大小 $M \times k_c \times$ elem_size（按微面板布局）。
2. 对每个 $M$ 方向上的块（步长 $m_r$）：
   (a) 对块内每一行 $i = 0, \ldots, m_r - 1$：
      i. 从 $\boldsymbol{A}$ 中提取行的 $k_c$ 个连续元素。
      ii. 按微内核的「列优先」顺序将该行的 $k_c$ 个元素交错地写入打包缓冲区的对应位置。
3. 打包后的布局：同一微面板的所有元素连续，在微面板内数据布局与微内核的数据访问模式精确匹配。

## 13.5 预取技术

### 13.5.1 硬件预取器

现代 CPU 在没有任何编程提示的情况下即可自动预取顺序访问的数据。x86 CPU 的常见预取器包括：

- **下一行预取器（Next-line prefetcher）**：当访问缓存行 $X$ 时，自动预取行 $X+1$。在 stride-1 遍历中效果最优。
- **步幅预取器（Stride prefetcher）**：检测固定步长的访问模式，预取 $X + s \times \text{stride}$。
- **流式预取器（Stream prefetcher）**：检测大块连续访问，预取多条缓存行。

对于打包矩阵（内存中已按顺序排布的块），硬件预取器对 80%-90% 的访存模式有效。但在以下场景中无效：

- 非连续、随机或稀疏的访问（如从 $\boldsymbol{A}$ 中非连续地读取列）。
- 步长超过一个页面（4KB），因为物理地址未必连续（虚拟地址连续 $\not\Rightarrow$ 物理地址连续）。

### 13.5.2　软件预取

当硬件预取无法覆盖时，程序员可通过软件预取指令显式提示。GCC 内建：

```
__builtin_prefetch(&addr, 0, 3);  // 预取到所有缓存级，针对读访问
```

或使用 intrinsics：`_mm_prefetch(char const* p, int hint)`，其中 `hint` 指定预取目标缓层和时序类型（非时序/时序）。

**预取距离的经验公式**

设所需数据的到达延迟为 $L$ 周期（L3 约 40 周期、主存约 200 周期），每迭代耗时 $T$ 周期。则预取应提前 $\tau = \lceil L/T \rceil$ 个迭代。在 GEMM 内循环中，常预取下一个 $\bar{\boldsymbol{B}}$ 微面板——距离约 4-8 个缓存行。

**距离过大**：预取的数据在真正使用前可能已被后续的缓存逐出替换，白白消耗了缓存和带宽。

**距离过小**：数据在使用时仍在传输中，CPU 需停顿等待（load-store ordering buffer 满）。

### 13.5.3　GPU 上的预取

在 GPU（CUDA）上，从全局内存到共享内存的异步拷贝（`cuda::memcpy_async`，CUDA 11+ 引入）提供了类似软件预取的机制。通过流水线化数据传输和计算，可以在线程组（warp）执行当前块的计算时，后台 DMA 引擎自动将下一个块的数据从全局内存迁入共享内存。

# 13.6 NUMA 优化

## 13.6.1 NUMA 架构

多路服务器（dual-socket）系统中，每个 CPU 插槽有自己直接连接的本地内存。访问本地内存延迟约为 80-100ns（DDR5），而跨插槽访问远程内存延迟为 150-200ns（需经过 QPI/UPI 互联）。这一非对称性称为 NUMA（Non-Uniform Memory Access）。

在数据并行（data-parallel）的推理中，将推理进程/线程绑定到模型权重复制所在的 NUMA 节点至关重要。错误的绑定可导致 1.5×-2× 的吞吐下降。

## 13.6.2 First-Touch 策略

Linux 默认的内存分配是懒惰的（lazy allocation）——首次写入（而非 mmap 时）才真正分配物理页面，且分配在首次写入线程所在的 NUMA 节点上。因此，初始化数据时应由后续将访问该数据的工作线程执行。如下模式：

```
// 错误：在主线程中初始化，所有页面分配到主线程所在的NUMA节点
float* data = (float*)malloc(N * sizeof(float));
memset(data, 0, N * sizeof(float));  // first-touch!

// 正确：用OpenMP在目标线程上初始化
# pragma omp parallel for
for (int i = 0; i < N; i++) data[i] = 0.0f;
```

## 13.6.3 numactl 与 libnuma

- `numactl --cpunodebind=0 --membind=0 ./inference_server`：将进程的 CPU 和内存绑定到 NUMA 节点 0。
- `numactl --interleave=all`：将内存页面以轮替方式在所有节点上分配——适合被所有线程均匀访问的共享数据。
- libnuma API：`numa_alloc_onnode()` 在指定节点上分配内存，`numa_run_on_node()` 绑定当前线程到指定 NUMA 节点。

**NUMA 与推理部署**

在实际的 LLM 推理服务部署中，推荐策略如下：

1. 将模型权重复制到每个 NUMA 节点的本地内存（replication）。复制后，每个节点独立服务一部分请求（数据并行推理）。
2. 若权重太大无法复制（单份权重 95GB，两个节点需要 190GB），则使用交错分配（interleave）——让权重均匀分布在所有节点上，每个访问均有 50% 的概率

为远程访问。交错分配的均摊远程访问比随机分配（可能 100% 远程）更为可预测。

3. 对 KV 缓存等动态数据，严格遵循 first-touch 策略。

## 13.7　页面大小优化

### 13.7.1　4KB 与巨页

默认的虚拟内存页面大小为 4KB。对于 GB 级别的大模型权重，4KB 页面意味着数百万个 TLB 条目——而 L2 TLB 仅可缓存约 2000 个条目。当 TLB 未命中率超过 1% 时，TLB 错失的开销（每次 30-100 周期）将显著拖慢推理速度。

巨页（Huge Pages）提供更大的页面尺寸：

- **2MB 巨页**：页面大小 $2^{21}$ 字节，是 4KB 的 512 倍。对于 32MB 的权重张量，仅需 16 个 TLB 条目（而非 8192 个）。
- **1GB 巨页**：页面大小 $2^{30}$ 字节，进一步减少 TLB 压力。但需要连续的 1GB 物理内存，分配可能失败。

### 13.7.2　透明巨页的局限性

Linux 的透明巨页（Transparent Huge Pages, THP）会自动尝试将 4KB 页面合并为 2MB 巨页。然而，THP 存在延迟问题——页面合并操作（khugepaged 线程）可能在任何时刻触发，在延迟敏感的推理服务中这可能引入不可预测的延迟尖刺。此外，THP 不能保证所有大分配都被合并。

### 13.7.3　显式巨页分配

**mmap 方式：**

```
void* ptr = mmap(NULL, size_mb * 1024 * 1024,
                 PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB,
                 -1, 0);
```

**hugetlbfs 方式**：挂载 hugetlbfs 文件系统，在挂载点上创建文件后 mmap 该文件——内存被映射为 2MB/1GB 巨页。

**预分配巨页（系统级）**：`sysctl vm.nr_hugepages=4096` 预留 4096 个 2MB 巨页（共 8GB），确保后续的巨页分配不会因内存碎片而失败。

**推荐实践的总结**

在现代 LLM 推理引擎中，推荐对所有模型权重和使用高频的 KV 缓存池启用显式 2MB 巨页分配。对于权重过度分散（如 GGUF 逐块量化格式）的场景，可以退化为 madvise(`MADV_HUGEPAGE`) 提示内核尝试合并。基准测试应使用 `perf stat -e dTLB-load-misses,iTLB-load-misses` 等硬件计数器验证 TLB 压力是否得到缓解。

## 13.8 访存优化的整体图景

**从缓存视角看推理引擎的访存流程**

在一次典型的 Transformer 前向推理中，内存访问呈现如下层次化模式：

1. **权重（只读，多次重用）：**权重按分块方案从主存进入 L3，在多个连续的前向传播中被重复使用。L2/L1 中的打包 A/B 微面板按需加载。
2. **激活（读写，单次使用）：**每层输出为下一层的输入。激活值偏向于流式（streaming）——即用即弃。若模型并行或各层串行执行，激活可以完全保持在 L2/L3 中，不对主存形成持续压力。
3. **KV 缓存（读写，跨 token 重用）：** K 和 V 矩阵随着序列长度的增长线性扩展，最终溢出缓存。在长序列场景下，KV 缓存访问常成为端到端瓶颈——此时 KV 缓存量化（第10.8节）和 KV 缓存的预取调度至关重要。
4. **临时缓冲区：**用于存储中间矩阵乘法结果和 softmax 的临时块应尽可能小，通过即时融合（operator fusion）来消除中间内存往返。

# 第十四章　JIT 编译

## 14.1　JIT 编译基础

### 14.1.1　JIT 在推理中的角色

JIT（Just-In-Time）编译是动态生成与编译代码并在运行时立即执行的技术。在推理引擎中，JIT 解决了一个根本矛盾：模型的计算图在编译期已知（层类型确定），但其具体的张量形状（batch size、序列长度、特征维度）和数据类型会因输入而变化。AOT（Ahead-of-Time）编译必须预生成所有可能组合的核函数变体，导致代码体积膨胀（组合爆炸）；而 JIT 编译仅在实际执行时，为当前确切的形状和类型生成专用代码。

**推理 JIT 的核心价值**

- **形状特化（Shape specialization）：**循环边界和分块参数被固化为常量，编译器可进行更激进的循环展开和常量折叠。
- **消除运行时分支：**数据类型分发、对齐检查等条件判断在编译时确定，生成代码中无分支。
- **零开销抽象：**编译时内联所有辅助函数和索引计算，运行时仅保留必要的算术和访存指令。
- **按需生成：**仅生成当前运行所需的核函数变体，避免组合爆炸。

### 14.1.2　AOT 与 JIT 的对比

| 特性 | AOT 编译 | JIT 编译 |
|---|---|---|
| 编译时机 | 构建时 | 运行时，首次执行前 |
| 形状特化 | 取决于模板/预生成 | 完全特化到具体形状 |
| 代码体积 | 可能很大（多组合） | 仅生成所用变体 |
| 预热延迟 | 无 | 首次 JIT 编译有延迟 |
| 编译器优化 | 完全（-O3） | 取决于 JIT 后端（LLVM 优化或轻量） |
| 可移植性 | 架构特定 | 在目标机器上为当前架构生成 |
| 缓存机制 | 文件系统 | 内存 + 磁盘缓存 |

**典型折衷：**对于形状已知、频率高的操作（如 GEMM），预编译手写微内核仍是最优；对于形状多变、频度低的操作（如 attention mask 生成、多头重组），JIT 编译可在不损失效率的前提下大幅简化代码。

## 14.2 JIT 编译架构

### 14.2.1 推理引擎中的 JIT 管线

一个完整的 JIT 编译管线包含七个步骤：

**JIT 推理核函数编译流程**

**输入：**操作类型（op_type）、张量描述符（形状、数据类型、步长）。
**输出：**编译后的核函数指针（compiled kernel function pointer）。

1. **图分析（Graph Analysis）：**从高层 IR 中提取要执行的操作，推算所有中间张量的形状（shape inference）。
2. **模板选择/生成（Template Selection）：**为当前操作类型选择或生成对应的核函数模板。
3. **参数特化（Parameter Specialization）：**将模板中的符号参数（$M$, $N$, $K$, dtype, unroll_factor, tile_size）替换为具体数值。
4. **代码生成（Code Generation）：**将特化后的模板编译为机器码。后端可能是 LLVM IR、直接机器码或中间表示。
5. **缓存查询（Cache Lookup）：**在编译缓存中查找（key = hash(op_type, shapes, dtypes, parameters)）是否有已编译的版本。
6. **首次编译或缓存命中：**若命中则返回缓存的函数指针；否则执行编译、缓存并返回。
7. **执行（Execution）：**调用核函数，传入输入/输出张量的数据指针。

### 14.2.2 JIT 代码缓存管理

JIT 编译本身是有开销的（从毫秒级到秒级，取决于编译器后端），因此缓存至关重要。
**缓存键设计：**缓存键需唯一标识核函数的所有编译期决定参数：

$$\text{key} = \text{hash}(\text{op_type}, \{M, N, K, \ldots\}, \text{dtype_a}, \text{dtype_b}, \text{tile_params}, \text{arch_features}) \tag{14.1}$$

其中 `arch_features` 包含 CPU 特性（如 AVX2、VNNI 支持等），确保不同机器上不会使用错误的编译版本。

**缓存策略：**

- **内存缓存（LRU）：**限容量的 LRU 淘汰策略，避免 JIT 代码膨胀消耗过多内存。
- **磁盘持久缓存：**将编译后的二进制保存到磁盘，跨进程重用。XLA 的 AOT 缓存和 NVRTC（CUDA）均采用此方式。
- **预编译（Warmup）：**对已知的常见形状组合（如 128, 256, 512, 1024, 2048 的序列长度），可提前在初始化阶段编译以消除首次推理的 JIT 延迟。

## 14.3　编译后端

### 14.3.1　C 级 JIT

最基础的 JIT 方式是运行时生成 C/C++ 源码，调用系统编译器（gcc/clang）编译为动态库（.so/.dylib），再动态加载。

- **优点：**充分利用编译器的所有优化（-O3），支持任何 C/C++ 特性。
- **缺点：**编译延迟极高（500ms-5s），不适合交互式推理场景。
- **适用：**离线批量优化（如 TVM 的 AOT 编译流程）或实验性质的原型验证。

### 14.3.2　LLVM JIT（ORC JIT）

LLVM 项目提供的 ORC JIT（On-Request Compilation JIT）是 C/C++ 程序中最成熟的 JIT 框架。使用流程：

1. 构造 LLVM IR 模块（Module），在其中定义核函数（使用 LLVM IRBuilder API 或直接从 IR 字符串解析）。
2. 将模块提交到 ORC JIT 的编译层（CompileLayer）。
3. ORC JIT 将 LLVM IR 优化（Optimization passes & machine code generation）并生成可执行代码。
4. 通过符号查找获取函数指针：

```
auto sym = jit->lookup("my_kernel");
auto* fn = (void(*)(float*, float*, float*, int, int, int))sym.getAddress();
fn(C, A, B, M, N, K);
```

LLVM JIT 的编译延迟通常在几毫秒到几十毫秒（取决于 IR 复杂度和优化级别），优化质量接近 AOT 编译。这是 XLA、TVM 等框架的主要 CPU 后端。

### 14.3.3 直接机器码生成

在极端追求启动速度和代码体积的场景（如移动端、浏览器 WASM 环境），可以使用直接机器码生成——将预格式化的指令模板中的操作数替换为具体值，直接写入可执行内存。

- **ASMJIT（C++ 库）**：提供了 x86/x64 架构的轻量 JIT 汇编 API。示例：

```
asmjit::JitRuntime rt;
asmjit::CodeHolder code;
code.init(rt.environment());
asmjit::x86::Assembler a(&code);

a.vmovaps(asmjit::x86::ymm0, asmjit::x86::ptr(asmjit::x86::rdi));
a.vfmadd231ps(asmjit::x86::ymm0, asmjit::x86::ymm1,
              asmjit::x86::ptr(asmjit::x86::rsi));
a.ret();

void (*fn)(float*, float*, float*) = nullptr;
rt.add(&fn, &code);
fn(A, B, C);
```

- **dynasm（LuaJIT 项目）**：最小化的预处理器式汇编器，以宏形式在 C 中嵌入机器码生成。极为轻量（单个头文件），但需深入理解指令编码。

**优势**：编译延迟极低（微秒级）——本质上只是将指令参数填入模板并写入内存。**劣势**：无高级优化、非可移植、需为每个架构编写独立的代码生成器。XNNPACK 和 QNNPACK 广泛采用直接机器码生成。

### 14.3.4 编译后端选择指南

**后端选择权衡**

| 后端 | 编译延迟 | 优化质量 | 可移植性 |
|---|---|---|---|
| C 系统编译器 | 500ms–5s | 极高 | 高（任何架构） |
| LLVM ORC JIT | 5–50ms | 高 | 中（LLVM 支持的架构） |
| ASMJIT | 1–50$\mu$s | 无 | 低（x86/x64） |
| 直接机器码 | $\sim$1$\mu$s | 无 | 极低（目标单一） |

选择原则：对于频繁但形状固定的操作，使用预编译的手工微内核（零 JIT 开销）；对于形状多变的操作，使用 LLVM JIT 以获得良好的优化；对于启动延迟极为敏感的

场景（如边缘设备上的 model serving），直接机器码生成是最佳选择。

## 14.4　核函数代码生成

### 14.4.1　可特化的参数

JIT 核函数的可特化参数包括但不限于：

- **矩阵维度：** $M, N, K$ 被特化为常量后，编译器可以计算精确的循环行程、消除边界判断、确定循环展开因子。
- **数据类型：** fp32, fp16, bf16, int8。特化后可以消除类型转换分支，生成针对该类型的专有 SIMD 指令序列。
- **首维步长（lda, ldb, ldc）：** 列主序或行主序的各种组合。
- **SIMD 宽度：** 在目标 CPU 上检测到的实际可用指令集（AVX2 vs AVX-512 vs NEON）。
- **分块参数：** $m_r, n_r, k_c$ 等微内核参数。
- **特殊标志：** 矩阵是否转置、是否需要 bias 加法、是否需要 ReLU 融合。

### 14.4.2　常量传播的价值

当矩阵维度被作为编译时常量嵌入核函数时，编译器可进行深度的常量传播和死代码消除。以 $M = 1$ 的 GEMV 为例：

- 外层 M 循环被完全消除（仅 1 次迭代）。
- 累加器从二维退化为一维，寄存器需求减半。
- 无需处理奇数 M 的尾循环。

**极端示例：** 将 $1 \times 4096$ 的 GEMV 特化后，代码从通用 6x16 微内核（ 100 条指令）缩减为仅加载一行权重、逐向量的 FMA 累积（ 20 条指令）——专用版本比通用版本快 2×-3×。

### 14.4.3　寄存器分配在 JIT 时的优势

在 AOT 编译中，编译器不知道矩阵的实际大小，只能为最坏情况（大 M 和大 N）分配寄存器，导致在 GEMV 场景中大量寄存器被浪费（分配给了不存在的行）。在 JIT 编译时，M 和 N 已被知悉——代码生成器可以根据实际所需的寄存器数量精确分配，避免虚假溢出。

# 14.5 基于模板的 JIT

## 14.5.1 XNNPACK 风格的模板 JIT

XNNPACK（Facebook/Meta 开发的移动端推理库）广泛使用了基于 C 宏的模板化 JIT。其核函数写作参数化的 C 宏：

```
#define XNN_IGEMM_MR 4
#define XNN_IGEMM_NR 8
#define XNN_IGEMM_KR 1
#define XNN_IGEMM_FUSED_OP  FUSED_RELU
#define XNN_IGEMM_ARCH_SIMD AVX2

// 核函数体使用了这些宏——循环边界、SIMD指令、后处理
// (代码省略——一个展开的微内核)
```

在运行时，XNNPACK 根据实际的操作需求设置这些宏定义，然后编译该源文件（使用微 JIT 编译器（micro-JIT compiler））生成具体的核函数。生成的.o 文件被动态链接到进程中执行。微 JIT 编译使用精简版的 C 编译器（如 TCC，Tiny C Compiler）或直接机器码生成器以最小化编译延迟。

## 14.5.2 缓存键与函数调度表

XNNPACK 维护一个全局的核函数缓存表（哈希映射）：

$$\text{cache}[\text{hash}(\text{op_type}, \text{params})] \rightarrow \text{function_pointer} \tag{14.2}$$

当算子分发层遇到一个之前未见过的形状/参数组合时，才触发 JIT 编译。后续相同参数的调用直接从缓存返回函数指针。这避免了编译的重复开销。

**模板 JIT 的参数组合爆炸分析**

即使是参数化模板，可特化的参数组合数量也可能是巨大的。假设：

- $M \in \{1, 2, \ldots, 4096\}$：约 4096 种可能。
- 数据类型组合：fp32, fp16, int8 → 约 9 种矩阵对。
- 转置标志：2 种。

全部组合可达 $4096 \times 9 \times 2 \approx 74\text{K}$ 种变体。然而实际推理中，形状遵循典型的分布（如批量大小为 2 的幂，序列长度遵循幂律分布），因此实际触发的组合数在几十到几百的范围内，不会导致组合爆炸。

## 14.6　高层 JIT 框架

### 14.6.1　Triton 语言方法

Triton 是由 OpenAI 开发的面向 GPU 的编程语言和编译器，它通过块级编程范式将 JIT 编译和自动调优结合。

**编程模型：**程序员以「程序块」（program block）为单位编写核函数，规定每个程序块应处理的数据范围，并声明数据从全局内存到 SRAM 的加载和写回。Triton 编译器自动管理：

- 将程序块映射到 CUDA 线程块（block）和线程束（warp）。
- 在共享内存（shared memory）中分配暂存空间并生成同步屏障。
- 向量化加载和存储（如 128 位加载）。
- 自动调优分块大小和并行化参数。

**编译管线：** Triton Python DSL → Triton IR → MLIR → LLVM IR → PTX → CUDA 二进制。Triton 的 JIT 在首次调用时编译，结果被缓存（基于 IR 的哈希 +GPU 架构）。

### 14.6.2　TVM/Apache TVM

TVM 是一个端到端的深度学习编译器堆栈，将模型从 DL 框架（PyTorch, ONNX, TensorFlow 等）直接编译为优化后的目标代码。

**核心抽象：**

- **Tensor Expression（TE）：**类 Halide 的张量表达式语言，声明计算逻辑。
- **Schedule（调度）：**独立于计算描述的变换集合（tiling, unrolling, vectorization, binding to threads），指导编译器如何映射计算到硬件。
- **AutoTVM：**使用机器学习（XGBoost cost model）自动搜索最优调度参数。在预定义的搜索空间（如分块因子、展开因子、向量化宽度）中采样配置，测量实际性能，训练成本模型指导后续搜索。
- **Ansor（AutoScheduler）：**替代 AutoTVM 的下一代自动调度器，使用进化搜索（evolutionary search）和基于学习的成本模型，在更大的搜索空间中自动推导调度结构。

### 14.6.3　MLIR 与 IREE

MLIR（Multi-Level Intermediate Representation）是 LLVM 生态中的新一代编译器中间表示，允许定义多层方言（dialect）并在其间逐步降级（lowering）。对于推理引擎：

- **Linalg 方言：**通用的线性代数操作（matmul, conv, generic ops），不绑定特定实现。
- **Vector 方言：**表示 SIMD 级别的操作和向量化变换。
- **SCF（Structured Control Flow）方言：**表示结构化循环和条件。
- **降级管线：** Linalg → SCF+Vector → LLVM IR → 机器码。

IREE（Intermediate Representation Execution Environment）基于 MLIR 构建，为模型推理提供统一的 JIT/AOT 编译和运行时。

## 14.7 JIT 缓存策略详解

### 14.7.1 磁盘持久缓存

磁盘缓存是 JIT 推理性能稳定性的关键：若每次进程重启都需要重新编译所有核函数，启动延迟将不可接受。

**缓存文件组织：**每个核函数的编译结果写入独立文件或数据库记录，以哈希为文件名：

```
~/.cache/my_inference_engine/jit/
    a1b2c3d4e5f6.o     # 编译后的目标文件
    a1b2c3d4e5f6.meta  # 元数据：编译参数、生成时间、来源模板版本
    ...
```

**版本管理与失效：**当核函数模板被修改或目标架构发生变化时，旧的缓存必须失效。通过在缓存键中包含模板内容的哈希（而非仅模板文件名）可实现自动版本管理。

### 14.7.2 内存缓存管理

内存缓存（in-process RAM cache）的容量限制至关重要。JIT 编译的代码驻留在特殊的内存区域（通过 `mmap(PROT_WRITE | PROT_EXEC)` 或 `mprotect` 设置可执行权限），而操作系统的 JIT 代码缓存（jit code cache）通常有容量限制。LRU 淘汰策略需配合引用计数（reference counting）以确保正在执行的核函数不会被淘汰。

### 14.7.3 预编译和热身

对推理服务而言，在加载模型后、开始接收请求前，可以预先针对常见形状（如批量大小 1, 2, 4, 8, 16, 32）触发 JIT 编译，将所有核函数产生到缓存中。这一「热身」阶段将 JIT 延迟前置，确保用户请求始终保持低延迟。

## 14.8　JIT 编译实践

**从零构建 JIT 推理引擎的核函数系统**

1. **定义核函数接口：**所有核函数共用一个类型擦除的函数签名（如 `void (*)(void** args)`），参数通过数组传入。这统一了调度接口。
2. **编写参数化模板：**对于 GEMM、卷积、激活、归一化等核心操作，编写 C intrinsics 的宏模板——参数包括块大小、数据类型、SIMD 宽度、融合操作等。
3. **实现代码生成器：**可采用 LLVM IRBuilder 或 ASMJIT 将参数化模板具体化为代码。对于简单操作，可使用预编译的指令模板填充。
4. **实现缓存层：**使用线程安全的哈希表作为内存缓存，在 `std::shared_mutex` 保护下并发访问。异步写入磁盘缓存。
5. **实现热身机制：**定义一个预设的 shape_list，在模型加载后顺序遍历，对每个 (op_type, shapes, dtype) 调用 JIT 编译 + 执行首个迭代。
6. **监控与降级：**为 JIT 编译设置超时（防止 LLVM 优化在极端形状上陷入过长编译）。超时后降级为未优化通用路径。
7. **测试与验证：**为每个核函数变体编写单元测试，比对其输出与 NumPy/PyTorch 参考输出的误差（fp32: 1e-5, fp16: 1e-3, int8: 允许 1 值误差）。

**JIT 的安全注意事项**

在运行时生成并执行机器码本质上是不安全的——若生成有缺陷的指令（如非法指令编码、错误的寄存器使用），将导致进程崩溃（SIGILL 或 SIGSEGV）。缓解措施：

- 在开发阶段启用指令验证（LLVM IR verify, ASMJIT error handler）。
- 将 JIT 编译限制在特定的进程（fork 子进程），崩溃不影响主进程。
- 在严格的生产环境中，对每个核函数变体执行在线测试：编译后先用已知的（小）参考输入执行一次，比对输出，验证正确性后再正式激活。

# 第十五章　稀疏优化

## 15.1　神经网络的稀疏性类型

### 15.1.1　非结构化稀疏

非结构化稀疏（Unstructured Sparsity）指权重矩阵中任意位置的单个权重值可被独立地置为零，不存在任何结构约束。这是最灵活的稀疏模式，可在给定压缩率下最大化模型精度——因为算法可以根据每个权重的绝对重要性自由选择哪些位置被剪枝。

对于一个稀疏度为 $s$（即 $s$ 比例的权重为零）的矩阵，非零元素的位置分布是完全随机的。存储此类稀疏矩阵需要同时存储值（values）和索引（indices），存储开销约为 $2\times(1-s)\times N$（对于 CSR 格式）。通用硬件的稀疏矩阵乘法（SpMM）的加速依赖于 $s$ 的大小——通常当 $s > 80\%$ 时才能获得超过稠密矩阵的效率。

**非结构化稀疏的挑战**

非结构化稀疏在实际部署中面临两大挑战：

1. **索引开销**：存储索引的比特数可能超过量化权重的比特数。例如，对于 int4 权重，每个 4 位值需要匹配一个 16 位列索引——元数据是权重的 4 倍。
2. **不规则访存**：稠密矩阵的行遍历是连续的内存访问（stride-1，缓存友好）；而稀疏矩阵的行遍历需要间接索引（gather），在 CPU 和 GPU 上都会导致缓存和 TLB 未命中。

### 15.1.2　结构化稀疏：2:4 模式

为克服非结构化稀疏的硬件效率问题，NVIDIA 在 Ampere 架构（A100, RTX 3090+）中引入了 **2:4 结构化稀疏**支持。其核心约束是：在每个连续的 4-元素组中，恰好 2 个权重为非零，2 个为零。这保证了 50% 的非结构化稀疏约束，同时使得稀疏矩阵乘法可以在张量核（Sparse Tensor Core）上以 2× 稠密速度执行。

2:4 稀疏的数学表示为：对权重矩阵做细粒度的列剪枝，对于每组 4 个连续的权重 $\{w_i, w_{i+1}, w_{i+2}, w_{i+3}\}$，仅保留绝对值最大的 2 个，其余 2 个置零。剪枝后，权重使用特殊的压缩格式存储（仅存储非零值和 2 位索引）。

**2:4 稀疏与 GPTQ/AWQ 的结合**

2:4 结构化稀疏和量化可以结合使用：首先使用 GPTQ 或 Magnitude Pruning 在 4 元素组内剪枝，然后在保留的 50% 权重上应用逐块 int4 量化。在 NVIDIA RTX 4090 上，这可以在保持大多数模型精度的情况下，实现约 2.7×-3.2× 的有效吞吐提升。

### 15.1.3 块稀疏

块稀疏（Block Sparsity）将矩阵划分为固定大小的块（如 $16 \times 16$ 或 $32 \times 32$），并以块为单位进行剪枝——要么整个块被保留（稠密），要么整个块被丢弃（全零）。这种粗粒度的稀疏性对硬件极其友好（可以直接跳过被丢弃块的整个 GEMM），但也更难以保持模型精度，因为每个块内的细粒度信息被一并丢弃。

块稀疏常见的应用场景包括：

- **稀疏注意力**：注意力矩阵中很多注意力头对某些位置的注意力权重接近零，可以直接跳过对应块的注意力计算。
- **混合专家（MoE）**：每个专家子网络为一个稠密块，被路由机制选择性激活（门控）。

### 15.1.4 激活稀疏与注意力稀疏

**激活稀疏（Dynamic Sparsity）**： ReLU、GELU 等激活函数在推理过程中普遍产出零值（ReLU 将负值全部置零，通常 50% 的激活值为零）。激活稀疏是动态的（取决于输入），因此不能像权重稀疏那样在编译期或训练后确定。但可以利用激活稀疏来跳过与零激活相关的计算——例如，在计算下一层的 GEMM 时，若输入向量中的第 $j$ 个元素为零，则可以跳过所有涉及第 $j$ 列的权重的乘法。

**注意力稀疏**：在大语言模型中，注意力矩阵 $\text{Attention} = \text{softmax}(QK^\top/\sqrt{d})$ 展现显著的稀疏性——大多数 token 仅对少数（局部或重要的长距离）token 有显著的注意力权重。利用这一稀疏性，可以大幅减少长序列注意力的 $O(n^2)$ 计算成本。

## 15.2 稀疏矩阵存储格式

### 15.2.1 CSR 格式

CSR（Compressed Sparse Row）是最广泛使用的通用稀疏矩阵格式，以行为单位压缩存储。

**CSR 存储结构**

对于稀疏矩阵 $\boldsymbol{A}$（$M$ 行 $\times$ $N$ 列，$nnz$ 个非零元素）：

- `values[nnz]`：非零元素的值，按行主序排列。

- `col_idx[nnz]`：每个非零元素对应的列索引。
- `row_ptr[M+1]`：`row_ptr[i]` 为第 $i$ 行第一个非零元素在 `values` 中的起始偏移；`row_ptr[i+1]` 为其结束偏移。`row_ptr[M] = nnz`。

存储需求：$nnz \times (\text{elem_size} + \text{idx_size}) + (M+1) \times \text{idx_size}$ 字节。

CSR 的核心优势在于行遍历的高效性:访问第 $i$ 行的所有非零元素只需要读取 `values[row_ptr[i] : row_ptr[i+1]]` 范围的连续内存。这使得 SpMV（稀疏矩阵-向量乘）在 CSR 上可以高效实现——对于每个非零元素，执行一个乘加操作。

**CSR 的访存特征：**在 SpMV 中，对 `values` 和 `col_idx` 的访问是顺序的（缓存友好），但对输入向量 $\boldsymbol{x}$ 的访问是间接的（`x[col_idx[k]]`），即 gather 操作——若 $\boldsymbol{x}$ 过大而无法全部放入缓存，这一步将成为性能瓶颈。

### 15.2.2　CSC 格式

CSC（Compressed Sparse Column）与 CSR 对称，但以列为主序压缩。对于 SpMM（稀疏矩阵-稠密矩阵乘），若稀疏矩阵为权重，则 CSC 格式更适合——因为权重向量的列访问与激活向量的列访问对齐。

### 15.2.3　COO 格式

COO（Coordinate List）是最简单的稀疏格式:每个非零元素存储一个三元组 (row_idx, col_idx, value)。三元组列表无需排序（但通常按行或按列排序以改善访存模式）。COO 易于构建（增量添加非零元素），但在计算时需要进行间接的、多次重复的索引访问，效率低于 CSR。

### 15.2.4　ELLPACK 格式

ELLPACK 将矩阵的每一行截断为固定 $K$ 个非零元素（$K$ 为所有行中非零数量的最大值），存储时使用两个稠密矩阵：`values[M][K]` 和 `col_idx[M][K]`。少于 $K$ 个非零元素的行用零填充。

ELLPACK 的优势在于行遍历的完全规则访存（所有行的数据宽度相同，stride-1），非常适合 GPU 和 SIMD。然而，当行的非零数量方差很大时（某些行稠密，多数行稀疏），$K$ 必须取最大行的非零数，导致大量无效的零填充和内存浪费。

### 15.2.5　混合格式与高级格式

**混合 ELL+COO（HYB）：**为 ELLPACK 选择适中的 $K$，超过 $K$ 的非零元素存放在补充的 COO 列表中。这平衡了 ELLPACK 的高效和 COO 的灵活性。

**分块 CSR（Blocked CSR）**：在块稀疏的场景中，将矩阵划分为固定块（$b \times b$），每个块要么全稠密，要么全零。CSR 索引块级（而非元素级）。块的乘法可以在寄存器级别高效执行。

**NVIDIA 2:4 稀疏格式**：在 Ampere 架构中，权重矩阵被存储为压缩格式，采用特殊的硬编码 2:4 解压缩逻辑支持。在 CUDA 编程中，通过 `cusparseSpMM` 和 cuSPARSELt 库访问。

## 15.3 稀疏矩阵乘法实现

### 15.3.1 CSR-based SpMM

SpMM（Sparse-Dense Matrix Multiplication）计算 $\boldsymbol{Y} = \boldsymbol{A}_{\text{sparse}} \times \boldsymbol{X}_{\text{dense}}$，其中 $\boldsymbol{A}$ 为 CSR 格式的权重矩阵，$\boldsymbol{X}$ 为稠密激活矩阵。

**CSR-SpMM 算法**

**输入：** CSR 权重 $\boldsymbol{A}$（$M \times K$），稠密激活 $\boldsymbol{X}$（$K \times N$）。
**输出：**稠密输出 $\boldsymbol{Y}$（$M \times N$），已初始化为零。

1. 对输出行 $i = 0, \ldots, M-1$：
   (a) 获取该行的非零范围：$p_{\text{start}} = \text{row_ptr}[i],\ p_{\text{end}} = \text{row_ptr}[i+1]$
   (b) 对该行每个非零元素 $k = p_{\text{start}}, \ldots, p_{\text{end}} - 1$：
      i. $j \leftarrow \text{col_idx}[k]$（该非零所在的列）。
      ii. $a \leftarrow \text{values}[k]$。
      iii. 对输出列 $n = 0, \ldots, N-1$（批量内积）：$\boldsymbol{Y}[i, n] \mathrel{+}= a \times \boldsymbol{X}[j, n]$。

内层 $n$ 循环将非零元素 $a$ 乘以 $\boldsymbol{X}[j, :]$ 的整行（$N$ 个元素），对其结果向量累加到 $\boldsymbol{Y}[i, :]$。这种方案适用于 SpMM 的批量情况（$N$ 为批量大小）。

### 15.3.2 块 SpMM

当权重在块级稀疏时，可以将块内的稠密矩阵乘法卸载给高效的稠密 GEMM 微内核。块 SpMM 的流程为：

1. 遍历行：$i = 0, \ldots, M/b$。
2. 遍历列：$j = 0, \ldots, K/b$。
3. 若块 $(i, j)$ 非零，则执行子块 GEMM：$\boldsymbol{C}_{\text{block}} \mathrel{+}= \boldsymbol{A}_{\text{block}} \times \boldsymbol{X}_{\text{block}}$。
4. 若块为零，跳过。

此方法将稀疏的开销压缩到块遍历层面，块内部的计算得到稠密微内核的全部效率。

### 15.3.3　向量化 SpMM

在 CPU 上实现 SpMM 的向量化面临核心挑战：CSR 行的每个非零元素在 $\boldsymbol{X}$ 中访问的位置（col_idx[$k$]）是不规则的。有两种策略：

**逐行向量化**：将一行中所有非零元素的乘积累加倍向量化——使用全循环展开和外积式累积，多次将 $\boldsymbol{X}$ 的各行加载到 SIMD 寄存器并执行 FMA。

**多行同步处理**：并行处理多行，在每一轮迭代中，将来自多条行的一个非零元素一起处理——但在实践中实现复杂度极高。

鉴于这些挑战，现代 CPU 推理引擎通常仅在稀疏度超过 90% 时才使用稀疏路径，否则退化为稠密 GEMM。

## 15.4　2:4 结构化稀疏

### 15.4.1　硬件加速机制

NVIDIA Ampere 架构中的 Sparse Tensor Cores 以 2× 速率处理 2:4 稀疏的 fp16/bf16 矩阵乘法。其工作原理如下：

- 权重矩阵以压缩格式存储（每个 4-元素组仅存储 2 个非零值和 2 位选择位）。
- 当发起到 Sparse Tensor Core 的矩阵乘法时，硬件自动解压缩选择的非零值，并将其映射到内侧维度中正确的位置。
- 激活矩阵保持为稠密格式，无需稀疏化。

**压缩格式**：原始 fp16 权重矩阵的每个 4-元素组 $(w_0, w_1, w_2, w_3)$ 中，仅 2 个元素非零。压缩存储为：

- 2 个 fp16 值（非零的两个权重值）。
- 1 个字节包含 2 个 2 位索引（选择位），标记非零值是组中的哪两个位置。

有效存储为 $2 \times 16 + 8 = 40$ 位/4 元素组，即 10 位/元素，约等于 fp16 的 62.5% 内存占用。

### 15.4.2　cuSPARSELt 库

NVIDIA cuSPARSELt 提供了一套完整的 2:4 稀疏矩阵乘法 API：

- `cusparseLtMatmul()`：执行压缩稀疏 × 稠密的矩阵乘法。
- `cusparseLtMatmulSearch()`：自动搜索最优算法配置（分块大小和线程调度方案）。

- 权重剪枝工具：对稠密权重矩阵执行 2:4 剪枝，生成压缩格式。

cuSPARSELt 在内部会进行额外的权重重排以优化 2:4 张量核的访存模式，并对结果反量化（若量化权重）。

### 15.4.3 2:4 剪枝到目标的流程

为将模型转换为可被 2:4 稀疏张量核加速的格式：

1. 从训练好的稠密模型出发。
2. 对每个权重矩阵中的每 4-元素组，保留绝对值最大的 2 个权重，其余置零。
3. 可选：在剪枝后进行微调（fine-tuning），允许剩余权重调整以补偿剪枝误差。
4. 将剪枝后的权重转换为 2:4 压缩格式（或使用 cuSPARSELt 的转换工具）。
5. 在推理时，使用 `cusparseLtMatmul` 替代 cuBLAS GEMM。

**2:4 稀疏的性能实测**

在 A100-SXM4 GPU 上，fp16 的 2:4 稀疏 GEMM 吞吐可达约 310 TFLOPS，而非稀疏的 fp16 张量核吞吐为 156 TFLOPS——即 1.98× 的加速，接近理论上的 2×。在实际的 LLaMA-13B 推理中，使用 2:4 稀疏的注意力投影权重可将解码阶段延迟降低约 30%。

## 15.5 稀疏注意力

### 15.5.1 局部/滑动窗口注意力

局部注意力将每个 token 的注意力范围限制为与其相邻的 $W$ 个 token（$W$ 为窗口大小），使注意力计算复杂度从 $O(n^2)$ 降至 $O(nW)$。这特别适用于长序列推理。

**滑动窗口注意力（Sliding Window Attention）**：以当前 token 位置为中心，取其前后各 $W/2$ 个 token 的对称窗口。超出窗口的 token 注意力权重被设为零。

**稀疏注意力的掩码矩阵**

局部注意力对应的掩码矩阵 $\boldsymbol{M}$ 带有一条宽度为 $W$ 的带状非零区域：

$$M_{ij} = \begin{cases} 0, & |i-j| \le W/2 \\ -\infty, & \text{否则} \end{cases} \tag{15.1}$$

在 FlashAttention 等融合实现中，$M_{ij} = -\infty$ 的块可以直接跳过——无需 load Q 或 K 值，无需计算点积。

### 15.5.2　组合稀疏模式

仅靠局部窗口会丢失长距离依赖。常见的拓展方案包括：

- **膨胀（Dilated）窗口**：在局部窗口上引入步长（dilation），覆盖更广的范围而保持参数量不变。
- **全局 + 局部注意力**：增加少量「全局 token」（如 CLS token 或专门学过远距离信息的 token），这些 token 可以与所有其他 token 进行注意力交互。全局 token 的数量远小于序列长度。
- **随机稀疏注意（Sparse attention with random patterns）**：如 BigBird 模型使用的随机 + 局部 + 全局组合。

### 15.5.3　FlashAttention 与块稀疏的集成

FlashAttention 的原始算法按块（tile）计算注意力，通过软最大值的分块累积来避免存储完整的 $n \times n$ 注意力矩阵。当与稀疏掩码结合时，可以在块级别跳过完全被掩码覆盖的块。FlashAttention-2 和 FlashAttention-3 进一步优化了块遍历顺序，使其对稀疏掩码更友好。

当掩码矩阵为块稀疏时（如某些区域全部被屏蔽），FlashAttention 的调度器可以直接跳过这些块的内循环，节省了从 HBM 加载 Q/K 子块和计算点积的带宽。

## 15.6　剪枝方法

### 15.6.1　幅值剪枝（Magnitude Pruning）

幅值剪枝是最简单、广泛有效的剪枝方法：按权重的绝对值排序，将最小的 $p\%$ 权重置为零。它基于一个合理假设——绝对值小的权重对输出的贡献小，因此移除它们引起的误差也小。

**一次性剪枝（One-shot）**：训练后单次剪枝，不需微调。适用于剪枝率不高（$< 30\%$）的情况。

**渐进式剪枝（Gradual Magnitude Pruning）**：在多个训练步骤中逐渐增加剪枝率，每次剪枝后给予模型一些步骤来恢复（微调）。最终达到比一次性剪枝更高的剪枝率而无严重精度损失。

### 15.6.2　基于 Hessian 的剪枝：SparseGPT

SparseGPT（与 GPTQ 类似但面向剪枝）使用二阶 Hessian 信息指导剪枝，在剪枝每个权重后更新剩余未剪枝的权重以补偿剪枝误差。其数学框架与 OBQ 相同：

- 计算 Hessian 逆（基于校准数据的激活统计）。

- 一次剪枝整列权重：移除某列的 $p\%$ 最小绝对值权重，使用 Hessian 补偿更新该列和后续列的剩余权重。

SparseGPT 可以在单次操作中实现 50%-60% 的无结构稀疏而不需任何微调，这在传统方法中是难以企及的。

### 15.6.3 移动剪枝（Movement Pruning）

移动剪枝（Sparse Fine-tuning 的变体）不依赖权重绝对值，而是学习每个权重的「重要性分数」。在微调过程中，权重更新遵循：

$$w_{t+1} = w_t - \eta \cdot (\nabla_w \mathcal{L} + \lambda \cdot \nabla_w \|w\|_1) \tag{15.2}$$

其中 L1 正则化项 $\lambda\|w\|_1$ 驱动不重要的权重趋向于零而被剪枝。重要性分数可与梯度方向结合，使得「错误」方向的权重更快被剪枝（即使其初始绝对值不小）。

## 15.7 运行时稀疏操作选择

### 15.7.1 阈值驱动的核函数选择

推理引擎在运行时需根据权重的实际稀疏度动态选择处理路径。典型的逻辑是：

```
if (sparsity > threshold) {
    use_sparse_kernel(mat, x, y);
} else {
    use_dense_kernel(mat, x, y);
}
```

其中阈值（threshold）通常在 80%-90% 之间，取决于架构和矩阵大小。对于 GPU 上的大矩阵，稀疏路径需要较高的稀疏度才比稠密张量核有优势（因为张量核算术密度极高，而稀疏路径的索引开销高）；对于 CPU 上的小矩阵或 GEMV，稀疏路径可能即使在 60% 稀疏度下也表现更好。

### 15.7.2 格式转换开销

剪枝后的权重以稠密格式存储是浪费的（一半空间存储零）。转换为 CSR 或 2:4 压缩格式需要一次性预处理。该预处理在模型加载期间进行，不进入推理的关键路径。部分引擎会在模型序列化阶段就完成格式转换，直接存储为稀疏格式。

### 15.7.3　稀疏的内存节省

对于内存受限的推理场景（如消费级 GPU 或移动端），稀疏格式带来的内存节省本身就是收益——即使计算速度相当。以 LLaMA-7B 为例：

- 稠密 fp16：约 14GB。
- 2:4 稀疏 fp16（50% 剪枝 + 压缩）：约 10GB（节省 28%）。
- int4 量化 +2:4 稀疏：约 5GB（节省 64%）。

## 15.8　推理引擎中的稀疏张量表示

### 15.8.1　稀疏张量的 IR 表示

在推理引擎的中间表示（IR）中，稀疏张量需要一种通用的表示方式以支持算子的稀疏感知调度。常见的方案包括：

- **稀疏存储类型**：将张量标记为 CSR、CSC、COO、Blocked-CSR 等存储类型，与稠密张量区分。
- **稀疏性传播**：在算子融合和形状推导中，跟踪稀疏模式的传递——例如，若输入激活的某些列为零（稀疏激活），则矩阵乘法中对应列的权重可被跳过。
- **元数据存储**：稀疏张量的元数据（行指针、列索引）随张量数据一起管理，在设备间拷贝时必须同步。

### 15.8.2　稀疏感知的操作分发

在算子分发层面，需要为每个支持稀疏的操作维护多个实现：

- 稠密-稠密路径（dense × dense）。
- 稀疏-稠密路径（sparse weight × dense activation），最常见。
- 稠密-稀疏路径（dense weight × sparse activation），适用于激活稀疏。
- 双稀疏路径（两操作数均稀疏），极不常见但理论可行。

分发器根据操作数的元数据（稀疏类型、稀疏度）选择对应路径，最好通过 JIT 编译（如第十四章所述）来避免 AOT 路径的维护负担。

**稀疏优化在推理引擎中的总览**

| 稀疏类型 | 来源 | 硬件支持 | 典型加速 |
|---|---|---|---|
| 非结构化稀疏（$>90\%$） | 幅值剪枝/SparseGPT | CPU CSR, GPU CUDA | $1.5\times$ – $3\times$ |
| 2:4 结构化稀疏 | 2:4 剪枝（组内） | NVIDIA Ampere Sparse TC | $\sim 2\times$ |
| 块稀疏 | MoE 门控/稀疏注意 | 通用 GEMM 跳过 | $2\times$ – $10\times$（高稀疏 |
| 激活稀疏 | ReLU/GELU | 动态列跳过 | $1.2\times$ – $1.5\times$ |
| 注意力稀疏 | 滑动窗口/局部模式 | FlashAttention 分块跳过 | $O(nW)/O(n^2)$ 加 |

### 15.8.3 混合执行策略

最先进的推理引擎采用了混合执行策略：

- 权重被剪枝和量化为紧凑的稀疏格式，在激活阶段，按列进行跳过（若激活列为零）或按块进行跳过（若块内所有权重为零）。
- 稀疏注意力的掩码在 JIT 阶段生成，与 FlashAttention 的块调度相结合。
- 在 KV 缓存溢出到 CPU 内存时，选择性仅对高频访问的近期 token 保持全精度，远期的 KV 对进行激进的稀疏化和量化。

**稀疏优化的未来方向**

随着模型规模的持续增长（百万 token 上下文窗口成为常态），稀疏优化将从可选的优化手段演变为必须的系统能力。注意力复杂度从 $O(n^2)$ 压缩到 $O(nW)$ 或 $O(n \log n)$，以及 KV 缓存的稀疏化管理，是支撑超长上下文推理的关键技术路径。硬件侧，NVIDIA Hopper 和后续架构对 FP8 和结构化稀疏的深度整合，预示着稀疏将成为第一等公民。

# 第四部分

# Transformer 专项优化篇

# 第十六章　Attention 机制专项优化

注意力机制（Attention Mechanism）是 Transformer 架构的核心计算单元，也是推理引擎性能瓶颈的主要来源。随着序列长度从 GPT-3 时代（2048 tokens）增长到 Claude 3（200K tokens）乃至 Gemini 1.5 Pro（10M tokens），注意力机制的平方复杂度 $O(n^2)$ 已成为制约长上下文推理的根本障碍。本章系统性地剖析注意力机制的优化技术，从 FlashAttention 的 Tiling 策略到 PagedAttention 的虚拟内存管理，再到投机解码的加速范式。

## 16.1　注意力的计算与存储瓶颈

### 16.1.1　标准注意力实现

考虑一个典型 Decoder-Only 模型中的自注意力层。给定输入 $\boldsymbol{X} \in \mathbb{R}^{n \times d}$，通过三个投影得到查询、键、值：

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{Q}, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{K}, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{V} \tag{16.1}$$

对于多头注意力（Multi-Head Attention, MHA），$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 被重塑为 $(b, h, n, d_h)$，其中 $b$ 为批次大小，$h$ 为注意力头数量，$n$ 为序列长度，$d_h = d/h$ 为每头维度。标准缩放点积注意力的计算流程如下：

**标准注意力逐步计算过程**

**步骤一：计算注意力分数矩阵**

将当前 token 的查询与所有历史 token 的键进行点积：

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^{\top} \in \mathbb{R}^{b \times h \times n_q \times n_{kv}} \tag{16.2}$$

矩阵 $\boldsymbol{S}$ 的元素 $S_{b,h,i,j}$ 表示批次 $b$ 中头 $h$ 的第 $i$ 个查询 token 与第 $j$ 个键 token 的相关性分数。对于因果注意力，$n_q = n_{kv} = n$（自回归场景），或 $n_q = 1$（增量解码场景，每次只计算一个新 token 的注意力）。

**步骤二：缩放（Scale）**

将分数除以每头维度的平方根以稳定训练和推理：

$$\boldsymbol{S}' = \boldsymbol{S}/\sqrt{d_h} \tag{16.3}$$

**步骤三：应用掩码（Mask）**

因果掩码确保 token $i$ 只能关注位置 $j \le i$ 的 token。掩码 $\boldsymbol{M}$ 中，$M_{i,j} = 0$（允许关注）当 $j \le i$，否则 $M_{i,j} = -\infty$：

$$\boldsymbol{S}'' = \boldsymbol{S}' + \boldsymbol{M} \tag{16.4}$$

$-\infty$ 经过 softmax 后变为 0。

**步骤四：Softmax 归一化**

沿键的维度（最后一维，即 $n_{kv}$ 维）应用 softmax：

$$\boldsymbol{P} = \text{softmax}(\boldsymbol{S}'') \in \mathbb{R}^{b \times h \times n_q \times n_{kv}} \tag{16.5}$$

矩阵 $\boldsymbol{P}$ 的第 $i$ 行是一个概率分布（$\sum_j P_{i,j} = 1$），表示查询 token $i$ 对每个键 token $j$ 的注意力权重。

**步骤五：加权聚合**

用注意力权重对值向量加权求和：

$$\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V} \in \mathbb{R}^{b \times h \times n_q \times d_h} \tag{16.6}$$

**步骤六：输出投影**

将所有头的输出拼接后进行一次线性变换：

$$\boldsymbol{O}_{\text{concat}} = \text{Concat}(\boldsymbol{O}_1, \boldsymbol{O}_2, \ldots, \boldsymbol{O}_h)\boldsymbol{W}^O \tag{16.7}$$

### 16.1.2 存储压力分析

标准注意力实现的核心问题是矩阵 $\boldsymbol{S}$（以及 $\boldsymbol{P}$）的显式存储。以 fp16 精度为例：

**注意力矩阵存储需求**

对于单个注意力头，$\boldsymbol{S}$ 和 $\boldsymbol{P}$ 各需要 $(b \times h \times n_q \times n_{kv})$ 个 fp16 元素。当 $b = 1, h = 32, n = 128\text{K}$（128K tokens），$n_q = n_{kv} = n$ 时：

- $\boldsymbol{S}$：$1 \times 32 \times 131072 \times 131072 \approx 5.50 \times 10^{11}$ 个元素
- 每个 fp16 元素 2 字节，总计 $\approx$ **1.1** TB 的中间张量

即使考虑因果掩码只存储下三角部分，仍需约 0.55 TB——远超任何现有 GPU 的显存容量（H100 为 80GB，B200 最大为 192GB）。

**存储瓶颈的本质**

注意力机制的计算量实际为 $O(n^2 \cdot d_h)$，而 FLOPs 可以通过矩阵乘法高效实现。真正的瓶颈在于**中间结果 $\boldsymbol{S}$ 和 $\boldsymbol{P}$ 的 HBM（高带宽内存）读写**。标准实现需要将 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 从 HBM 读入 SRAM（片上内存），计算 $\boldsymbol{S}$，写回 HBM；再从 HBM 读出 $\boldsymbol{S}$ 计算 softmax 得到 $\boldsymbol{P}$，写回 HBM；最后从 HBM 读出 $\boldsymbol{P}$ 和 $\boldsymbol{V}$ 完成最终乘法。这一过程中，数据在 HBM 和计算单元之间往返多次，带宽成为限制因素。对于 $n = 128\text{K}$，HBM 读写量约 $O(n^2) \approx 1.7 \times 10^{10}$ 个元素，以 H100 的 2TB/s 带宽计算，单是数据搬运就需要约 17 秒——这还不包括计算时间。

## 16.2　FlashAttention：分块计算的革命

FlashAttention（Dao et al., 2022）的核心洞察是：注意力矩阵 $\boldsymbol{S}$ 和 $\boldsymbol{P}$ 不需要被完整地写出到 HBM。通过将输入数据分块（tile），在每个分块内部完成局部注意力的计算，并将中间结果保持在 SRAM 中，可以将 HBM 访问量从 $O(n^2)$ 降至 $O(n^2 d_h / M)$，其中 $M$ 为 SRAM 容量。

### 16.2.1　在线 Softmax 算法

FlashAttention 的核心数学基础是在线 Softmax（online softmax），它允许在不完全知晓所有输入的情况下，增量地计算 softmax 结果并修正已有输出。

**在线 Softmax 的数学推导**

考虑一个分块处理的场景：向量 $\boldsymbol{x}$ 被分为两段 $\boldsymbol{x}^{(1)}$ 和 $\boldsymbol{x}^{(2)}$。我们需要在观察完 $\boldsymbol{x}^{(1)}$ 后得到一个初始的 softmax 结果，然后在观察到 $\boldsymbol{x}^{(2)}$ 时修正这个结果。

标准的数值稳定 softmax 计算：

$$\text{softmax}(\boldsymbol{x})_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}, \quad m = \max(\boldsymbol{x}) \tag{16.8}$$

**第一段处理：**计算 $\boldsymbol{x}^{(1)}$ 的局部最大值 $m^{(1)}$ 和局部指数和 $\ell^{(1)} = \sum_{j \in \text{块 } 1} \exp(x_j - m^{(1)})$。初始 softmax 输出为：

$$o_i^{(1)} = \exp(x_i - m^{(1)}) / \ell^{(1)}, \quad i \in \text{块 } 1 \tag{16.9}$$

**第二段处理（修正）：**当 $\boldsymbol{x}^{(2)}$ 到来时，更新全局最大值和指数和：

$$m^{\text{new}} = \max(m^{(1)}, m^{(2)}) \tag{16.10}$$

$$\ell^{\text{new}} = \exp(m^{(1)} - m^{\text{new}}) \cdot \ell^{(1)} + \exp(m^{(2)} - m^{\text{new}}) \cdot \ell^{(2)} = \exp(m^{(1)} - m^{\text{new}}) \cdot \ell^{(1)} + \sum_{j \in \text{块 } 2} \exp(x_j - m^{\text{ne} \tag{16.11}$$

然后**修正旧输出**：

$$o_i^{(1)\text{new}} = o_i^{(1)} \times \frac{\ell^{(1)}}{\ell^{\text{new}}} \times \exp(m^{(1)} - m^{\text{new}}) = o_i^{(1)} \times \frac{\exp(m^{(1)} - m^{\text{new}}) \cdot \ell^{(1)}}{\ell^{\text{new}}} \tag{16.12}$$

新块的输出直接使用新参数计算即可。

**关键性质**：修正因子仅依赖两个标量（$\ell^{\text{new}}$ 和 $m^{\text{new}}$），与第一段的具体 token 数量无关。这使得分块计算在数学上是精确的，没有任何近似。

### 16.2.2 FlashAttention-v1 分块算法

FlashAttention 将 $\boldsymbol{Q}$ 沿序列维度划分为 $T_r$ 个块（每个块 $B_r$ 行），将 $\boldsymbol{K}$ 和 $\boldsymbol{V}$ 沿序列维度划分为 $T_c$ 个块（每个块 $B_c$ 行）。对于每个 $\boldsymbol{Q}$ 块，遍历全部 $\boldsymbol{K}, \boldsymbol{V}$ 块，使用在线 Softmax 增量地计算部分注意力并累积输出。

**FlashAttention-v1 前向传播算法**

**输入**：$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$，块大小 $B_r$（Q 块行数），$B_c$（KV 块行数）

**输出**：$\boldsymbol{O} \in \mathbb{R}^{n \times d}$

**步骤 1**：将 $\boldsymbol{Q}$ 划分为 $T_r = \lceil n/B_r \rceil$ 个块 $\boldsymbol{Q}_1, \boldsymbol{Q}_2, \ldots, \boldsymbol{Q}_{T_r}$，每个 $\boldsymbol{Q}_i \in \mathbb{R}^{B_r \times d}$。同理划分 $\boldsymbol{K}$ 和 $\boldsymbol{V}$ 为 $T_c = \lceil n/B_c \rceil$ 个块。

**步骤 2**：将 $\boldsymbol{O}$ 划分为对应的 $T_r$ 个块 $\boldsymbol{O}_1, \boldsymbol{O}_2, \ldots, \boldsymbol{O}_{T_r}$，每个初始化为零矩阵。

**步骤 3**：为每个 $\boldsymbol{O}$ 块分配标量存储：归一化因子 $\ell_i \in \mathbb{R}^{B_r}$ 初始化为 0，最大分数 $m_i \in \mathbb{R}^{B_r}$ 初始化为 $-\infty$。

**步骤 4**：对每个 Q 块 $i \in \{1, 2, \ldots, T_r\}$：

1. 将 $\boldsymbol{Q}_i$ 从 HBM 加载至片上 SRAM。
2. 对每个 KV 块 $j \in \{1, 2, \ldots, T_c\}$：
   (a) 将 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 从 HBM 加载至片上 SRAM。
   (b) 在片上计算分块注意力分数：$\boldsymbol{S}_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$。
   (c) 应用因果掩码（如果需要）：将 $S_{ij}[p, q]$ 置为 $-\infty$ 当对应位置不可见时。
   (d) 计算当前块的行最大值：$\tilde{m}_{ij} = \text{row_max}(\boldsymbol{S}_{ij}) \in \mathbb{R}^{B_r}$。
   (e) 更新全局最大值：$m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij})$。
   (f) 计算当前块的未归一化 softmax 分数：$\boldsymbol{P}_{ij} = \exp(\boldsymbol{S}_{ij} - m_i^{\text{new}})$。
   (g) 计算当前块的归一化因子行和：$\tilde{\ell}_{ij} = \text{row_sum}(\boldsymbol{P}_{ij}) \in \mathbb{R}^{B_r}$。
   (h) 更新全局归一化因子：$\ell_i^{\text{new}} = \exp(m_i - m_i^{\text{new}}) \odot \ell_i + \tilde{\ell}_{ij}$。
   (i) 修正已有输出并累加新输出：$\boldsymbol{O}_i = \text{diag}(\exp(m_i - m_i^{\text{new}})) \cdot \boldsymbol{O}_i + \boldsymbol{P}_{ij} \boldsymbol{V}_j$。

(j) 更新状态：$m_i \leftarrow m_i^{\text{new}}$，$\ell_i \leftarrow \ell_i^{\text{new}}$。

3. 最终归一化：$\boldsymbol{O}_i = \text{diag}(1/\ell_i) \cdot \boldsymbol{O}_i$。

4. 将 $\boldsymbol{O}_i$ 从 SRAM 写回 HBM。

**FlashAttention 的 IO 复杂度分析**

设 SRAM 容量为 $M$（对 A100 约 192KB per SM，H100 约 228KB），每个注意力头的维度为 $d$，单精度元素大小为 $s$ 字节。为了使一个 Q 块和一个 KV 块能同时放在 SRAM 中，需要满足：

$$B_r \cdot d \cdot s + B_c \cdot d \cdot s \cdot 2 \leq M \tag{16.13}$$

（三个张量 $\boldsymbol{Q}_i, \boldsymbol{K}_j, \boldsymbol{V}_j$，还有 $\boldsymbol{S}_{ij}$ 和 $\boldsymbol{P}_{ij}$ 也需要空间）。
在这种设置下，HBM 读取次数约为：

$$\Theta\left(\frac{n^2 d}{M}\right) = \Theta\left(\frac{n^2 d^2}{M}\right) \cdot \frac{1}{d} \tag{16.14}$$

相比标准实现的 $\Theta(nd + n^2)$（即 $\Theta(n^2)$ 当 $n \gg d$），FlashAttention 将 HBM 访问量降低了 $O(M/d)$ 倍。
**具体数值**：对于 H100，SRAM 为 228KB，$d_h = 128$（LLaMA-70B 单头维度），$s = 2$（fp16），若取 $B_r = B_c = 128$：

- $\boldsymbol{Q}_i$：$128 \times 128 \times 2 = 32$ KB
- $\boldsymbol{K}_j$：$128 \times 128 \times 2 = 32$ KB
- $\boldsymbol{V}_j$：$128 \times 128 \times 2 = 32$ KB
- $\boldsymbol{S}_{ij}$：$128 \times 128 \times 2 = 32$ KB
- 总计约 128KB，可以装入 228KB 的 SRAM 中。

此时 HBM 访问量降低约 $M/(d \cdot s) = 228\text{K}/(128 \times 2) \approx 890$ 倍（相对于完整 $\boldsymbol{S}$ 写出）。

## 16.3　FlashAttention-2：并行度与调度优化

FlashAttention-v1 的主要局限在于并行策略：它将前向传播的并行化放在批次和注意力头维度上（线程块映射到 $(b, h)$ 对），每个线程块顺序处理所有 Q 块。当序列长度很大而批次和头数有限时，并行度不足。FlashAttention-2（Dao, 2023）通过将并行化转移到序列长度维度，大幅提升了长序列场景下的 GPU 利用率。

**FlashAttention-2 的关键改进**

**改进一：序列长度维度的并行化**

不再将线程块映射到 $(b, h)$ 对，而是映射到 Q 块索引 $(i)$ 上。每个线程块负责一个特定的 $\boldsymbol{Q}_i$ 块，独立遍历所有 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 块。这意味着对于 $n = 128\text{K}$，可以启动 $T_r = n/B_r$ 个线程块（例如 $B_r = 128$ 时约为 1024 个），远多于 v1 的 $b \times h$ 个（例如 $b = 1, h = 32$ 时仅 32 个）。

**改进二：减少非矩阵乘法操作**

v1 中，每次遍历 KV 块时都需要进行重缩放（rescaling）操作。v2 通过重新组织计算顺序，将重缩放的频率从 $O(T_c)$ 降低到 $O(T_c)$ 但每次的开销更小。具体来说，在最终归一化之前的中间结果不以全精度存储，而是用更少的重缩放步累积。

**改进三：Warp 级工作划分**

在单一线程块内部，将工作更均匀地分配给各个 warp。对于 $\boldsymbol{S}_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^\top$ 的矩阵乘法（$B_r \times d$ 矩阵乘 $d \times B_c$ 矩阵），将输出矩阵 $\boldsymbol{S}_{ij}$ 按行或列分配给不同 warp。这样每个 warp 只计算 $\boldsymbol{S}_{ij}$ 的一部分，提高了 warp 级并行度。

**改进四：减少共享内存通信**

v1 在计算 attention 分数、softmax 和值乘法的过程中，需要在线程块内的 warp 之间进行多次同步和共享内存读取。v2 通过合并操作（fuse the scaling into the matmul）和重新组织数据流，减少了 warp 间同步的次数。

**FlashAttention-2 在推理中的特殊考量**

对于推理场景（尤其是增量解码），$n_q = 1$（每次只生成一个新 token）。此时：

- $\boldsymbol{Q} \in \mathbb{R}^{1 \times d_h}$（退化为向量），$B_r$ 实际上为 1。
- FlashAttention 的块内并行度降低：只有 1 个 Q 行需要处理。
- 此时 $\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top \in \mathbb{R}^{1 \times n_{kv}}$，直接计算反而更简单。
- v2 的序列并行优势主要体现在 prefill 阶段（$n_q$ 较大），对增量解码的加速有限。

正因为如此，许多推理引擎（如 vLLM）在 prefill 阶段使用 FlashAttention/FlashInfer，在增量解码阶段使用专门优化的单查询注意力 kernel。

## 16.4 FlashAttention-3：Hopper 架构特化

FA3（Shah et al., 2024）充分利用了 NVIDIA Hopper 架构（H100/H200）的新硬件特性，实现了最高可达 H100 峰值算力 740 TFLOPs 的 75% 以上的实际利用率。核心创新包括异步 Tensor Core 运算和硬件加速的数据搬运。

**Hopper 架构关键特性在 FA3 中的应用**

**WGMMA（Warp-Group Matrix Multiply-Accumulate）**

Hopper 引入了 wgmma 指令，允许单个 warp-group（4 个 warp，128 个线程）直接对寄存器中的矩阵片段执行 Tensor Core 矩阵乘法。与 Ampere 的 mma（需要显式的共享内存加载）相比，wgmma 的源操作数可以直接来自共享内存或寄存器，减少了指令级开销。

**TMA（Tensor Memory Accelerator）**

TMA 是一个独立的硬件单元，可以异步地将多维张量块从 HBM 复制到共享内存，无需占用 CUDA 核心或 Tensor Core 的计算资源。FA3 使用 TMA 将下一个 $\boldsymbol{K}_{j+1}, \boldsymbol{V}_{j+1}$ 块提前预取到共享内存，同时当前块的计算正在进行——实现计算与数据搬运的完美流水线重叠。

**异步流水线设计**

FA3 的前向传播采用了生产者-消费者流水线模式：

1. **TMA 生产者：**负责将 KV 块从 HBM 异步预取到共享内存缓冲区（双缓冲）。
2. **计算消费者：**对当前缓冲区中的 KV 块与驻留的 Q 块执行 wgmma 操作。
3. 两个阶段通过异步屏障（async barrier）同步，几乎消除数据搬运延迟。

**FP8 低精度支持**

FA3 原生支持 FP8 数据格式（E4M3 和 E5M2），利用 H100 的 FP8 Tensor Core 实现 2 倍于 FP16 的吞吐量。这对于推理场景尤其重要，因为模型权重和 KV Cache 都可以量化为 FP8 以节省显存带宽。

**FA3 异步流水线伪代码**

**初始化：**

- 分配双缓冲共享内存：SMEM_K_Buf[0], SMEM_K_Buf[1]
- 分配双缓冲共享内存：SMEM_V_Buf[0], SMEM_V_Buf[1]
- 将 $\boldsymbol{Q}_i$ 加载到共享内存（持久化在整个内循环中）

**流水线执行：**

```
// 预取第一个KV块
TMA_Load(SMEM_K_Buf[0], K_block[0]);
TMA_Load(SMEM_V_Buf[0], V_block[0]);

for j = 0 to T_c - 1:
    buf_cur = j % 2;
```

```
    buf_next = (j + 1) % 2;

    // 等待当前块的数据加载完成
    cp_async_barrier();

    // 预取下一个块（如果存在）
    if j + 1 < T_c:
        TMA_Load(SMEM_K_Buf[buf_next], K_block[j+1]);
        TMA_Load(SMEM_V_Buf[buf_next], V_block[j+1]);

    // 对当前块执行计算
    // wgmma: S_ij = Q_i @ K_j^T
    wgmma(S_frag, Q_frag, SMEM_K_Buf[buf_cur]);
    // mask + scale + exp (在寄存器中进行)
    online_softmax_update(O_frag, S_frag, SMEM_V_Buf[buf_cur]);
```

## 16.5 PagedAttention 与 vLLM

PagedAttention（Kwon et al., 2023）是 vLLM 推理引擎的核心创新，将操作系统的虚拟内存分页思想引入 KV Cache 管理，解决了可变长度序列下 KV Cache 的内存碎片问题。

### 16.5.1 KV Cache 碎片问题

在传统的 KV Cache 管理中，每个请求的 KV Cache 被分配为一个连续的内存块。然而，由于不同请求的序列长度不同，且输出长度无法预先知道，这种分配方式导致严重的内存碎片。

**KV Cache 碎片的具体表现**

假设 GPU 显存中 KV Cache 总容量为 $M$ 字节。三个请求依次到达：

- 请求 A：输入 100 tokens，预估输出 500 tokens → 分配 600 tokens 的 KV Cache。
- 请求 B：输入 500 tokens，预估输出 2000 tokens → 分配 2500 tokens 的 KV Cache。
- 请求 C：输入 50 tokens，预估输出 100 tokens → 分配 150 tokens 的 KV Cache。

如果请求 A 实际输出了 800 tokens（超出预留），需要扩展 KV Cache 但相邻空间已被请求 B 占用，产生**内部碎片**（internal fragmentation）。如果请求 B 只输出了 1000 tokens，剩余的 1500 tokens 空间成为**外部碎片**（external fragmentation），在请求 B

完成前无法被其他请求使用。当这些碎片累积时，有效 KV Cache 利用率可能低至 20%-40%。

### 16.5.2 PagedAttention 的分页实现

**PagedAttention 核心设计**

**分块（Block）**：KV Cache 被划分为固定大小的块（block/page），每个块包含 $B$ 个 token 的键和值向量。典型块大小为 $B = 16$（一个块包含 16 个 token 的 K 和 V）。
**块表（Block Table）**：每个请求维护一张块表，将逻辑块索引映射到物理块索引。类比操作系统的页表（page table）。
**逻辑到物理映射：**

$$\text{物理块 ID} = \text{BlockTable}[\text{请求 ID}][\text{逻辑块索引}] \tag{16.15}$$

其中逻辑块索引 $\lfloor \text{token 位置}/B \rfloor$。
**内存分配**：当请求需要新空间时，从全局空闲块池中分配一个物理块，将物理块 ID 记录到该请求的块表中。释放时，块归还到空闲池。

**PagedAttention 单头注意力计算**

**输入**：查询向量 $\boldsymbol{q} \in \mathbb{R}^{1\times d_h}$，块表 BT（$N_B$ 个逻辑块），物理 KV Cache 池
**输出**：注意力输出 $\boldsymbol{o} \in \mathbb{R}^{1\times d_h}$
初始化：$m = -\infty$（运行最大值），$\ell = 0$（运行指数和），$\boldsymbol{o} = \boldsymbol{0}$
**对所有非空的逻辑块** $b = 0, 1, \ldots, N_B - 1$：

1. 通过块表获取物理块 ID：$\text{phys_id} = \text{BT}[b]$。
2. 从物理 KV Cache 加载 $\boldsymbol{K}_b, \boldsymbol{V}_b \in \mathbb{R}^{B\times d_h}$。
3. 计算分数：$\boldsymbol{s}_b = \boldsymbol{q}\boldsymbol{K}_b^\top \in \mathbb{R}^{1\times B}$。
4. 应用缩放：$\boldsymbol{s}_b = \boldsymbol{s}_b/\sqrt{d_h}$。
5. 应用掩码（如果当前块部分超出有效序列长度，对超出部分置 $-\infty$）。
6. 在线 Softmax 更新（与前节相同）：

$$m_{\text{new}} = \max(m, \max(\boldsymbol{s}_b)) \tag{16.16}$$

$$\ell_{\text{new}} = \exp(m - m_{\text{new}}) \cdot \ell + \sum_j \exp(s_{b,j} - m_{\text{new}}) \tag{16.17}$$

$$\boldsymbol{o} = \exp(m - m_{\text{new}}) \cdot \boldsymbol{o} + \exp(\boldsymbol{s}_b - m_{\text{new}})\boldsymbol{V}_b \tag{16.18}$$

7. $m \leftarrow m_{\text{new}}$，$\ell \leftarrow \ell_{\text{new}}$。

**最终归一化：** $\boldsymbol{o} = \boldsymbol{o}/\ell$。

### 16.5.3 Copy-on-Write 内存共享

PagedAttention 的一个重要优势是支持多请求之间的 KV Cache 共享。当多个请求共享相同的前缀时（如系统提示词、few-shot 示例），前缀的物理 KV 块可以被所有请求共享。

**前缀共享的 COW 机制**

**场景：** 系统提示词长度为 $P$ 个 token。所有请求共享这个前缀。

**初始状态：** $P$ 个 token 的 KV Cache 存储在 $\lceil P/B \rceil$ 个物理块中，引用计数为该块的共享请求数。

**Copy-on-Write：** 当请求 A 需要向共享块中的某个位置写入新的 KV 值时（例如，由于该请求的后续生成改变了注意力的计算路径——实际上在因果注意力中不会发生，共享块是只读的），系统会复制该物理块，让请求 A 使用新副本。在纯自回归解码中，所有请求共享的前缀块不会被修改，因此 COPY-ON-WRITE 几乎不触发实际复制。

**内存节省：** 对于 100 个并发请求共享 512 tokens 的系统提示词（$B = 16$ 时需 32 个物理块），传统方案需要 $100 \times 32 = 3200$ 个物理块用于前缀存储，而 PagedAttention 仅需 32 个块，节省 99% 的前缀 KV Cache 内存。

**GQA/MQA 对 PagedAttention 的影响**

分组查询注意力（GQA）和多查询注意力（MQA）减少了键值头的数量，这对 PagedAttention 非常有利：

- 每个 KV 块的物理大小与 KV 头数成正比。MQA（1 个 KV 头）相比 MHA（32 个 KV 头），块大小减少 32 倍。
- 块表条目数不变（仍然按逻辑块索引），但每个逻辑块对应的物理数据量大幅减少。
- 更小的块意味着在共享内存中可以容纳更多块的 KV 数据，减少块遍历的轮数。

# 16.6　KV Cache 优化技术总览

## 16.6.1　多头注意力的变体

**MQA、GQA 与 MHA 的对比**

设查询头数为 $h$，键值头数为 $h_{kv}$。每层的 KV Cache 大小为：

$$\text{KV Cache Size} = 2 \times n \times h_{kv} \times d_h \times \text{bytes_per_element} \tag{16.19}$$

**MHA（Multi-Head Attention）**：$h_{kv} = h$。如 LLaMA-2-7B 使用 $h = 32$，每层 KV Cache 约 $2 \times n \times 32 \times 128 \times 2 = 16,384n$ 字节。当 $n = 4$K 时，单层约 65MB，32 层约 2GB；当 $n = 128$K 时，32 层约 65GB。

**GQA（Grouped-Query Attention）**：$h_{kv} = h/g$，其中 $g$ 为每组共享同一 KV 头的查询头数。LLaMA-3-70B 使用 $h = 64, h_{kv} = 8$（$g = 8$）。KV Cache 减少 8×。

**MQA（Multi-Query Attention）**：$h_{kv} = 1$。所有查询头共享同一对 KV 头。KV Cache 减少 $h$×。计算时，KV 需要广播到所有查询头。

表 16.1: MQA/GQA/MHA 的计算与存储对比

| 属性 | MHA | GQA (g=8) | MQA |
|---|---|---|---|
| K/V 头数 | $h$ | $h/8$ | 1 |
| KV Cache 大小 | $2nhd_h$ | $2nhd_h/8$ | $2nd_h$ |
| 每个查询头的 KV 计算量 | 相同 | 相同 | 相同 |
| 质量损失 | 基线 | 极小 | 可接受 |
| 代表模型 | LLaMA-2 | LLaMA-3, Qwen2 | Falcon-180B |

## 16.6.2　KV Cache 压缩技术

**主流 KV Cache 压缩方法**

**StreamingLLM（滑动窗口 + 注意力汇聚）**：保留初始的 $s$ 个 token（注意力汇聚 token，attention sink）和最近的 $w$ 个 token。中间的” 被遗忘”token 的 KV 直接从缓存中移除。理论基础：初始 token 承载了大量全局信息，最近 token 承载了局部上下文。

**H2O（Heavy-Hitter Oracle）**：基于注意力权重的累积分数选择保留哪些 KV 对。对于每个查询头，维护一个” 重击者集合”（heavy-hitter set），保留累积注意力分数最高的 $k$ 个 token 的 KV 对。直觉：少数 token 在多数查询中获得了不成比例的高注意力权重。

**SnapKV**：在生成过程中，定期对 KV Cache 进行” 快照压缩”。通过观察最近几个 token 的注意力模式，识别” 重要” 的 KV 位置（注意力权重持续高的位置），只保留

这些位置的 KV 对。对不重要位置的 KV 通过加权平均进行” 压缩合并”。

**量化 KV Cache：**

- **Int8 KV：**将 Key 和 Value 从 fp16 量化为 int8，减少 50% 存储。使用逐通道（per-channel）或逐 token（per-token）量化以保持精度。
- **KV-Int4：**进一步量化到 4-bit，但需要更精细的量化策略（如分组量化）。
- **ZipCache（键值联合量化）：**注意到 K 和 V 的分布特性不同（K 通常需要更高精度），可以对 K 和 V 采用不同的量化位宽。K 通常保留 int8 或更高精度，V 可以降至 int4。

**层间 KV 共享：**

- **CLA（Cross-Layer Attention）：**相邻层之间共享 KV Cache。例如层 $i$ 计算的 KV 同时被层 $i$ 和层 $i+1$ 使用。将 KV Cache 大小减少近 50%。
- **YOCO（You Only Cache Once）：**设计一个特殊的 KV 编码器-解码器结构，其中解码器部分不需要 KV Cache（自注意力被替换为仅依赖固定大小状态的机制）。
- **MiniCache（深度上采样）：**在相邻层的 KV 之间进行插值或融合，以更少的物理存储覆盖更多逻辑层的 KV。

**CPU 卸载（Offloading）：**将不活跃请求的 KV Cache 从 GPU HBM 卸载到 CPU 内存（通过 PCIe 或 NVLink-C2C）。当请求恢复活跃时再加载回来。适用于长尾延迟不敏感的批处理场景。

## 16.7 投机解码

投机解码（Speculative Decoding）是一种在不损失输出质量的前提下加速自回归生成的范式。核心思想：用一个小而快的” 草稿模型”（draft model）预测多个未来 token，然后用大而准的” 目标模型”（target model）一次性验证所有草稿 token。

### 16.7.1 基本投机解码框架

**投机解码的数学原理**

**设置：**

- 目标模型 $M_q$：参数分布 $q(x_t \mid x_{<t})$，每步生成一个 token。
- 草稿模型 $M_p$：参数分布 $p(x_t \mid x_{<t})$，每步生成一个 token（但速度快得多）。

**步骤：**

1. **草稿阶段**：从当前前缀开始，用 $M_p$ 自回归生成 $K$ 个草稿 token $x_1, \ldots, x_K$，其中每个 $x_t \sim p(\cdot \mid x_{<t})$。
2. **验证阶段**：用 $M_q$ 进行一次前向传播，以 $x_1, \ldots, x_k$ 为输入，并行计算 $q(\cdot \mid x_{<1}), q(\cdot \mid x_{<2}), \ldots, q(\cdot \mid x_{<K})$。
3. **接受/拒绝阶段**：对每个草稿 token $x_t$，以概率 $\min\left(1, \frac{q(x_t|x_{<t})}{p(x_t|x_{<t})}\right)$ 接受。
   - 若接受，继续验证下一个草稿 token。
   - 若拒绝，从修正分布中采样替代 token：$\text{norm}(\max(0, q(\cdot) - p(\cdot)))$，并丢弃所有后续草稿 token。

**质量保证**：无论草稿模型的质量如何，上述接受/拒绝过程保证最终生成的 token 序列与仅使用 $M_q$ 自回归生成具有**相同的分布**。这是投机解码的核心理论保证。

**加速分析**：期望的每步生成 token 数（接受率 $\alpha$）：

$$\mathbb{E}[\text{每步 token 数}] = \frac{1 + \sum_{k=1}^{K-1} \prod_{i=1}^{k} \alpha_i}{1 + \text{验证开销比例}} \tag{16.20}$$

其中 $\alpha_i$ 为第 $i$ 个草稿 token 被接受的概率。当草稿模型与目标模型高度一致时，$\alpha_i \to 1$，加速比接近 $K$。

### 16.7.2　投机解码的演进

**投机解码的变体**

**自投机解码（Self-Speculative Decoding）**：不需要独立的草稿模型。使用目标模型自身的部分层作为草稿：取前 $L/2$ 层的输出作为草稿模型的表示，附加一个小的预测头来生成草稿 token。或者使用目标模型中 Dropout 开启的模式产生不同的预测路径。

**Medusa（多头草稿）**：在目标模型上额外附加 $K$ 个预测头（每个头预测一个未来位置的 token），所有 $K$ 个头在一次前向传播中并行预测 $K$ 个草稿 token。不需要独立的草稿模型，额外参数极少（通常 $< 1\%$）。草稿 token 的验证仍使用目标模型的完整前向传播。

**Eagle（特征级投机）**：不直接预测草稿 token，而是预测目标模型在对应位置的隐层特征。然后基于这些隐层特征用目标模型的 LM 头生成草稿 token。特征级预测比 token 级预测更稳定，接受率更高。Eagle-2 进一步引入了动态草稿树结构。

**树形注意力（Tree Attention）**：当草稿模型生成多个候选分支时（树形结构而非链式结构），验证阶段需要同时评估树上所有节点的 token。树形注意力通过专门的注意力掩码实现这一目标——每个节点的查询只能关注其祖先路径上的键。掩码形状不再

是简单三角，而是由树拓扑决定的。

**Sequoia**：在投机解码框架中引入推测性树形结构的动态构建。使用蒙特卡洛树搜索（MCTS）策略选择最可能的新 token 序列路径，在有限的草稿预算下最大化期望接受 token 数。

**SpecInfer**：使用多个草稿模型（集成）来生成草稿 token。不同的草稿模型可以捕获不同的语言特征，组合使用可提高草稿质量。验证阶段使用联合概率进行接受/拒绝决策。

**投机解码在工程实践中的经验**

- **草稿模型选择**：草稿模型应比目标模型小 20-100×（参数量）。常用选择包括：LLaMA-68M 用于 LLaMA-7B，JackFram/Llama-160M 用于 Llama-2-7B。过大的草稿模型会抵消投机加速。
- **草稿长度 $K$ 的选择**：$K$ 不是越大越好。接受率随 $K$ 递减（远离已知上下文后预测困难），一般 $K = 3 \sim 5$ 在大多数场景下加速比最优。
- **树形结构的收益**：在 $K = 5$ 时，树形结构（分支因子 2-3）通常比链式结构多接受 20%-30% 的草稿 token。
- **草稿生成的批处理**：一次验证前向传播可以并行处理 $b \times K$ 个位置（$b$ 为批次中的请求数，$K$ 为草稿长度）。这使得验证阶段的计算效率接近普通的 prefill。
- **与 PagedAttention 的兼容性**：投机解码和 PagedAttention 可以共存。PagedAttention 管理物理 KV 块，投机解码在 $K$ 个后续位置产生额外的查询——每个额外查询需要访问对应位置的 KV。这些 KV 可以预先分配在" 推测块" 中。

## 16.8 注意力优化在推理引擎中的实践

### 16.8.1 混合注意力算子选择策略

在实际推理引擎中，不能对所有情况使用同一种注意力实现。以下是一个典型的算子选择逻辑：

**注意力算子的运行时选择策略**

1. **Prefill 阶段（$n_q \gg 1$，$n_{kv} \gg 1$）：**
   - $n_{kv} \leq 512$ 且无 GQA：使用标准 cuBLAS batched GEMM + softmax kernel。
   - $n_{kv} > 512$：使用 FlashAttention-2/3（或 FlashInfer 实现）。
   - 对极长上下文（$n_{kv} > 32\text{K}$）：使用带有 KV 缓存压缩的注意力（如

StreamingLLM + FA2）。

2. **增量解码阶段（$n_q = 1$，$n_{kv}$ 逐渐增长）：**
   - $n_{kv} \leq 4096$：使用优化的 GEMV 内核（Q 为向量，$K^\top$ 为矩阵转置的向量-矩阵乘法）。
   - $n_{kv} > 4096$：使用 FlashDecoding 或 FlatSplitAttention（将 KV 序列拆分到不同 SM 并行处理）。
   - 若使用 PagedAttention：使用分块注意力内核，按物理 KV 块遍历。
3. **有树形验证的投机解码：**使用专门的树形注意力内核。

### 16.8.2　预填充与解码的混合调度

推理引擎需要同时处理处于 prefill 阶段（首次处理输入 prompt）和 decode 阶段（逐 token 生成）的请求。这两个阶段对注意力算子的需求截然不同。

**Prefill 与 Decode 的混合调度策略**

**问题：**prefill 是计算密集型（高 arithmetic intensity），decode 是内存带宽密集型（每 token 只需要少量 FLOPs 但对 KV Cache 的访问量大）。若在同一个批次中混合 prefill 和 decode，GPU 的计算资源和内存带宽会被同时争抢，效率降低。

**Chunked Prefill（分块预填充）：**将长 prompt 的 prefill 拆分为多个 chunk（每个 chunk 固定 token 数，如 512 或 1024）。每个推理迭代中，一部分 GPU 时间分配给 prefill chunk 处理，一部分分配给 decode token 处理。好处：

- 避免单个长 prefill 请求阻塞所有 decode 请求（降低首 token 延迟和后续生成延迟的抖动）。
- 使计算负载更均匀，GPU 利用率更稳定。

**分离式 Prefill-Decode 架构：**使用不同的 GPU（或 GPU 组）分别处理 prefill 和 decode 请求（如 DeepSeek 的 Mooncake 架构）：

- Prefill 集群：接收新请求，执行 prefill，生成 KV Cache，通过 RDMA 将 KV Cache 传输给 Decode 集群。
- Decode 集群：维护活跃请求的 KV Cache，执行增量解码。
- 优势：prefill 和 decode 可以使用不同的硬件配置和优化策略。Prefill 需要高计算吞吐量（H100 理想），Decode 需要大容量 HBM（H200 或 B200 更优）。

**注意力优化的”不可能三角”**

在长上下文推理中，注意力优化存在一个根本性的权衡：

- **精度**：保留所有 KV 对的完整注意力 → 无损质量。
- **速度**：跳过不必要的注意力计算 → 更快的推理。
- **内存**：压缩或丢弃 KV 对 → 更少的显存占用。

当前不存在同时满足这三者的完美方案。FlashAttention 在内存和速度方面做出巨大改进但保持精度无损；KV Cache 压缩在内存方面取得优势但牺牲了精度；稀疏注意力在速度方面有优势但同样损失精度。推理引擎设计者需要根据具体应用场景进行取舍。

# 第十七章　动态形状与批处理

现代 AI 推理服务面临的核心工程挑战之一，是如何高效地处理形状各异的输入序列和动态变化的请求流。与训练阶段" 固定批次、固定序列长度" 的规整形态不同，推理服务必须应对在线请求的不可预测性——有的请求仅包含几个 token 的查询，有的请求则携带数十万 token 的上下文；有的请求只需生成几个 token 的简短回答，有的则可能需要输出数千 token 的长篇文章。本章系统性地探讨动态形状推理的优化技术和批处理调度策略。

## 17.1　动态形状问题的本质

### 17.1.1　可变序列长度的挑战

在实际推理服务中，每个请求的输入序列长度（prompt 长度）各不相同。假设一个批次包含三个请求，序列长度分别为 $[n_1 = 128, n_2 = 2048, n_3 = 64]$，最朴素的方案是将所有序列 padding 到最大长度 $n_{\max} = 2048$。

**Padding 的代价分析**

对于上例中的三个序列，Padding 方案的量化代价：

- **计算浪费率：** $\frac{3\times2048-(128+2048+64)}{3\times2048} = \frac{6144-2240}{6144} \approx 63.5\%$ 的计算是无意义的。
- **显存浪费：** 所有中间激活的序列维度也扩展到 2048，显存开销增加 2.74×。
- **注意力浪费：** 注意力矩阵从 $\sum_i n_i \times n_i$（每序列自身的因果注意力的有效计算量）膨胀到 $3 \times 2048 \times 2048$（约 12.6M 个有效元素 vs 12.6M——注意这是每序列独立计算的情况；但若使用 batch matrix multiply，仍需处理 padding 位置）。

当序列长度分布极不均匀时（如 99% 的请求 $<$ 512 tokens，但有 1% 的请求 $>$ 32K tokens），padding 到 $n_{\max}$ 的资源浪费率可超过 95%。

### 17.1.2　序列打包

序列打包（Sequence Packing）是消除 Padding 浪费的直接方案。核心思想是将多个短序列沿长度维度拼接成一个长序列，通过注意力掩码隔离不同序列的 token。

**序列打包的形式化描述**

**设定：** $N$ 个请求，长度分别为 $n_1, n_2, \ldots, n_N$。总有效 token 数 $T = \sum_{i=1}^{N} n_i$。

**打包操作：**

$$\boldsymbol{X}_{\text{packed}} = \text{Concat}(\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_N) \in \mathbb{R}^{T \times d} \tag{17.1}$$

**注意力掩码：** 打包后的注意力掩码 $\boldsymbol{M} \in \mathbb{R}^{T \times T}$ 是一个分块对角矩阵（block-diagonal matrix），每个对角块的大小为 $n_i \times n_i$，块内为因果注意力模式（下三角）。不同序列之间的注意力权重被设为 $-\infty$，确保信息不会在不同请求之间泄漏。

$$M_{p,q} = \begin{cases} 0 & \text{如果 token } p \text{ 和 token } q \text{ 属于同一序列且} q \leq p \\ -\infty & \text{否则} \end{cases} \tag{17.2}$$

**配套数据结构：** 需要维护一个 `cu_seqlens` 数组（累积序列长度），例如对于 $n = [128, 2048, 64]$：

```
cu_seqlens = [0, 128, 2176, 2240]
```

该数组在变长注意力、变长 Softmax 等算子中被广泛使用。

**序列打包的工程实现要点**

- **嵌入层处理：** 每个序列的 token 在打包后仍然可以独立进行嵌入查找（embedding lookup），因为嵌入查找不涉及序列间的交互。
- **RoPE 位置编码：** 每个序列使用各自的位置索引（0 到 $n_i - 1$），需要在打包序列上按段应用不同的位置 ID。
- **损失/输出提取：** 对于自回归生成，每个序列的 logits 从 packed tensor 中对应位置提取。需要 `cu_seqlens` 来定位。
- **KV Cache 管理：** 打包后的序列在 prefill 阶段共享同一个前向传播，但在增量解码阶段退化为各自的独立状态（每个请求独立管理 KV Cache）。

## 17.2 连续批处理

连续批处理（Continuous Batching），又称飞行中批处理（In-flight Batching）或迭代级批处理（Iteration-level Batching），是现代高性能推理引擎（vLLM、TensorRT-LLM、SGLang 等）的标志性特性。

### 17.2.1　静态批处理 vs 连续批处理

**两种批处理范式的对比**

**静态批处理（Static Batching）：**

1. 等待收集 $B$ 个请求组成一个批次。
2. 对批次中的所有请求执行完整的 prefill（处理全部输入 prompt）。
3. 对批次中的所有请求执行自回归生成，直到所有请求都完成（输出终止符或达到最大长度）。
4. 返回结果，回到步骤 1 收集下一个批次。

**静态批处理的效率分解：**设等待收集 $B$ 个请求的时间为 $T_{\text{wait}}$，prefill 时间为 $T_{\text{prefill}}$，每个 decode step 时间为 $T_{\text{decode}}$，最长的请求需要 $L_{\max}$ 步 decode。

$$T_{\text{total}} = T_{\text{wait}} + T_{\text{prefill}} + L_{\max} \times T_{\text{decode}} \tag{17.3}$$

当 $L_{\max}$ 远大于平均输出长度时，已经完成生成的那些请求的"slot" 被迫等待最慢的请求——GPU 在此期间对这些 slot 只计算 padding token，资源严重浪费。

**连续批处理（Continuous Batching）：**

1. 维护一个**活跃请求集合** $\mathcal{A}$。
2. 每次迭代（一次前向传播）：
   - 为 $\mathcal{A}$ 中每个请求生成一个 token（decode），或为新加入请求的 prompt 处理一个 chunk（prefill）。
   - 检测新到达的请求，将其加入 $\mathcal{A}$（或排队）。
   - 检测已完成的请求（生成终止符或达到最大长度），将其从 $\mathcal{A}$ 中移除并返回结果。
3. 当 $\mathcal{A}$ 为空且无新请求到达时，停止。

**连续批处理的效率分析：**

$$T_{\text{total}} \approx \frac{\sum_i (n_i^{\text{prompt}} + L_i)}{B_{\text{eff}} \times \text{throughput_per_token}} \tag{17.4}$$

其中 $B_{\text{eff}}$ 为平均活跃批次大小。由于已完成请求的 slot 立即被新请求填充（或释放），GPU 几乎不会处理 padding token。理论利用率接近 100%（减去 kernel 启动开销和调度开销）。

### 17.2.2 迭代级调度器

**连续批处理调度器的核心算法**

**状态：**

- $\mathcal{W}$：等待队列（新到达尚未开始处理的请求）。
- $\mathcal{A}$：活跃请求集合（正在 prefill 或 decode 的请求）。
- $\mathcal{F}$：已完成请求集合（等待返回结果的请求）。

**每次迭代（iteration）的执行逻辑：**

*Phase 1:* 调度决策

1. 计算当前可用资源：KV Cache 剩余容量 $M_{\text{free}}$，最大批次大小 $B_{\max}$。
2. 从 $\mathcal{W}$ 中选择请求加入 $\mathcal{A}$（prefill 调度）：
   - 选择策略：FCFS（先到先服务）、最短 prompt 优先（减少等待队列）、按需混合。
   - 检查每个候选请求的 prompt 长度是否能在当前 KV Cache 容量下容纳。
   - 可执行的 prefill 总 token 数受限于计算预算（不能饿死 decode 请求）。
3. 如果启用了 Chunked Prefill，将一个长 prompt 拆分为多个 chunk。

*Phase 2:* 执行

1. 将 $\mathcal{A}$ 中所有请求的当前 token（prefill chunk 或下一个 decode token）打包为一个批次。
2. 执行一次前向传播，获得每个请求的下一个 token 的 logits。
3. 对每个请求进行采样/贪心解码。

*Phase 3:* 更新

1. 将新生成的 token 追加到对应请求的 KV Cache。
2. 检测终止条件（EOS token 或达到 max_new_tokens），将完成的请求移至 $\mathcal{F}$。
3. 释放已完成请求的 KV Cache 资源。

**防止饥饿（Starvation Prevention）：**

- 为每个请求维护等待时间和已执行迭代数。
- 当某个请求的等待时间超过阈值时，提升其调度优先级。
- 在 decode 密集型负载中，为新到达的 prefill 请求预留最小计算份额（minimum prefill budget）。

### 17.2.3 Prefill-Decode 混合调度

**Prefill 与 Decode 的计算特性差异**

表 17.1: Prefill 与 Decode 的计算特性对比

| 特性 | Prefill | Decode |
|---|---|---|
| 输入形状 | $(n_{\text{prompt}}, d)$ | $(1, d)$ |
| 序列维度 | $n_{\text{prompt}} \gg 1$ | 1 |
| 计算类型 | 矩阵-矩阵乘法（GEMM） | 矩阵-向量乘法（GEMV） |
| 算术强度 | 高（compute-bound） | 低（memory-bandwidth-bound） |
| 注意力模式 | 并行（$n_{\text{prompt}} \times n_{\text{prompt}}$） | 增量（$1 \times n_{\text{kv}}$） |
| 延迟敏感度 | 中等（首 token 时间影响 TTFT） | 高（每 token 时间影响 TPOT） |
| 显存访问模式 | 计算密集，HBM 带宽利用率高 | 带宽密集，HBM 带宽易成为瓶颈 |

**混合调度的冲突**：当 prefill 和 decode 在同一批次中执行时：

- Prefill 部分需要大量 Tensor Core 计算资源，对 HBM 带宽压力小。
- Decode 部分需要大量 HBM 带宽来读取 KV Cache，对计算资源压力小。
- 两者同时执行时，GPU 的 L2 缓存和 HBM 带宽被竞争，decode 部分的延迟可能增加 $2 \sim 5\times$。

**Chunked Prefill（分块预填充）**：将长 prompt 的 prefill 分解为大小为 $C$ token 的多个 chunk。每个推理迭代处理一个 prefill chunk（与其他请求的 decode token 一起），而非一次性完成整个 prompt 的 prefill。$C$ 的选择需要权衡：

- $C$ 太小：prefill 总迭代数增加，首 token 延迟增加。
- $C$ 太大：单次迭代中 prefill 计算占比太高，decode 延迟增加。
- 经验值：$C = 512 \sim 2048$ tokens，取决于 GPU 型号和模型大小。

## 17.3　预填充-解码分离架构

### 17.3.1　Mooncake 式分离架构

**分离式 Prefill-Decode 架构的设计原理**

**核心思想**：将 prefill 和 decode 分配到不同的 GPU 实例上，使两类工作负载可以独立优化。

**Prefill 实例（Prefill Instance, PI）：**

- 配置：高计算吞吐 GPU（如 H100，高 Tensor Core 利用率），较少 HBM 容量要求。
- 任务：接收新请求，完成全部 prompt 的 prefill，生成初始 KV Cache。
- 优化重点：高吞吐量地处理 prefill（使用 FlashAttention-3 的全序列并行处理）。
- 完成后：将生成的 KV Cache 通过高速互联（RDMA over InfiniBand/RoCE）传输给 Decode 实例。

**Decode 实例（Decode Instance, DI）：**

- 配置：大 HBM 容量 GPU（如 H200 有 141GB HBM，B200 有 192GB HBM），可容纳更多并发请求的 KV Cache。
- 任务：维护活跃请求的 KV Cache，执行逐 token 增量解码。
- 优化重点：高并发批处理、PagedAttention、投机解码。

**KV Cache 传输：**

- 传输量：$2 \times L_{\text{layers}} \times n_{\text{prompt}} \times h_{kv} \times d_h \times \text{bytes_per_elem}$。
- 对于 70B 模型（80 层，GQA $h_{kv} = 8$，$d_h = 128$，fp16），2048 token prompt 的 KV Cache 约 $2 \times 80 \times 2048 \times 8 \times 128 \times 2 \approx 1.34$ GB。
- 使用 400 Gbps InfiniBand 传输约需 27ms。这段传输时间可以被后续 decode 步骤中的计算重叠。

**负载均衡：**

- PI 和 DI 的数量不需要相等。通常 PI:DI 比率较低（如 1:3 到 1:5），因为 decode 的持续时间远长于 prefill。
- 动态调整：根据请求队列的长度和分布动态增减 PI 或 DI 实例。

### 17.3.2 SGLang 的 RadixAttention

**基于前缀树的 KV Cache 共享**

SGLang 的 RadixAttention 将 PagedAttention 的 KV Cache 共享机制推广到基于前缀树（radix tree）的自动共享。

**Radix 树结构：**

- 树中每个节点表示一个前缀 token 序列（通常对应一个或多个 KV Cache 物理块）。
- 所有具有相同前缀的请求在树中共享该前缀的节点。
- 当一个请求的 prompt 匹配树中的某个前缀路径时，其 KV Cache 直接复用已缓存的节点，只需要计算剩余部分。

**树维护操作：**

- **插入**：新请求的 prompt 在树中匹配到最长前缀后，将剩余 token 作为新节点插入。
- **引用计数**：每个节点维护被多少个活跃请求引用的计数。计数降到 0 时，节点可以作为候选被驱逐。
- **驱逐（Eviction）**：当 KV Cache 容量不足时，使用 LRU（最少最近使用）或 LFU（最不频繁使用）策略从叶子节点开始驱逐。

**应用场景：**

- **多轮对话**：同一对话的所有轮次共享对话历史的 KV Cache。
- **Few-shot 提示**：多个请求使用相同的 few-shot 示例前缀。
- **并行采样（Parallel Sampling）**：同一 prompt 生成多个候选输出（如 beam search、self-consistency）。
- **搜索结果增强**：同一查询的不同搜索结果共享查询前缀。

**Cache 命中率分析：**在典型的在线推理负载中（如 Chatbot Arena），RadixAttention 的 KV Cache 命中率可达到 50%-80%，意味着约一半到三分之二的 prompt token 不需要重新计算 KV，直接节省对应的 prefill 计算和显存。

## 17.4　动态形状的算子支持

### 17.4.1　形状无关的内核

**三种动态形状处理策略**

**策略一：即时编译（Just-In-Time Compilation, JIT）**
当遇到新的形状组合时，在运行时编译针对该形状优化的 CUDA kernel。典型实现：TensorFlow XLA、PyTorch 2.0 的 TorchInductor、JAX。

- 优点：每个形状都能获得最优化的代码（循环展开、寄存器分配等）。
- 缺点：首次编译延迟（cold start latency）可能达到数秒。对于推理服务，如果形状空间太大，编译开销累积不可接受。
- 缓解措施：编译缓存（持久化到磁盘）、预热（warm-up with known shapes）、异步编译（使用 fallback kernel 直到编译完成）。

**策略二：形状分桶（Shape Bucketing）**
将无限连续的形状空间划分为有限个”桶”（bucket），每个桶使用一个预编译的 kernel。

- 分桶策略：例如对序列长度使用指数分桶 $[1,2),[2,4),[4,8),...,[2^k,2^{k+1})$。桶内的形状统一 padding 到桶的上界。
- 桶的数量权衡：桶太少 →padding 浪费大；桶太多 → 编译开销大。
- 典型应用：TensorRT、ONNX Runtime 的推理优化。
- vLLM 的做法：按 $(batch_size, max_seqlen)$ 的二维桶空间进行分桶，约 20-50 个常用桶覆盖 90% 以上的流量。

**策略三：符号形状追踪（Symbolic Shape Tracking）**
使用符号表达式而非具体数值来表示张量的形状关系。例如：$n$ 个 token 的序列打包后，注意力输出的 batch 维为 $n$，但具体数值在运行时才知道。

- 优点：能推理形状间的关系（如 $n_{\text{out}} = n_{\text{in}}$），进行更多编译时优化。
- 挑战：当形状表达式变得复杂时（包含取模、除法等），符号推理可能超时或失败。
- 典型框架：PyTorch 的 SymInt（符号整数）、ONNX Runtime 的符号维度。

### 17.4.2 动态形状的内存规划

**动态形状下的内存管理策略**

**最坏情况预留（Worst-Case Reservation）：**在每次推理开始前，根据当前批次中所有序列的最大长度预留全部中间张量的内存。这是最简单的方案，但浪费巨大。

**增量分配（Incremental Allocation）：**在计算图中动态追踪每个算子的实际输入形状，按需分配输出张量内存。需要框架支持动态内存分配器（如 PyTorch 的 Caching Allocator、TensorFlow 的 BFC Allocator）。

**KV Cache 的分配策略：**

- **预分配上限：**每个请求预分配 $L_{\max}$ 个 token 的 KV Cache（$L_{\max}$ 为最大允许生成长度）。优点是各请求 KV Cache 空间连续（简化 PagedAttention），缺点是限制并发数。
- **按需增长：**先分配一个初始块（如 16 个 token），每生成 16 个 token 后再申请一个新块。这是 PagedAttention 的基础。
- **可溢出分配：**当 GPU KV Cache 满时，将低优先级请求的 KV Cache 卸载到 CPU 内存。

**内存碎片处理：**PagedAttention 的分页机制天然解决外部碎片问题（所有块大小相同）。内部碎片问题（块内未用完的空间）通过选择合适的块大小来平衡：块小 $\rightarrow$ 碎片少但元数据开销大；块大 $\rightarrow$ 元数据开销小但碎片多。经验最优块大小通常为 16-64 tokens。

### 17.4.3 变长注意力核函数

对于打包/ragged 序列的注意力计算，需要专门的变长注意力核函数。

**变长注意力的分块实现**

**输入：**

- $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{T \times d}$（$T = \sum_i n_i$，所有序列拼接）
- `cu_seqlens` $= [0, n_1, n_1 + n_2, \ldots, T]$（累积序列长度）
- 最大序列长度 $n_{\max} = \max_i n_i$

**算法：**

1. 对每个序列 $s$（由 `cu_seqlens`$[s]$ 到 `cu_seqlens`$[s+1] - 1$ 标识）：
   (a) 提取子矩阵 $\boldsymbol{Q}_s, \boldsymbol{K}_s, \boldsymbol{V}_s$，形状为 $(n_s, d)$。

(b) 在子矩阵上执行标准（或 Flash）注意力。

(c) 将输出写回 $\boldsymbol{O}$ 的对应位置。

**高效实现要点：**

- **使用 CUBLAS 的 Batched GEMM：**将所有序列的 $\boldsymbol{Q}_s\boldsymbol{K}_s^\top$ 组织为 batch GEMM。每个序列是 batch 的一个样本。需要 stride 数组来指示各序列矩阵在内存中的偏移。
- **自定义变长 Softmax 核函数：**在 CUDA kernel 中，每个线程块处理一个序列的一个行块。使用 `cu_seqlens` 来确定每行的有效长度（即 softmax 归一化的分母计算边界）。
- **与 PagedAttention 结合：**对于已经分块的 KV Cache，变长注意力外层是序列遍历，内层是物理块遍历。两层循环可以通过 GPU 的 grid-stride loop 并行化。

## 17.5　批处理的效率分析

### 17.5.1　吞吐量建模

**推理吞吐量的解析模型**

设模型有 $L$ 层，每层参数 $P_{\text{per_layer}}$，每层激活大小 $A_{\text{per_layer}}(n, b)$，HBM 带宽 $B_{\text{HBM}}$，计算峰值 $F_{\text{peak}}$。

**单次前向传播的延迟：**

$$T_{\text{fwd}}(b, n) = \max\left(\frac{\text{Total FLOPs}}{F_{\text{peak}}}, \frac{\text{Total HBM bytes}}{B_{\text{HBM}}}\right) \tag{17.5}$$

对于 decode 阶段（$n = 1$），算术强度 $\frac{\text{FLOPs}}{\text{HBM bytes}}$ 极低，延迟由 HBM 带宽主导：

$$T_{\text{decode}} \approx \frac{2 \times b \times L \times P_{\text{per_layer}} \text{ bytes}}{B_{\text{HBM}}} \tag{17.6}$$

（2× 因为需要从 HBM 读取权重并写回 KV Cache/中间结果）。

**吞吐量（Token per Second, TPS）：**

$$\text{TPS} = \frac{b}{T_{\text{decode}}} \approx \frac{B_{\text{HBM}}}{2 \times L \times P_{\text{per_layer}}} \tag{17.7}$$

**关键观察：**decode 阶段的 TPS 与批次大小 $b$ 几乎关系不大——因为 $T_{\text{decode}}$ 也大致正比于 $b$（权重读取次数相同，但 batch 维度上的计算可共享权重读取）。实际上，TPS 会随着 $b$ 增大而轻微增加，因为更大的 $b$ 使 GEMV 变成 GEMM，算术强度略有提高。但收益递减——当 $b$ 增大到某个阈值后，HBM 带宽仍是瓶颈。

**批处理效率的边际递减效应**

对于 decode 阶段，批处理的效率提升主要来自**权重共享**——一批请求共享同样的模型权重，只需要一次 HBM 读取。但是，随 $b$ 增大，KV Cache 的 HBM 读取量也线性增长（每个请求有自己的 KV Cache）。当 $b$ 大到 KV Cache 读取量超过权重读取量时，继续增大批次不再提升 TPS。

**最大有效批次大小的估算：**

$$b_{\max} \approx \frac{P_{\text{per_layer}}}{2 \times n_{\text{kv_avg}} \times h_{kv} \times d_h} \tag{17.8}$$

其中 $n_{\text{kv_avg}}$ 为平均 KV Cache 长度。例如，对于 LLaMA-3-70B（$P_{\text{per_layer}} \approx 700$M 参数 =1.4GB fp16，$h_{kv} = 8, d_h = 128$），$n_{\text{kv_avg}} = 4096$ 时：$b_{\max} \approx \frac{1.4 \times 10^9}{2 \times 4096 \times 8 \times 128 \times 2} \approx 83$。超过 83 的批次大小几乎不再增加 TPS。

### 17.5.2　请求分布的负载均衡

**分布式推理的请求调度**

在多 GPU 或多节点部署中，请求的合理分配对整体吞吐量至关重要。

**轮询调度（Round-Robin）**：请求依次分配到每个 GPU 实例。简单但无法处理异构负载——当某个请求恰巧落到正在处理长 prompt 的实例时，延迟显著增加。

**最少连接调度（Least-Connection）**：将新请求发送到当前活跃请求数最少的实例。比轮询更好地平衡负载，但不考虑请求的计算量差异。

**幂等选择（Power of Two Choices）**：随机选取两个实例，将请求发送给其中负载较低的。理论上接近最优负载均衡，且实现简单、状态量少。

**前缀感知调度（Prefix-Aware Scheduling）**：将具有相同前缀的请求路由到同一实例（或同一组共享 KV Cache 的实例），最大化 RadixAttention 的缓存命中率。

**异构硬件下的调度**：当部署集群中存在不同型号的 GPU 时（如部分 H100 和部分 A100），需要根据硬件性能和当前负载情况动态决策。通常将延迟敏感的请求分配给高性能 GPU，将批量吞吐型请求分配给较慢但数量更多的 GPU。

**批处理设计中的常见陷阱**

1. **Prefill 饥饿**：在连续批处理中，如果 decode 请求持续占据所有资源，新到达的 prefill 请求可能长时间得不到处理（首 token 延迟飙升）。必须为 prefill 预留最小计算份额。

2. **长尾延迟放大**：批次中有一个极长 prompt 时，即使使用了 chunked prefill，其处理仍会拖慢同批次 decode 请求的每 token 时间。

3. **内存碎片累积**：即使使用 PagedAttention，如果块大小选择不当或驱逐策略不完善，长时间运行后 KV Cache 利用率仍可能下降。

4. **投机解码的批处理交互**：投机解码产生多个草稿 token 后需要在验证阶段并行处理，这增加了单次迭代的计算量。如果调度不当，可能导致迭代延迟的显著波动（jitter）。

# 第五部分

# 分布式与硬件篇

# 第十八章　分布式推理

当单张 GPU 的显存不足以容纳整个模型，或单 GPU 的吞吐量不能满足服务级别协议（SLA）时，分布式推理成为必然选择。本章从张量并行、流水线并行、专家并行、数据并行四个维度系统性地剖析分布式推理的技术体系，涵盖通信原语、负载均衡策略以及混合并行的自动搜索方法。

## 18.1　张量并行

张量并行（Tensor Parallelism, TP）将单个算子的权重矩阵按列或行切分到多个 GPU 上，每个 GPU 计算部分结果后通过集合通信（collective communication）合并。TP 的通信发生在每层的前向传播内部，因此需要高带宽低延迟的互联（如 NVLink）。

### 18.1.1　线性层的张量并行

**列并行线性层（Column-Parallel Linear）**

给定输入 $\boldsymbol{X} \in \mathbb{R}^{b\times n\times d}$，权重 $\boldsymbol{W} \in \mathbb{R}^{d\times d_{\text{out}}}$（忽略 batch 和 seq 维度以简化）。将 $\boldsymbol{W}$ 按列均匀切分为 $p$ 个 GPU：

$$\boldsymbol{W} = [\boldsymbol{W}_1 \mid \boldsymbol{W}_2 \mid \cdots \mid \boldsymbol{W}_p], \quad \boldsymbol{W}_k \in \mathbb{R}^{d\times (d_{\text{out}}/p)} \tag{18.1}$$

每个 GPU $k$ 计算其负责的部分：

$$\boldsymbol{Y}_k = \boldsymbol{X}\boldsymbol{W}_k \in \mathbb{R}^{b\times n\times (d_{\text{out}}/p)} \tag{18.2}$$

**通信需求：**$\boldsymbol{X}$ 在每个 GPU 上都需要完整的副本。如果 $\boldsymbol{X}$ 来自前一层的输出且前一层已经完成通信整合，则 $\boldsymbol{X}$ 天然在所有 GPU 上一致。如果 $\boldsymbol{X}$ 来自不同 GPU 的部分结果（如 attention heads 分散在不同 GPU），则需要 **All-Gather** 操作将所有 GPU 的 $[\boldsymbol{Y}_1, ..., \boldsymbol{Y}_p]$ 拼接为完整的 $\boldsymbol{Y} \in \mathbb{R}^{b\times n\times d_{\text{out}}}$。

**通信量：**All-Gather 的每 GPU 通信量为 $\frac{p-1}{p} \cdot b \cdot n \cdot d_{\text{out}} \cdot \text{bytes}$。

**行并行线性层（Row-Parallel Linear）**

给定权重 $\boldsymbol{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$，按行切分：

$$\boldsymbol{W} = \begin{bmatrix} \boldsymbol{W}_1 \\ \boldsymbol{W}_2 \\ \vdots \\ \boldsymbol{W}_p \end{bmatrix}, \quad \boldsymbol{W}_k \in \mathbb{R}^{(d_{\text{in}}/p) \times d_{\text{out}}} \tag{18.3}$$

输入 $\boldsymbol{X}$ 也按最后一维切分（通常来自列并行层的输出，或输入本身就是分布式存储的）：

$$\boldsymbol{X} = [\boldsymbol{X}_1 \mid \boldsymbol{X}_2 \mid \cdots \mid \boldsymbol{X}_p] \tag{18.4}$$

每个 GPU $k$ 计算：

$$\boldsymbol{Y}_k = \boldsymbol{X}_k \boldsymbol{W}_k \in \mathbb{R}^{b \times n \times d_{\text{out}}} \tag{18.5}$$

**通信需求：**每个 GPU 的 $\boldsymbol{Y}_k$ 是部分和（partial sum），需要 **All-Reduce** 将所有 GPU 的 $\boldsymbol{Y}_k$ 相加得到完整输出 $\boldsymbol{Y} = \sum_k \boldsymbol{Y}_k$。
**通信量：**All-Reduce 的每 GPU 通信量为 $2 \cdot \frac{p-1}{p} \cdot b \cdot n \cdot d_{\text{out}} \cdot \text{bytes}$（发送 + 接收）。

### 18.1.2 Transformer 层的张量并行

**Megatron-LM 风格的 Transformer 层张量并行**

Megatron-LM（Shoeybi et al., 2019）建立了 LLM 张量并行的标准范式。一个 Transformer 层的张量并行结构如下：
**自注意力模块：**

1. **QKV 投影：**列并行线性层。$\boldsymbol{W}^Q, \boldsymbol{W}^K, \boldsymbol{W}^V$ 按列切分到 $p$ 个 GPU。每个 GPU 计算部分注意力头（$h/p$ 个头）的 Q、K、V。由于注意力头之间天然独立，无需额外通信。

2. **注意力计算：**每个 GPU 独立计算其负责的注意力头（$h/p$ 个头），输出 $\boldsymbol{Y}_k^{\text{attn}} \in \mathbb{R}^{b \times n \times (h/p) \times d_h}$。

3. **输出投影：**行并行线性层。$\boldsymbol{W}^O$ 按行切分。每个 GPU 计算其部分，然后 All-Reduce 求和。

**前馈网络（FFN）模块：**

1. **第一层线性（升维）：**列并行。$\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$ 按列切分。每个 GPU 计算部分输出。

2. **激活函数：**$\boldsymbol{A}_k = \text{GELU}(\boldsymbol{Y}_{1,k})$。激活是逐元素的，不同 GPU 间无交互。

3. **第二层线性（降维）**：行并行。$\boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ 按行切分。每个 GPU 计算部分和，然后 All-Reduce 求和。

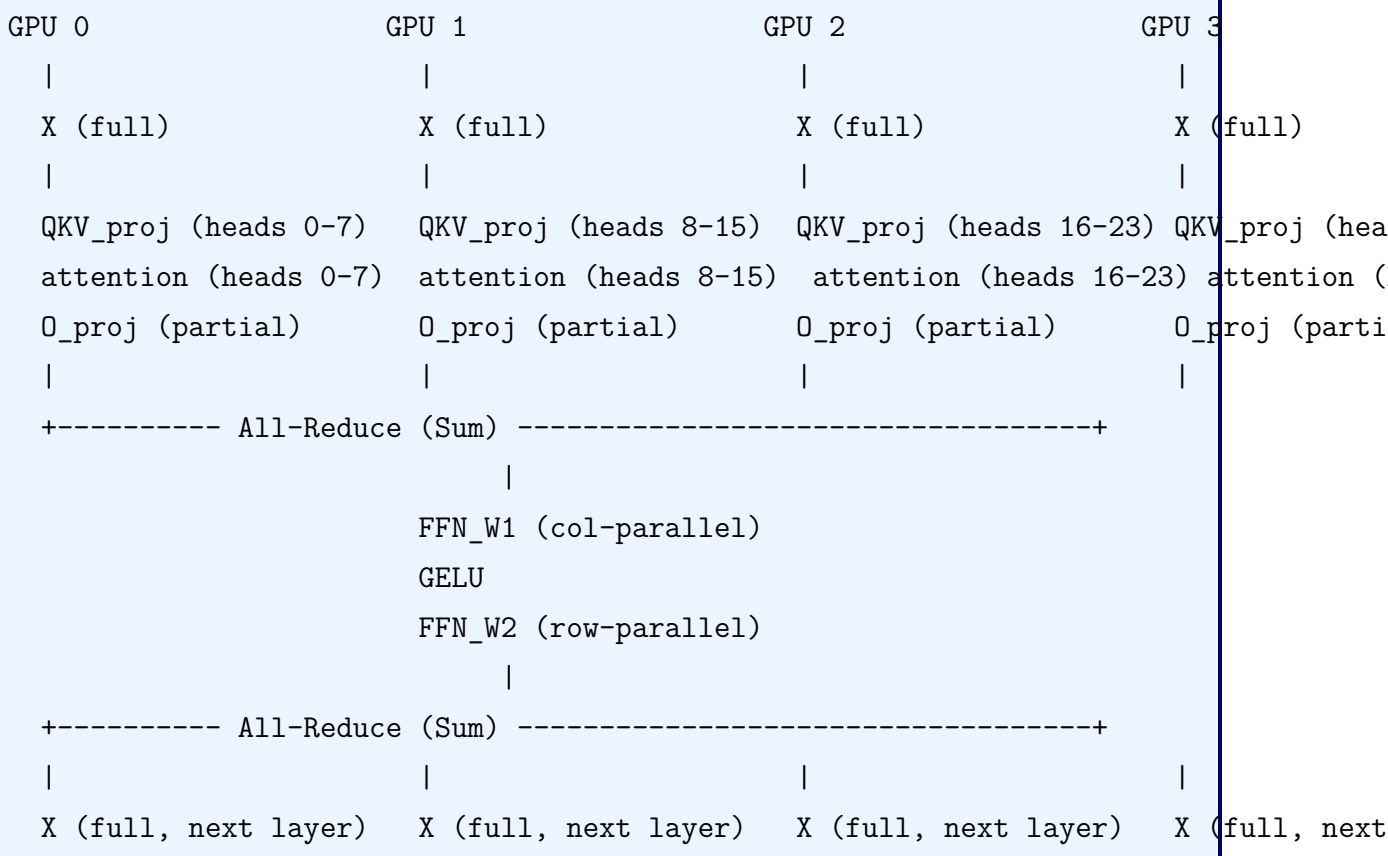



图 18.1: 4-way 张量并行的 Transformer 层数据流（Megatron-LM 风格）

**每层通信次数：**共需要 2 次 All-Reduce 操作（一次在注意力输出投影后，一次在 FFN 第二层线性后）。

### 18.1.3　张量并行的通信分析

**张量并行的通信带宽需求**

设模型参数量为 $P$，层数为 $L$，单层参数量约为 $P/L$。忽略 KV Cache 和激活内存，每次前向传播中每个参数的权重被读取一次。对于张量并行度 $p$：

**每层的通信量**（以 fp16 为例，bytes_per_elem $= 2$）：

$$\text{通信量}_{\text{All-Reduce}} = 2 \times \frac{p-1}{p} \times b \times n \times d \times 2 \text{ bytes} \tag{18.6}$$

$$\approx 4 \times b \times n \times d \text{ bytes} \quad (\text{当}p\text{较大时}, \frac{p-1}{p} \to 1) \tag{18.7}$$

对于 $b = 1$（单请求推理），$n = 1$（decode 阶段），$d = 8192$（LLaMA-3-70B 的中间维度）：

$$\text{通信量}_{\text{单次 All-Reduce}} \approx 4 \times 1 \times 1 \times 8192 \times 2 = 65,536 \text{ bytes} \approx 64 \text{ KB} \tag{18.8}$$

两次 All-Reduce 共约 128 KB。在 900 GB/s 的 NVLink 4.0 上，这部分通信只需约 0.14 $\mu$s。

**然而**，对于 prefill 阶段（$b = 1, n = 8192$）：

$$\text{通信量} \approx 4 \times 1 \times 8192 \times 8192 \times 2 = 536,870,912 \text{ bytes} \approx 512 \text{ MB} \tag{18.9}$$

在 900 GB/s 上约需 0.57 ms。如果模型有 80 层，每层 2 次 All-Reduce，总通信量约 $80 \times 2 \times 512$ MB $= 80$ GB，约 89 ms。

**通信受限判断**：在 decode 阶段（$n = 1$），通信量极小，TP 几乎不受通信限制。在 prefill 阶段（$n$ 大），通信量显著，需要高带宽互联（NVLink/NVSwitch）以保持效率。

### 18.1.4 序列并行

**序列并行与张量并行的结合**

序列并行（Sequence Parallelism, SP）是张量并行的扩展——将序列维度也分布到不同 GPU 上，以减少冗余的激活存储。

**动机**：在张量并行中，$\boldsymbol{X}$（即每一层的输入激活）在所有 GPU 上都是完整副本。对于长序列 prefill，这造成 $p\times$ 的激活内存冗余。

**实现**：

1. $\boldsymbol{X} \in \mathbb{R}^{b \times n \times d}$ 按序列维度切分：每个 GPU 持有一部分 $n/p$ 连续的序列 token。
2. 在 `LayerNorm`/RMSNorm 前，执行 All-Gather 将序列维度聚合（因为 LayerNorm 需要完整的 $d$ 维数据，而不是序列维）。
3. 在注意力计算中，每个 GPU 独立处理其部分序列（$n/p$ 个查询），但需要完整的 $n$ 个 K 和 V——这需要 All-Gather 或 Ring Attention 来获取完整 KV。
4. 在 FFN 中，由于 FFN 运算在 $d$ 维度（每 token 独立），天然支持序列并行——每个 GPU 处理 $n/p$ 个 token 的 FFN，无需通信。

**内存节省**：激活存储从 $b \times n \times d \times p$ 减少到 $b \times n \times d$（每个 GPU 只存储 $n/p$ 部分）。

**2D 并行**：TP（切分 $d$ 维）+ SP（切分 $n$ 维）构成 2D 并行。对于极长序列（如 $n = 128$K），2D 并行可在更多 GPU 上分摊内存。

## 18.2　流水线并行

流水线并行（Pipeline Parallelism, PP）将模型按层切分到不同 GPU 上。与张量并行不同，PP 的通信只发生在层间边界（点对点，发送激活到下一 GPU），单次通信量较小但需要整个模型串行执行。

### 18.2.1　流水线并行的基本概念

**PP 的延迟分解与气泡**

**模型切分：**将 $L$ 层模型均匀分配给 $p$ 个 GPU。GPU $k$ 负责层 $[k\cdot L/p, (k+1)\cdot L/p-1]$。GPU $k$ 的输入是 GPU $k-1$ 的输出（最后一层的激活）。

**朴素流水线：**一次前向传播中，GPU $k$ 等待 GPU $k-1$ 完成其层的前向计算后，接收激活并开始自己的计算。总前向延迟为所有 $p$ 个 GPU 的层计算之和（即完整模型的前向延迟）。

**流水线气泡（Pipeline Bubble）：**在朴素流水线中，某时刻只有 1 个 GPU 在工作，其余 $p-1$ 个 GPU 空闲。这种空闲时间称为" 气泡"。

**微批次（Micro-Batch）：**将原始批次拆分为 $m$ 个微批次。每个微批次依次注入流水线。当微批次 $i$ 在 GPU $k$ 上完成计算并送到 GPU $k+1$ 后，GPU $k$ 可以立即开始处理微批次 $i+1$。

```
时间 →
GPU 0: | F0,1 | F0,2 | F0,3 | F0,4 | idle | idle | idle |
GPU 1: | idle | F1,1 | F1,2 | F1,3 | F1,4 | idle | idle |
GPU 2: | idle | idle | F2,1 | F2,2 | F2,3 | F2,4 | idle |
GPU 3: | idle | idle | idle | F3,1 | F3,2 | F3,3 | F3,4 |
```

图 18.2: GPipe 调度（$p=4, m=4$）。$\mathrm{F}_{k,i}$ 表示 GPU $k$ 上第 $i$ 个微批次的前向传播。

### 18.2.2　流水线调度策略

**主流流水线调度方案**

**GPipe 调度：**将所有 $m$ 个微批次依次全部完成前向传播，然后依次全部完成后向传播（用于训练）。对于推理场景（仅需前向传播），$m$ 个微批次的注入阶段和排空阶段各占 $(p-1)$ 个时间单位。

$$\text{气泡占比} = \frac{p-1}{m+p-1} \approx \frac{p-1}{m} \quad (\text{当} m \gg p) \tag{18.10}$$

要控制气泡率在 10% 以下，需要 $m \geq 10(p-1)$。对于 $p=8$，需要 $m \geq 70$ 个微批次。

**1F1B（One-Forward-One-Backward）调度：**交替执行前向和反向传播，以减少激活内存（训练场景的主要动机）。在推理场景中不适用（无反向传播），但概念上可

以理解为交错执行不同请求的 prefill 和 decode。

**交错式 1F1B（Interleaved 1F1B）:**每个 GPU 负责多个不连续的流水线阶段（stage）。例如，一个 4 层模型分布在 2 个 GPU 上，GPU 0 负责层 0 和层 2，GPU 1 负责层 1 和层 3。这进一步减小了气泡，因为每个 GPU 有更多机会在处理不同阶段之间切换。

**推理场景中的 PP 优化：**

- 对于纯推理（无反向传播），前向传播的流水线调度简化为：按请求（而非微批次）交错。请求 $i+1$ 可以在请求 $i$ 离开 GPU $k$ 后立即注入。
- 请求级别的流水线：设置最大" 流水线深度"$D$（同时处于流水线中的请求数）。每个 GPU 维护一个工作队列，完成一个请求的当前层处理后立即从队列中取下一个。

**PP 在推理中的适用场景**

PP 的主要优势在于**扩大模型容量**——使得超过单 GPU 显存的超大模型可以运行。

PP 的劣势在于**延迟不成比例增加**——流水线气泡使每个 token 的生成延迟增加。

**典型使用策略：**

- 对于模型大小远大于单 GPU 显存的情况，PP 是必需的（如 405B 模型装载在 8×80GB A100 上）。
- PP 通常与 TP 结合使用：TP（高带宽开销，低延迟）用于节点内（NVLink），PP（低带宽开销，有气泡）用于跨节点。
- 对于延迟敏感的在线服务，尽量使用 TP 而非 PP。对于吞吐量优先的离线批处理，PP 可以与 TP 混合使用。

## 18.3 专家并行

混合专家（Mixture-of-Experts, MoE）模型将 FFN 层的参数量扩大了 $E$ 倍（$E$ 为专家数量），但每个 token 只激活其中的 $k$ 个专家（通常 $k=1$ 或 $k=2$），从而实现参数量暴增但计算量可控。MoE 的分布式策略即为专家并行（Expert Parallelism, EP）。

### 18.3.1　MoE 路由与专家分配

**MoE 的门控与路由**

一个 MoE 层的计算流程为：

1. **门控（Gating）**：对每个 token $i$，计算路由分数 $\boldsymbol{g}_i = \text{softmax}(\boldsymbol{x}_i \boldsymbol{W}_g) \in \mathbb{R}^E$。
2. **Top-$k$ 选择**：选择分数最高的 $k$ 个专家。$\mathcal{E}_i = \text{TopK}(\boldsymbol{g}_i, k)$。
3. **归一化**：对选中的专家分数重新归一化：$g'_{i,e} = g_{i,e} / \sum_{e' \in \mathcal{E}_i} g_{i,e'}$。
4. **专家计算**：$\boldsymbol{y}_i = \sum_{e \in \mathcal{E}_i} g'_{i,e} \cdot \text{FFN}_e(\boldsymbol{x}_i)$。

**专家并行**：每个专家 $\text{FFN}_e$ 分配到一个 GPU（或多个 GPU 通过 TP 进一步拆分）。门控网络确定每个 token 需要访问哪些专家，然后通过 All-to-All 通信将 token 发送到对应的专家 GPU。

### 18.3.2　All-to-All 通信

**MoE 中的 All-to-All 通信模式**

对于 $p$ 个 GPU、$E$ 个专家（假设 $E = p$，每个 GPU 一个专家），All-to-All 通信分两个阶段：

**第一阶段（Token Dispatch，分发）**：每个 GPU 计算其本地 token 的路由决策，将属于专家 $e$ 的 token 发送到 GPU $e$。

$$\text{发送量}_{\text{GPU } i} = \sum_{e \neq i} (\text{GPU } i \text{ 上分配到 GPU } e \text{ 的 token 数}) \times d \tag{18.11}$$

**第二阶段（Token Combine，合并）**：每个 GPU $e$ 收到来自所有 GPU 的 token 后，执行专家 $\text{FFN}_e$ 的计算，然后将结果返回给原始 GPU。

$$\text{发送量}_{\text{GPU } e} = \sum_{i \neq e} (\text{GPU } i \text{ 发来的 token 数}) \times d \tag{18.12}$$

**总通信量**：每个 token（若不考虑负载均衡）需要被发送到 $k$ 个不同的 GPU 并返回。总 All-to-All 通信量为 $2 \times k \times (\text{total tokens}) \times d \times \text{bytes}$。

**通信带宽需求**：对于千亿级 MoE 模型（如 Mixtral-8x7B：8 个专家，$k = 2$，$d = 4096$），若每个 batch 处理 4096 个 token：

$$\text{通信量} \approx 2 \times 2 \times 4096 \times 4096 \times 2 \approx 128 \text{ MB} \tag{18.13}$$

需要 All-to-All 在微秒级完成，要求高带宽互联（NVLink 或 InfiniBand）。

### 18.3.3 专家负载均衡

**负载不均衡问题与解决方案**

**问题**：如果某些专家被远比其他专家更多的 token 选中，这些 GPU 会成为瓶颈（其他 GPU 空闲等待）。

**容量因子（Capacity Factor, CF）**：每个专家的容量 $C = \text{CF} \times \frac{k \times T}{E}$，其中 $T$ 为总 token 数。CF 通常设为 1.25-1.5。超过容量的 token 被" 溢出"（routed to residual）或丢弃。

**辅助损失（Auxiliary Loss）**：在训练中增加负载均衡损失：

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_{e=1}^{E} f_e \cdot P_e \tag{18.14}$$

其中 $f_e$ 为分配给专家 $e$ 的 token 占比，$P_e$ 为门控分配给专家 $e$ 的平均概率，$\alpha$ 为权重系数。最小化此损失鼓励均匀分配。

**DeepSeek-V3 的辅助损失自由负载均衡**：DeepSeek-V3 引入了动态偏置（bias）调整机制，在每个训练步动态调整每个专家的门控偏置，使得：

- 过载专家的偏置减小，降低未来 token 选中它的概率。
- 负载不足专家的偏置增大，增加未来 token 选中它的概率。
- 无需辅助损失项，避免了辅助损失对模型质量的负面影响。

**共享专家（Shared Expert）**：DeepSeek-V3 引入了一个" 共享专家"——所有 token 始终经过此共享专家，再加上 $k$ 个" 路由专家"。共享专家捕获通用知识，路由专家专注于领域特定知识。共享专家不需要路由决策，始终在所有 GPU 上计算。

## 18.4 数据并行

**推理场景中的数据并行**

在推理中，数据并行（Data Parallelism, DP）是最简单的分布式策略：每个 GPU 拥有完整的模型副本，独立处理不同的请求。

**推理 DP 与训练 DP 的区别**：

- **训练 DP**：每个 GPU 处理不同的数据批次，在反向传播后需要 All-Reduce 梯度。通信量大且频繁。
- **推理 DP**：每个 GPU 独立处理请求，**前向传播期间不需要任何通信**（除非使用了 TP/PP）。这使 DP 在推理中成为近乎完美线性扩展的策略。

**吞吐量扩展**：设单 GPU 吞吐量为 $\text{TPS}_1$，使用 $p$ 个 GPU 的 DP 集群吞吐量理论上

为：

$$\mathrm{TPS}_p = p \times \mathrm{TPS}_1 \tag{18.15}$$

前提是请求负载均衡、无 KV Cache 共享需求、每个 GPU 有足够的内存存放完整模型。

**DP 的限制：**

- 每个 GPU 必须有足够显存放完整模型。对于 400B+ 模型，单 GPU 无法容纳。
- KV Cache 不能跨 GPUs 共享，多轮对话的上下文共享需要在应用层处理。
- 当使用共享前缀优化（RadixAttention）时，DP 的不同实例无法共享同一物理 KV Cache 池。

## 18.5　混合并行与自动搜索

### 18.5.1　TP+PP+DP+EP 的组合策略

**混合并行的典型配置**

大规模 LLM 推理通常采用"3D 并行" 或"4D 并行"：

表 18.1: 混合并行各维度的特性对比

| 维度 | 切分目标 | 通信频率 | 通信带宽需求 | 典型互联 |
|---|---|---|---|---|
| TP | 层内矩阵 | 每层 2-4 次 | 极高（TB/s 级） | NVLink/NVSwitch |
| PP | 层间 | 每层 1 次 | 中等（GB/s 级） | InfiniBand/RoCE |
| DP | 请求 | 无（推理） | 无 | 无 |
| EP | 专家权重 | 每 MoE 层 2 次 | 高（百 GB/s 级） | NVLink/IB |

**典型配置示例（8 节点 × 8 GPU = 64 GPU 部署 LLaMA-3-405B）：**

- **节点内：** $p_{\text{TP}} = 8$（使用 NVSwitch 全互联）。每 GPU 分得 $405/8 \approx 50.6$B 参数的权重。
- **跨节点：** $p_{\text{PP}} = 8$（每个节点负责 16 层中的 2 层，或使用 interleaved 策略）。
- **DP：** $p_{\text{DP}} = 1$（每个 64 GPU" 副本" 独立处理请求）。或 $p_{\text{DP}} = 2$（两套独立的 64 GPU 副本）。

**资源映射：**

$$\text{GPU 总数} = p_{\text{TP}} \times p_{\text{PP}} \times p_{\text{DP}} \times p_{\text{EP}} \tag{18.16}$$

其中 $p_{\text{EP}} = 1$（或等于专家数 $E$，每个 GPU 一个或多个专家）。

### 18.5.2 自动并行策略搜索

**自动并行化方法**

手动为每个模型和硬件配置选择最优并行策略极为困难。自动化方法旨在自动搜索最优并行配置。

**Alpa（Stanford, 2022）：**将并行策略搜索分解为两个层次：

1. **算子间并行（Inter-Operator）：**决定哪个算子（层）放在哪个设备（PP 风格）。使用动态规划最小化通信成本。
2. **算子内并行（Intra-Operator）：**对于每个算子，决定 TP 切分方式。使用整数线性规划（ILP）优化。

**成本模型：**

$$\text{Cost} = \sum_{\text{所有算子}} (T_{\text{compute}} + T_{\text{communication}}) + \sum_{\text{流水线气泡}} T_{\text{bubble}} \tag{18.17}$$

其中：

- $T_{\text{compute}}$：基于 roofline model 的算子计算时间估计。
- $T_{\text{communication}}$：基于通信量/带宽的估计。
- $T_{\text{bubble}}$：基于流水线调度的气泡估计（使用 micro-batch count 和 stage count）。

**搜索空间：**

- 每个线性层的切分方式（列并行、行并行、无并行）。
- 每个 Transformer 块放置在哪个设备上。
- 每个注意力层和 FFN 的并行度。

**简化策略：**在实践中，多数推理引擎使用简单的启发式规则：

- 优先用 DP 满足吞吐量需求。
- 当模型太大单 GPU 放不下时，先用 TP（节点内），再用 PP（跨节点）。
- 对于 MoE，EP 在节点间使用（若专家数 > 节点内 GPU 数），TP 在节点内使用。

## 18.6　通信集合原语

### 18.6.1　NCCL 与集合通信算法

**核心集合通信原语**

**All-Reduce：**每个 GPU 提供一个数组 $\boldsymbol{x}_i$，所有 GPU 获得 $\sum_i \boldsymbol{x}_i$。
**Ring All-Reduce：**

1. 将 $\boldsymbol{x}_i$ 切分为 $p$ 个块（scatter-reduce 阶段）。
2. 每个 GPU 将其中的一个块发送给环形拓扑的下一个 GPU，收到上一个 GPU 的对应块后累加到本地。
3. 经过 $p-1$ 步后，每个 GPU 拥有一个完整块的全局和。
4. All-Gather 阶段：每个 GPU 将其完整块沿环传播，$p-1$ 步后所有 GPU 拥有完整结果。

总通信量：$2(p-1)/p \cdot |\boldsymbol{x}|$ 每个 GPU。带宽最优（在下界意义上）。
**Tree All-Reduce：**构建一棵二叉树（或双二叉树以实现双向带宽利用）。叶子节点发送数据到父节点，父节点累加后向上传递。根节点拥有完整结果后沿树向下广播。延迟（步数）为 $O(\log p)$，但对带宽利用率不如 ring。

**Recursive Halving-Doubling（递归折半加倍）**：专为大消息设计的 all-reduce 算法。通过递归地对半交换数据实现全归约。步数为 $2\log_2(p)$。
**NCCL 的算法选择**：NCCL 根据消息大小自动选择算法：

- 小消息（$<$ 256 KB）：树形算法（低延迟优先）。
- 中等消息（256 KB $\sim$ 4 MB）：Ring 算法。
- 大消息（$>$ 4 MB）：Recursive Halving-Doubling。

### 18.6.2 互联技术

**GPU 互联技术栈**

**NVLink（GPU-to-GPU）：**

- NVLink 3.0（A100）：600 GB/s 双向带宽 per link，最多 12 links。
- NVLink 4.0（H100）：900 GB/s 双向带宽 per link，最多 18 links。总带宽 900 GB/s。
- NVSwitch：所有 GPU 通过 NVSwitch 全互联（non-blocking crossbar）。DGX H100 中 8 个 GPU 全互联，任意两 GPU 间带宽 900 GB/s。

**PCIe：**

- PCIe 5.0 x16：约 64 GB/s 双向带宽。
- 用于 GPU-CPU 通信和跨节点 NVLink 无覆盖时的 GPU 间通信。
- 带宽远低于 NVLink，不适合频繁的张量并行通信。

**InfiniBand（节点间互联）：**

- NDR400：400 Gbps（50 GB/s）单向带宽 per port。
- RDMA（Remote Direct Memory Access）：GPU 可以直接读写远程节点的 GPU 内存，无需 CPU 介入。
- GPUDirect RDMA：NVIDIA 的优化，允许 NIC（网卡）直接访问 GPU 显存。

**NVLink-C2C（Grace-Hopper）：**

- CPU（Grace）和 GPU（Hopper）之间的高速互联。
- 900 GB/s 双向带宽。

- 实现 CPU 和 GPU 的统一内存访问，对于 KV Cache 的 CPU 卸载非常有利。

**NVLink Switch（跨节点扩展）：**

- 将多个 DGX 节点通过 NVSwitch 互联，形成更大的 GPU 域。
- 例如，NVIDIA DGX SuperPOD 连接 32 个 DGX H100 节点（256 个 GPU）。

### 18.6.3　通信与计算的重叠

**通信-计算重叠技术**

通过将通信和计算分配到不同的 CUDA 流（stream），可以在 GPU 执行计算的同时进行通信，隐藏通信延迟。

**All-Reduce 重叠：**将张量 $\boldsymbol{A}$ 切分为多个子张量。当最后一个子张量计算完成时，立即对该子张量启动 All-Reduce（使用单独 stream），同时计算继续在下一个算子中进行。

**具体实现：**在行并行线性层的输出中，将 $\boldsymbol{Y}$ 按行切分为两个子张量 $\boldsymbol{Y}_a$ 和 $\boldsymbol{Y}_b$：

```
// Stream 0: 计算 Y_a
Y_a = X_a @ W_a;
// 启动 Y_a 的 All-Reduce（Stream 0）
// 同时 Stream 1: 计算 Y_b
Y_b = X_b @ W_b;
// 等待 Y_a All-Reduce 完成
// 对 Y_a 的后续操作可以开始（如 residual add, LayerNorm）
// 同时 Y_b All-Reduce 仍在进行
```

**收益分析：**假设 All-Reduce 通信时间为 $T_{\text{comm}}$，可重叠的计算时间为 $T_{\text{comp}}$。如果 $T_{\text{comm}} \leq T_{\text{comp}}$，通信延迟可以被完全隐藏。在 decode 阶段（$n = 1$），All-Reduce 通信量极小（几 KB），$T_{\text{comm}}$ 远小于计算时间，重叠的收益有限。在 prefill 阶段（$n$ 大），通信量大，重叠收益显著。

**FP8 通信：**使用 FP8 进行 TP 通信（quantize activations to FP8 before All-Reduce, then dequantize），将通信量减半。H100 支持 FP8 Tensor Core，通信和计算都能利用 FP8。

### 18.6.4 分布式 KV Cache

**分布式环境中的 KV Cache 管理**

**TP 下的 KV Cache：**张量并行中，注意力头被分布到不同 GPU。每个 GPU 只存储其负责的注意力头的 KV Cache。因此，每个 GPU 的 KV Cache 量是全模型的 $1/p_{\text{TP}}$。这是 TP 的重要附带优势。

**PP 下的 KV Cache：**流水线并行中，每个 GPU 存储其负责层的 KV Cache。不同层的 KV Cache 分布在不同 GPU 上。总 KV Cache 量在全 GPU 集合中，但每 GPU 只有 $1/p_{\text{PP}}$ 层。

**分离式架构下的 KV Cache 传输：**当使用 prefill-decode 分离架构时，KV Cache 需要从 prefill 实例传输到 decode 实例。传输格式和压缩策略至关重要：

- **未压缩传输：**以原始 fp16 格式传输完整 KV Cache。
- **量化传输：**传输前将 K 和 V 量化为 int8（减少 50% 传输量），decode 实例接收后再反量化回 fp16 用于计算。FP8 压缩可进一步减少 50%。
- **层间压缩传输：**利用相邻层的 KV 相关性，只传输部分层的 KV，decode 实例通过插值恢复中间层的 KV（如 CLA/层间共享）。

**KV Cache 的路由：**在多节点部署中，一些请求可能需要在不同节点间迁移（如节点故障转移、负载均衡调整）。KV Cache 的迁移需要：

1. 从源节点读出所有层的完整 KV Cache。
2. 打包为连续消息，通过 RDMA 发送到目标节点。
3. 目标节点将 KV Cache 写回对应的层位置。
4. 更新路由表，使后续请求到达新节点。

对于 80 层、$h_{kv} = 8$、$d_h = 128$、fp16、$n_{\text{kv}} = 8192$ 的情况，迁移量约为 $2 \times 80 \times 8192 \times 8 \times 128 \times 2 \approx 2.68$ GB。使用 400Gbps InfiniBand 约需 54ms。

**分布式推理的协调开销**

分布式的引入并非无代价的。随着并行度增加，以下开销非线性增长：

- **通信同步开销：**即使通信时间被计算重叠，同步屏障（barrier）的固定开销随 GPU 数量增长。
- **负载不均衡：**不同 GPU 的计算时间不可能完全相等。最慢的 GPU 决定整个迭代的延迟（straggler）。
- **启动延迟：**CUDA kernel 启动、NCCL 集合通信初始化的延迟在 $n = 1$（decode）

场景下占比显著。

- **故障域扩大：**一个 GPU 故障可能导致整个 TP 组或 PP 流水线崩溃。

在实践中，推理服务的 TP 通常不超过 8（单节点 NVLink 域），PP 不超过 8（控制气泡和延迟），DP 根据请求量线性扩展。

# 第十九章　GPU 编程与 CUDA 优化

推理引擎的性能天花板最终由 GPU 程序的执行效率决定。无论上层框架设计得多么精巧，若底层 CUDA kernel 不能有效利用 GPU 的计算和存储资源，一切优化都将事倍功半。本章从 GPU 硬件架构出发，深入探讨 CUDA kernel 的编写技巧、Tensor Core 编程范式以及内存访问优化策略，为构建高性能推理引擎提供底层技术支撑。

## 19.1　GPU 架构与执行模型

### 19.1.1　流多处理器架构

**NVIDIA H100 的 SM 架构详解**

H100 GPU 包含 132 个流多处理器（Streaming Multiprocessor, SM），其中 108 个始终可用（其余用于冗余和功耗管理）。每个 SM 的内部结构：

表 19.1: H100 SM 的关键资源

| 资源 | 规格 |
|---|---|
| CUDA 核心（FP32/INT32） | 128 个（4 个处理块，每块 32 个核心） |
| 第四代 Tensor Core | 4 个（每处理块 1 个） |
| 寄存器文件 | 65,536 个 32 位寄存器 |
| 共享内存/L1 缓存（统一可配） | 最多 228 KB |
| 最大线程数 | 2048 |
| 最大 Warp 数 | 64 |
| 最大线程块数 | 32 |
| 最大线程块线程数 | 1024 |
| L1 缓存带宽 | 约 4 TB/s per SM |
| L2 缓存 | 50 MB（所有 SM 共享），约 12 TB/s 带宽 |
| HBM3 | 80 GB，约 3.35 TB/s 带宽 |

**SM 内的执行组织：**

- CUDA 核心被组织为 4 个处理块（processing partition），每块有 32 个 FP32

核心和 16 个 FP64 核心。

- 每个处理块有各自的 Warp Scheduler 和 Dispatch Unit。
- Tensor Core 独立于 CUDA 核心，用于专门的矩阵乘法加速。
- 共享内存被组织为 32 个 bank（每 bank 4 字节宽），以实现单周期并发访问。

### 19.1.2 Warp 与 SIMT 执行模型

**Warp 的核心概念**

NVIDIA GPU 最核心的执行单元是 Warp——由 32 个线程组成的线程束。同一 Warp 内的所有线程在任意时刻执行相同的指令（SIMT: Single Instruction, Multiple Threads）。

**Warp 发散（Warp Divergence）**：当 Warp 内线程执行条件分支时：

```
if (threadIdx.x < 16) {
    // 路径 A: 前16个线程执行
} else {
    // 路径 B: 后16个线程执行
}
```

前 16 个线程执行路径 A 时后 16 个线程被屏蔽（masked），然后后 16 个线程执行路径 B 时前 16 个线程被屏蔽。这导致执行时间翻倍。应尽量避免 Warp 内的数据依赖分支。

**线程块（Thread Block）**：一个线程块（Thread Block, TB）包含若干 Warp，被调度到同一个 SM 上执行。TB 内的线程可以通过共享内存进行通信和同步（`__syncthreads()`）。同一 TB 的所有线程共享同一份共享内存和 L1 缓存。

**Grid**：一个 CUDA kernel 的所有线程块组成 Grid。Grid 可以是一维、二维或三维的。不同 TB 之间不能直接同步（只能通过全局内存原子操作实现有限的协调）。

### 19.1.3 显存层级

**GPU 显存层级的速度与容量模型**

GPU 的显存层级自顶向下形成了容量递增、速度和带宽递减的金字塔结构：

表 19.2: GPU 显存层级特性（H100 典型值）

| 层级 | 容量 | 带宽 | 延迟 | 作用域 | 管理方式 |
|---|---|---|---|---|---|
| 寄存器 | 256 KB/SM | ∼8 TB/s | ∼0 cycles | 单线程 | 编译器自动分配 |
| 共享内存 | 228 KB/SM | ∼4 TB/s | ∼20 cycles | 单 TB | 程序员显式管理 |
| L1 缓存 | 256 KB/SM | ∼4 TB/s | ∼30 cycles | 单 SM | 硬件自动管理 |
| L2 缓存 | 50 MB | ∼12 TB/s | ∼200 cycles | 全 GPU | 硬件自动管理 |
| HBM3 | 80 GB | ∼3.35 TB/s | ∼300 cycles | 全 GPU | 程序员管理（cudaMalloc） |

**显存层级优化原则：**

- **时间局部性（Temporal Locality）**：对多次使用的数据，尽可能保持在寄存器或共享内存中。
- **空间局部性（Spatial Locality）**：让同一 Warp 的线程访问连续的内存地址（合并访问，coalesced access）。
- **减少 HBM 访问**：HBM 访问的延迟约 300 个周期，期间如果 Warp 没有其他指令可执行就会 stall。通过高占用率（occupancy）让其他 Warp 在这期间执行来隐藏延迟。

**Roofline 模型**：算术强度（arithmetic intensity）$I = \text{FLOPs/bytes}$。当 $I$ 低于机器平衡点时，kernels 受限于显存带宽。当 $I$ 高于平衡点时，受限于峰值算力。
对于 H100（FP16 Tensor Core，989 TFLOPS，HBM 带宽 3.35 TB/s）：

$$I_{\text{balance}} = \frac{989 \times 10^{12}}{3.35 \times 10^{12}} \approx 295 \text{ FLOPs/byte} \tag{19.1}$$

这意味着每个从 HBM 加载的字节需要约 295 次浮点运算才能充分利用算力。绝大多数推理 kernel 的 $I$ 远低于此值——它们本质上是带宽受限的。

## 19.2　CUDA Kernel 优化技术

### 19.2.1　占用率优化

**占用率的计算与优化**

占用率（Occupancy）定义为 SM 上活跃 Warp 数与理论最大 Warp 数的比值：

$$\text{Occupancy} = \frac{\#\text{Active Warps per SM}}{\text{Max Warps per SM}} \tag{19.2}$$

高占用率允许 GPU 在部分 Warp 因访存而 stall 时，切换到其他就绪的 Warp 执行——这是 GPU 隐藏内存延迟的核心机制。

**限制占用率的三种资源：**

1. **寄存器数量**：每个线程使用的寄存器越多，每个 SM 可容纳的线程越少。H100 中每 SM 有 65536 个寄存器，每个线程最大可用 255 个。若 kernels 中每线程使用 128 个寄存器，则每 SM 最多约 512 线程（16 个 Warp），占用率约 25%。
2. **共享内存**：每个 TB 使用的共享内存越多，每个 SM 可容纳的 TB 越少。H100 中每 SM 最多 228KB 共享内存。若每 TB 使用 64KB，则每 SM 最多 3 个 TB。
3. **线程块大小**：每 SM 最多 32 个 TB 或 2048 线程，取较小者。如果每 TB 有 512 线程，则最多 4 个 TB（=2048 线程）；如果用 1024 线程，则最多 2 个 TB。

**占用率计算示例**：一个 kernel 使用每线程 64 个寄存器，每 TB 256 线程（8 Warps），每 TB 48KB 共享内存：

- 寄存器限制：65536 / 64 = 1024 线程 → 4 个 TB
- 共享内存限制：228 / 48 ≈ 4.75 → 4 个 TB
- 线程块限制：max 32 TB
- 线程限制：$4 \times 256 = 1024 \leq 2048$

有效：4 个 TB × 8 Warp/TB = 32 个活跃 Warp，占用率 32/64 = 50%。

**使用 CUDA Occupancy Calculator**：通过`--ptxas-options=-v` 编译 flags 查看寄存器使用量，然后使用 CUDA Occupancy Calculator API（`cudaOccupancyMaxActiveBlocksPerMultiprocessor`）精确计算占用率并调整 TB 大小。

### 19.2.2 共享内存编程

**共享内存的使用模式**

共享内存（Shared Memory, SMEM）是程序员手动管理的片上高速暂存器。正确使用共享内存可以将 HBM 访问转换为 SMEM 访问，获得 10 ∼ 30× 的带宽提升。

**使用模式一：数据复用（Data Reuse）**

对于多次访问的 HBM 数据，先加载到共享内存：

```
__shared__ float tile[BLOCK_SIZE][BLOCK_SIZE];

// 协同加载：每个线程加载一个元素到共享内存
```

```
int row = blockIdx.y * BLOCK_SIZE + threadIdx.y;
int col = blockIdx.x * BLOCK_SIZE + threadIdx.x;
tile[threadIdx.y][threadIdx.x] = global_data[row * N + col];
__syncthreads();

// 需要时从共享内存读取（比HBM快30倍）
float val = tile[threadIdx.y][threadIdx.x];
```

**使用模式二：线程间通信（Inter-Thread Communication）**
当 TB 内的线程需要交换数据时（如归约操作），共享内存是唯一的低延迟通道。
**银行冲突（Bank Conflict）**：共享内存分为 32 个银行（每银行 4 字节），连续的 4 字节字落在连续的银行上。当同一 Warp 的多个线程访问同一银行的不同地址时，发生银行冲突——这些访问被顺序化。

```
// 2路银行冲突示例（Warp内线程0和16访问同一银行）
__shared__ float data[32][32];
// 线程i访问 data[i][i] —— 无冲突（i不同但地址在同一银行的不同偏移）
// 线程i访问 data[threadIdx.x][0] —— 无冲突（所有线程访问不同银行）
// 线程i访问 data[0][threadIdx.x*2] —— 2路冲突（线程0和16访问bank 0的不同地址）
```

**避免银行冲突**：将共享内存数组的列维度填充 1：

```
__shared__ float data[32][33];  // 32+1=33，不是32的倍数
// data[i][j] 在 bank = (i * 33 + j) % 32
```

### 19.2.3　合并内存访问

**全局内存的合并访问规则**

全局内存（HBM）的访问以 32 字节、64 字节或 128 字节的”事务”（transaction）为单位。当 Warp 中 32 个线程访问的地址落在尽可能少的 L2 缓存行（128 字节）中时，访问是**合并的（coalesced）**，达到接近峰值的带宽利用率。
**合并访问条件**（对 128 字节的 L2 缓存行）：Warp 中的所有线程访问的地址应落在同一 128 字节对齐的缓存行内，且地址连续。

```
// 合并访问（线程i访问 base + i*4）
float val = global_array[threadIdx.x + blockIdx.x * blockDim.x];

// 非合并访问（线程i访问 base + i*stride*4, stride>1）
float val = global_array[(threadIdx.x * stride) + base];
```

```
// 当stride=2时，线程覆盖256字节范围，需要2个128字节事务（利用率50%）
// 当stride=32时，线程分散在4096字节范围，需要32个事务（利用率约3%）
```

**向量化加载（Vectorized Load）**：使用更宽的数据类型一次加载更多字节，减少事务数：

```
// 每个线程加载 float4（16字节），代替4次单独的 float 加载
float4 val = reinterpret_cast<const float4*>(&global_array[base])[threadIdx.x];
// 4个float × 32线程 = 128字节 = 恰好1个L2缓存行事务
```

向量化加载还减少指令数（1 条 ldg.128 代替 4 条 ldg.32），降低指令缓存压力。

### 19.2.4 CUDA 流与异步执行

**CUDA 流的并发编程模型**

CUDA 流（Stream）是一系列按顺序执行的异步 CUDA 操作。不同流之间的操作可以并发执行，实现计算与数据传输的重叠。

**基本用法：**

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

// 流1: kernel计算
// 流2: HBM CPU数据传输
kernel<<<grid, block, 0, stream1>>>(d_input, d_output);
cudaMemcpyAsync(h_output, d_output, size, cudaMemcpyDeviceToHost, stream2);

cudaStreamSynchronize(stream2);  // 等待特定流完成
```

**默认流与非阻塞流：**

- **默认流（Stream 0）**：隐式同步流。在默认流上的操作会等待所有其他流的操作完成。
- **非阻塞流：** 使用 `cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking)` 创建。不同非阻塞流之间可真正并行。

**CUDA 事件（Event）**：使用事件在流之间建立精确的同步点：

```
cudaEvent_t event;
cudaEventCreate(&event);
```

```
cudaEventRecord(event, stream1);  // stream1到达此处时记录事件
cudaStreamWaitEvent(stream2, event, 0);  // stream2等待event发生
```

**CUDA 图（CUDA Graph）**：将一系列 kernel 启动和内存操作捕获为一个图结构，之后可以选择性地重用（replay）。这消除了重复 kernel 启动的 CPU 开销（对于小 kernels 尤为显著）。

```
cudaGraph_t graph;
cudaGraphExec_t instance;

// 捕获阶段
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
// ... 一系列 kernel 启动和 memcpy ...
cudaStreamEndCapture(stream, &graph);

// 实例化和复用
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
cudaGraphLaunch(instance, stream);  // 单次API调用启动整个graph
```

CUDA Graph 对于推理引擎中的固定计算图（每次迭代相同 kernel 序列）带来了显著的 CPU 侧延迟降低。

## 19.3　Tensor Core 编程

### 19.3.1　Tensor Core 基础

**Tensor Core 的计算模型**

Tensor Core 是 NVIDIA GPU 中专用于矩阵乘法加速的硬件单元。与 CUDA Core（一次只算一个标量 FMA）不同，Tensor Core 一次计算一个小的矩阵乘法块，吞吐量远超 CUDA Core。

**Tensor Core 的演化：**

表 19.3: Tensor Core 各代规格

| 架构 | FP16 mma 形状 | TF32 mma 形状 | FP8 支持 | 稀疏加速 |
|---|---|---|---|---|
| Volta (V100) | m8n8k4 | - | - | - |
| Turing (T4) | m8n8k4 | - | INT8/INT4 | - |
| Ampere (A100) | m16n8k8 | m16n8k4 | - | 2:4 结构化 |
| Hopper (H100) | m16n8k8 | m16n8k4 | FP8 E4M3/E5M2 | 2:4 结构化 |
| Blackwell (B200) | m16n8k8 | m16n8k4 | FP8/FP6/FP4 | 2:4 结构化 |

**Ampere mma 指令（PTX ISA）：**

```
mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16
    {d0, d1, d2, d3}, {a0, a1, a2, a3}, {b0, b1}, {c0, c1, c2, c3};
```

在一条 mma 指令中，4 个线程协作完成 $16 \times 8 \times 16$ 的矩阵乘法（使用线程间寄存器数据共享）。

**WMMA API（Warp Matrix Multiply-Accumulate）：**更高级的 C++ API，封装了底层 mma 指令：

```
#include <cuda_fp16.h>
#include <mma.h>
using namespace nvcuda;

wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, half> c_frag;

wmma::load_matrix_sync(a_frag, a_ptr, lda);
wmma::load_matrix_sync(b_frag, b_ptr, ldb);
wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);  // C += A × B
wmma::store_matrix_sync(c_ptr, c_frag, ldc, wmma::mem_row_major);
```

### 19.3.2　Hopper 的异步 Tensor Core

**Hopper 架构的高级 Tensor Core 特性**

**WGMMA（Warp-Group MMA）：**Hopper 引入 wgmma 指令，操作数可以不经过共享内存直接从寄存器或共享内存读取。wgmma 以 warp-group（4 个连续 warp，128 线程）为单位调度异步矩阵乘法。

```
// wgmma: 异步启动，返回后直接继续执行
asm volatile(
    "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 "
    "{%0, %1, %2, %3, ..., %15}, "
    "%16, %17, "
    "%18, %19, %20, %21, "
    "1, 1, 1, 1;\n"
    : "+f"(d0), "+f"(d1), ...
    : "r"(desc_a), "r"(desc_b), "f"(c0), "f"(c1), ...
);
```

关键点是 wgmma 是**异步的**——指令启动后，Warp-Group 立即返回，可继续执行其他操作（如加载下一块数据），无需等待矩阵乘法完成。通过 `wgmma.wait_group` 进行显式同步。

**TMA（Tensor Memory Accelerator）：**TMA 是 Hopper 引入的独立硬件单元，可以异步地将多维张量块从 HBM 复制到共享内存，或从共享内存复制到 HBM。TMA 支持的复制模式包括 2D tile、3D box 复制，以及边界处理。

```
// 创建 TMA 描述符
CUtensorMap tma_desc;
// 设置描述符：形状、步幅、数据格式等
// 使用 cp.async.bulk.tensor 指令发起 TMA 复制
// 这是硬件加速的2D拷贝，无需占用CUDA核心
```

**TMA+WGMMA 的流水线：**理想情况下，使用双缓冲（或多缓冲）共享内存：

1. TMA 将下一个数据块从 HBM 异步加载到共享内存 Buffer [next]。
2. 同时，WGMMA 使用当前 Buffer [cur] 执行矩阵乘法。
3. 通过异步屏障（async barrier）同步，将 [next] 和 [cur] 交换。

这种模式几乎完全隐藏了 HBM 数据搬运的延迟。

# 19.4 Kernel 融合

## 19.4.1 Kernel 融合的原理与收益

**Kernel 融合的核心价值**

在未融合的执行中，每个算子作为独立的 CUDA kernel 执行：

```
// Kernel 1: MatMul (X @ W1) -> write to HBM
// Kernel 2: BiasAdd + GELU -> read from HBM, compute, write to HBM
// Kernel 3: Dropout -> read from HBM, compute, write to HBM
// Kernel 4: MatMul (X @ W2) -> read from HBM, compute, write to HBM
```

每个 kernel 写入 HBM 的输出立刻被下一个 kernel 读回——这就是”读写往返”（round-trip）。这种往返既浪费带宽又增加延迟。

**融合后的 kernel**：将上述操作合并为一个 kernel：

```
// Fused Kernel:
// 从HBM读取X和W1
// 分块计算 X@W1，保存在共享内存
// 直接在共享内存/寄存器中应用 bias + GELU
// 直接从共享内存读取并计算 @W2
// 将最终结果写入HBM（一次写入）
```

**融合收益的量化**：假设每层 FFN 的参数为 $P_{FFN} = 4d^2$（$d = 8192$ 时约 268M = 1GB fp16），每层激活为 $b \times n \times d$（$b = 1, n = 1$ 时仅 16KB）。

未融合时的 HBM 读写量（decode 阶段）：

- 读取 W1（列并行）：约 286MB（$d \times d_{ff}/p_{TP}$）
- 写入中间激活：$b \times n \times d_{ff}$ 0（对于 n=1 可忽略）
- 读取中间激活（再次）：同上
- 读取 W2（行并行）：约 286MB
- 写入输出：$b \times n \times d$
- **总 HBM 流量** 572MB（主要由权重读取主导）

融合后的 HBM 读写量：

- 读取 W1：286MB
- 读取 W2：286MB（可部分与 W1 重叠）

- 写入输出：$b \times n \times d$　0
- **总 HBM 流量**　572MB（权重的读取仍是主导）

在 decode 阶段（$n = 1$），由于激活极小，融合的收益不在于中间张量的节省（中间张量本身微不足道），而在于减少 kernel 启动延迟和在权重读取间建立更好的流水线。在 prefill 阶段（$n$ 大），融合的收益更为显著——避免了完整中间激活张量的 HBM 往返。

### 19.4.2　协作组

**使用协作组实现灵活的同步**

CUDA 9 引入的协作组（Cooperative Groups）提供了比 `__syncthreads()` 更灵活的同步原语。

**粒度类型：**

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

// 获得外层线程块组
cg::thread_block tb = cg::this_thread_block();

// 将线程块划分为更小的tile组
cg::tiled_partition<32>(tb);                    // 每个warp一个组
cg::tiled_partition<16>(tb);                    // 每16个线程一个组
cg::tiled_partition<64>(tb);                    // 每2个连续warp一个组

// 在组内同步（仅组内成员等待）
cg::sync(tile_group);
```

**在融合 kernel 中的应用：**例如，在融合 MLP 中：

1. 将线程块划分为两个协作组：组 A 负责第一层线性 + 激活，组 B 负责第二层线性。
2. 组 A：加载 W1 分块 $\rightarrow$ 计算 $Y_1 = XW_1$ $\rightarrow$ 在共享内存中应用激活 $\rightarrow$ 标记完成。
3. 组 B：等待组 A 的完成信号 $\rightarrow$ 从共享内存读取 $Y_{\text{act}}$ $\rightarrow$ 加载 W2 分块 $\rightarrow$ 计算 $Y_2 = Y_{\text{act}}W_2$ $\rightarrow$ 写入 HBM。
4. 使用协作组同步（而非全局 `__syncthreads()`）避免不必要的全线程块同步。

## 19.5　多 GPU 推理的 CUDA 实现

### 19.5.1　NCCL 与 CUDA 的集成

**NCCL 的 CUDA 集成模式**

NCCL（NVIDIA Collective Communications Library）是 NVIDIA 提供的多 GPU 集合通信库，与 CUDA 流紧密集成。

**基本用法：**

```
ncclComm_t comm;
ncclCommInitRank(&comm, world_size, unique_id, rank);

// 在特定CUDA流上执行All-Reduce
ncclAllReduce(sendbuff, recvbuff, count, ncclFloat16, ncclSum, comm, stream);
```

**通信与计算的 CUDA 流重叠：**

```
cudaStream_t comp_stream, comm_stream;
cudaStreamCreate(&comp_stream);
cudaStreamCreate(&comm_stream);

// 计算流：执行主要的矩阵乘法
matmul_kernel<<<grid, block, 0, comp_stream>>>(...);

// 通信流：同时执行上一个分块的All-Reduce
ncclAllReduce(prev_output, prev_output_reduced, size, ..., comm, comm_stream);

// 确保通信完成后再使用结果
cudaStreamWaitEvent(comp_stream, comm_done_event, 0);
```

**NCCL 的 Ring All-Reduce 与 CUDA：**在 NCCL 的实现中，Ring All-Reduce 的每个”步”被封装为一个 CUDA kernel，在 NCCL 内部管理的一个 CUDA 流上执行。这些 kernel 执行简单的数组复制和累加操作。NCCL 内部会管理跨 GPU 的同步和流依赖。

### 19.5.2 MIG 与 GPU 分区

**A100/H100 的 MIG 技术**

MIG（Multi-Instance GPU）允许将单个物理 GPU 切分为多个独立的 GPU 实例，每个实例拥有隔离的计算和显存资源。这对推理服务尤其有用——多个模型或模型的多个副本可以共享同一物理 GPU 而互不干扰。

**MIG 分区配置**（以 A100-80GB 为例）：

- 7 个 GPU 实例（每个 10GB 显存，14 个 SM）：适合小模型或 KV Cache 受限的 decode。
- 3 个 GPU 实例（每个 20GB 显存，28 个 SM）+ 1 个（10GB，14 个 SM）。
- 2 个 GPU 实例（每个 40GB 显存，56 个 SM）。
- 1 个完整 GPU（80GB 显存，108 个 SM）：不使用 MIG。

**在 CUDA 中使用 MIG：**

```
// 创建MIG-aware的CUDA上下文
CUcontext ctx;
CUdevice dev;
cuDeviceGetByPCIBusId(&dev, "0000:CB:00.0");
// 根据MIG UUID选择实例
cuDeviceGetAttribute(&is_mig_enabled, CU_DEVICE_ATTRIBUTE_MIG_MODE, dev);
cuCtxCreate(&ctx, 0, dev);
```

**MIG 对推理的适用性：**

- **多租户服务**：不同用户/模型的推理服务在 MIG 实例中完全隔离，消除”吵闹的邻居”（noisy neighbor）问题。
- **小模型多副本**：可以为多个 LLM 实例提供独立的 MIG 分区。
- **Prefill-Decode 分离**：在同一个 GPU 上，一个 MIG 实例用于 prefill，另一个用于 decode。
- **限制**：MIG 实例之间无法使用 NVLink 通信（只有完整 GPU 之间有 NVLink）。MIG 不支持 P2P 通信。

## 19.6　推理 Kernel 的最佳实践

### 19.6.1　GEMV 优化（增量解码核心）

**单查询注意力的 GEMV 优化**

增量解码阶段的注意力计算本质上是矩阵-向量乘法：$\boldsymbol{s} = \boldsymbol{q}\boldsymbol{K}^\top$（单查询头），这是推理中最常见的 kernel 之一。对于此操作：

**常规 GEMV 的瓶颈：**

- $\boldsymbol{q}$ 很小（$d_h$ 维，如 128），可以完全放在寄存器中。
- $\boldsymbol{K}$ 很大（$n_{kv} \times d_h$，如 $4096 \times 128$）。HBM 读取量为 $O(n_{kv} \times d_h)$，计算量为 $O(n_{kv} \times d_h)$（每元素一次 FMA）。
- 算术强度 $I = \frac{2 \times n_{kv} \times d_h}{2 \times n_{kv} \times d_h} = 1$ FLOP/byte——极低，明显受限于 HBM 带宽。

**优化策略：**

1. **向量化加载：**让每个线程加载 float4（16 字节），减少内存事务数。
2. **多级归约：**每个 warp 计算一个部分内积，然后在 warp 内归约（warp shuffle），最后在线程块内跨 warp 归约。
3. **KV 分块：**将 $\boldsymbol{K}$ 按 $n_{kv}$ 维度分块，每个 TB 处理一个块，最终合并结果。这与 PagedAttention 的物理块遍历天然匹配。
4. **持久化 kernel：**当 $n_{kv}$ 很小时，使用持久化线程块（persistent thread block）避免重复的 kernel 启动开销。

**FlashDecoding（Tri Dao, 2023）：**FlashDecoding 将 KV 序列沿 $n_{kv}$ 维度拆分为多个 chunk，每个 chunk 在不同的 SM 上独立计算部分注意力（使用在线 softmax），最后通过一次全局归约合并所有 chunk 的结果。在 $n_{kv} = 128\text{K}$ 时，FlashDecoding 使 decode 吞吐量提升约 8×。

### 19.6.2　激活函数与归一化的向量化实现

**逐元素 Kernel 的优化**

RMSNorm、LayerNorm、GELU、SiLU 等逐元素操作虽然计算量小，但因每个 token 都需要执行，kernel 启动延迟成为主要开销。

**RMSNorm 的向量化实现：**

$$\text{RMSNorm}(\boldsymbol{x}) = \boldsymbol{x} \odot \frac{1}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot \boldsymbol{\gamma} \tag{19.3}$$

实现策略：

1. 每个线程处理连续的内存段（向量化，一次加载 16 字节）。
2. 先扫描计算平方和（使用 warp reduction）。
3. 使用 Warp Shuffle（`__shfl_xor_sync`）在 warp 内归约平方和。
4. 使用共享原子操作跨 warp 归约（或分两级：SMEM 归约 + 单个原子操作）。
5. 计算缩放因子后，每个线程向量化地应用归一化。

**融合优化：**将 RMSNorm 与后续的线性层融合。在归一化 kernel 的输出不写出到 HBM，而是直接作为下一层 GEMV 的输入（通过共享内存传递）。

**推理 Kernel 开发的调试与性能分析工具**

- **NCU（NVIDIA Nsight Compute）：**对单个 kernel 进行详细的性能分析。提供占用率、显存带宽利用率、指令吞吐量、Warp Stall 原因等详尽指标。使用 `ncu --set full -o output report.ncu-rep ./my_program`。
- **NSYS（NVIDIA Nsight Systems）：**提供系统级的时间线视图，显示 CPU-GPU 交互、kernel 执行顺序、NCCL 通信、CUDA API 调用等。用于分析端到端性能瓶颈。
- **Roofline 分析：**使用 NCU 的实测 FLOPs 和带宽数据，在 Roofline 图上标出 kernel 的位置，判断是计算受限还是带宽受限。
- **占用率调优：**使用 `cudaOccupancyMaxPotentialBlockSize` 在运行时动态选择最优的 TB 大小和共享内存配置。
- **PTX/SASS 检查：**检查编译后的 PTX 或 SASS（机器码），确认关键计算确实使用了 Tensor Core 指令而非回退到 CUDA Core 模拟。

# 第二十章　专用硬件与异构计算

随着 AI 推理需求的爆炸式增长，通用 GPU 已不再是唯一的选择。从 NVIDIA 各代架构的持续演进，到 Google TPU、AMD MI300X、Apple Silicon、乃至专用 AI 推理芯片（如 Groq LPU、Cerebras WSE），硬件生态正朝着极度多样化的方向发展。本章系统性地介绍各类 AI 推理硬件的架构特性、编程模型及其在异构计算中的协同策略。

## 20.1　NVIDIA GPU 架构演进

### 20.1.1　从 Volta 到 Blackwell

**NVIDIA 数据中心 GPU 各代关键特性对比**

表 20.1: NVIDIA 数据中心 GPU 代际对比

| 特性 | V100 (Volta) | A100 (Ampere) | H100 (Hopper) | H200 (H |
|---|---|---|---|---|
| 制程 | 台积电 12nm | 台积电 7nm | 台积电 4nm | 台积电 |
| FP16 TFLOPS | 125 | 312 | 989 | 989 |
| 显存 | 16/32 GB HBM2 | 40/80 GB HBM2e | 80 GB HBM3 | 141 GB H |
| 显存带宽 | 900 GB/s | 2 TB/s | 3.35 TB/s | 4.8 T |
| NVLink | 300 GB/s (6 links) | 600 GB/s (12 links) | 900 GB/s (18 links) | 900 G |
| Tensor Core 代 | 第一代 | 第三代 | 第四代 | 第四 |
| 低精度支持 | FP16 | FP16/BF16/TF32 | FP8/FP16/BF16/TF32 | FP8/FP16/B |
| SM 数量 | 80 | 108 | 132 (108 可用) | 132 (108 |
| L2 缓存 | 6 MB | 40 MB | 50 MB | 50 M |
| TDP | 300W | 400W | 700W | 700W |
| PCIe 版本 | PCIe 3.0 | PCIe 4.0 | PCIe 5.0 | PCIe |

**对推理的影响分析：**

- **V100→A100：**FP16 算力提升 2.5×，显存带宽提升 2.2×，显存容量提升 2.5×。支持结构化稀疏（2:4）可额外获得 2× 有效算力。TF32 的引入使大多数混合精度推理无需手动精度控制。

- **A100→H100**：FP16 算力提升 3.2×，但更关键的是 FP8 的引入——FP8 Tensor Core 的峰值算力为 1979 TFLOPS，是 FP16 的 2×。FP8 量化的推理（权重和激活）可将延迟减半。新的 Transformer Engine 可动态管理低精度格式。
- **H100→H200**：算力不变，但显存容量提升 76%（80→141 GB），带宽提升 43%（3.35→4.8 TB/s）。这对 KV Cache 密集型推理（长上下文）是巨大的利好——可容纳的并发请求数或最大上下文长度提升近 2×。H200 实际上是为推理优化的"H100+"。
- **H200→B200**：FP4 和 FP6 低精度支持的引入将推理的极致量化推至新高度。双 die 设计（两个计算 die 通过 NVLink-C2C 互联），算力大幅提升。192 GB HBM3e 显存和 8 TB/s 带宽使 B200 成为长上下文推理的理想硬件。

### 20.1.2 Transformer Engine 与 FP8

**NVIDIA Transformer Engine 的 FP8 推理**

H100 及之后的 GPU 引入了 Transformer Engine（TE），这是一个用于自动管理 FP8 精度计算的软件-硬件协同库。

**FP8 格式：**

- **E4M3**（4 位指数，3 位尾数）：指数范围 $[-6, 8]$（以 2 为底）。适用于前向传播的张量（需要更高精度以维护数值稳定性）。
- **E5M2**（5 位指数，2 位尾数）：指数范围 $[-14, 15]$。适用于梯度和需要宽动态范围的后向张量。

**逐张量缩放（Per-Tensor Scaling）**：FP8 的有限动态范围（E4M3 仅约 448 个离散值）使得直接量化 LLM 的张量会导致严重精度损失。TE 使用逐张量缩放因子 $s$ 将张量值映射到 FP8 的表示范围内：

$$\boldsymbol{x}_{\text{FP8}} = \text{quantize}\left(\frac{\boldsymbol{x}_{\text{FP16}}}{s}\right), \quad s = \frac{\max(|\boldsymbol{x}|)}{\max(\text{FP8_range})} \tag{20.1}$$

**延迟缩放（Delayed Scaling）**：TE 在推理中使用延迟缩放策略：缩放因子基于历史张量统计信息计算，而非当前张量的实际值。这避免了对当前张量的完整扫描开销，但可能在张量分布突然变化时引入误差。TE 通过回退机制（检测到溢出时回退到 FP16）来处理异常情况。

**与推理引擎的集成**：vLLM、TensorRT-LLM 等主流推理引擎已集成 TE 的 FP8 推理路径。典型流程：

1. 将模型权重离线量化为 FP8（使用校准数据集确定每层的缩放因子）。
2. 推理时，矩阵乘法使用 FP8 Tensor Core（峰值算力 1979 TFLOPS）。

3. 激活在层间以 FP16/BF16 传递，进入下一层前重新量化为 FP8（或保持在 FP16 用于敏感层如 LayerNorm 和 Attention）。

4. GEMM 完成后，输出累加器为 FP32（Tensor Core 的累加器天然是 FP32），然后转换为 FP16。

**精度影响：**在 Llama-2-70B 上，FP8 推理的相对质量退化通常 $\Delta_{\text{perplexity}} < 1\%$，在多数下游任务上无统计显著的性能下降。

## 20.2　AMD GPU 与 ROCm 生态

### 20.2.1　MI250X 与 MI300X 架构

**AMD Instinct 系列推理特性**

表 20.2: AMD Instinct GPU 关键参数

| 特性 | MI250X (CDNA2) | MI300X (CDNA3) |
|---|---|---|
| 计算单元 (CU) | 220 | 304 |
| FP16 TFLOPS | 383 | 1300 |
| FP8 TFLOPS | 不支持 | 2600 |
| 显存 | 128 GB HBM2e | 192 GB HBM3 |
| 显存带宽 | 3.2 TB/s | 5.3 TB/s |
| Infinity Fabric 带宽 | 400 GB/s (跨 die) | 896 GB/s (跨 die) |
| 制程 | 台积电 6nm | 台积电 5nm+6nm |
| PCIe 版本 | PCIe 4.0 | PCIe 5.0 |
| TDP | 560W | 750W |

**AMD GPU 的推理特性：**

- **Matrix Core：**AMD 的 Tensor Core 等效物。MI300X 的 Matrix Core 峰值算力为 FP16 1300 TFLOPS 和 FP8 2600 TFLOPS，略超 H100 的对应数值。
- **高显存容量和带宽：**192 GB HBM3 和 5.3 TB/s 带宽（vs H100 的 80 GB 和 3.35 TB/s），使其在 KV Cache 密集型长上下文推理中具有天然优势。
- **ROCm 软件栈：**AMD 的 GPU 计算平台，提供 HIP（Heterogeneous-Compute Interface for Portability）作为 CUDA 的语法转换工具。
- **MI300X 的 chiplet 设计：**8 个计算 die + 4 个 IO die 通过 Infinity Fabric 互联。跨 die 的显存访问对带宽敏感应用的优化至关重要。

### 20.2.2 HIP 与 CUDA 的互操作性

**从 CUDA 迁移到 HIP 的实践**

HIP（Heterogeneous-Compute Interface for Portability）是 AMD 提供的 CUDA-to-ROCm 移植工具。

**HIP 的层次：**

1. **HIP Runtime API：**语义上映射到 CUDA Runtime API（如 `hipMalloc` `cudaMalloc`，`hipMemcpy` `cudaMemcpy`）。
2. **HIP Kernel Language：**在 CUDA kernel 源码上使用 `hipify` 工具自动将 CUDA 语法转换为 HIP 语法（大多数转换是机械替换，如

`threadIdx.x`→`hipThreadIdx_x`）。

3. **HIP-Backend 兼容**：HIP 代码可以编译到 NVIDIA GPU（通过`--offload-arch=native`）或 AMD GPU（通过`--offload-arch=gfx942` for MI300X）。

**推理 kernel 迁移的关键差异：**

- **Warp 大小**：NVIDIA Warp 为 32 线程，AMD Wavefront 为 64 线程。Kernel 的占用率计算、同步边界、共享内存 bank 冲突模式都有差异。
- **Tensor Core vs Matrix Core**：AMD 的 Matrix Core 编程使用 `rocwmma` 库（类似 NVIDIA 的 wmma），但 tile 大小和吞吐量特性不同。
- **共享内存容量**：MI300X 的每个 CU 有 64KB 共享内存（vs H100 的 228KB），导致分块大小和占用率需要重新调优。
- **浮点舍入行为**：AMD GPU 的 FMA（乘加）指令的舍入行为与 NVIDIA 不完全一致，可能导致微小的数值精度差异。

**实践中支持多后端的策略：**

- 使用抽象层（如 PyTorch 的 ATen 调度、JAX 的 XLA 后端）将算子实现与硬件解耦。
- 维护多套手写 kernel（CUDA、HIP、Metal 等），运行时根据检测到的硬件选择。
- 使用代码生成框架（如 Triton）编写一次，自动编译到不同硬件后端。

## 20.3　专用 AI 推理硬件

### 20.3.1　Google TPU

**TPU 的脉动阵列架构**

Google TPU（Tensor Processing Unit）自 2015 年第一代发布以来已迭代至 v5p。TPU 的架构核心是**脉动阵列（Systolic Array）**——一个二维的乘加单元网格，数据在阵列中以脉动方式流动。

**TPU v5p 的关键参数：**

- 单芯片 FP16/BF16 算力：459 TFLOPS。
- 单芯片 Int8 算力：918 TOPS。

- HBM 容量：95 GB per chip。
- HBM 带宽：2.76 TB/s。
- 互联：ICI（Inter-Chip Interconnect），4.8 Tbps per chip。
- 拓扑：3D 环形——每芯片通过 ICI 与相邻芯片直连。

**脉动阵列的工作原理：**一个 $128 \times 128$ 的脉动阵列中，权重从上向下流动，激活从左向右流动，部分和在每个阵列单元中累积后向右下传递。经过 $128 + 128 - 1 = 255$ 个周期后，完整的 $128 \times 128$ 矩阵乘法结果从右下角流出。脉动阵列的优势在于：

- 极简的控制逻辑（无复杂指令调度），面积效率极高。
- 所有数据流预知且固定，功耗可控。
- 规模化容易——大型矩阵乘法通过分块映射到脉动阵列。

**TPU 的推理性能：**Google 报告在 TPU v5e（经济版）上运行 LLaMA-2-70B 推理，单芯片 decode 吞吐量达到约 35 tokens/s（bf16）。4 芯片 TPU v5e（256 GB 总 HBM）可容纳完整的 70B 模型，吞吐量线性扩展到约 140 tokens/s。

### 20.3.2 Groq LPU

**Groq LPU 的确定性架构**

Groq 的语言处理单元（Language Processing Unit, LPU）采取了一种与传统 GPU 截然不同的设计哲学。

**核心设计原则：**

- **全 SRAM：**不使用 HBM。所有模型权重和数据驻留在分布在各计算单元之间的片上 SRAM 中。这消除了 HBM 带宽瓶颈，但限制了可部署的模型大小（受 SRAM 总容量限制）。
- **确定性执行：**程序的指令调度在编译时完全确定（类似 VLIW 架构）。没有分支预测、乱序执行或缓存未命中——每周期执行的指令是预先确定的。这提供了可精确预测的延迟。
- **数据流架构：**计算通过数据流图描述，芯片内部通过确定性的片上网络（NoC）传递数据。

**LPU 的推理特性：**

- **极高吞吐量：**Groq 第一代 LPU 在 LLaMA-2-70B 上达到约 300 tokens/s 的 decode 速度（每个用户）。

- **极低延迟**：确定性架构使得每 token 时间高度一致（无抖动）。
- **模型大小限制**：第一代 LPU 约 230MB 片上 SRAM，通过多芯片扩展可容纳更大模型。Groq 通过” 模型分区” 将模型权重分布到多个 LPU 芯片。
- **编译时间**：程序的确定性调度需要在编译时求解复杂的调度问题，编译时间可达数分钟甚至数小时。

**与 GPU 的比较**：LPU 在延迟可预测性和能效方面远超 GPU（无 HBM 访问能耗），但 GPU 在灵活性、模型大小适应性和生态成熟度方面遥遥领先。LPU 最适合延迟极度敏感、模型固定、请求模式可预测的推理场景（如实时对话系统）。

### 20.3.3 Cerebras WSE

**晶圆级引擎的极致计算密度**

Cerebras 的晶圆级引擎（Wafer-Scale Engine, WSE）将整个晶圆作为一个芯片，在单个器件上集成了海量的计算和存储资源。

**WSE-3 的关键参数：**

- 晶体管数：4 万亿。
- AI 核心数：约 900,000 个。
- 片上 SRAM：44 GB（分布在各核心）。
- 片上内存带宽：约 21 PB/s (!)。
- 功耗：约 23 kW。

**WSE 的架构特点：**

- 每个核心包含一个小的数据流处理单元和本地 SRAM（约 48KB per core）。
- 核心通过 2D Mesh NoC 互联，数据以确定性的流水线方式在核心间传递。
- 适合将大型模型的每一层映射到不同核心组，以数据流流水线方式执行——模型各层的数据在核心组间流动（类似于极致的流水线并行）。

**WSE 的推理特性：**

- 极致的片上带宽 → 几乎无带宽瓶颈 → 适合计算密集 + 数据密集混合的工作负载。
- 模型每一层可以完全加载在片上 SRAM 中 → 无需从 HBM 反复加载权重。
- 主要限制：晶圆的单一性和成本（单器件极其昂贵）。扩展只能通过增加 WSE 器件的数量（而非像 GPU 集群那样灵活增加）。

### 20.3.4 其他专用硬件

**AI 推理硬件全景**

**Habana Gaudi 系列（现属 Intel）：**

- Gaudi3 关键参数：FP8 算力 1835 TFLOPS，128 GB HBM2e，3.7 TB/s 带宽。
- 独特特性：集成了 24 个 100 Gbps RoCE RDMA 网卡（直接做在芯片上），跨节点通信无需额外的 NIC。
- 推理生态：通过 Optimum Habana（HuggingFace 集成）和 SynapseAI 软件栈支持主流模型。

**Graphcore IPU（Intelligence Processing Unit）：**

- 大量简单核心（IPU-POD256 约 59,000 个核心）+ 本地 SRAM（每核心 256KB）的众核架构。
- MIMD（多指令多数据）执行模型，而非 GPU 的 SIMT/SIMD。
- 适合稀疏计算和细粒度数据流图。
- 挑战：生态成熟度、市场份额、持续经营的不确定性。

**Qualcomm Hexagon/AI Engine（移动端）：**

- 集成在 Snapdragon 移动平台中的 AI 加速器。
- Hexagon NPU 面向 INT8/INT4 推理优化，峰值算力可达数十 TOPS。
- 软件栈：Qualcomm AI Engine Direct SDK。
- 典型场景：端侧 LLM 推理（如 LLaMA-2-7B 的 INT4 量化版本在 Snapdragon 8 Gen 3 上运行）。

**Apple Neural Engine (ANE)：**

- Apple A 系列/M 系列芯片中的专用 AI 推理硬件。
- A17 Pro 的 ANE 提供约 35 TOPS（INT8）。
- ANE 与 CPU、GPU、AMX（Apple Matrix coprocessor）共享统一内存（unified memory）。
- 软件栈：Core ML、MLX（Apple 的开源 ML 框架）、Metal Performance Shaders（MPS）。
- 典型场景：端侧 LLM 推理（MLX 在 M3 Max 上运行 Mistral-7B 达 15 tokens/s）。

**FPGA（可编程逻辑）：**

- Xilinx（现属 AMD）和 Intel（前 Altera）提供可编程逻辑门阵列。
- DPU（Deep learning Processing Unit）：Xilinx 的预构建深度学习加速器 IP，可配置为专注推理的脉动阵列结构。
- 优势：可针对特定模型架构定制硬件数据路径，极低延迟（微秒级），极低功耗（瓦级）。
- 劣势：开发周期长（数月），需要硬件设计专业知识，每性能成本高于 GPU。
- 适合场景：对延迟和功耗有极端要求的边缘推理（自动驾驶、实时视频分析）。

## 20.4 异构计算架构

### 20.4.1 算子放置策略

**异构环境中的算子放置问题**

在包含 CPU、GPU、NPU 等多种计算设备的系统中，每个算子需要在特定设备上执行。算子放置问题可以形式化为一个图分割优化问题。

**形式化：**给定一个计算图 $G = (V, E)$（其中 $V$ 为算子集合，$E$ 为数据依赖边），和 $D$ 个可用设备，决定分配函数 $f : V \to \{1, 2, \ldots, D\}$ 来最小化总执行时间。

**总执行时间：**

$$T_{\text{total}} = \sum_{v \in V} T_{\text{exec}}(v, f(v)) + \sum_{(u,v) \in E, f(u) \neq f(v)} T_{\text{transfer}}(\text{data}_{u \to v}, f(u), f(v)) \tag{20.2}$$

其中：

- $T_{\text{exec}}(v, d)$：算子 $v$ 在设备 $d$ 上的执行时间。
- $T_{\text{transfer}}(\text{data}, d_1, d_2)$：数据从设备 $d_1$ 到 $d_2$ 的传输时间。

**成本模型：**对于每个算子在每种设备上的执行时间，使用基于设备特性的 roofline 模型估计：

$$T_{\text{exec}}(v, d) = \max\left(\frac{\text{FLOPs}_v}{F_{\text{peak},d}}, \frac{\text{Memory_bytes}_v}{B_{\text{mem},d}}\right) \tag{20.3}$$

**解决方案：**

1. **启发式策略：**将大矩阵乘法（GEMM）分配到 GPU/NPU，将小规模操作（Embedding、Sampling）放在 CPU。这是 llama.cpp 的 GPU 卸载策略的基础。
2. **动态规划：**将计算图视为线性链（序列模型），使用 DP 求解最优分割点。每个分割点对应于从设备 A 切换到设备 B 的层位置。
3. **成本模型 + 贪心：**按计算密度排序，将高密度算子优先分配给加速设备，直到其内存或计算资源满载。

### 20.4.2 llama.cpp 的异构执行

**llama.cpp 的 GPU 卸载策略分析**

llama.cpp 是一个广泛使用的 C/C++ LLM 推理框架，支持在 CPU 和 GPU 之间灵活分配模型层。

**卸载策略：**

- `-ngl N` 参数：指定卸载到 GPU 的层数。层 0 到层 `N-1` 在 GPU 上运行，层 `N`

到层 `L-1` 在 CPU 上运行。

- 当 `N = L` 时，全部模型在 GPU 上运行（前提是 GPU 显存够大）。
- 当 `N = 0` 时，全部模型在 CPU 上运行（纯 CPU 推理）。
- 中间值：部分层在 GPU 上，部分在 CPU 上。

**数据流：**

- **嵌入层**：通常在 CPU 上执行（需要大量内存 + 带宽小），输出通过 PCIe 复制到 GPU（当 GPU 层数 >0 时）。
- **GPU 层**：在 GPU 上顺序执行。每层的输入/输出在 GPU 显存中，权重预加载在 GPU 显存。
- **CPU 层**：GPU 的输出（激活，$b \times n \times d$）通过 PCIe 传回 CPU。CPU 执行剩余层。最终 logits 在 CPU 上，直接用于采样。

**分割点的选择：**分割点决定了 PCIe 传输次数和数据量。在每次前向传播中，GPU 和 CPU 交界处需要传输激活张量（约 $b \times n \times d \times 2$ bytes fp16）。对于 $b = 1, n = 1$，仅约 $d \times 2$ bytes（如 $d = 4096$ 时约 8KB）。传输量极小，分割点的选择主要由显存容量决定——尽可能将更多层卸载到 GPU。

**多后端支持：**llama.cpp 支持多种 GPU 后端：

- **CUDA**：NVIDIA GPU。
- **Metal**：Apple Silicon GPU（通过 Metal Performance Shaders）。
- **Vulkan**：跨平台的 GPU API（AMD、Intel、NVIDIA 均支持）。
- **SYCL**：Intel GPU（通过 oneAPI）。
- **ROCm**：AMD GPU。

每个后端提供相同的接口（如 `ggml_backend_t`），框架可以统一调度。

# 20.5 移动端与边缘推理

## 20.5.1 移动端推理的特殊约束

**移动端推理的约束模型**

移动设备上的 LLM 推理面临一系列硬约束：

表 20.3: 移动端与服务器端推理对比

| 约束维度 | 服务器端 | 移动端 |
|---|---|---|
| 可用内存 | 80-192 GB HBM | 2-8 GB（统一内存） |
| 内存带宽 | 2-8 TB/s | 50-100 GB/s |
| 计算峰值 | 1000+ FP16 TFLOPS | 5-35 FP16 TFLOPS |
| 功耗预算 | 300-1200W | 3-10W（被动散热） |
| 散热方式 | 主动风冷/液冷 | 被动散热（无风扇） |
| 网络依赖 | 数据中心网络 | WiFi/5G（不稳定，高延迟） |
| 延迟要求 | TTFT $<$ 1s | TTFT $<$ 3s |

**移动端的核心挑战：**

1. **内存限制：**7B 模型仅权重就需 14GB（fp16），超过大多数手机的可用内存。需要 INT4 量化（权重降至 3.5GB）或使用更小的模型（如 Phi-3-mini-3.8B，TinyLlama-1.1B）。

2. **带宽限制：**移动端内存带宽（约 50-100 GB/s）比 H100 低 30 $\sim$ 70×。这使得 decode 阶段（带宽受限）成为移动端推理的主要瓶颈——decode 每 token 时间通常 $>$ 50ms。

3. **功耗与散热：**持续推理导致芯片升温，触发 DVFS（动态电压频率调节）降频——性能进一步下降。需要利用 NPU 等低功耗专用硬件。

4. **存储限制：**模型文件需要几百 MB 到几 GB 的存储空间。量化 + 压缩可缓解。应用商店的安装包大小限制也是一个考量。

### 20.5.2　移动端推理框架对比

**移动端 LLM 推理框架评析**

表 20.4: 主流移动端推理框架

| 框架 | 平台 | 后端 | 量化支持 | 代表应用 |
|---|---|---|---|---|
| llama.cpp | iOS/Android | Metal/Vulkan | INT4/INT8 | 众多端侧 LLM |
| MLX (Apple) | iOS/macOS | Metal/ANE | INT4/INT8 | MLX Chat |
| MNN (阿里) | iOS/Android | OpenCL/Vulkan/ANE | INT8/FP16 | 电商推荐 |
| ncnn (腾讯) | iOS/Android | Vulkan/OpenCL/ANE | INT8/FP16 | 微信 AI 功能 |
| TensorFlow Lite | iOS/Android | GPU/CPU/NNAPI | INT8/FP16 | Google Assista |
| Executorch (Meta) | iOS/Android | Metal/Vulkan/CoreML | INT4/INT8/FP16 | Llama 部署 |
| MediaPipe (Google) | Android/iOS | GPU/CPU | INT8/FP16 | Gemini Nan |

**选择框架的关键考量：**

- **模型格式：**GGUF（llama.cpp）、ONNX、TFLite FlatBuffer、MLX safetensors 各有生态。GGUF 因 llama.cpp 的广泛使用成为端侧 LLM 的事实标准格式。
- **量化质量：**不同框架的量化实现质量差异显著。INT4 在移动端的精度损失与实现方式密切相关——分组量化（group-wise quantization）通常优于逐张量量化。
- **后端支持：**iOS 上 Metal Performance Shaders 使用最广泛；Android 上 Vulkan 是跨厂商的最佳选择（而 OpenCL 已逐渐被废弃）。
- **自定义算子支持：**当模型的注意力机制或 FFN 结构特殊时，需要框架支持自定义算子的桥接。

## 20.6　硬件感知 Kernel 设计

### 20.6.1　跨硬件的编程抽象

**多硬件后端的 Kernel 抽象策略**

构建一个可跨多种硬件运行的推理引擎的核心挑战在于：如何在保持各硬件最优性能的同时，尽量减少代码重复。

**策略一：高级 DSL（如 Triton）：**OpenAI 的 Triton 语言允许用 Python 编写 GPU kernel，编译器将其生成优化后的 PTX/CUDA 或 HIP。Triton 3.0 已初步支持 AMD GPU（通过 ROCm 后端）和 Intel GPU（通过 XPU 后端）。

```
@triton.jit
def matmul_kernel(
    A, B, C,
    M, N, K,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr
):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    # ... tile-based matmul using TL ops (自动映射到Tensor Core) ...
```

优势：写一次，跨 NVIDIA/AMD/Intel 运行。劣势：对硬件特定特性的利用不如手写 kernel 极致。

**策略二：统一中间表示（如 MLIR）：**MLIR（Multi-Level Intermediate Representation）允许在不同层次的 IR 上进行优化和代码生成。一个推理计算图可以在 MLIR 的"Linalg" 层表示（高级、硬件无关），逐步下降到"GPU" 层（硬件相关），最终生成指定后端的代码。

**策略三：宏适配层：**使用 C++ 模板/宏在不同硬件 API 之间适配：

```
#ifdef __CUDACC__
    #define DEVICE_INLINE __device__ __forceinline__
    #define WARP_SIZE 32
    #define SYNC_WARP __syncwarp()
#elif defined(__HIPCC__)
    #define DEVICE_INLINE __device__ __forceinline__
    #define WARP_SIZE 64
    #define SYNC_WARP __syncwarp()
#endif
```

这是大多数手写 kernel 多后端支持的方法，最灵活但维护成本高。

### 20.6.2 硬件自动调优

**推理 kernel 参数的自动搜索**

不同硬件的最优 kernel 参数（如分块大小、线程块大小、展开因子）差异巨大。手动为每种硬件调优是不现实的。

**自动调优框架：**

- **AutoTVM / Ansor (Apache TVM)：**使用机器学习引导的搜索（代价模型 + 进化搜索 / 迭代调度探索）为每个算子找到最优调度。搜索过程中在目标硬

件上实际执行候选 kernel 并记录性能。

- **Triton Autotuner**：对 Triton kernel 的 `@triton.autotune` 装饰器，自动搜索 `BLOCK_M`、`BLOCK_N`、`BLOCK_K`、`num_warps` 等参数的最优组合。
- **CUTLASS profiler**：NVIDIA 的 CUTLASS 库提供基于性能模型和运行时测试的 kernel 参数选择。

**调优的参数空间**：对于矩阵乘法的自动调优，典型的参数空间包括：

- $B_r$（寄存器块大小，如 $64, 128, 256$）
- $B_c$（另一维度的寄存器块大小）
- 线程块维度（$(128, 1, 1)$、$(128, 2, 1)$、$(256, 1, 1)$ 等）
- 每线程处理的 $k$ 维度迭代数（$4, 8, 16$）
- 是否使用共享内存双缓冲
- 向量化加载宽度（float2、float4）

**部署时的硬件检测**：推理引擎启动时检测具体的硬件型号（通过 PCI 设备 ID、CUDA compute capability、ROCm gfx arch 等），加载预调优的 kernel 参数数据库（或执行在线调优并缓存结果）。

## 20.7　新兴趋势

### 20.7.1　超低精度推理

**FP4 和 FP6 的推理前景**

NVIDIA Blackwell 架构（B200）引入了对 FP4（E2M1，2 位指数 +1 位尾数）和 FP6（E3M2/E2M3 变体）的原生支持。这标志着推理量化从 INT8/FP8 继续下探到 4-bit 浮点。

**FP4 的特性**：

- 总共 16 个可表示值（$2^4$）：-0、+0 以及少量正负浮点数。
- 优点：相比 FP8，模型权重存储需求再减少 50%（7B 模型 FP8 需 7GB，FP4 需 3.5GB）。
- 挑战：极低的精度需要精细的量化策略——分组量化（group size 32-128）、激活感知量化（AWQ/SmoothQuant）、通道级量化。

- B200 的 FP4 Tensor Core 算力是 FP8 的 2 倍。

**MX 格式（Microscaling）**：微软、AMD、Intel、NVIDIA 等联合提出的 MX（Microscaling）标准系列（MXFP8、MXFP6、MXFP4）：

- 将张量划分为小的 block（32 个元素），每个 block 共享一个 8-bit 共享指数（shared exponent）。
- 相比每元素独立指数，MX 格式在精度和存储效率间实现了更好的平衡。
- 已被纳入 OCP（Open Compute Project）标准，可能成为下一代 AI 硬件的事实标准。

### 20.7.2 稀疏计算的硬件支持

**结构化稀疏与细粒度稀疏**

**NVIDIA 2:4 结构化稀疏**：在每 4 个连续的权重值中，至少 2 个必须为 0。这保证了 50% 的稀疏率。Ampere+ Tensor Core 可以以 2× 有效算力执行 2:4 稀疏矩阵乘法（通过跳过零值对应的计算）。

**细粒度非结构化稀疏**：通过剪枝（pruning）得到的权重稀疏模式通常是非结构化的（零值位置无规律）。传统硬件无法有效加速非结构化稀疏（需要复杂的索引和数据重组）。Cerebras WSE 的细粒度数据流架构可以原生支持非结构化稀疏。

**激活稀疏**：ReLU/GELU 等激活函数产生大量自然零值（特别是在 FFN 的中间层）。利用激活稀疏性（跳过零值激活对应的计算）可以将推理计算量降低 30-50%。这需要支持动态稀疏模式的硬件和 kernel——如 NVIDIA 的 Sparse Tensor Core（仅支持静态权重稀疏，不支持激活稀疏）和 DejaVu 等稀疏注意力方法。

**Mixture-of-Depths**：一种新型稀疏范式——不仅路由至专家，还可以完全跳过某些层的计算（当该层的输出对当前 token 不重要时）。类似激活稀疏但作用在层级别。

### 20.7.3 存内计算与光学互联

**超越传统冯诺依曼架构的硬件技术**

**存内计算（Processing-In-Memory, PIM）**：传统架构中，数据必须在 HBM 和计算单元之间搬运——即”冯诺依曼瓶颈”。PIM 在内存芯片内部集成计算单元，使得数据在存储位置附近即可被处理。

- Samsung HBM-PIM：在 HBM 的每个 bank 中集成了浮点计算单元（FP16 加法器和乘法器），可以在 bank 内执行简单的 Reduce 和 Multiply-Add 操作。
- PIM 对推理的潜力：KV Cache 的注意力分数计算（$QK^\top$）可以在 KV 所在的

HBM bank 内执行，只需将小的 Q 向量广播到各 bank 并从各 bank 收集部分积。这可将注意力阶段的 HBM 带宽需求降低几个数量级。

- 现状：仍处于早期验证阶段。软件栈、编程模型和标准化尚未成熟。

**光学互联（Optical Interconnect）**：传统 NVLink/NVSwitch 使用铜缆或 PCB 走线，带宽随距离衰减严重。光学互联通过光信号传输数据，提供更高的带宽和更低的功耗。

- Ayar Labs：与 NVIDIA 合作的光学 I/O chiplet，提供 Tbps 级带宽的 chip-to-chip 互联。
- 应用前景：光学互联可使 GPU-to-GPU 通信跨越更远的物理距离（机柜间甚至楼栋间），使分布式推理的 TP/EP 跨节点成为可能。
- 与分离式 Prefill-Decode 架构的结合：光学互联的低延迟高带宽特性使 KV Cache 跨节点传输的延迟可被忽略。

**Chiplet 与先进封装**：从 MI300X 的 8 个计算 die 到 B200 的双 die 设计，chiplet 技术将多个较小的芯片 die 通过先进封装（硅中介层、EMIB）集成到同一封装中。

- 优势：突破单一 die 的制程限制（reticle limit），可以在单一”芯片”上集成更多晶体管。
- 对推理的影响：更大的”有效”显存容量和带宽（通过多 HBM 堆栈）、更高的计算密度、更好的能效比。
- 挑战：跨 die 通信的 NUMA（非统一内存访问）特性需要更智能的内存分配策略。

**推理硬件选择的决策框架**

部署 LLM 推理服务时的硬件选择涉及多个维度的权衡。以下是面向不同场景的推荐策略：

1. **低延迟在线对话（Chatbot）**：H100 TensorRT-LLM + FP8。若预算允许，B200 的 FP4 推理可提供更低延迟。
2. **长上下文推理（文档分析、代码审查）**：H200 或 MI300X，利用大 HBM 容量支撑长 KV Cache。
3. **高吞吐批处理（离线评估、数据标注）**：多张 A100 或 H100 组成集群，通过 DP 最大化吞吐量。

4. **端侧推理（手机、笔记本）：**Apple Silicon 用 MLX（M3 Max 以上），Android 用 llama.cpp（Vulkan 后端）。模型选择 INT4 量化的 7B 以下模型。

5. **成本敏感部署：**使用较老型号（V100、A100）配合激进量化（INT8/INT4）+ 投机解码以提升等效吞吐量。CPU-only 方案（llama.cpp 纯 CPU）可用于极低成本场景。

6. **多租户 SaaS 平台：**MIG 分区的 H100/H200，或使用多张小容量 GPU（L40S）为不同租户提供隔离实例。

# 第六部分

# 框架深度剖析篇

# 第二十一章　llama.cpp 框架深度剖析

## 21.1　项目概览与设计哲学

### 21.1.1　起源与动机

llama.cpp 由 Georgi Gerganov 于 2023 年 3 月创建，最初的动机是让 LLaMA 模型能够在消费级 CPU 上高效运行。在 LLM 推理领域，当时几乎所有的高性能推理方案都深度依赖 NVIDIA GPU 和 CUDA 生态系统。Gerganov 提出了一个激进但朴素的问题：能否用纯 C/C++（无任何外部依赖）实现一个足够高效的 LLM 推理引擎？

**从零开始的哲学**

llama.cpp 的一个重要设计原则是“零依赖”。整个项目的核心部分不依赖任何第三方 BLAS 库、数学库或框架。这不仅意味着极致的可移植性——只要有一个 C++11 编译器就能在任何平台运行——也意味着对每一条计算路径拥有完全的控制权。

项目的命名既是致敬 Meta 的 LLaMA 模型，也暗含了技术路线：通过量化（quantization）技术将模型权重的存储精度从 FP32/FP16 压缩到 4-bit 甚至更低，使得原本需要几十 GB 显存的模型能够在普通笔记本的 16 GB 内存中运行。这种“先量化，后推理”的技术路线后来被证明极具前瞻性。

**llama.cpp 核心设计原则**

1. **极简主义：**核心推理引擎（ggml）仅包含约 5 万行 C 代码。所有的矩阵乘法、注意力计算、归一化操作都以“原生”方式实现，不依赖外部 BLAS。

2. **量化优先：**从张量类型系统到算子实现，每个环节都围绕量化精度设计。GGML 类型系统定义了数十种量化格式（Q4_0、Q4_1、Q5_0、Q8_0、K-quant 系列、IQ 系列等）。

3. **可移植性：**支持 x86（AVX2、AVX-512）、ARM（NEON）、Apple Silicon（Accelerate）、RISCV 等架构。通过自研后端抽象层支持 GPU（CUDA、Metal、Vulkan、SYCL）。

4. **实用主义：**交互式聊天、服务器模式、批量推理、语法约束生成——功能随着用

户需求自然生长。

### 21.1.2 架构总览

llama.cpp 的代码库可以划分为以下几个层次：

1. **ggml 张量库（`ggml.h/ggml.c`）**：底层张量运算引擎。定义张量数据结构、计算图、操作原语。
2. **后端层（`ggml-backend*`）**：硬件抽象接口。CPU、CUDA、Metal、Vulkan、SYCL 后端各自实现相同的张量操作集合。
3. **LLaMA 模型层（`llama.h/llama.cpp`）**：模型加载、tokenizer、前向推理、KV 缓存管理。
4. **应用层（`main`、`server`、`llama-bench` 等）**：命令行聊天、HTTP 服务器、性能基准测试工具。

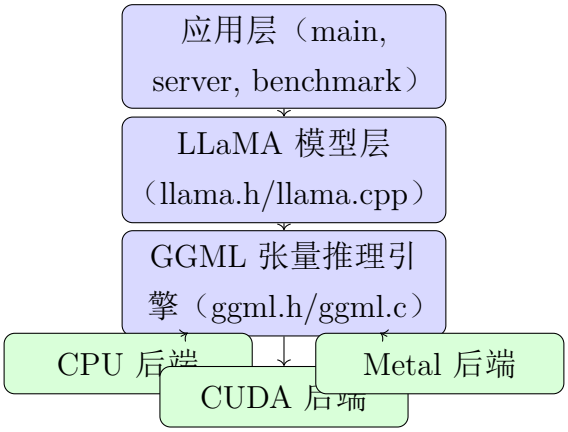



图 21.1: llama.cpp 架构层次

### 21.1.3 支持的模型家族

截至 2026 年，llama.cpp 已支持超过 100 种模型架构，包括但不限于：

- **LLaMA 系列**：LLaMA 1/2/3，LLaMA 3.1/3.2，LLaMA 4
- **Mistral 系列**：Mistral 7B，Mixtral 8x7B/8x22B（MoE）
- **其他密集模型**：Gemma 1/2/3，Phi-2/3/4，Qwen 1/2/2.5，DeepSeek-V2/V3/R1
- **视觉语言模型**：LLaVA，MobileVLM，Qwen-VL
- **SSM 架构**：Mamba，Mamba-2

- **其他特色架构**：Granite，Command R，DBRX，OLMo，Arctic

这种丰富的模型支持得益于 llama.cpp 的灵活架构——模型层的代码为每种架构定义了“构建函数”，将 GGML 操作节点组装成计算图。

## 21.2　GGUF 格式深度剖析

### 21.2.1　从 GGML 到 GGUF 的演进

最初的 llama.cpp 使用 GGML 格式存储模型权重。GGML 格式有一个严重的缺陷：模型架构信息（层数、维度、注意力头数等）和 tokenizer 配置与格式本身耦合较松，需要在加载时通过文件名或其他约定来推断。GGUF（GGML Universal Format）是 2023 年 8 月引入的替代格式，其核心改进是：

- **自描述性**：所有模型元数据（架构名称、超参数、tokenizer 配置、分词表、聊天模板等）都以键值对形式嵌入文件头部。
- **前向兼容性**：新增的元数据键可以被旧版加载器忽略而不影响模型加载。
- **严格的二进制规范**：字段类型、对齐要求、字节序都有明确规定。

**格式兼容性**

GGUF 格式的版本号从 v1 到 v3，每个版本在字段编码方式上有变化。v3 是当前主流版本，它改进了字符串的编码方式（使用立即长度前缀）。加载器必须正确解析版本号并采用对应的解码逻辑。

### 21.2.2　二进制格式规范

GGUF 文件的二进制布局如下：

表 21.1: GGUF 文件二进制布局

| 区域 | 大小（字节） | 描述 |
|---|---|---|
| Magic Number | 4 | `0x47 0x47 0x55 0x46`（“GGUF”） |
| Version | 4 | 版本号（uint32，小端。v1=1, v2=2, v3=3） |
| Tensor Count | 8 | 张量数量（uint64） |
| Metadata KV Count | 8 | 元数据键值对数量（uint64） |
| Metadata KV Pairs | 可变 | 按顺序排列的键值对 |
| Tensor Info Array | 可变 | 每个张量的名字、维度、类型、偏移量 |
| Tensor Data | 可变 | 原始张量数据区块 |

#### 21.2.2.1 元数据键值对编码

每个元数据键值对包含以下字段：

表 21.2: 元数据键值对字段

| 字段 | 类型 | 描述 |
|---|---|---|
| Key Length | uint64 | 键字符串的长度 |
| Key | char[Key Length] | 键字符串（通常是如 `"general.architecture"` 这样的点分路径） |
| Value Type | uint32 | 值的 GGUF 类型枚举 |
| Value | 可变 | 根据类型编码的值 |

GGUF 值类型枚举包括：`UINT8=0`, `INT8=1`, `UINT16=2`, `INT16=3`, `UINT32=4`, `INT32=5`, `FLOAT32=6`, `BOOL=7`, `STRING=8`, `ARRAY=9`, `UINT64=10`, `INT64=11`, `FLOAT64=12`。数组类型通过 OR 标记（`0x8000`）表示该类型是数组。

#### 21.2.2.2 关键元数据键

以下是最重要的元数据键：

1. `general.architecture`（string）：模型架构名称，如 `"llama"`、`"mistral"`、`"phi3"`。加载器根据此值选择对应的架构构建函数。
2. `general.name`（string）：模型名称。
3. `general.quantization_version`（uint32）：量化版本号。
4. `general.file_type`（uint32）：文件类型（1= 全部 F32, 2= 大部分 F16, 3= 部分量化, 7= 全部量化等）。
5. `llama.context_length`（uint32）：上下文长度（$n_ctx$）。
6. `llama.embedding_length`（uint32）：嵌入维度（$d_model$）。
7. `llama.block_count`（uint32）：Transformer 层数（$n_layer$）。
8. `llama.feed_forward_length`（uint32）：FFN 中间层维度（$d_ff$）。
9. `llama.attention.head_count`（uint32）：注意力头数（$n_head$）。
10. `llama.attention.head_count_kv`（uint32）：KV 头数（用于 GQA/MQA）。
11. `llama.attention.layer_norm_rms_epsilon`（float32）：RMSNorm $\epsilon$。
12. `llama.rope.dimension_count`（uint32）：RoPE 维度。
13. `llama.rope.freq_base`（float32）：RoPE 基础频率。

14. `tokenizer.ggml.model`（string）：tokenizer 类型（`"llama"`、`"gpt2"` 等）。

15. `tokenizer.ggml.tokens`（array[string]）：词汇表。

16. `tokenizer.ggml.scores`（array[float32]）：token 分数。

17. `tokenizer.ggml.token_type`（array[uint32]）：token 类型。

18. `tokenizer.ggml.bos_token_id`（uint32）：BOS token ID。

19. `tokenizer.ggml.eos_token_id`（uint32）：EOS token ID。

20. `tokenizer.chat_template`（string）：Jinja2 聊天模板。

**GGUF 与其他格式的对比**

表 21.3: 模型格式对比

| 特性 | GGUF | SafeTensors | ONNX |
|---|---|---|---|
| 自描述性 | 完整元数据 | 仅张量名 + 形状 | 完整计算图 |
| 量化支持 | 内建多种量化类型 | 无 | QDQ 格式 |
| 内存映射 | 支持 | 支持 | 部分支持 |
| 二进制大小 | 紧凑 | 较大 | 取决于 Protobuf |
| 生态支持 | llama.cpp 专用 | HuggingFace | 通用互操作 |

### 21.2.3　张量信息数组

在元数据之后，紧接着是 $N$ 个张量信息条目（$N$ = Tensor Count）。每个条目包含：

1. **名称（string）**：张量名称，如 `"blk.0.attn_q.weight"`。

2. **维度数（uint32）**：$n_dims$。

3. **维度数组（uint64[$n_dims$]）**：从内到外的维度大小（GGML 的行优先约定：$ne[0]$ = 最后一维，$ne[n_dims-1]$ = 第一维）。

4. **类型（uint32）**：GGML 张量类型枚举（如 0=F32, 1=F16, 2=Q4_0, 等等）。

5. **偏移量（uint64）**：从文件开始到该张量数据的字节偏移。

数据区的每个张量按 32 字节对齐（填充到下一个 32 字节边界，在 v3 中调整为 64 字节）。量化类型的张量数据包含量化的 scale 和 bias 信息，这些信息以紧凑的块格式编码在张量数据中。

表 21.4: 常用 GGML 量化类型

| 类型 | 枚举值 | bits/elem | 块大小 |
|---|---|---|---|
| F32 | 0 | 32 | 1 |
| F16 | 1 | 16 | 1 |
| Q4_0 | 2 | 5.0 | 32 |
| Q4_1 | 3 | 4.5 + 1 | 32 |
| Q5_0 | 6 | 5.5 | 32 |
| Q5_1 | 7 | 5.0 + 1 | 32 |
| Q8_0 | 8 | 9.0 | 32 |
| Q8_1 | 9 | 8.5 + 1 | 32 |
| Q2_K | 10 | 2.5625 | 256 |
| Q3_K | 11 | 3.4375 | 256 |
| Q4_K | 12 | 4.5 | 256 |
| Q5_K | 13 | 5.5 | 256 |
| Q6_K | 14 | 6.5625 | 256 |
| IQ2_XXS | 20 | 2.0625 | 256 |
| IQ2_XS | 21 | 2.3125 | 256 |
| IQ3_XXS | 22 | 3.0625 | 256 |
| IQ4_XS | 23 | 4.25 | 256 |

#### 21.2.3.1　GGML 量化类型速查表

**Bytes-per-element 的含义**

表中的 bits/elem 包含了量化的 scale 和 bias 字节。例如 Q8_0 的 9.0 bits/elem = 8 bits 量化值 + $\frac{32 \text{ bits}}{32 \text{ elements}}$ scale 分摊 = 9 bits。

### 21.2.4　字节序和对齐

GGUF 统一使用小端（little-endian）字节序。所有多字节整数（uint32、uint64、float32）都按小端编码。在 v3 格式中，张量数据对齐到 64 字节边界是为了兼容内存映射（mmap）和 SIMD 操作的对齐要求。

## 21.3　GGML 张量库

### 21.3.1　核心数据结构

GGML 的核心是 `ggml_tensor` 结构体：

表 21.5: ggml_tensor 结构体字段

| 字段 | 类型 | 描述 |
|---|---|---|
| type | ggml_type | 张量元素类型（F32, F16, Q4_0 等） |
| backend | ggml_backend_t | 后端类型 |
| buffer | ggml_backend_buffer_t | 后端缓冲区指针 |
| data | void* | 数据指针 |
| name[64] | char | 张量名称 |
| n_dims | int | 维度数量 |
| ne[GGML_MAX_DIMS] | int64_t | 各维度大小（ne[0]=width, ne[1]=height, ne[2]= |
| nb[GGML_MAX_DIMS] | size_t | 各维度的步长（stride，字节数） |
| op | ggml_op | 操作类型 |
| op_params[16] | int32_t | 操作参数 |
| flags | int32_t | 标志位 |
| src[GGML_MAX_SRC] | ggml_tensor* | 源张量（操作数） |
| view_src | ggml_tensor* | 视图源（view 操作的原始张量） |
| view_offs | size_t | 视图偏移量（字节） |

#### 21.3.1.1　多维数组的布局约定

GGML 使用行优先（row-major）布局，但维度顺序与通常约定的“外部到内部”相反：

- $ne[0]$：最内层维度（连续维），通常是嵌入维度或特征维度。
- $ne[1]$：第二维，通常是序列长度或行数。
- $ne[2]$：第三维，通常是注意力头数。
- $ne[3]$：第四维（批处理维）。

这意味着对于矩阵 $\boldsymbol{W} \in \mathbb{R}^{n_embd \times n_vocab}$，存储为 $ne[0] = n_vocab$, $ne[1] = n_embd$。在内存中，$n_vocab$ 个元素是连续的。

#### 21.3.1.2 步长和视图

$nb[i]$ 表示沿维度 $i$ 前进一个单位需要跳过的字节数。对于常规的连续存储：

$$nb[0] = \text{element_size}, \quad nb[1] = nb[0] \cdot ne[0], \quad nb[2] = nb[1] \cdot ne[1], \quad \ldots$$

GGML 支持“零拷贝”视图操作（reshape、permute、slice），这些操作不复制数据，只修改 $ne[]$ 和 $nb[]$。这大幅减少了内存开销。

**视图操作示例：Reshape**

将一个 $(d, 2d)$ 的矩阵 reshape 为 $(d, d, 2)$ 的张量：

- 原始：$ne = \{d, 2d\}$, $nb = \{\texttt{elem_size}, \texttt{elem_size} \cdot d\}$
- Reshape 后：$ne = \{d, d, 2\}$, $nb = \{\texttt{elem_size}, \texttt{elem_size} \cdot d, \texttt{elem_size} \cdot d^2\}$

注意 $ne[0] \cdot ne[1] \cdot ne[2] = d \cdot d \cdot 2 = 2d^2$，与原矩阵的元素总数一致。这完全是零复制的——只修改了 $ne$ 和 $nb$ 元数据。

### 21.3.2 操作类型系统

GGML 定义了超过 100 种操作（枚举 `ggml_op`），涵盖：

- **算术操作**：`MUL`, `ADD`, `SUB`, `DIV`, `MUL_MAT`（矩阵乘法）
- **一元操作**：`ABS`, `NEG`, `STEP`, `RELU`, `GELU`, `SILU`, `TANH`, `GELU_QUICK`
- **归约操作**：`SUM`, `SUM_ROWS`, `MEAN`, `ARGMAX`, `SOFT_MAX`, `ROPE`
- **归一化操作**：`RMS_NORM`, `LAYER_NORM`, `GROUP_NORM`
- **张量操作**：`REPEAT`, `REPEAT_BACK`, `CPY`（类型转换/拷贝）, `RESHAPE`, `VIEW`, `PERMUTE`, `TRANSPOSE`, `GET_ROWS`, `GET_ROWS_BACK`, `DIAG_MASK_INF`（因果掩码）
- **注意力操作**：`FLASH_ATTN_EXT`（Flash Attention）

- **卷积操作**：CONV_1D_1S, CONV_1D_2S, CONV_2D
- **SSM 操作**：SSM_CONV, SSM_SCAN（Mamba）

每个操作都有参数化的行为。例如 GGML_OP_RMS_NORM 携带 $\epsilon$ 作为参数；GGML_OP_ROPE 携带 $n_dims$, $mode$, $n_ctx$, $freq_base$ 等参数。

### 21.3.3　计算图的构建与执行

GGML 采用“构造-执行”分离模式：

1. **图构建阶段**：调用各种工厂函数（如 ggml_mul_mat()）创建操作节点。每个工厂函数分配一个 ggml_tensor 并设置其操作类型和源张量。
2. **图的展开**：调用 ggml_build_forward_expand(cgraph, tensor) 将目标张量及其依赖的整个计算图递归地添加到 ggml_cgraph 中。
3. **执行**：调用 ggml_graph_compute_with_ctx(cgraph, cplan, n_threads) 按拓扑顺序依次执行每个操作节点。

**GGML 图构建算法**

```
void ggml_build_forward_expand(ggml_cgraph *cgraph, ggml_tensor *tensor) {
    if (tensor 已访问)  return;
    标记 tensor 为已访问;
    if (tensor->op != GGML_OP_NONE) {
        递归处理 tensor->src[0];
        递归处理 tensor->src[1];
    }
    将 tensor 添加到 cgraph->nodes[];
    更新 cgraph->n_nodes++;
}
```

#### 21.3.3.1　执行调度

执行时，GGML 遍历 cgraph->nodes[] 数组并调用相应后端的计算实现：

```
for i in 0..n_nodes:
    node = cgraph->nodes[i]
    node->backend->iface.compute_forward(backend, node)
```

每个后端提供其实现的所有操作的调度表。如果某个操作在目标后端上没有实现（例如 CUDA 后端不支持某些小众操作），可以回退到 CPU 后端。

### 21.3.4 内存管理

GGML 使用多种内存策略：

- **全局缓冲区（hash set）**：使用哈希表管理已分配的张量缓冲区，确保相同形状和类型的张量复用同一内存（对象精确复用）。`ggml_hash_set` 结构维护已分配缓冲区的集合。
- **图级 scratch 缓冲区**：每个计算图关联一个临时缓冲区，用于存储中间计算结果。执行完成后释放。
- **后端缓冲区**：每个后端（CPU、CUDA 等）有自己的内存分配器。`ggml_backend_buffer_t` 是后端管理的连续内存块。
- **内存映射**：模型权重文件通过 mmap 映射到虚拟地址空间，操作系统按需加载页面（缺页中断），避免了将整个模型加载到内存的需要。

## 21.4 后端系统

### 21.4.1 后端抽象层

GGML 的后端系统定义了一个统一的接口，使不同的硬件加速器可以“即插即用”。核心接口包括：

**ggml_backend 接口**

```
struct ggml_backend_i {
    const char *get_name(ggml_backend_t backend);
    void free(ggml_backend_t backend);
    ggml_backend_buffer_type_t get_default_buffer_type(backend);
    void set_tensor_async(...);        // 异步拷贝数据到张量
    void get_tensor_async(...);        // 异步读取张量数据
    bool cpy_tensor_async(...);        // 张量间拷贝
    void synchronize(backend);         // 同步（等待所有操作完成）
    ggml_backend_graph_plan_t graph_plan_create(backend, cgraph);
    void graph_plan_free(backend, plan);
    ggml_status graph_plan_compute(backend, plan);
    bool supports_op(backend, tensor); // 查询是否支持某操作
    bool offload_op(backend, tensor);  // 是否卸载到本后端
    // 事件同步接口
    ggml_backend_event_t event_new(backend);
    void event_free(event);
```

```
    void event_record(event);
    void event_wait(backend, event);
    void event_synchronize(event);
};
```

### 21.4.2　CPU 后端

CPU 后端是最基础和最重要的后端，它在纯 C 代码中实现了所有 GGML 操作，并通过预处理器宏和运行时检测选择最优的 SIMD 路径。

#### 21.4.2.1　SIMD 指令集支持

表 21.6: CPU 后端 SIMD 支持

| 指令集 | 架构 | 向量宽度 | 关键操作 |
|---|---|---|---|
| AVX2 | x86_64 | 256 bit | 8×F32, 16×F16 |
| AVX-512 (F/VNNI/BF16) | x86_64 | 512 bit | 16×F32, 32×F16 |
| AVX-512 VNNI | x86_64 | 512 bit | 4×u8s8 dot prod |
| NEON | ARM64 | 128 bit | 4×F32, 8×F16 |
| SVE (可伸缩向量) | ARM64 | 128-2048 bit | 架构可变 |
| SSSE3 | x86_64 | 128 bit | 量化操作后备 |
| WASM SIMD | WebAssembly | 128 bit | WASM 目标 |

运行时检测使用 `ggml_cpu_has_avx2()`, `ggml_cpu_has_avx512()` 等函数，根据 CPUID 指令的结果选择最优实现。

#### 21.4.2.2　vec_dot 微内核

对于量化矩阵乘法（如 Q4_0 × F32），GGML 的核心是最内层的点积循环。每个量化类型、每个 SIMD ISA 都有独立的 `ggml_vec_dot_*` 实现。例如：

```
ggml_vec_dot_q4_0_q8_0(int n, float *s,
    const block_q4_0 *x, const block_q8_0 *y)
```

这里 $n$ 是块的数量（不是元素数量），$x$ 是量化的权重（每块 32 元素），$y$ 是即时量化的激活值（Q8_0 格式）。微内核在循环内部：

1. 从 $x$ 中解量化的 scale 和 4-bit 值。
2. 从 $y$ 中读取 8-bit 值和 scale。
3. 执行 SIMD 的整数乘加（mad）或 dot 指令（如 AVX-512 VNNI 的 `_mm512_dpbusd_epi32`）。
4. 累积结果并乘以 scale。

### 21.4.3 CUDA 后端

CUDA 后端将选定的计算图节点卸载到 NVIDIA GPU 上执行。关键实现点：

- **后端初始化**：使用 CUDA Runtime API 创建流、加载 PTX/JIT 编译内核。
- **张量拷贝**：`set_tensor_async` 使用 `cudaMemcpyAsync` 将张量数据传输到 GPU 内存。
- **同步点**：只有部分操作在 GPU 上执行。当 GPU 后端遇到不支持的操作时，需要在 GPU 和 CPU 之间插入数据传输节点。GGML 通过在图中自动插入拷贝节点来处理这一点。
- **图计划**：`graph_plan_create` 可以编译 CUDA Graph（`cudaGraphLaunch`），将重复执行的模式（如每个 decode 步骤）捕获为 CUDA Graph 以减少 kernel launch 开销。

支持的 GPU 操作主要包括：`MUL_MAT`（矩阵乘法，通过 cuBLAS 或自定义内核）、`SOFT_MAX`、`RMS_NORM`、`ROPE`、`FLASH_ATTN_EXT`、`ADD`、`MUL`、`CPY` 等。

### 21.4.4 Metal 后端

Metal 后端通过 Apple 的 Metal Shading Language (MSL) 实现 GPU 加速，支持所有 Apple Silicon 设备（M1、M2、M3、M4 系列及后续）。它使用 Metal Performance Shaders (MPS) 进行矩阵乘法优化。

### 21.4.5 Vulkan 后端

Vulkan 后端通过 SPIR-V 计算着色器提供跨平台 GPU 加速，支持 NVIDIA、AMD、Intel GPU。使用 `kompute` 库管理 Vulkan 资源。

### 21.4.6 多后端执行与张量分割

在多 GPU 配置中，GGML 支持将张量沿特定维度分割到不同的 GPU 上（tensor parallelism）。

**多后端图分割**

`tensor_split` 参数是一个浮点数组，指定总计算量在每个 GPU 上的分配比例。GGML 通过以下方式分割计算图：

1. 在 `MUL_MAT` 节点插入 split 操作：将权重矩阵沿 $ne[1]$（列）维度分割到多个 GPU。
2. 每个 GPU 对其负责的部分执行矩阵乘法。
3. 使用 `ADD` 或 `ALL_REDUCE` 操作聚合结果。

4. 插入必要的同步节点（events/barriers）。

## 21.5　模型加载与架构实例化

### 21.5.1　加载流程

模型加载的核心函数是 `llama_model_load()`，其执行流程如下：

**模型加载流程**

1. **打开 GGUF 文件**：通过 mmap 或标准文件 IO 打开。
2. **解析头部**：验证 magic number 和版本，读取 tensor count 和 metadata KV count。
3. **解析元数据**：遍历所有键值对，填充 `llama_hparams` 和 `llama_cparams`。
4. **选择架构**：根据 `general.architecture` 选择架构构建函数。
5. **分配模型结构**：创建 `llama_model`，包括 $n_layer$ 个 `llama_layer`。
6. **构建计算图**：为每个层调用架构特定的构建函数，创建 GGML 操作节点（`MUL_MAT`, `RMS_NORM`, `ROPE` 等）。
7. **绑定权重**：将 GGUF 文件中的张量（通过偏移量 mmap 访问）分配给对应的 GGML 层权重张量。
8. **分配 KV 缓存**：根据上下文长度和层数创建 KV 缓存数据结构和缓冲区。
9. **返回模型句柄**：`llama_model*` 指针。

**延迟加载（mmap）的优势**

使用 mmap 而非显式的 `read()` 系统调用有以下好处：

- **按需分页**：只有被访问的页面才会从磁盘加载到物理内存。对于量化模型中的大部分权重可能永远不会被完全读入——只有活跃层的权重在分页中。
- **零拷贝访问**：GGML 张量直接指向 mmap 的区域，无需额外的数据缓冲区。
- **操作系统缓存**：MMAP 的内容由 OS 的页面缓存管理，多次加载同一模型更快。
- **多个进程共享**：多个 llama.cpp 进程可以共享同一 mmap 的物理页面（只读共享），大幅减少总体内存占用。

### 21.5.2 核心数据结构

表 21.7: llama.cpp 核心数据结构

| 结构体 | 描述 |
|---|---|
| `llama_hparams` | 模型超参数：$n_vocab$, $n_embd$, $n_layer$, $n_head$, $n_head_kv$, $n_ff$, $n_embd_head_k$, $n_embd_head_v$, $n_rot$, $f_norm_eps$, $f_rope_freq_base$ 等 |
| `llama_cparams` | 上下文参数：$n_ctx$, $n_batch$, $n_ubatch$, $n_seq_max$, $n_threads$, $rope_scaling_type$, $rope_freq_scale$, $yarn_ext_factor$ 等 |
| `llama_layer` | Transformer 层：包含所有该层的 GGML 张量（`attn_norm`, `wq`, `wk`, `wv`, `wo`, `ffn_norm`, `ffn_gate`, `ffn_up`, `ffn_down`） |
| `llama_model` | 模型：包含 $n_layer$ 个层、词嵌入表 `embd`、输出投影 `output`、tokenizer 等 |

### 21.5.3 架构适配

不同模型架构的主要差异体现在层内的运算组合方式。llama.cpp 为每种架构定义了构建函数。例如：

- **LLaMA**：标准架构。RMSNorm → Q/K/V 投影 → RoPE → 注意力 → RMSNorm → FFN（SiLU 门控）。
- **Mistral**：类似 LLaMA，但使用滑动窗口注意力（SWA），在偶数层和所有层之间交替使用 SWA 和全局注意力。
- **Mixtral**：MoE 架构。FFN 替换为多个专家网络 + 路由网络（Top-K 软门控）。
- **Phi-3**：使用部分 RoPE、偏置维度、特殊的块稀疏注意力模式。
- **Gemma 2**：使用 GeGLU 激活、pre-norm 和 post-norm 混合、logit soft-capping。

每种变体通过在模型加载时选择不同的“层构建函数”来处理，这些函数使用相同的 GGML 操作原语组合出不同的计算图。

### 21.5.4 量化加载

llama.cpp 支持在加载时或加载后进行权重量化：

- **预量化模型：**直接加载 GGUF 文件中的量化权重（最常见的用法，如从 HuggingFace 下载的 `.gguf` 文件）。

- **即时量化：**加载 FP16 模型并在推理前使用重要性矩阵（imatrix）进行量化。K-quant 和 IQ-quant 系列类型使用 imatrix 信息来为重要 token 分配更多精度。

imatrix（importance matrix）记录每个权重元素在模型前向传播中的重要性（基于梯度或激活值的统计），使量化器能更好地保留重要权重的精度。

## 21.6　前向传播实现

### 21.6.1　推理主循环

推理的核心函数是 `llama_decode()`，它处理输入 token 批次并更新 KV 缓存和 logits。

推理主循环（简化版）

```
int llama_decode(llama_context *ctx, llama_batch batch) {
    const int n_tokens = batch.n_tokens;
    // 步骤1: 为每个token分配KV缓存位置
    llama_kv_cache_find_slot(kv_self, batch);
    // 步骤2: 构建前向计算图
    for each token in batch:
        // 词嵌入查找
        cur = ggml_get_rows(embd, tokens[token]);
        // 遍历每一层
        for each layer in 0..n_layer:
            // RMSNorm
            cur = ggml_rms_norm(attn_norm[layer], cur);
            // Q/K/V 投影
            Q = ggml_mul_mat(wq[layer], cur);
            K = ggml_mul_mat(wk[layer], cur);
            V = ggml_mul_mat(wv[layer], cur);
            // RoPE 位置编码
            Q = ggml_rope(Q, position);
            K = ggml_rope(K, position);
            // 更新KV缓存并构建注意力
            cur = ggml_flash_attn_ext(Q, K, V,
                kv_cache, mask, scale);
            // RMSNorm + FFN (SiLU门控)
            cur = ggml_add(cur,
                ggml_mul_mat(ffn_down[layer],
                    ggml_silu(
                        ggml_mul_mat(ffn_gate[layer],
                            ggml_rms_norm(ffn_norm[layer], cur))
                    ) * ggml_mul_mat(ffn_up[layer],
                        ggml_rms_norm(ffn_norm[layer], cur))
                ));
        // 最终RMSNorm + 输出投影
        cur = ggml_rms_norm(output_norm, cur);
        logits = ggml_mul_mat(output, cur);
    // 步骤3: 执行计算图
    ggml_backend_graph_compute(ctx->backend, cgraph);
    return 0;
}
```

1.

2.

3.

**KV 缓存布局**

物理存储是一个四维张量的集合，每层对应一对 (K, V) 缓存：

$$\underbrace{\boldsymbol{K}_{\text{cache}}}_{\text{形状: } (n_embd_k, n_head_kv, n_ctx)} \qquad \underbrace{\boldsymbol{V}_{\text{cache}}}_{\text{形状: } (n_embd_v, n_head_kv, n_ctx)}$$

在 GGML 的维度约定中：

- $ne[0] = n_embd_head_k$（每个头的 K 维度，即 $d_k$）
- $ne[1] = n_ctx$（最大上下文长度）
- $ne[2] = n_head_kv$（KV 头数）
- $ne[3] = n_layer$ 或独立分配

1.

2.

3.

**llama.cpp Flash Attention（简化的核心逻辑）**

```
// 对每个query头和每个注意力块
for each query_head h:
    for each KV block b:
        // 加载Q块 (1×d_k)
        // 加载K块 (block_size × d_k)
        // 加载V块 (block_size × d_v)
        scores = Q[h] @ K[b].T
        scores = scores * scale
        scores = scores + mask[b]
        // 在线softmax更新
        max_new = max(max_old, max(scores))
        sum_new = sum_old * exp(max_old - max_new)
                + sum(exp(scores - max_new))
        output = output * (sum_old/sum_new) * exp(max_old-max_new)
               + exp(scores-max_new) @ V[b]
        max_old = max_new; sum_old = sum_new
```

- 
- 
- 

1.

2.

3.

- 

- 

- 

1.

2.

- 

- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 

表 21.8: LLaMA 3 7B (Q4_K_M) CPU 推理性能

| 硬件 | 线程数 | pp 速度 (t/s) | tg 速度 (t/s) |
|---|---|---|---|
| Apple M2 Max | 8 | 120 | 45 |
| Apple M3 Max | 12 | 180 | 65 |
| AMD Ryzen 7950X (AVX-512) | 16 | 95 | 38 |
| Intel i9-13900K (AVX2) | 24 | 70 | 28 |
| AMD Ryzen 5600X | 6 | 42 | 16 |

表 21.9: LLaMA 3 70B (Q4_K_M) 混合 CPU/GPU 推理

| GPU 卸载层数 | tg 速度 (t/s) | GPU 内存使用 |
|---|---|---|
| 0 (纯 CPU) | 4.2 | 0 GB |
| 10 层 | 8.7 | 8 GB |
| 20 层 | 11.3 | 14 GB |
| 40 层 | 15.1 | 22 GB |
| 全部 80 层 | 18.6 | 38 GB |

表 21.10: 不同量化类型的困惑度（perplexity）对比（LLaMA 3 8B, WikiText-2）

| 量化类型 | 模型大小 (GB) | 困惑度 | 质量损失 |
|---|---|---|---|
| FP32 | 32.0 | 6.14 | –（基线） |
| FP16 | 16.0 | 6.14 | 0.00 |
| Q8_0 | 9.0 | 6.14 | 0.00 |
| Q6_K | 7.0 | 6.15 | +0.01 |
| Q5_K_M | 6.0 | 6.16 | +0.02 |
| Q4_K_M | 5.0 | 6.19 | +0.05 |
| Q3_K_M | 4.0 | 6.31 | +0.17 |
| Q2_K | 3.2 | 6.85 | +0.71 |
| IQ4_XS | 5.0 | 6.17 | +0.03 |
| IQ3_XXS | 3.8 | 6.30 | +0.16 |

- 
- 
- 
- 

**不是教条，而是实用选择**

这并不意味着 C 在所有场景中都优于 C++。llama.cpp 的许多贡献者使用 C++ 编写 CUDA 内核（部分 CUDA 后端代码是 `.cu`）。关键是在接口层保持 C ABI 的简洁性。

1. 

2. 

3.

4.

- 
- 
- 
- 
- 
- 
- 

**llama.cpp 的工程智慧**

1. **mmap 是一切：**内存映射不仅节省内存，还避免了显式的序列化/反序列化。权重数据在文件和内存中是同一个物理页面。
2. **延迟量化：**在加载时根据可用资源和精度要求动态选择量化策略。
3. **微内核狂热：**对性能最关键的路径（vec_dot），为每个量化类型 × 每个 SIMD ISA 都手写优化。

4. **C API 作为契约：**所有外部交互通过 C API，内部可以任意重构。
5. **增量计算：**每次只处理一个 (或一批) token，将状态维护在 KV 缓存中。这使推理与生成的长度无关（在缓存容量内）。
6. **社区驱动：**支持 100+ 模型架构不是几个人能做到的，而是通过清晰的架构抽象让社区贡献者容易添加新模型。

**本章核心要点**

1. GGUF 格式是实现高效推理的文件基础——自描述、量化原生、mmap 友好。
2. GGML 张量库是 llama.cpp 的灵魂——从零构建的、量化优先的、操作丰富的张量引擎。
3. 后端系统实现了从纯 CPU 到多 GPU 的无缝扩展，关键在于操作粒度的统一抽象。
4. KV 缓存是推理的性能核心，llama.cpp 通过量化和按需分配优化了其内存使用。
5. 量化不是权宜之计，而是让 AI 推理走向普惠的关键技术路径。

**ONNX Runtime 的设计目标**

1. **框架无关：**接受任何能导出 ONNX 的框架（PyTorch、TensorFlow、Keras、scikit-learn、JAX 等）的模型。
2. **硬件无关：**通过 Execution Provider 抽象支持 CPU、GPU、NPU、FPGA 等各类加速器。
3. **生产就绪：**提供 C、C++、C#、Python、Java、JavaScript/WebAssembly、Rust 等多语言 API。
4. **企业特性：**模型加密、量化和优化工具链、跨平台构建系统、与 Azure/Windows ML 的深度集成。

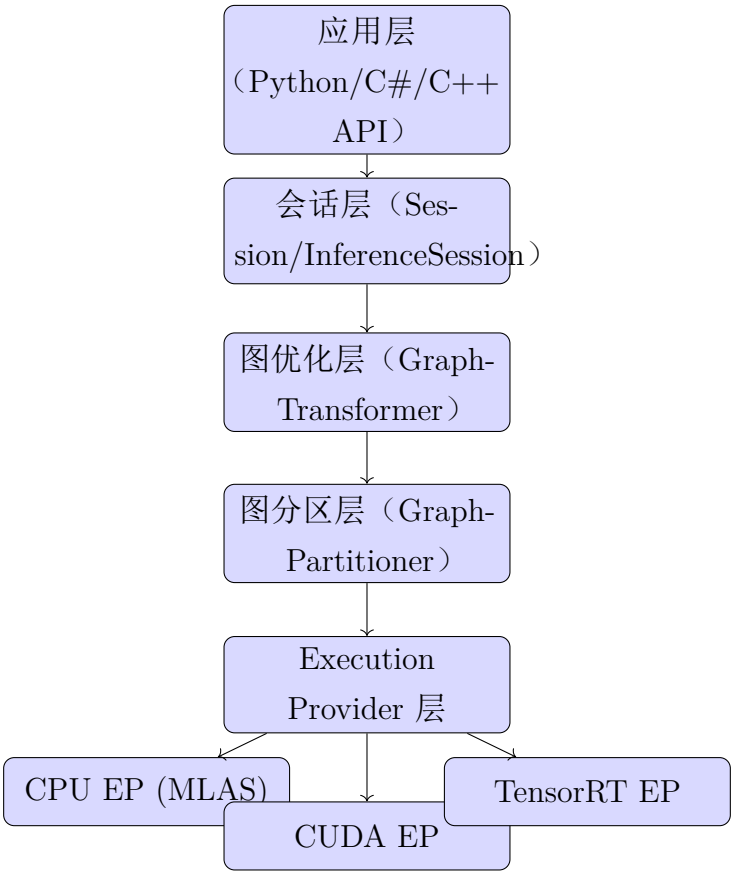



图 22.1: ONNX Runtime 架构层次

表 22.1: ONNX Runtime 核心模块

| 模块 | 路径 | 功能 |
|---|---|---|
| Graph | `onnxruntime/core/graph/` | ONNX 模型解析、中间表示（IR）构建、图遍历 |
| Optimizer | `onnxruntime/core/optimizer/` | 图级优化：规则匹配、节点融合、常量折叠 |
| Execution Provider | `onnxruntime/core/providers/` | 后端抽象、内核注册、内存管理 |
| Session | `onnxruntime/core/session/` | 会话生命周期、执行计划构建、状态管理 |
| MLAS | `onnxruntime/core/mlas/` | CPU 线代库：SGEMM、QGEMM、卷积等 |
| QDQ | `onnxruntime/core/optimizer/qdq_transformer/` | QDQ 格式的量化插入和优化 |

- 
- 
- 
- 
- 
- 
- 
- 
- 
- 

- 
- 
- 
- 

- 
- 

**Initializer vs Input**

在 ONNX 图中，权重通过 `initializer` 而非 `input` 存储。这允许 ORT 在运行时直接将 initializer 分配给设备端存储器并进行优化（如预包装、量化），而输入是在每次推理时由用户提供的数据。

- 
- 
- 
- 

**EP 核心接口**

```
class IExecutionProvider {
    // 获取EP名称、类型
    virtual std::string Type() const = 0;

    // 能力查询：是否支持特定节点
    virtual std::vector<NodeIndex> GetCapability(
        const GraphViewer& graph,
        const IKernelLookup& kernel_lookup) const = 0;

    // 为支持的节点编译内核
    virtual common::Status Compile(
        const std::vector<FusedNodeAndGraph>& fused_nodes,
        std::vector<NodeComputeInfo>& node_compute_funcs);

    // 内存管理
    virtual AllocatorPtr GetAllocator(
        OrtDevice device) const = 0;

    // 数据拷贝
    virtual common::Status OnRunStart() = 0;
    virtual common::Status OnRunEnd() = 0;
```

```
};
```

表 22.2: ONNX Runtime 内置 Execution Provider

| **EP 名称** | **代号** | **目标硬件** | **关键特征** |
|---|---|---|---|
| CPU | CPU | x86/ARM CPU | MLAS 驱动，全算子覆盖，回退基础 |
| CUDA | CUDA | NVIDIA GPU | cuBLAS/cuDNN，部分自定义内核 |
| TensorRT | TRT | NVIDIA GPU | 子图编译为 TRT 引擎 |
| OpenVINO | OV | Intel CPU/GPU/VPU | Intel 硬件加速 |
| DirectML | DML | Windows GPU (DX12) | 任何 DX12 GPU（NVIDIA/AMD/Intel） |
| CoreML | CoreML | Apple Neural Engine | iOS/macOS 加速 |
| XNNPACK | XNNPACK | ARM/x86 CPU | 移动端优化，来自 Google |
| ROCm | ROCM | AMD GPU | HIP/rocBLAS/MIOpen |
| QNN | QNN | Qualcomm NPU | 骁龙设备上的 AI Engine |
| SNPE | SNPE | Qualcomm DSP | 骁龙设备上的 DSP 加速 |
| ACL | ACL | ARM CPU | Arm Compute Library（NEON 优化） |
| VitisAI | VitisAI | Xilinx FPGA | Xilinx DPU 加速 |

**基于能力的图分区（简化）**

1. 对于每个 EP（按优先级排序）：
   a. 调用 EP->GetCapability(graph)
      → 返回 EP 可执行的节点索引列表
   b. 将可执行节点标记为 "assigned_to_EP"
2. 对于未被任何 EP 覆盖的节点：
   a. 使用默认 EP（通常为 CPU）的 GetCapability 查询
   b. 标记为 CPU EP（确保所有节点都有 EP）
3. 构造分区子图：
   a. 遍历原始图的节点拓扑
   b. 连续的、属于同一 EP 的节点合并为一个子图
   c. 在子图边界插入 "MemcpyToDevice" / "MemcpyFromDevice" 节点
      （当子图的 EP 与前驱/后继的 EP 不同时）
4. 每个子图发送给对应 EP 的 Compile() 方法

- 
- 
- 
- 

- 
- 
- 

**优化规则的执行**

每个 `RewriteRule` 包含：

- 一个模式匹配函数：识别图中特定的节点组合（如 `Conv` → `BatchNorm` → `Relu`）。
- 一个重写函数：将匹配的模式替换为融合节点（如 `Conv+BN+Relu` 融合为一个 `FusedConv`）。

表 22.3: 关键图优化和融合模式

| 优化名称 | 输入模式 | 输出（融合后） |
|---|---|---|
| ConvBNReluFusion | Conv+BN+Relu | FusedConv |
| MatMulAddFusion | MatMul+Add | Gemm (FusedMatMul) |
| LayerNormFusion | ReduceMean+Sub+Pow+...+Mul+Add | LayerNormalization |
| SkipLayerNormFusion | Add+LayerNorm | SkipLayerNormalization |
| AttentionFusion | MatMul+Add+Softmax+MatMul | Attention (MultiHeadAttention) |
| GeluFusion | Mul+Mul+Add+Mul+Tanh | Gelu |
| FastGeluFusion | Mul+Erf+Add+Mul+Mul | FastGelu |
| BiasGeluFusion | Add+Gelu | BiasGelu |
| QDQPropagation | Q+DeQ+Op+Q+DeQ | QOp (量子化操作) |

- 
- 
- 
- 
- 
- 
- 

**MLAS SGEMM 的分块和包装策略**

**矩阵乘法 $\boldsymbol{C} = \boldsymbol{A} \cdot \boldsymbol{B}^T$ (或 $\boldsymbol{B}$)**

1. **分块：**将 $\boldsymbol{A}$ 按行分块为 $m \times k$ 的子矩阵，将 $\boldsymbol{B}$ 按列分块为 $k \times n$ 的子矩阵。$\boldsymbol{C}$ 的块尺寸为 $m \times n$。
2. **包装（Packing）：**
   - $\boldsymbol{A}$ 按 k 维度（最内层）包装为连续存储，使得内积可以顺序访问内存。
   - $\boldsymbol{B}$ 按 n 维度包装，使得可以并行存取多个输出列的中间结果。
3. **内核计算：**对每个 $m \times n$ 的 $\boldsymbol{C}$ 块，从包装后的 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 中加载数据，使用 SIMD FMA 指令（`_mm256_fma_ps` 或 `_mm512_fma_ps`）进行乘加操作。
4. **展开循环：**手动展开最内层的 k 循环以增加指令级并行性（ILP）。典型的展开因子为 4-8。
5. **线程调度：**使用 OpenMP 或自定义线程池将 $m$ 维度的工作分配到多个核心。每个核心处理一组连续的输出行。

- 
- 
- 
- 

表 22.4: MLAS 平台特定优化

| 平台 | 指令集 | 关键优化 | 向量宽度 |
|---|---|---|---|
| x86 Server | AVX-512 VNNI | int8 点积加速 | 512 bit |
| x86 Desktop | AVX2 + FMA | FP32 乘加 | 256 bit |
| ARM Server | NEON SDOT | int8 点积 → int32 | 128 bit |
| ARM Mobile | NEON MLA | FP32 乘加 | 128 bit |
| POWER | VSX | FP32 向量操作 | 128 bit |

**关键微内核示例：AVX2 SGEMM**

```
// 典型的微内核（8个累加器 × 12个输出列）
// A 行加载 (1×K), B 列加载 (12×K)
for (kk = 0; kk < K; kk += 4) {
    // 加载4个B元素到12个寄存器
    __m256 b0 = _mm256_broadcast_ss(B + kk + 0);
    __m256 b1 = _mm256_broadcast_ss(B + kk + 1);
    __m256 b2 = _mm256_broadcast_ss(B + kk + 2);
    __m256 b3 = _mm256_broadcast_ss(B + kk + 3);
    // 对12个输出列各执行一次FMA
    c0 = _mm256_fmadd_ps(_mm256_load_ps(A + kk), b0, c0);
    c1 = _mm256_fmadd_ps(_mm256_load_ps(A + kk), b1, c1);
    // ... (c2 到 c11 类似)
    A += 4; B += 4;
}
```

1.

2.

3.

- 
- 
- 

- 
- 
- 

1. 
2. 
3. 

- 

- 

- 

**TensorRT EP 的陷阱**

- **编译时间：**大型模型的 TRT 引擎编译可能需要 10-60 分钟。
- **算子覆盖：**某些操作不在 TensorRT 的原生算子集中（需要 ONNX-TRT 映射或回退到 CUDA EP）。
- **动态形状：**在 TensorRT 中处理动态形状需要 profile 定义和较复杂的配置。
- **版本锁定：**TRT 引擎与具体的 CUDA/cuDNN/TensorRT 版本绑定，环境更新后需要重新编译。

1. 

2. 

3. 

4. 

5. 

- 

表 22.5: SessionOptions 关键参数

| 参数 | 描述 |
| --- | --- |
| execution_mode | ORT_SEQUENTIAL（顺序执行）vs ORT_PARALLEL（图级并行） |
| graph_optimization_level | ORT_DISABLE_ALL, ORT_ENABLE_BASIC, ORT_ENABLE_EXTENDED, ORT_ENABLE_ALL |
| intra_op_num_threads | 单个操作内的线程数 |
| inter_op_num_threads | 不同操作间的并行度 |
| optimized_model_filepath | 保存优化后模型的路径（用于下次快速加载） |
| enable_cuda_graph | 启用 CUDA Graph 的自动捕获 |

- 
- 
- 
- 
- 
- 

- 
- 

1. 

2. 

3. 

- 
- 

1.

2.

3.

4.

- 

- 

- 

- 

- 

- 
- 
- 
- 
- 

**本章核心要点**

1. ONNX Runtime 是一个工业级的跨平台推理引擎，其核心价值在于——框架无关的模型表示 → 图优化 → 硬件适配的完整管线。
2. Execution Provider 抽象实现了真正的即插即用硬件支持，图分区算法是其中的关键技术。
3. 图优化（规则融合、常量折叠、布局变换）是 ORT 相比直接使用底层库的差异化价值。
4. MLAS 体现了为性能而生的极简数学库哲学——针对每个平台用手写 SIMD 优化核心循环。
5. QDQ 格式提供了一种可移植的量化表示，使得量化模型可以跨 EP 和跨平台地

优化和执行。

6. 生产部署中的关键实践：图缓存、IO Binding、动态批处理、并发推理。

**vLLM 的设计哲学**

1. **最大化 GPU 利用率：**GPU 是 LLM 服务中最昂贵的资源，vLLM 的每一个设计决策都围绕“让 GPU 更忙、让内存更有效”展开。
2. **虚拟内存式 KV 缓存：**不按最大序列长度预分配，而是按需分配固定大小的块——消除了内部碎片和外部碎片。
3. **连续批处理（Continuous Batching）：**在每次 forward pass 中动态调整批处理成员，消除静态批处理的空闲等待。
4. **开放且兼容：**提供 OpenAI 兼容的 API 接口，支持 HuggingFace 生态中几乎所有流行的 LLM。

**vLLM 的影响**

vLLM 在 2023 年 6 月发布后迅速成为 LLM serving 的事实标准——被 LMSYS（Chatbot Arena）、Anthropic、Databricks、AWS Bedrock 等组织采用或参考。其 PagedAttention 和连续批处理设计被后续框架（SGLang、TGI、TensorRT-LLM）广泛采纳。

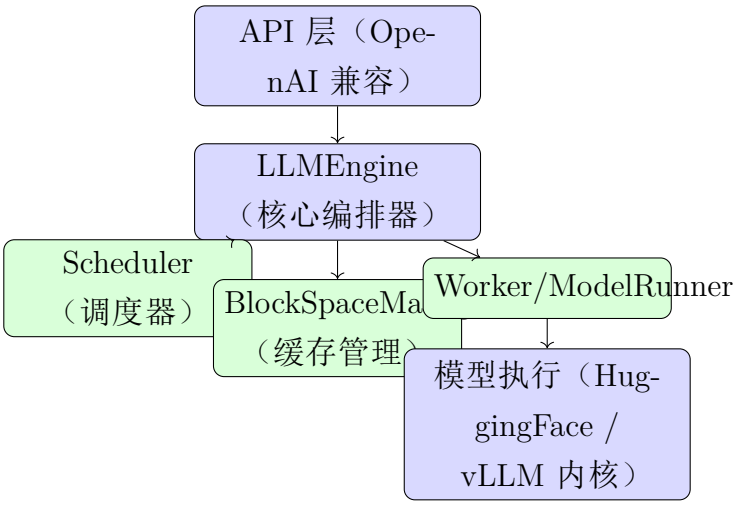



图 23.1: vLLM 架构概览

1.

2.

3.

**KV 缓存的内存占比**

在典型部署中，KV 缓存占 GPU 总内存的 20%-40%。在一个 80 GB A100 上运行 LLaMA 3 70B（约 36 GB 权重），剩余的 44 GB 中约 35 GB 用于 KV 缓存。预分配策略意味着只有少量请求能同时服务（例如 10-20 个），尽管 GPU 的计算能力还能处理更多。

•

- 
- 
- 

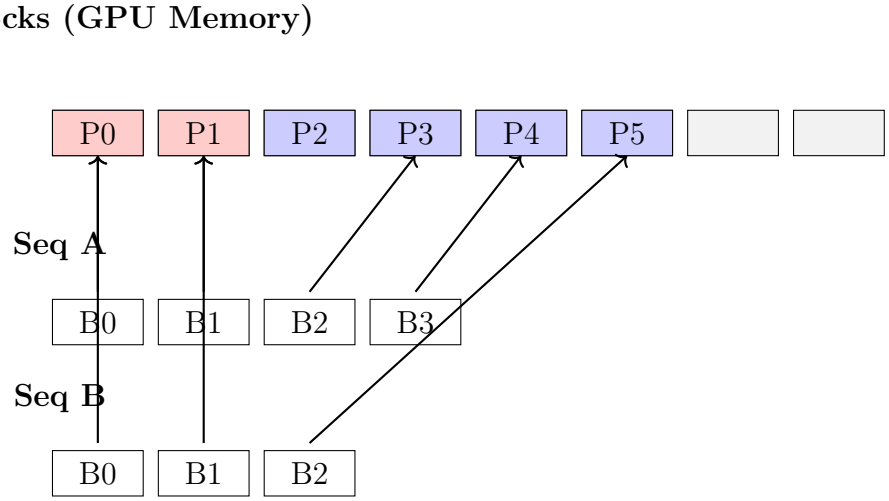



图 23.2: PagedAttention 示意图：Seq A 和 Seq B 共享前缀块 P0 和 P1（红色标为共享块），后续块各自独立分配。逻辑块通过块表映射到物理块。

表 23.1: 块大小的权衡

| **权衡维度** | 小 $B$（如 **8**） | 大 $B$（如 **32**） |
|---|---|---|
| 内部碎片 | **少**：浪费的 token 槽少 | **多**：块内未使用的槽更多 |
| 块表大小 | **大**：$n_tokens/B$ 更大 | **小**：块表开销小 |
| GPU 内核效率 | **低**：更多小块的循环 | **高**：块内向量化更好 |
| 共享粒度 | **细**：可共享更短的前缀 | **粗**：共享粒度粗 |

- 

-

-

**PagedAttention 前向内核**

```
// 输入: query[n_heads, d_head]
// 输入: key_cache[num_blocks, block_size, n_kv_heads, d_head]
// 输入: value_cache[num_blocks, block_size, n_kv_heads, d_head]
// 输入: block_table[request_id, max_blocks]
// 输入: context_lens[request_id]

for each request in batch:
    n_kv_tokens = context_lens[request]
    num_blocks = ceil(n_kv_tokens / BLOCK_SIZE)

    // 对每个query头
    for each head h in n_heads:
        kv_head = h // (n_heads / n_kv_heads)  // GQA映射
        acc = zeros[d_head]  // 累加器
        m = -inf, l = 0      // 在线softmax状态

        // 遍历所有块
        for block_idx in 0..num_blocks:
            phys_block = block_table[request][block_idx]
            k_block = key_cache[phys_block, :, kv_head, :]
            v_block = value_cache[phys_block, :, kv_head, :]

            // 计算部分点积
            scores = Q[h] @ k_block.T  // (block_size,)
            scores = scores * scale + mask[block_idx * BLOCK_SIZE:]

            // 在线softmax更新
            m_new = max(m, max(scores))
            l_new = l * exp(m - m_new) + sum(exp(scores - m_new))
            acc = acc * (l/l_new) * exp(m - m_new)
                + exp(scores - m_new) @ v_block
            m = m_new; l = l_new

        output[h] = acc
```

- 
- 
- 
- 
- 

1. 

2. 

-

-

-

**连续批处理迭代循环**

```
while True:
    // 1. 收集新的到达请求（如果有）
    从请求队列中取出所有等待的请求，
    分配到可用的块表中。

    // 2. 选择要处理的请求
    scheduled = scheduler.schedule(
        running_requests + waiting_requests,
        free_blocks)

    // 3. 一个前向传播
    model_output = model_runner.execute(
        scheduled.seq_groups)

    // 4. 处理输出token
    for each seq_group in scheduled.seq_groups:
        token = sample(model_output[seq_group])
        seq_group.append_token(token)
        更新 KV 缓存块（分配新块如果需要）

        if token == EOS:
            seq_group.mark_finished()
            释放序列的块

    // 5. 从running中移除已完成的序列
    remove_finished_sequences()

    // 6. 检查是否所有请求都完成
    if len(running_requests) == 0 and len(waiting) == 0:
        break
```

-

-

**Prefill-Decode 调度策略**

vLLM 支持以下调度策略：

1. **Priority-based：**每个请求有优先级。prefill 阶段的请求优先于 decode，以快速发出第一个 token。
2. **Chunked Prefill：**长 prompt 的 prefill 分成多个 chunk，每个 chunk 在单独的迭代中执行。这使得 prefill 工作与 decode 交错，避免了因一个长 prefill 导致的 decode 延迟抖动。
3. **Preemption：**如果新请求的 prefill 会超过内存预算，则抢占现有 running 请求。

-

-

-

1.

2.

3.

4.

5.

1.

2.

3.

- 
- 
- 
- 

表 23.2: Sequence 对象的关键字段

| 字段 | 描述 |
| --- | --- |
| `seq_id` | 序列的唯一标识 |
| `prompt_token_ids` | Prompt token ID 数组 |
| `output_token_ids` | 已生成的输出 token ID 数组 |
| `logical_token_blocks` | 逻辑块编号列表（块表） |
| `block_table` | 物理块表（int 数组） |
| `state` | 序列状态：WAITING, RUNNING, SWAPPED, FINISHED_STOPPED, FI |
| `status` | 完成状态 |

- 

-

-

-

1.

2.

3.

4.

-

-

- 
- 
- 
- 

**张量并行的通信模式**

关键通信操作：

- **All-Reduce**：用于行并行线性层和注意力输出聚合。
- **All-Gather**：用于列并行线性层的输出拼接和 embedding 层。
- **Reduce-Scatter**：用于行并行线性层的输入分发。这些操作通过 NCCL（NVIDIA Collective Communications Library）实现，跨 GPU 使用 NVLink 或 InfiniBand。

- 
- 
- 
- 
- 

表 23.3: GPU 内存使用分布（LLaMA 3 70B, A100 80GB）

| 内存用途 | 大小 (GB) | 占比 |
|---|---|---|
| 模型权重（FP16） | 36.0 | 45% |
| KV 缓存 | 32.0 | 40% |
| 激活/临时缓冲 | 8.0 | 10% |
| CUDA 上下文/其他 | 4.0 | 5% |
| **总计** | **80.0** | **100%** |

- 
- 
- 

- 
- 
- 
- 
- 
- 

**CUDA Graph 捕获与执行**

1. **预热运行：**用实际 batch shape 执行一次完整的前向传播，记录所有内核启动和依赖。
2. **图捕获：**在另一次前向传播中启用 CUDA Graph 捕获模式，将整个 decode 前向过程记录为一个 CUDA Graph。
3. **运行时启动：**后续具有相同 batch size 和 shape 的 decode 步骤，只需一次 `cudaGraphLaunch()` 调用。
4. **参数更新：**CUDA Graph 在每次启动前通过更新图实例化参数来调整输入 token 的值和位置，而图的拓扑结构不变。

- 
- 
- 
- 
- 
- 

**OpenAI 兼容 API 的意义**

OpenAI 兼容 API 使 vLLM 可以无缝替代任何使用 OpenAI SDK 的应用——不需要修改客户端代码，只需将 `base_url` 指向 vLLM 服务器即可。这在企业部署中消除了迁移成本。

- 
- 
- 
- 
- 
- 
- 

- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 

表 23.4: LLaMA 3 8B serving 性能对比

| 指标 | vLLM | HF + Static Batching | 提升倍数 |
|---|---|---|---|
| 最大吞吐量 (tokens/s) | 3850 | 1200 | 3.2× |
| 同时服务请求数 | 256 | 32 | 8× |
| 平均 TTFT (ms) | 45 | 120 | 2.7× |
| 平均 TBT (ms) | 12 | 28 | 2.3× |
| GPU 内存利用率 | 92% | 45% | 2.0× |

表 23.5: KV 缓存浪费率对比

| 场景 | 传统方法 | PagedAttention |
|---|---|---|
| $\mu_{seq} = 256$, max_seq=2048, 32 requests | 87.5% | 3.1% |
| $\mu_{seq} = 512$, max_seq=4096, 64 requests | 87.5% | 5.2% |
| $\mu_{seq} = 128$, max_seq=1024, 128 requests | 87.5% | 2.8% |

- 
- 
- 
- 

- 
- 
- 
- 
- 

**本章核心要点**

1. PagedAttention 是 vLLM 的皇冠明珠——将操作系统虚拟内存原理应用于 KV 缓存管理，消除了预分配浪费，实现了块级共享。
2. KV 缓存分为固定大小的块（默认 16 token），通过块表映射逻辑块到物理块，按需分配，引用计数，COW 共享。
3. 连续批处理消除了静态批处理的空闲插槽问题——每次迭代动态选择参与 forward pass 的请求。
4. Chunked Prefill 通过将长 prompt 的 prefill 分片执行，平衡了 prefill 的延迟敏感性和 decode 的吞吐需求。
5. 张量并行与 PagedAttention 的融合实现了跨 GPU 的高效推理，通过 NCCL 通信原语管理数据分发和聚合。
6. 融合内核、CUDA Graph、前缀缓存等优化将 GPU 利用率推向理论极限。
7. 生产级特性（OpenAI API 兼容、流式输出、Multi-LoRA、结构化输出）使 vLLM 成为 LLM serving 的事实标准。

1.

2.

3.

4.

5.

6.

7.

8.

9. https://github.com/ggerganov/llama.cpp

10. https://github.com/microsoft/onnxruntime

11. https://github.com/vllm-project/vllm

12.

13.

14.

15.

16.

17.

18.

19.

20.

21.

22.

23.

24.

25.

26.